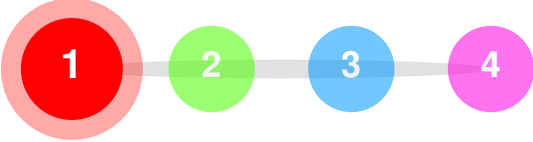


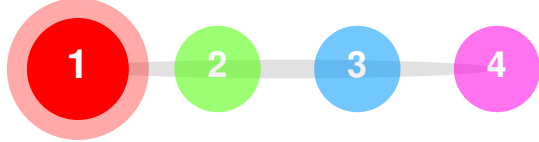
Efficient Extraction from Effectful E-Graphs



Oliver Flatt, Anjali Pal, Yihong Zhang, Ryan Tjoa, Kirsten Graham, Alex Fischman,
Chandrakana Nandi, Eli Rosenthal, Zachary Tatlock, Haobin Ni



Motivate Effectful E-Graphs



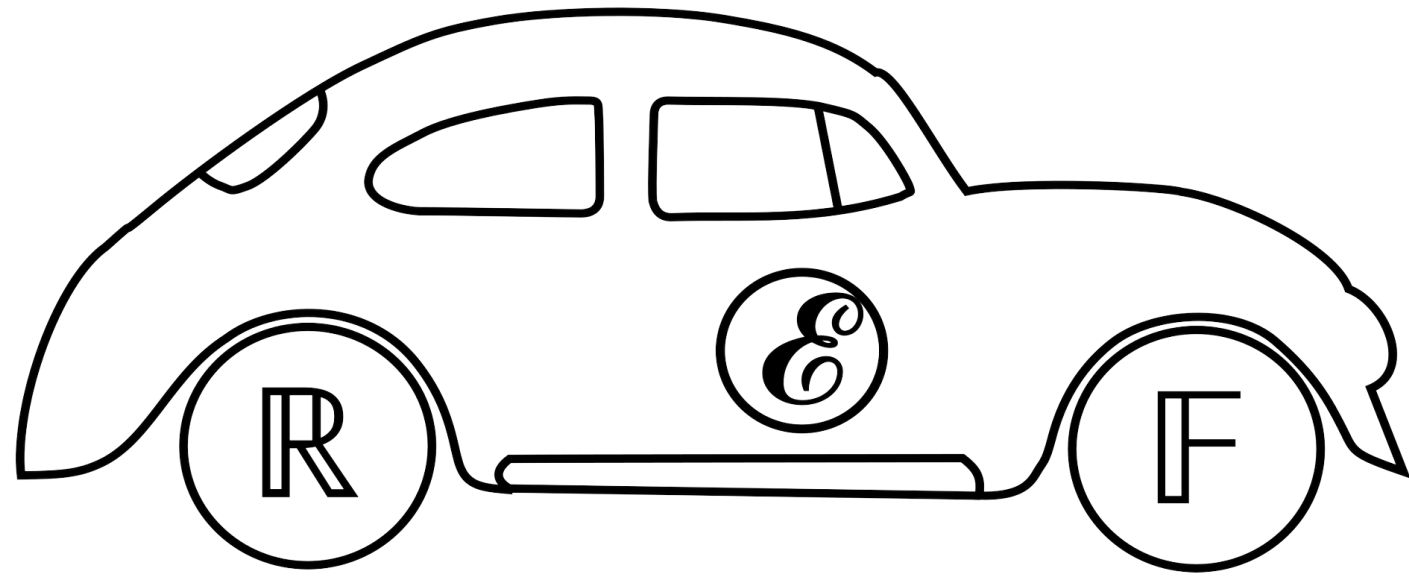
Motivate Effective E-Graphs

E-Graphs are Super Useful!



E-Graphs are Super Useful!

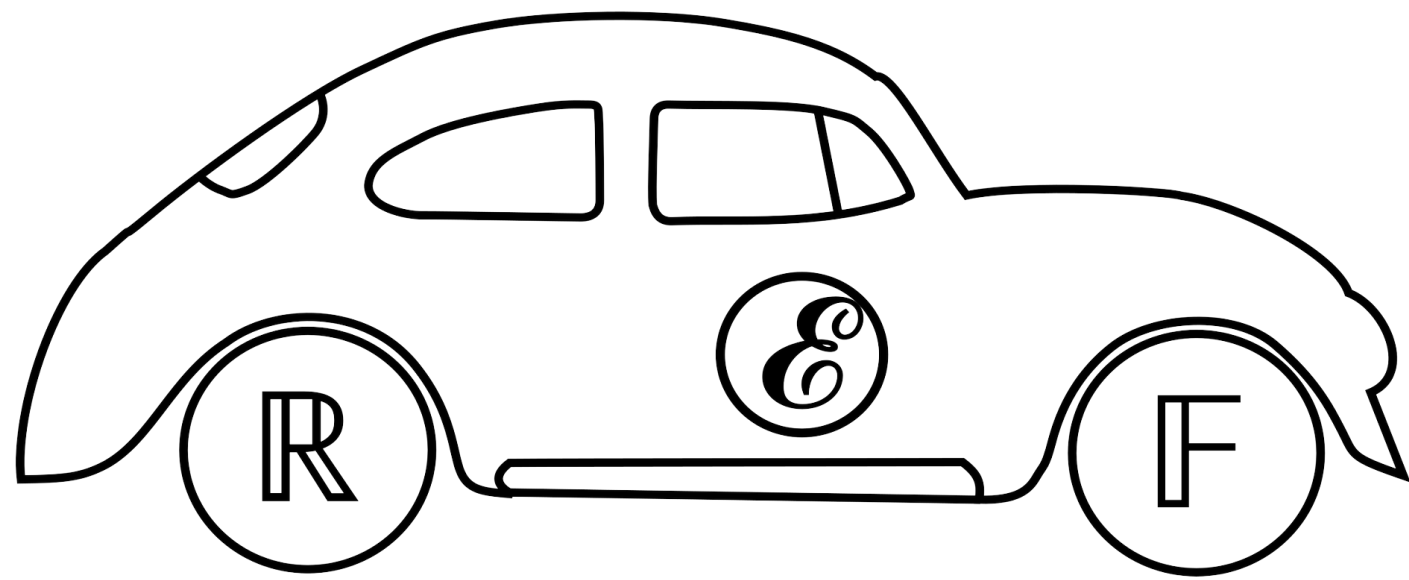
Floating point





E-Graphs are Super Useful!

Floating point

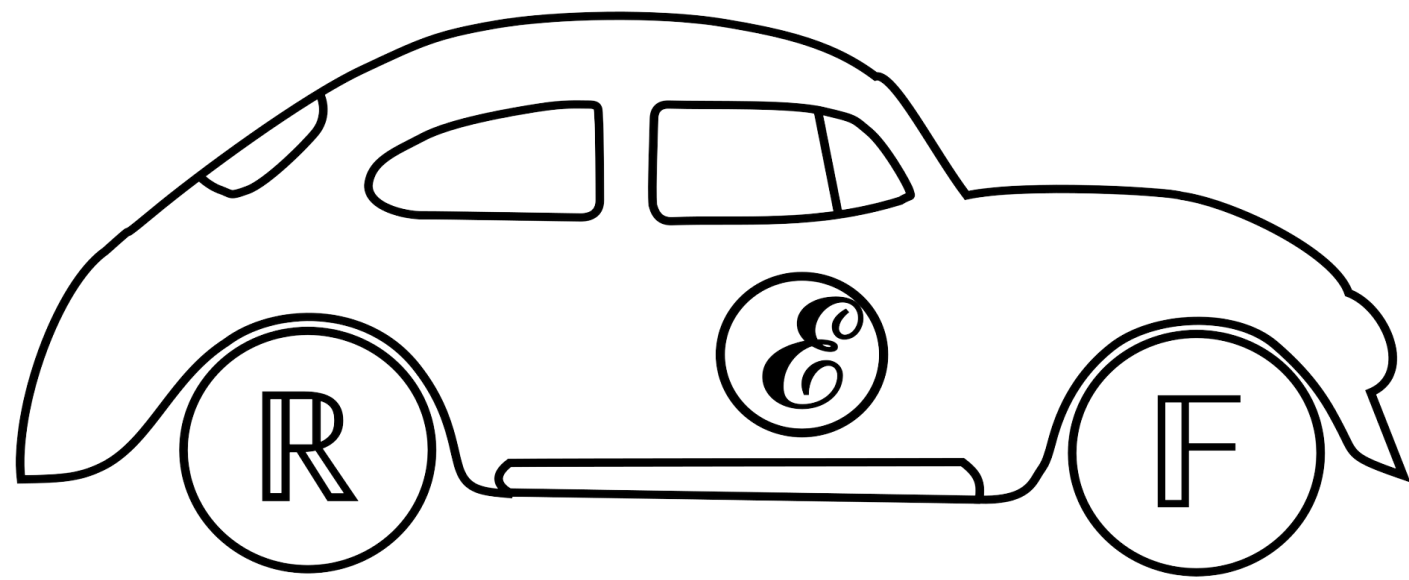


Python Simulations



E-Graphs are Super Useful!

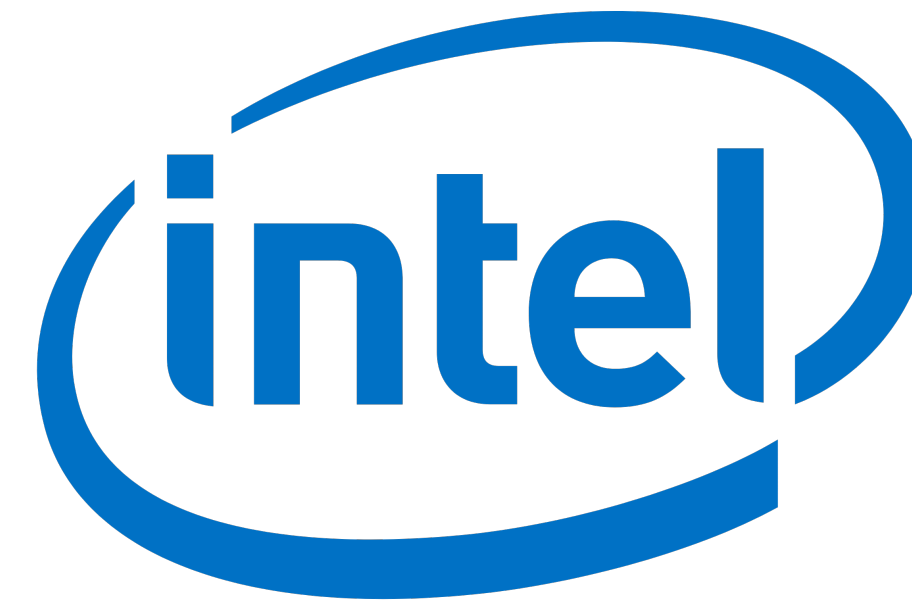
Floating point



Python Simulations

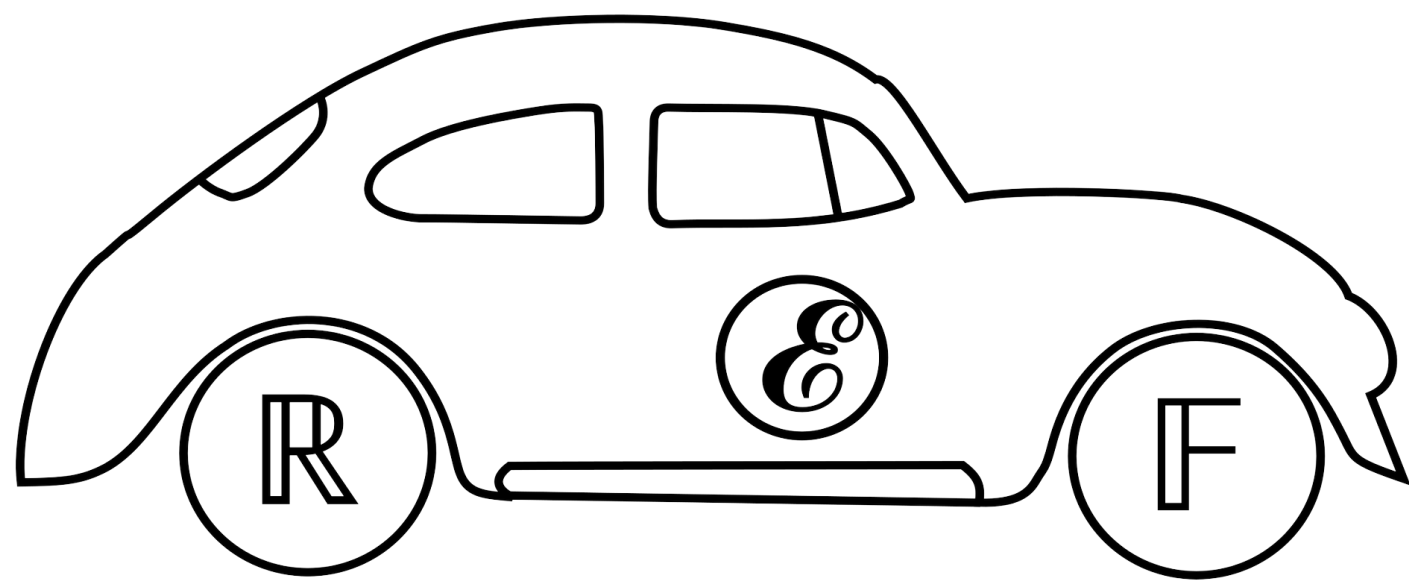


RTL Optimization



E-Graphs are Super Useful!

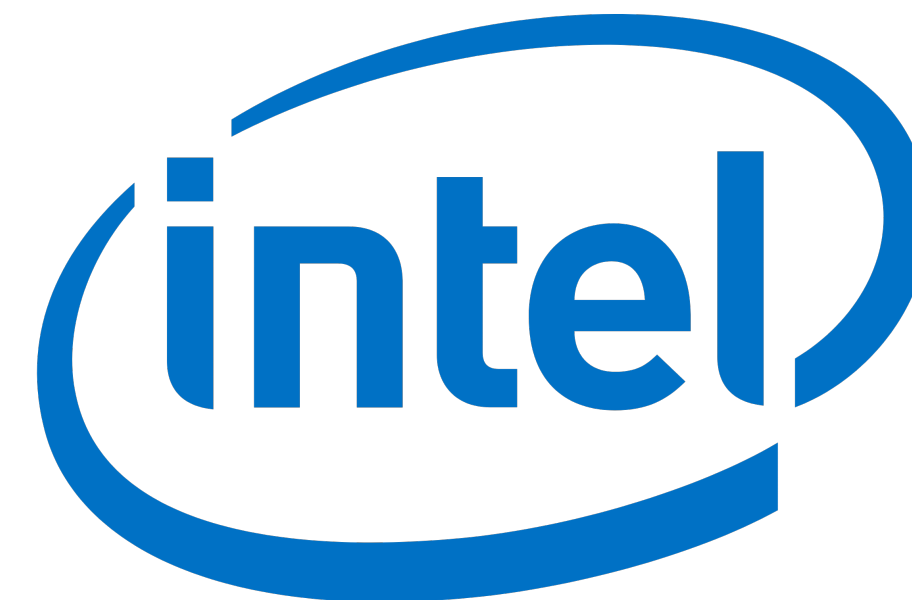
Floating point



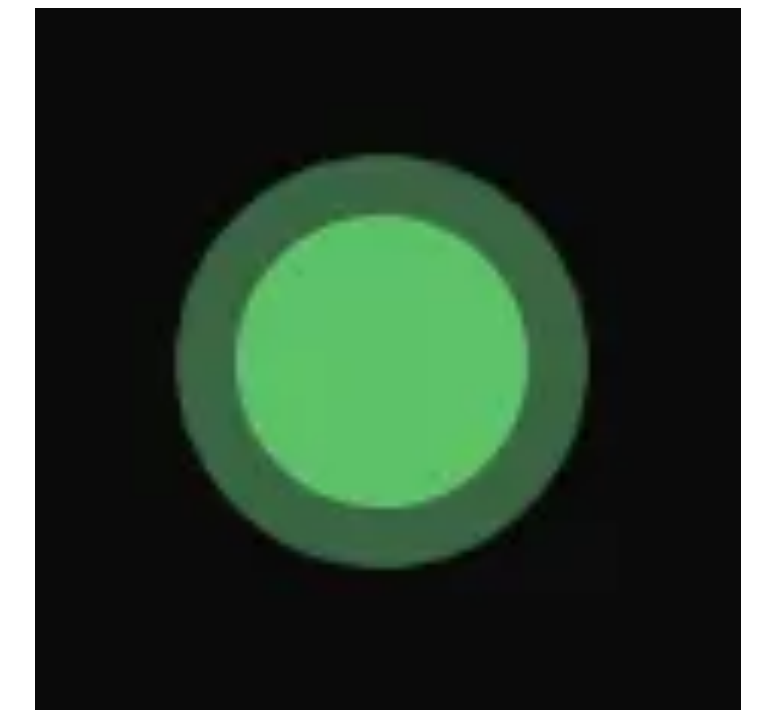
Python Simulations



RTL Optimization



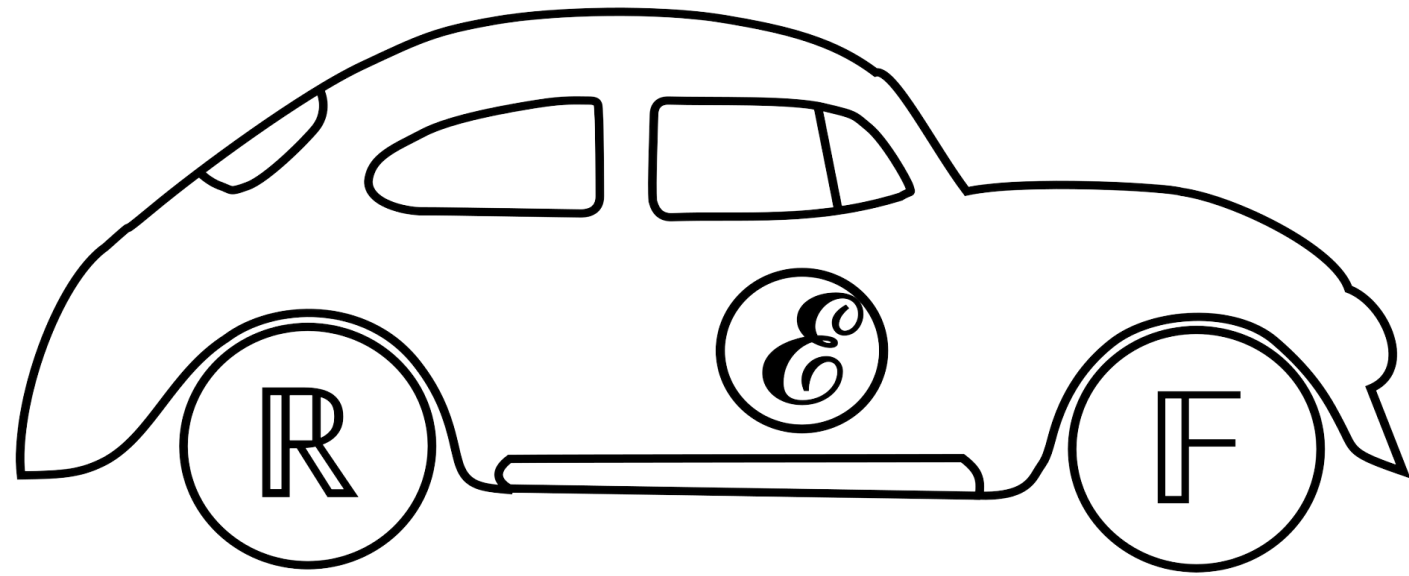
ML Kernels



Luminal

E-Graphs are Super Useful!

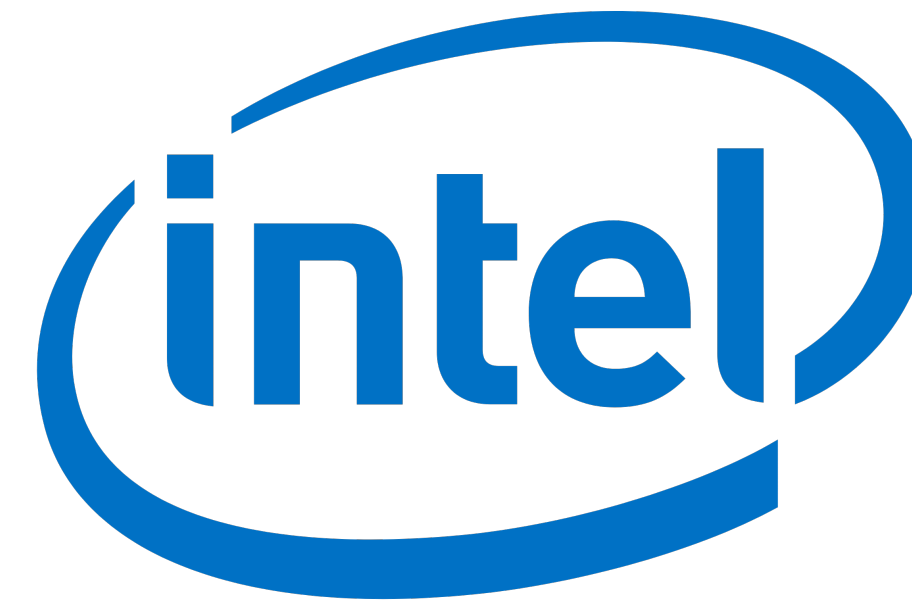
Floating point



Python Simulations



RTL Optimization



ML Kernels



Luminal

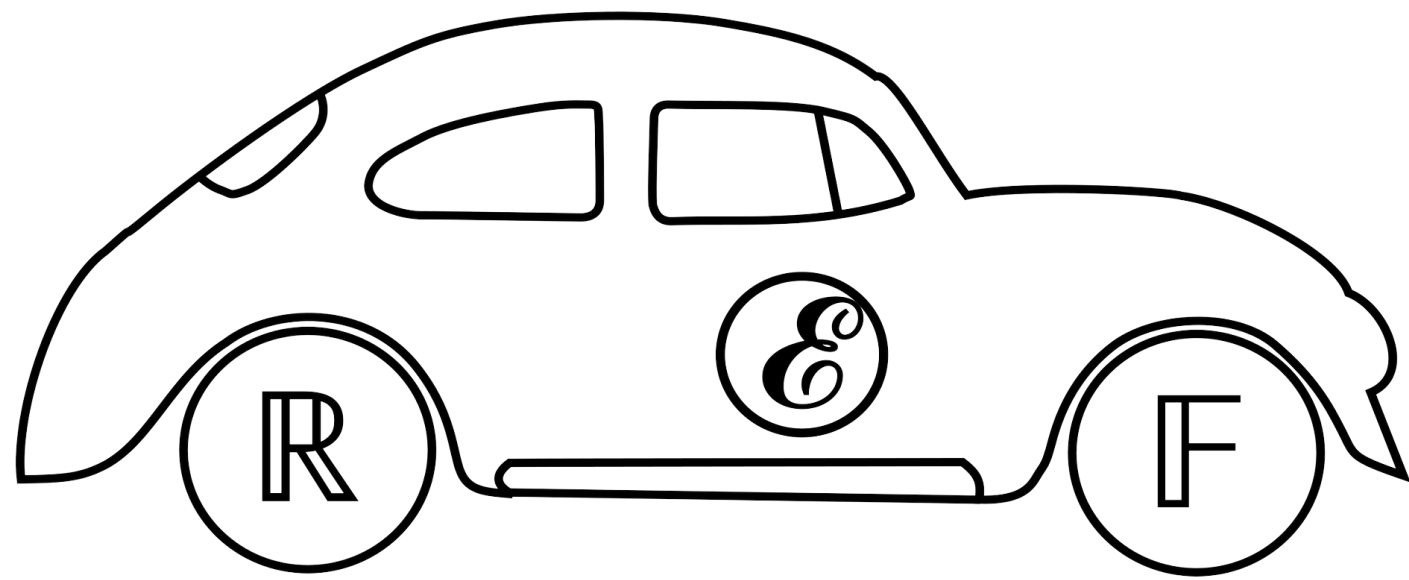
SQL



RisingWave

E-Graphs are Super Useful!

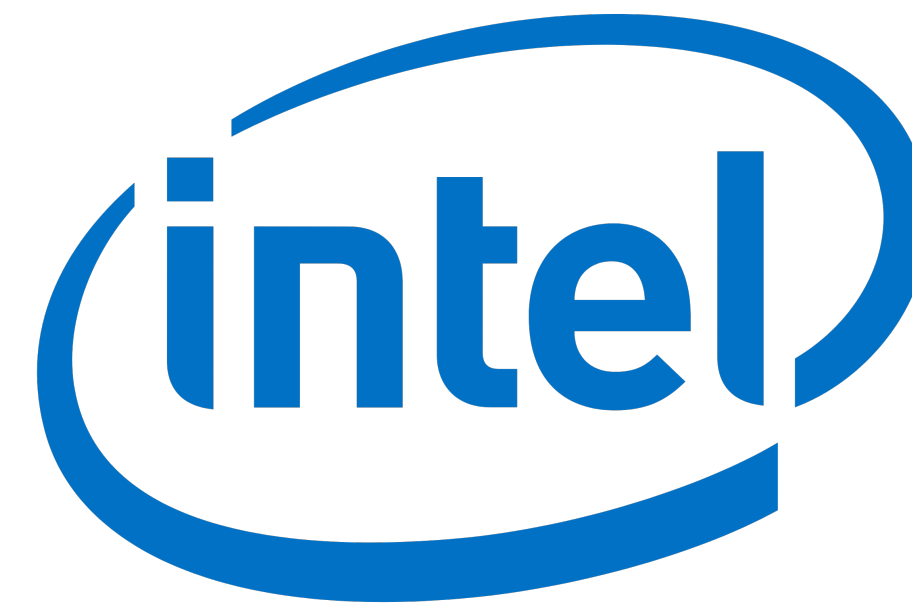
Floating point



Python Simulations



RTL Optimization



ML Kernels



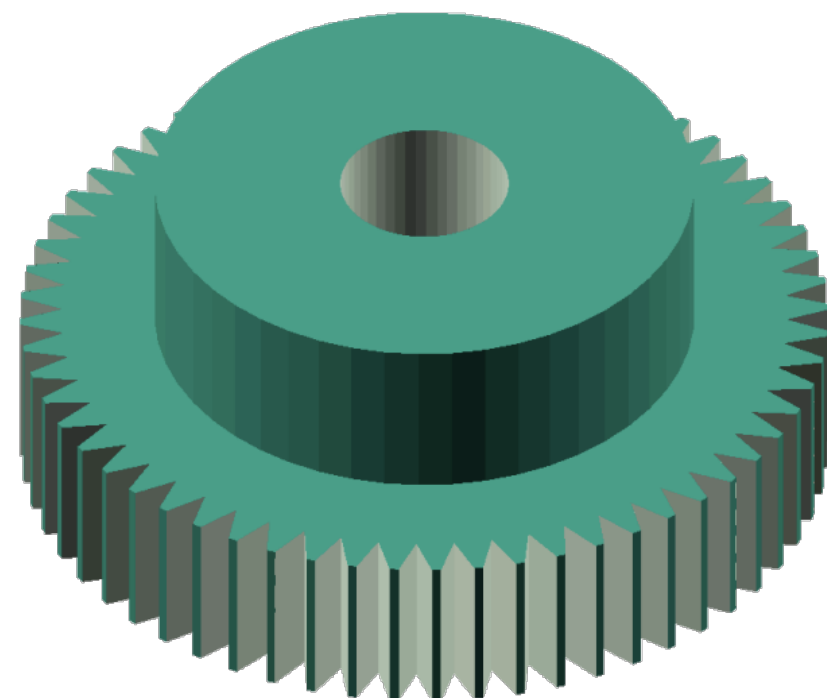
Luminal

SQL



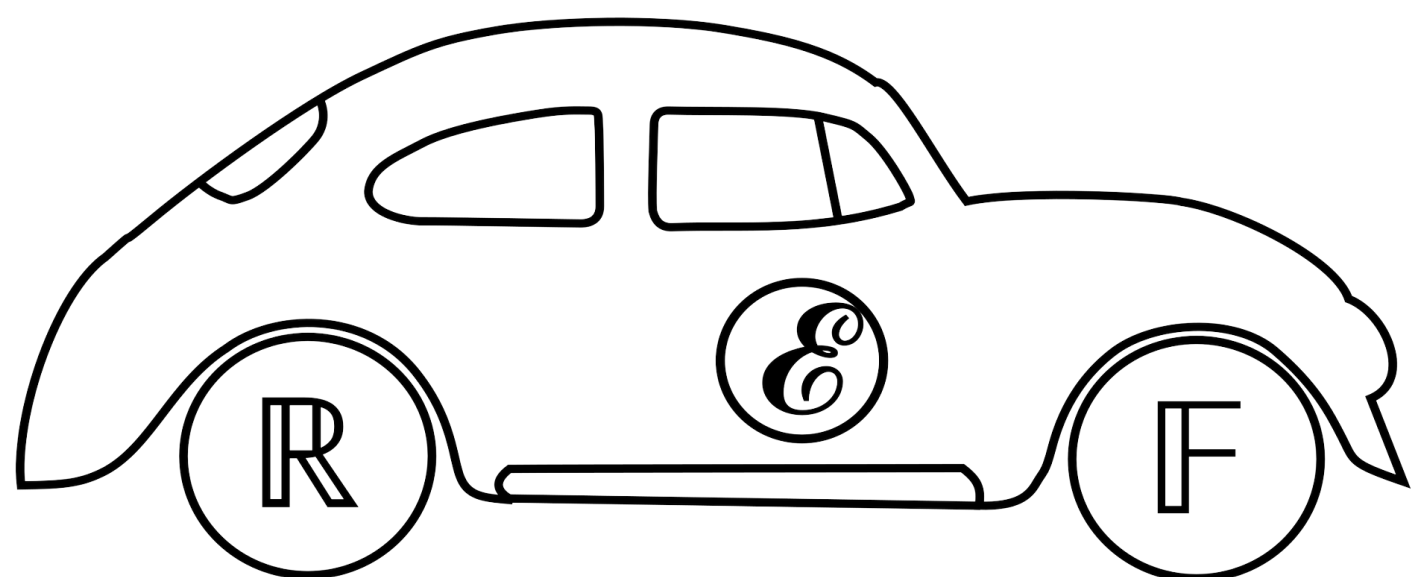
RisingWave

CAD



E-Graphs are Super Useful!

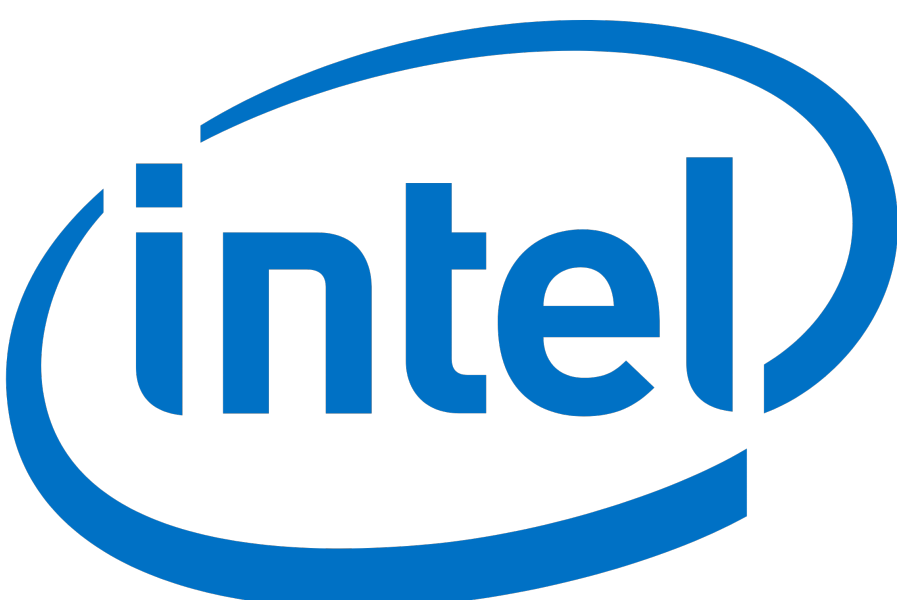
Floating point



Python Simulations



RTL Optimization



ML Kernels



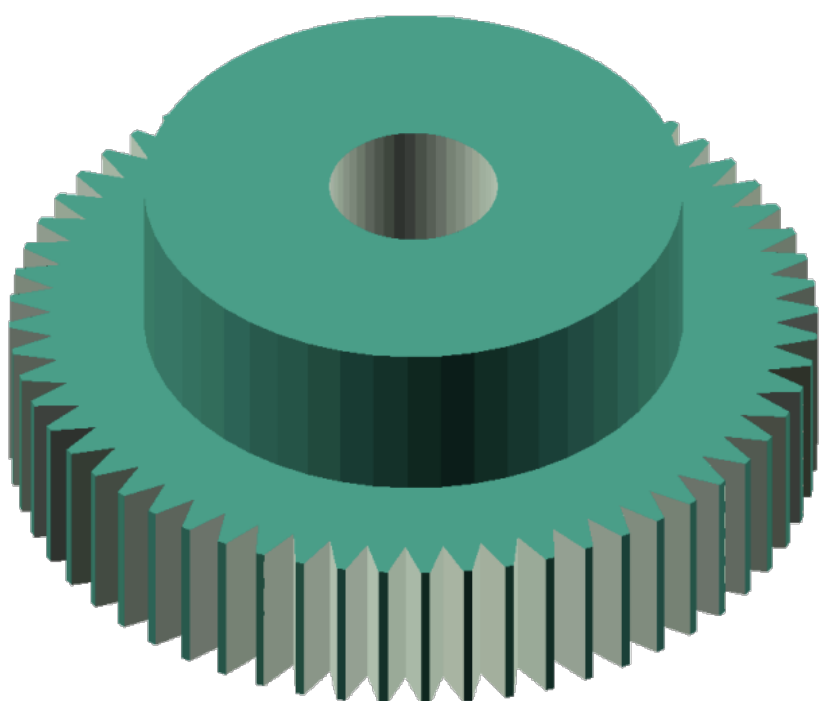
Luminal

SQL



RisingWave

CAD

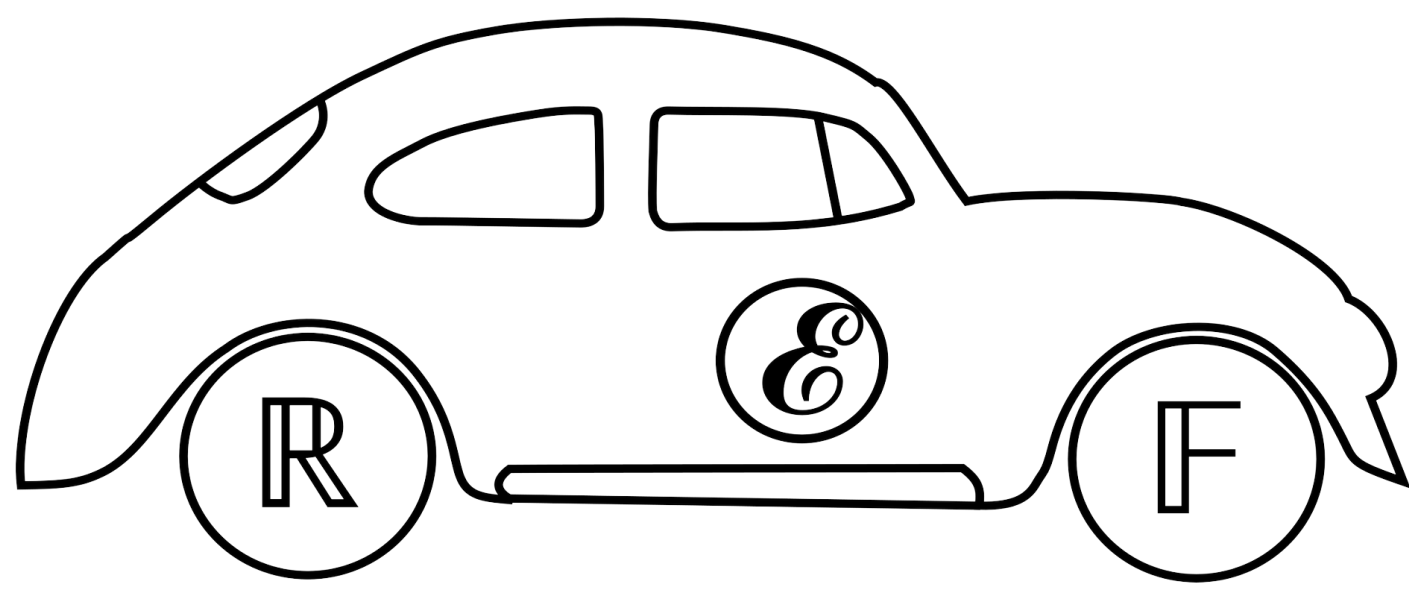


WASM



E-Graphs are Super Useful!

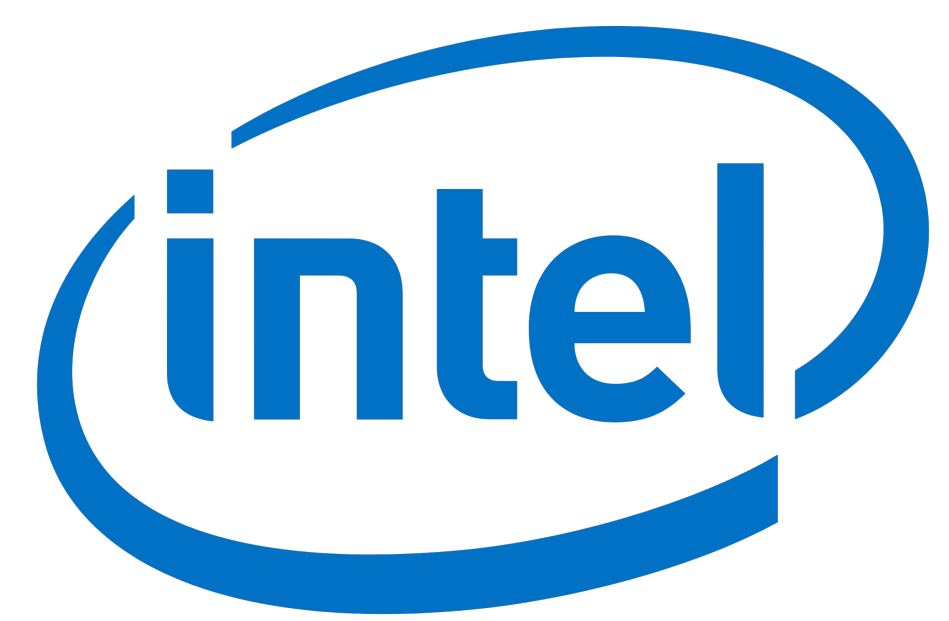
Floating point



Python Simulations



RTL Optimization



ML Kernels

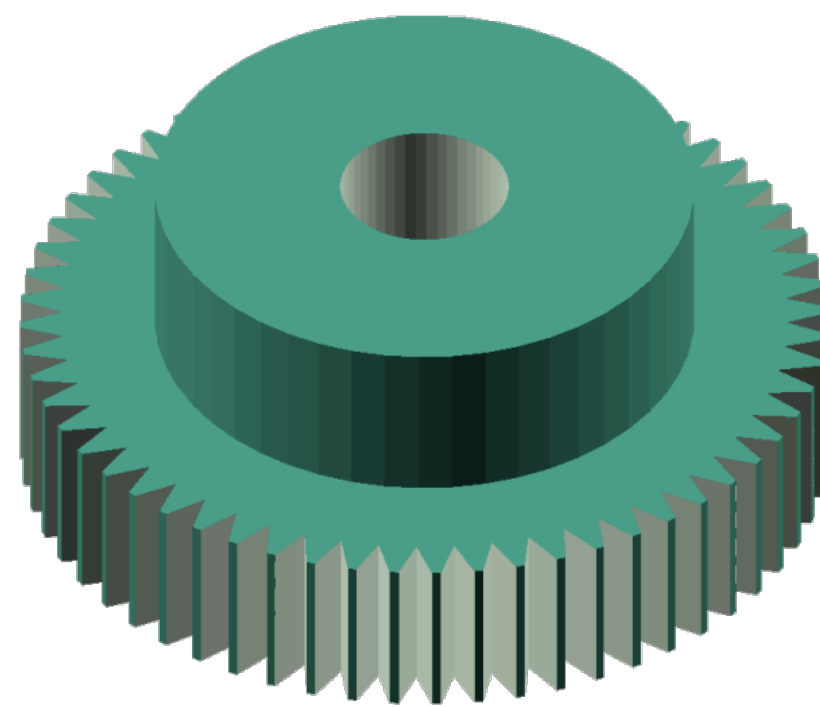


SQL



RisingWave

CAD



WASM

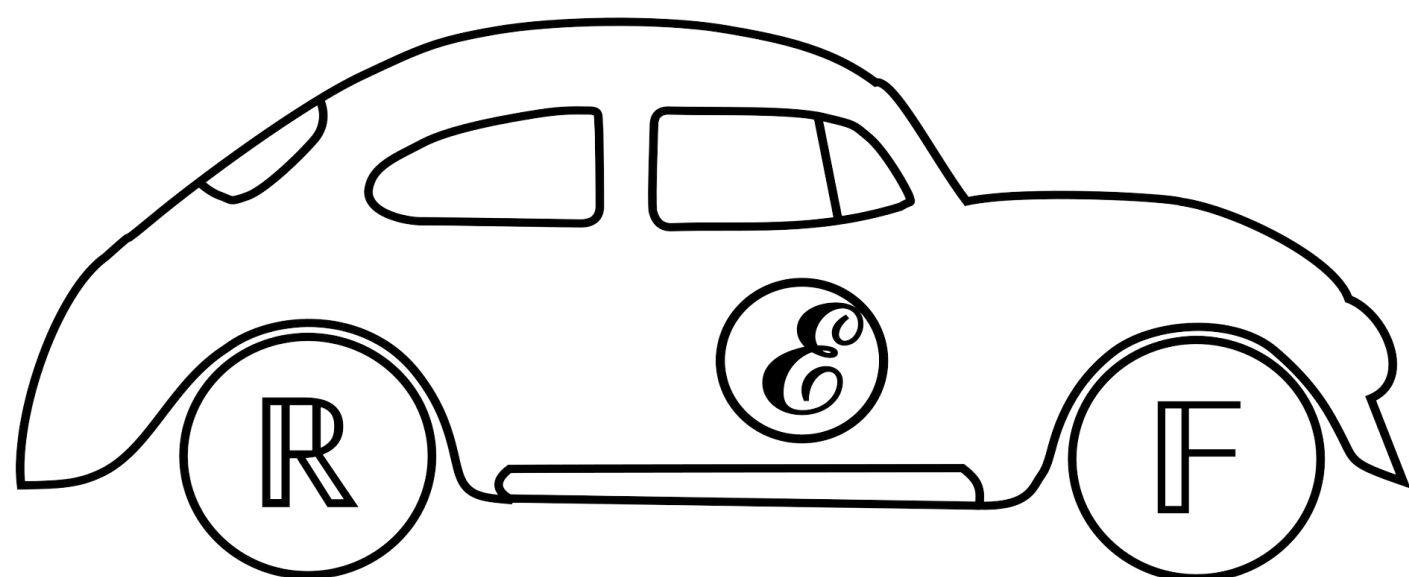


Luminal
User DSL Optimizations



E-Graphs are Super Useful!

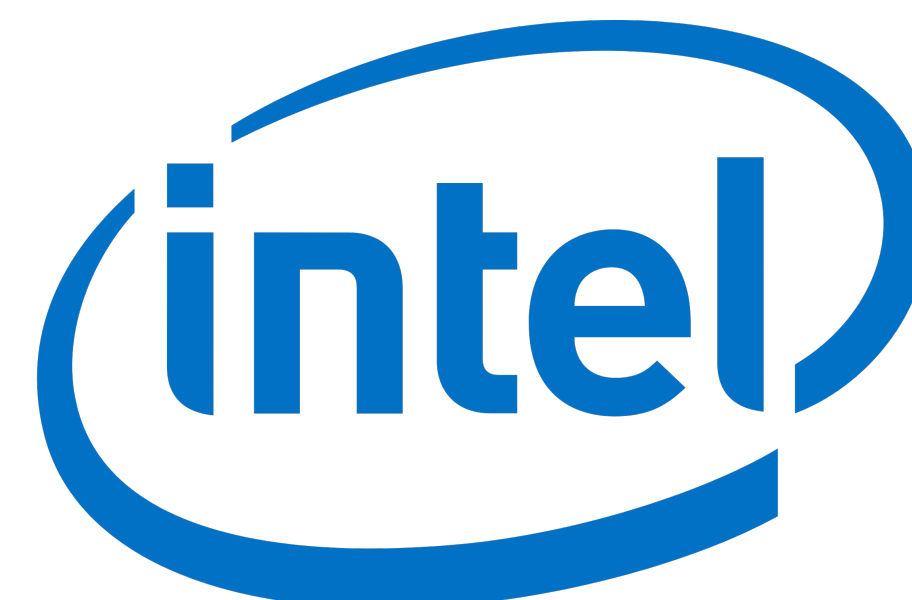
Floating point



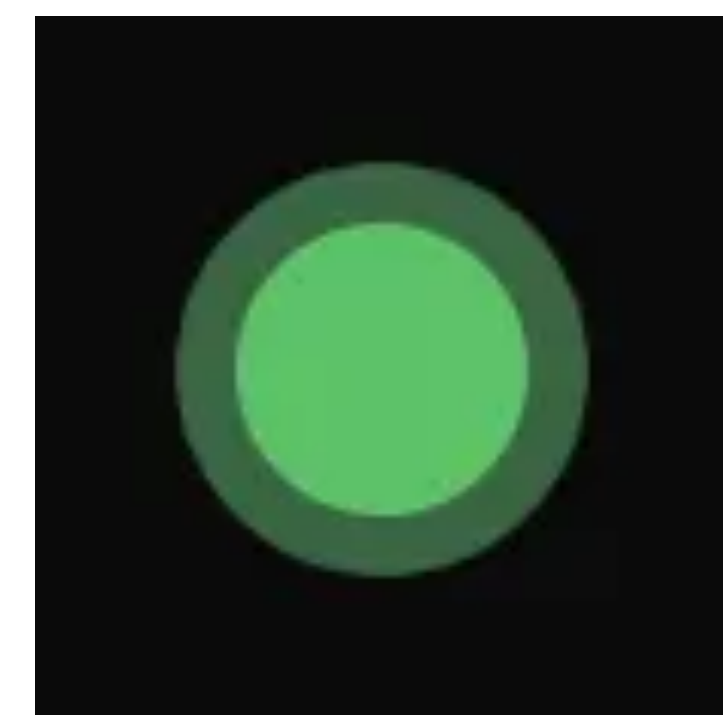
Python Simulations



RTL Optimization



ML Kernels

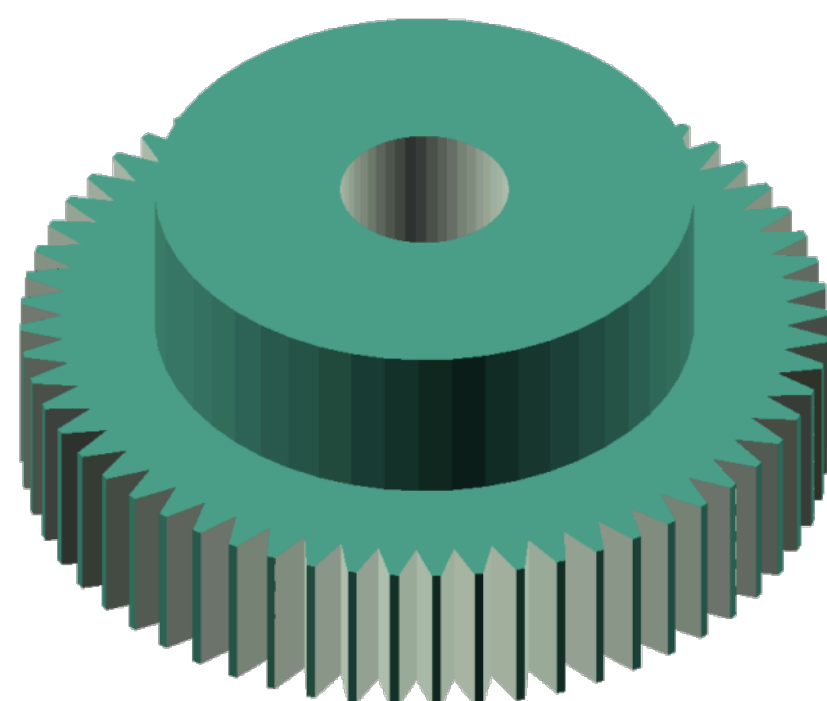


SQL

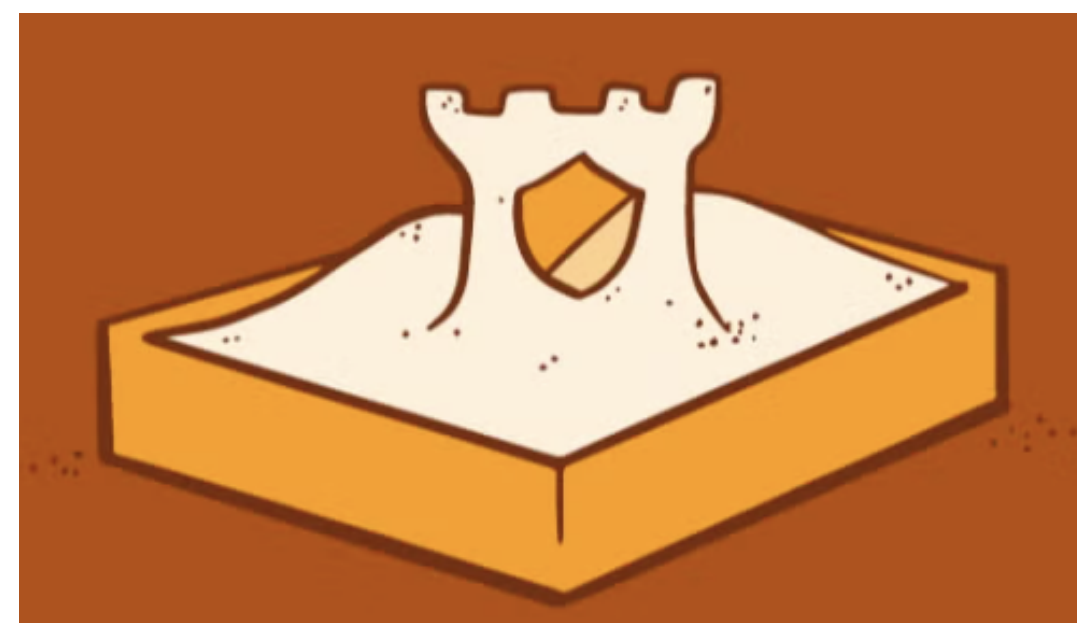


RisingWave

CAD



WASM



User DSL Optimizations



...and more!

1

2

3

4

Motivate Effective E-Graphs

E-Graphs Explore a Big Space

Input Program

$(a + 1) - 1$

E-Graphs Explore a Big Space

Input Program

$(a + 1) - 1$

Optimization Rules

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

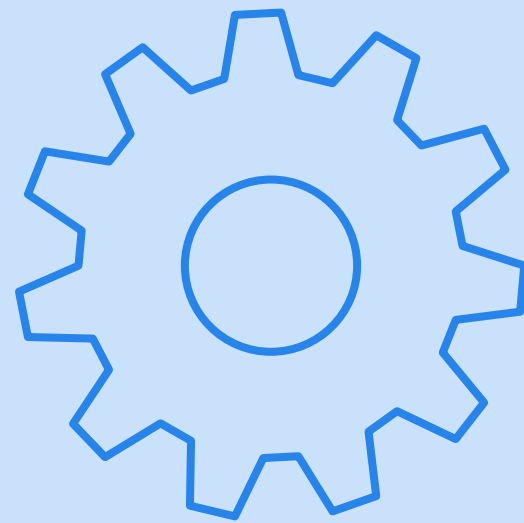
$\alpha - \alpha \rightarrow 0$

E-Graphs Explore a Big Space

Input Program

$(a + 1) - 1$

E-Graph

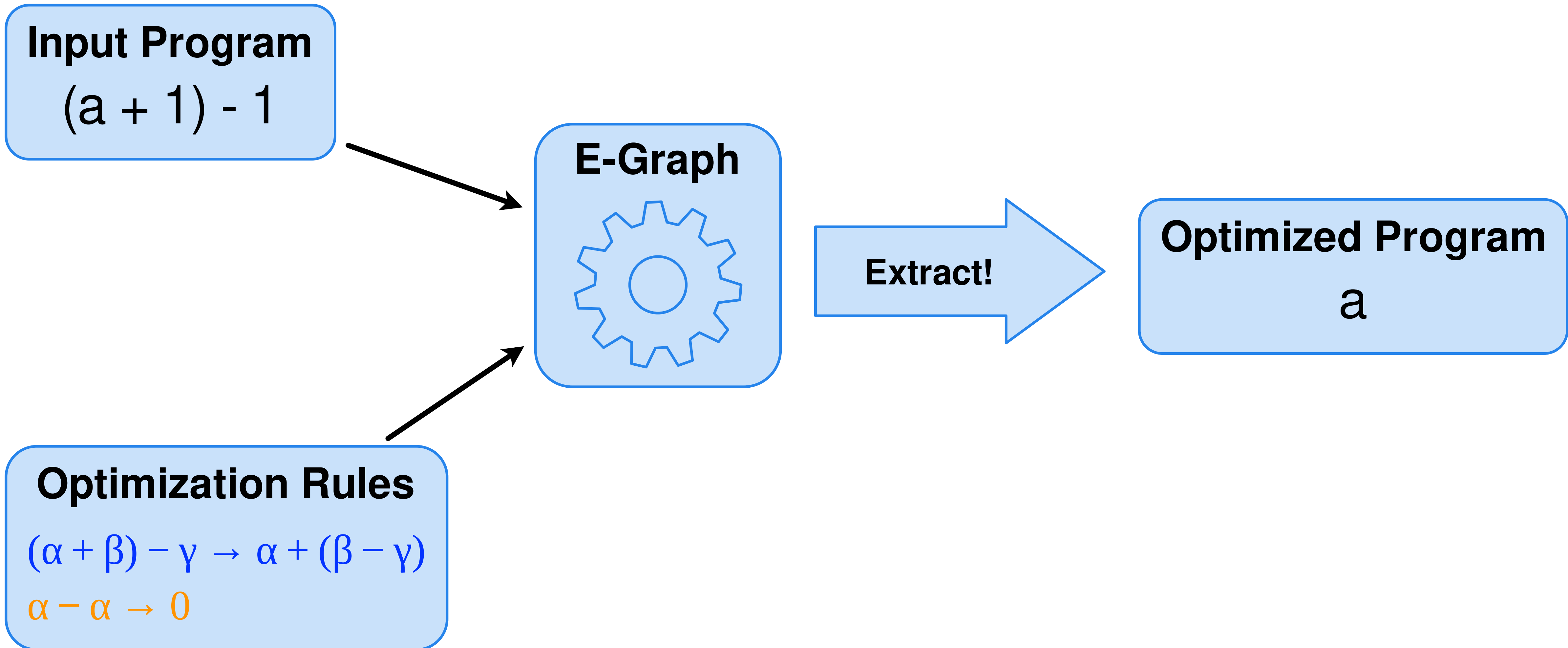


Optimization Rules

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

$\alpha - \alpha \rightarrow 0$

E-Graphs Explore a Big Space



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma) \quad (a + 1)$$

$$\alpha - \alpha \rightarrow 0$$

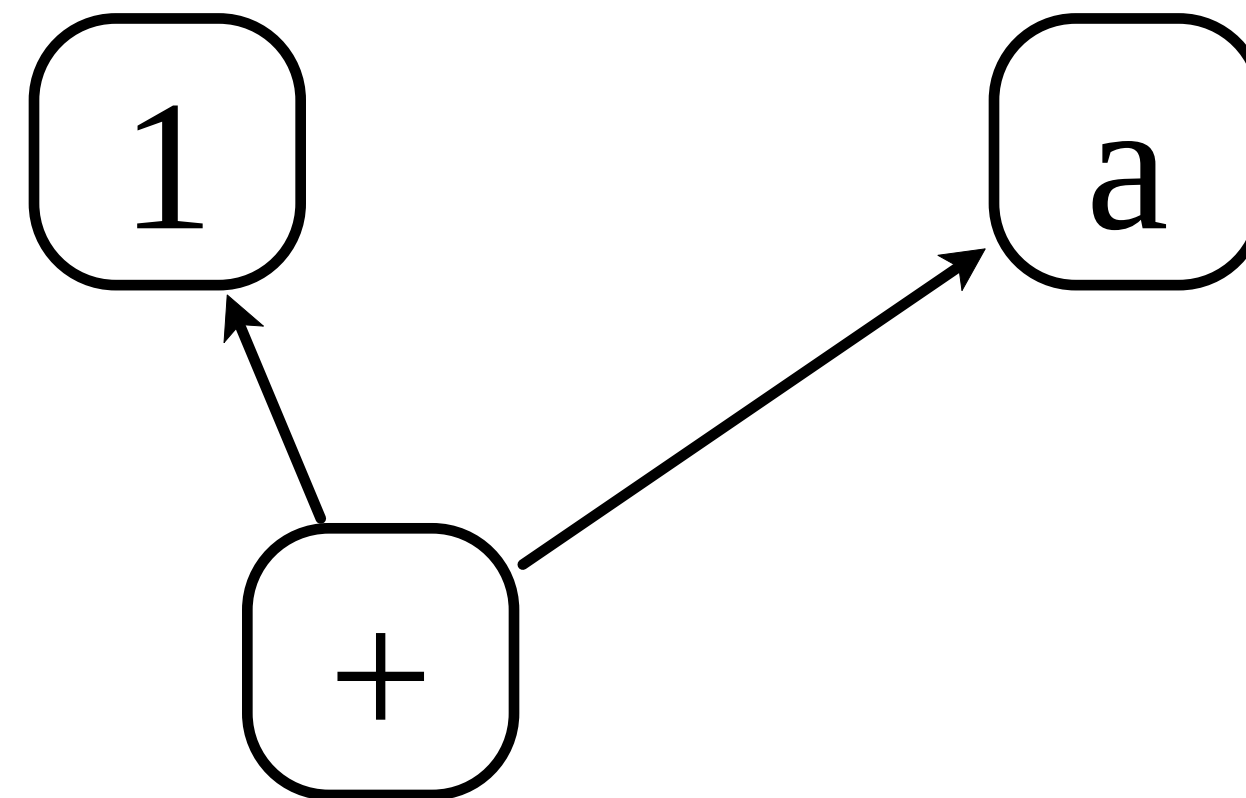
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



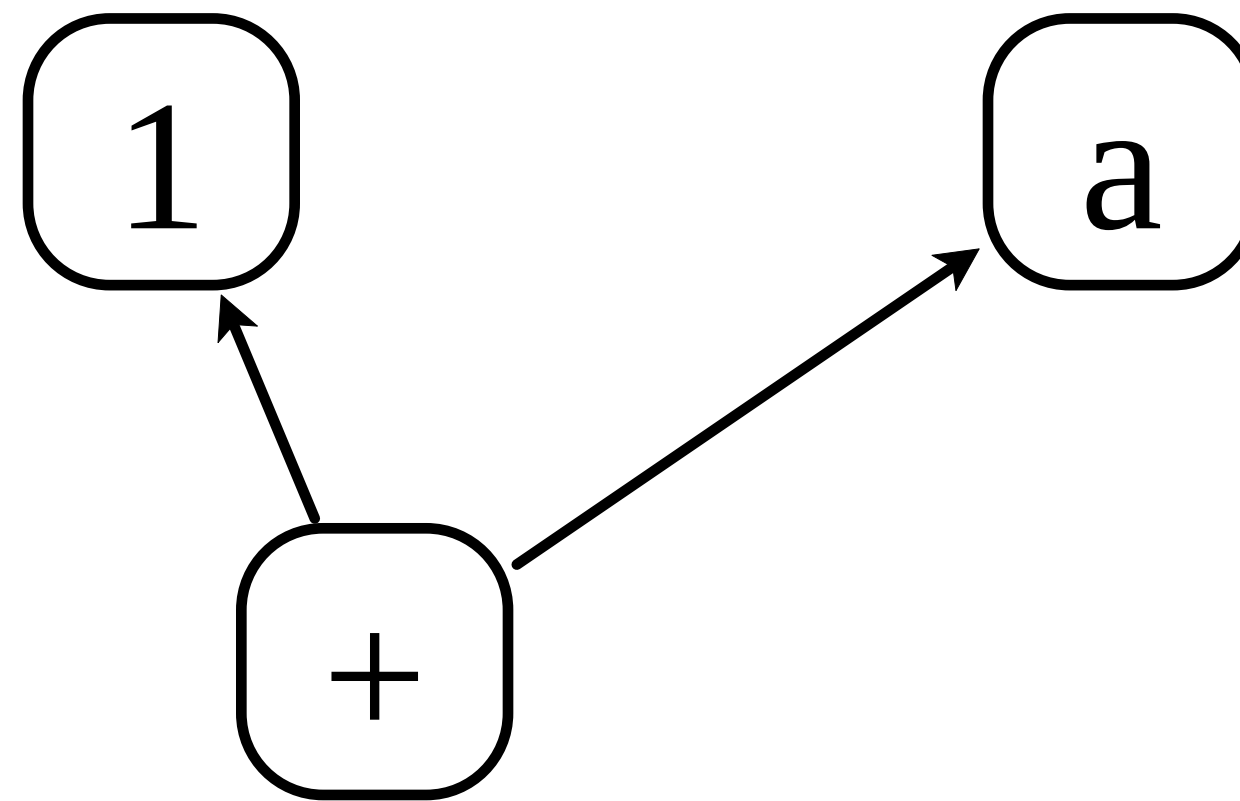
E-Graphs Explore a Big Space

$(a + 1) - 1$

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

$\alpha - \alpha \rightarrow 0$

data flow



E-Graphs Explore a Big Space

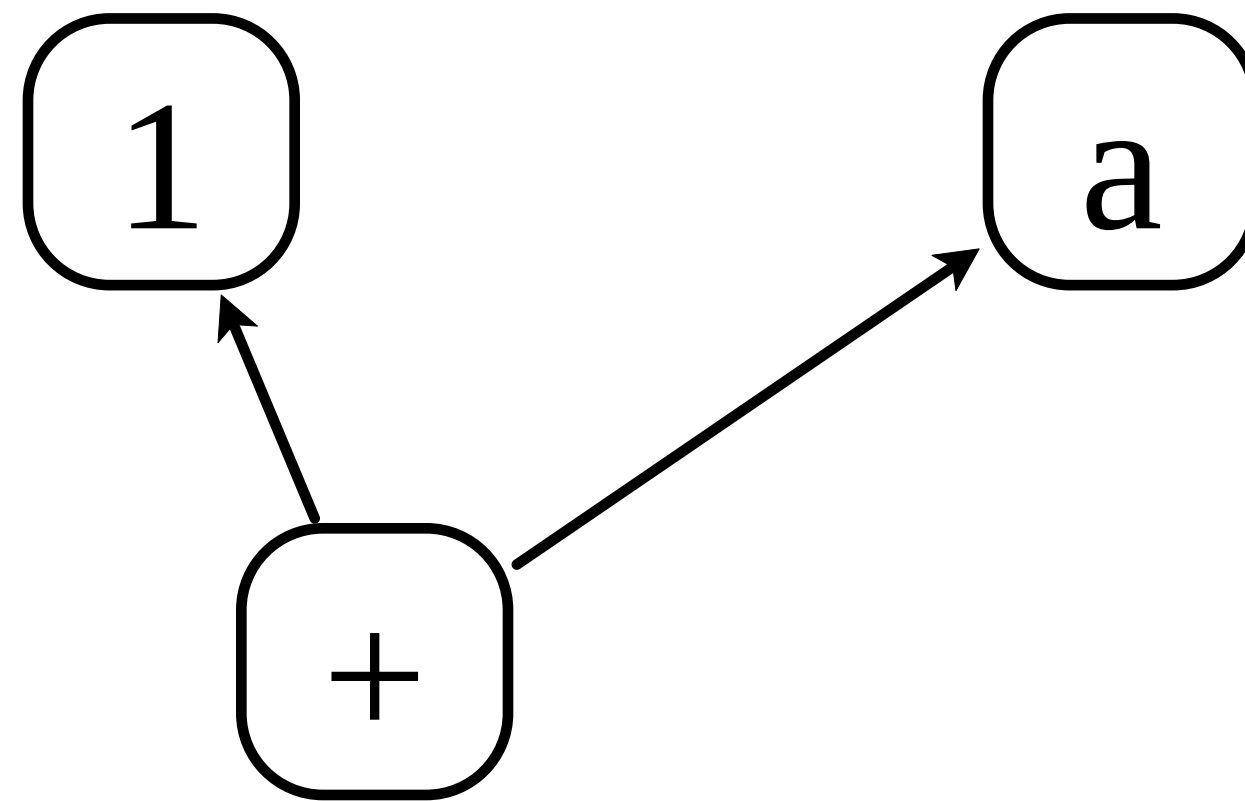
$(a + 1) - 1$

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

$\alpha - \alpha \rightarrow 0$

$(a + 1) - 1$

data flow



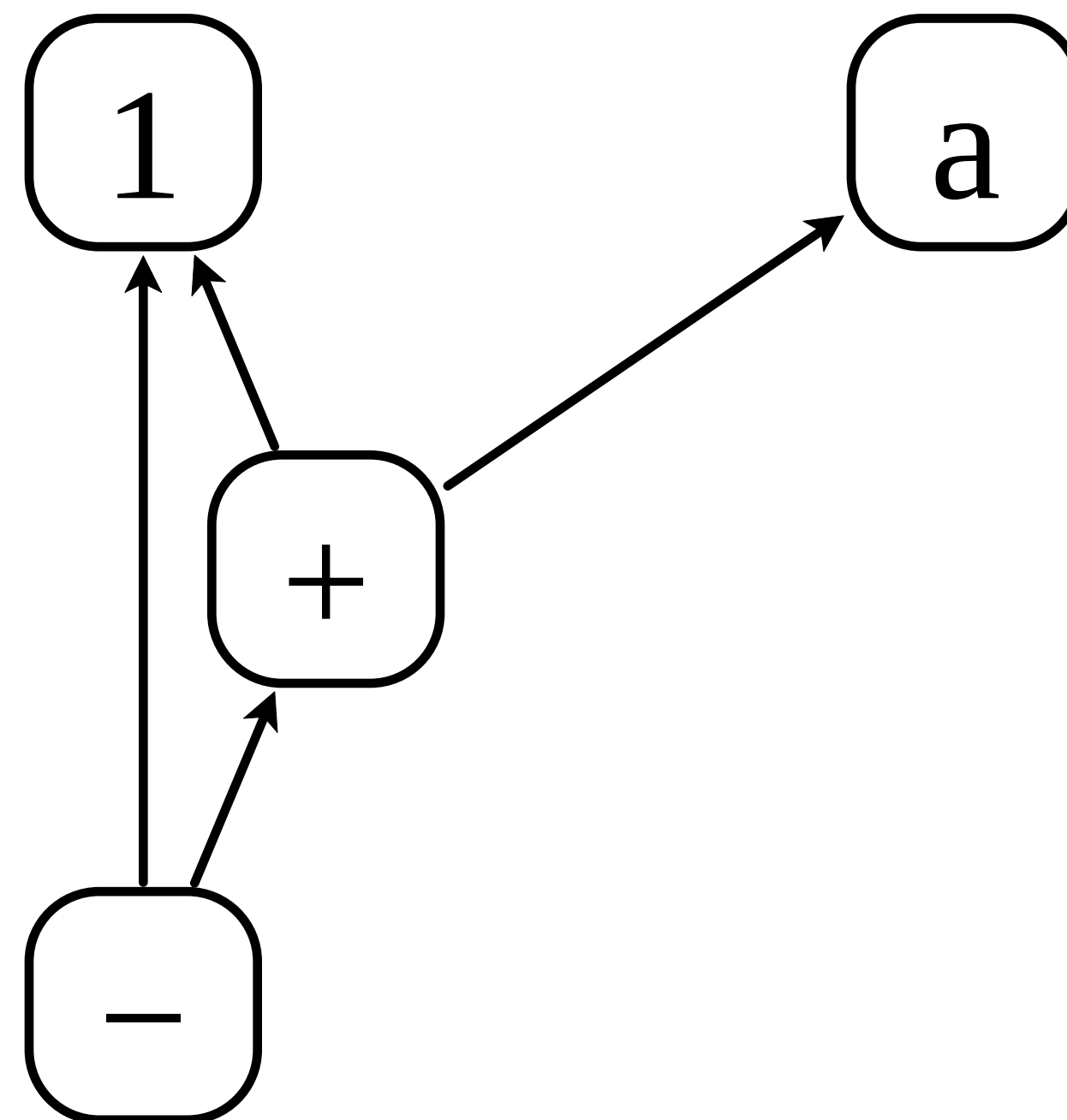
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



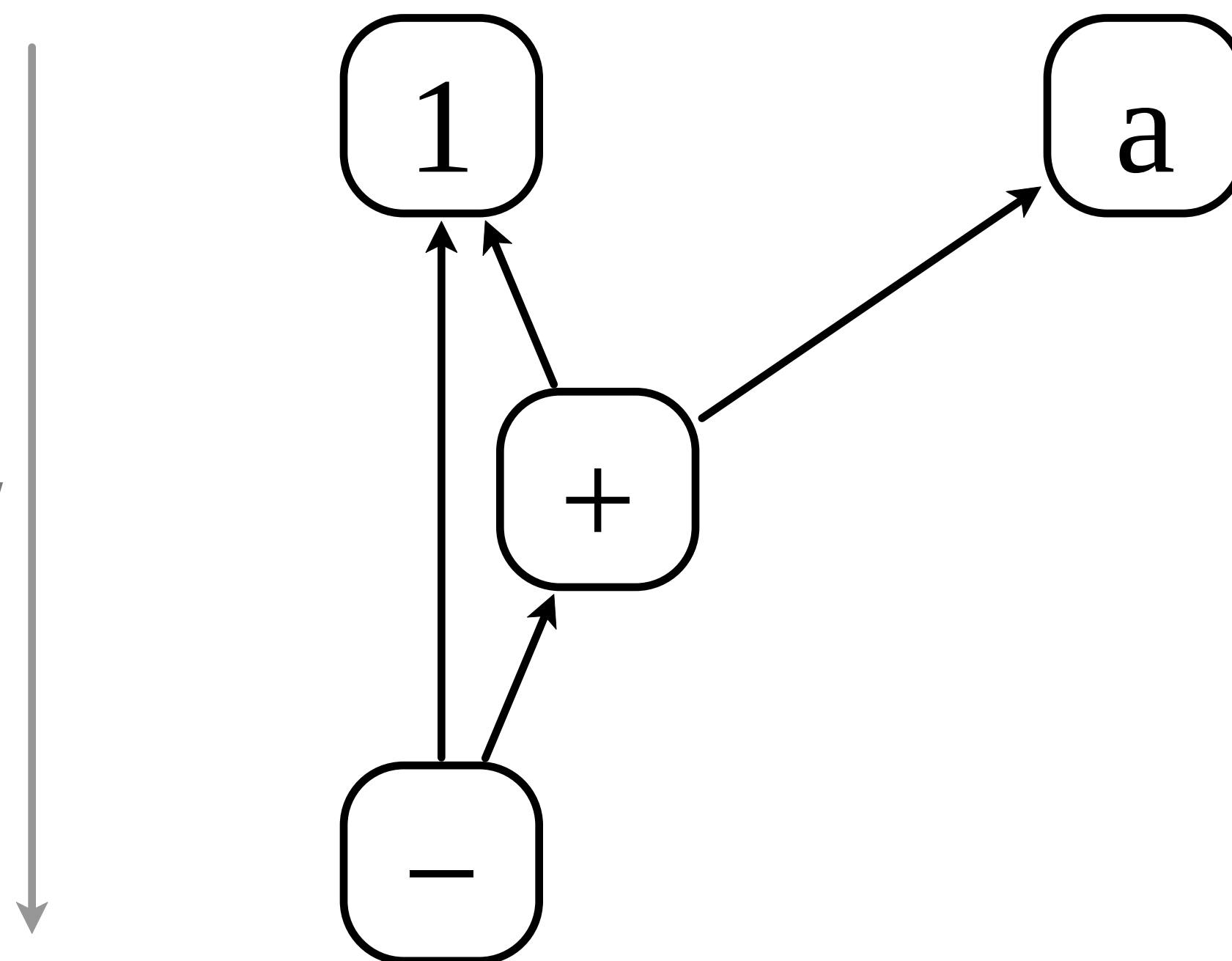
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



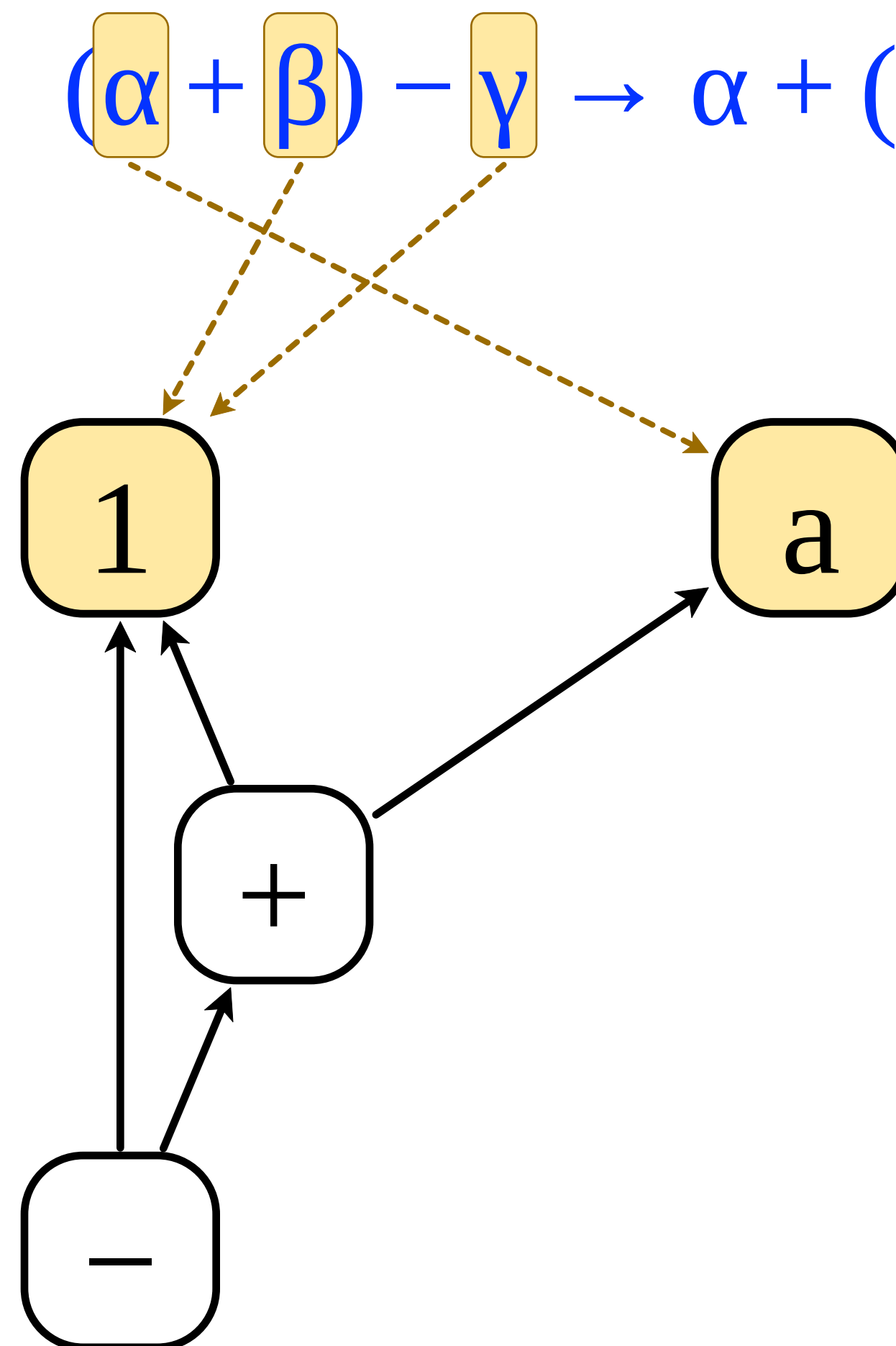
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

data flow

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$



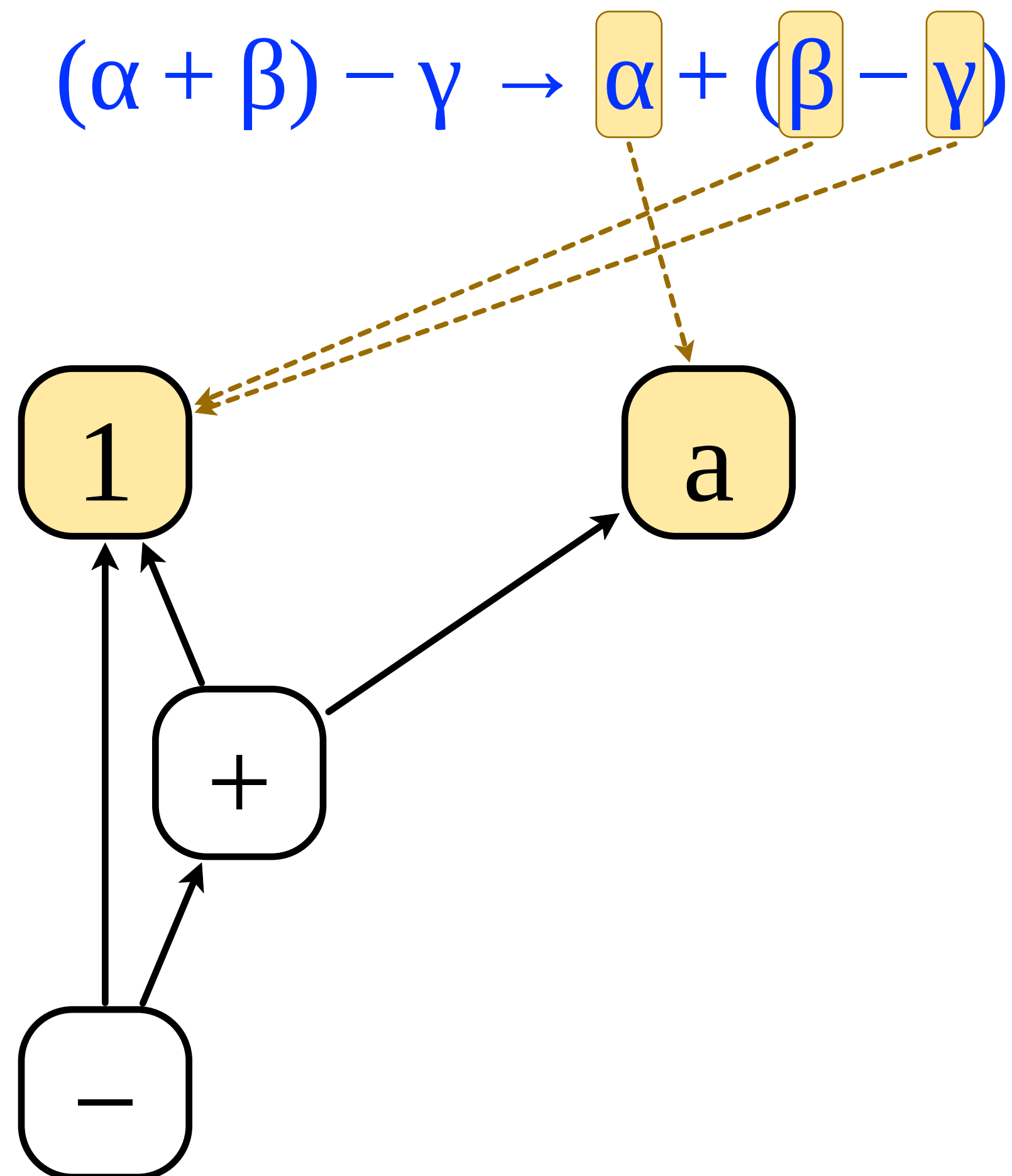
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

data flow

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

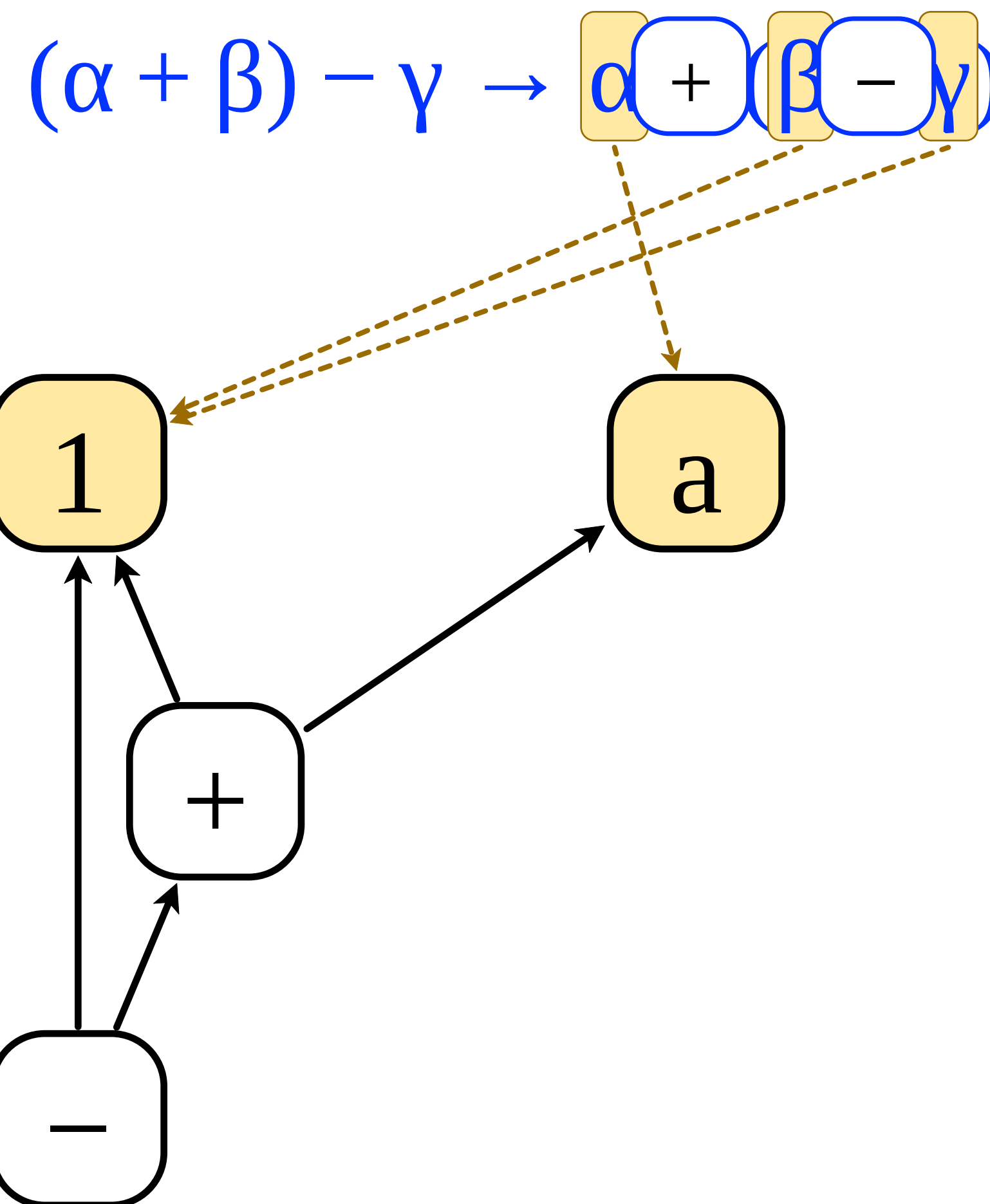


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

data flow



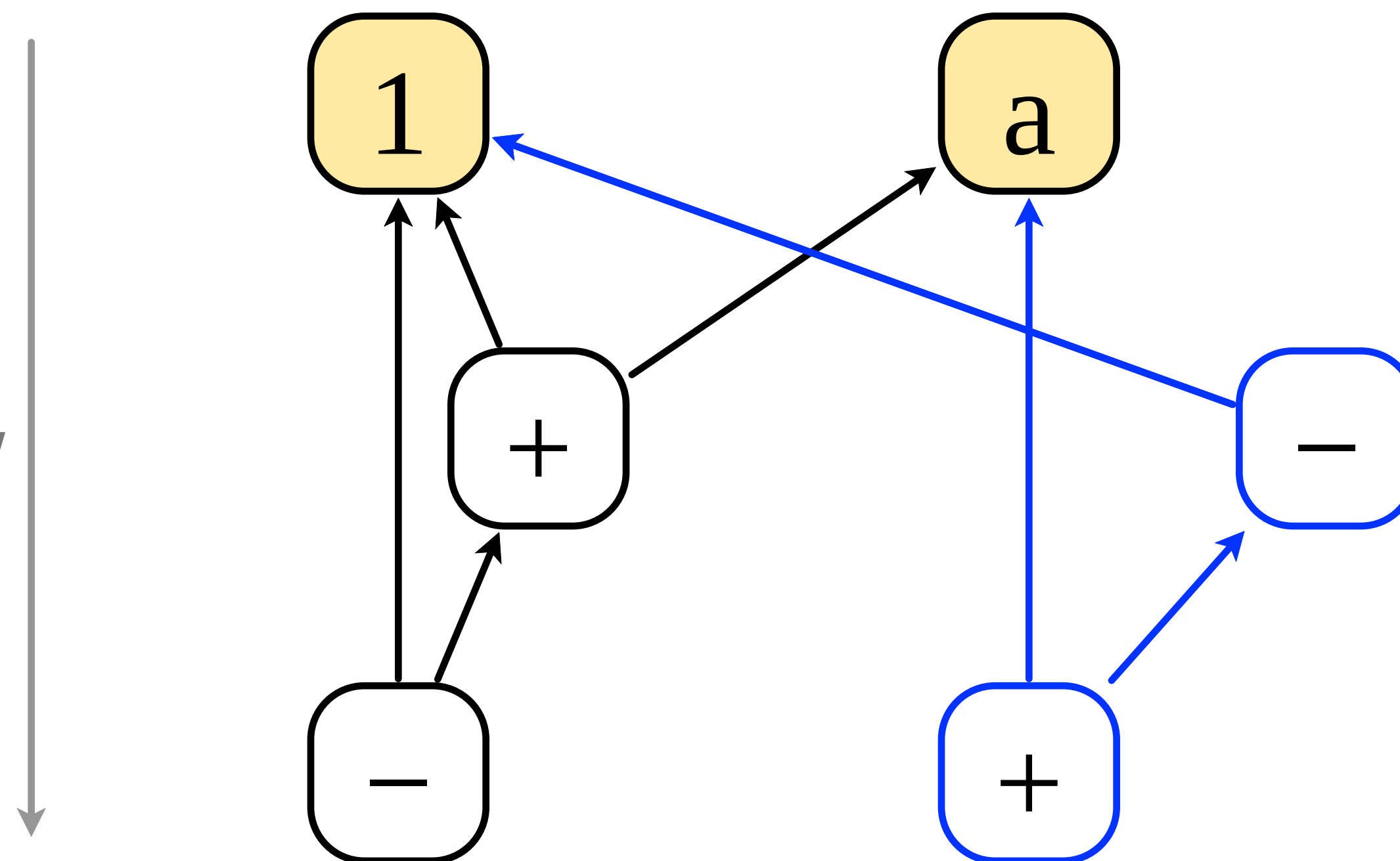
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \boxed{\alpha} + (\boxed{\beta} - \boxed{\gamma})$$

$$\alpha - \alpha \rightarrow 0$$

data flow



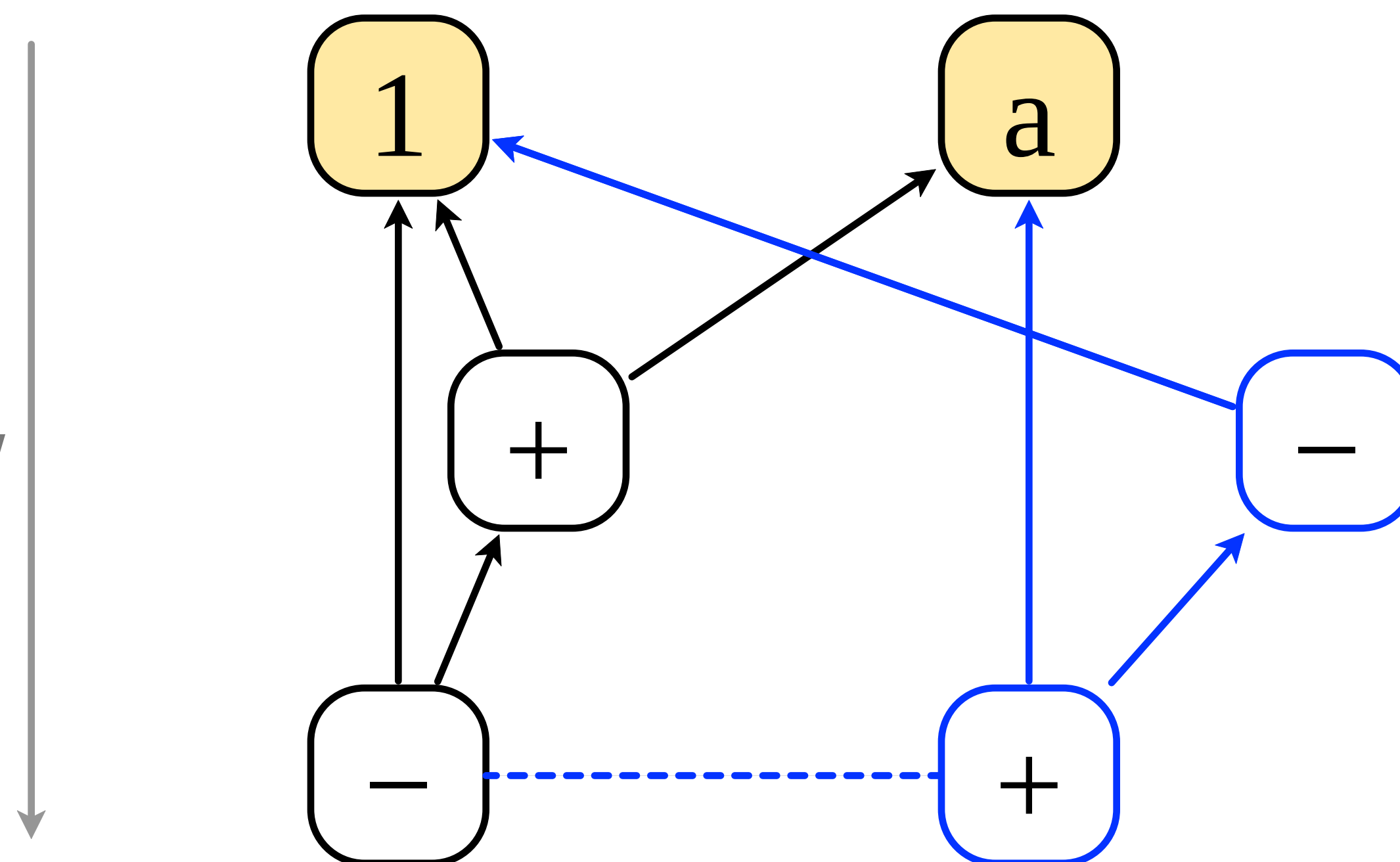
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



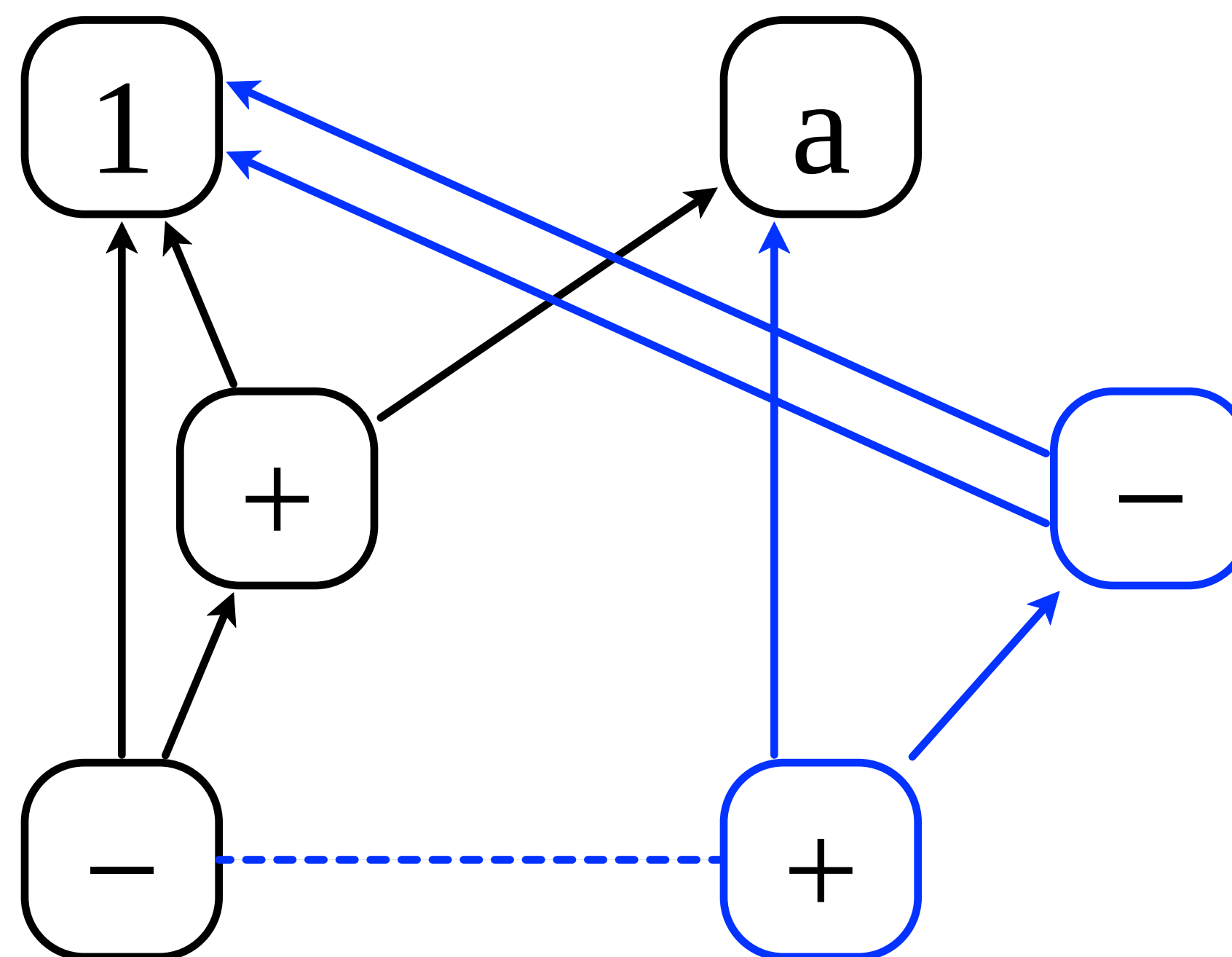
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow

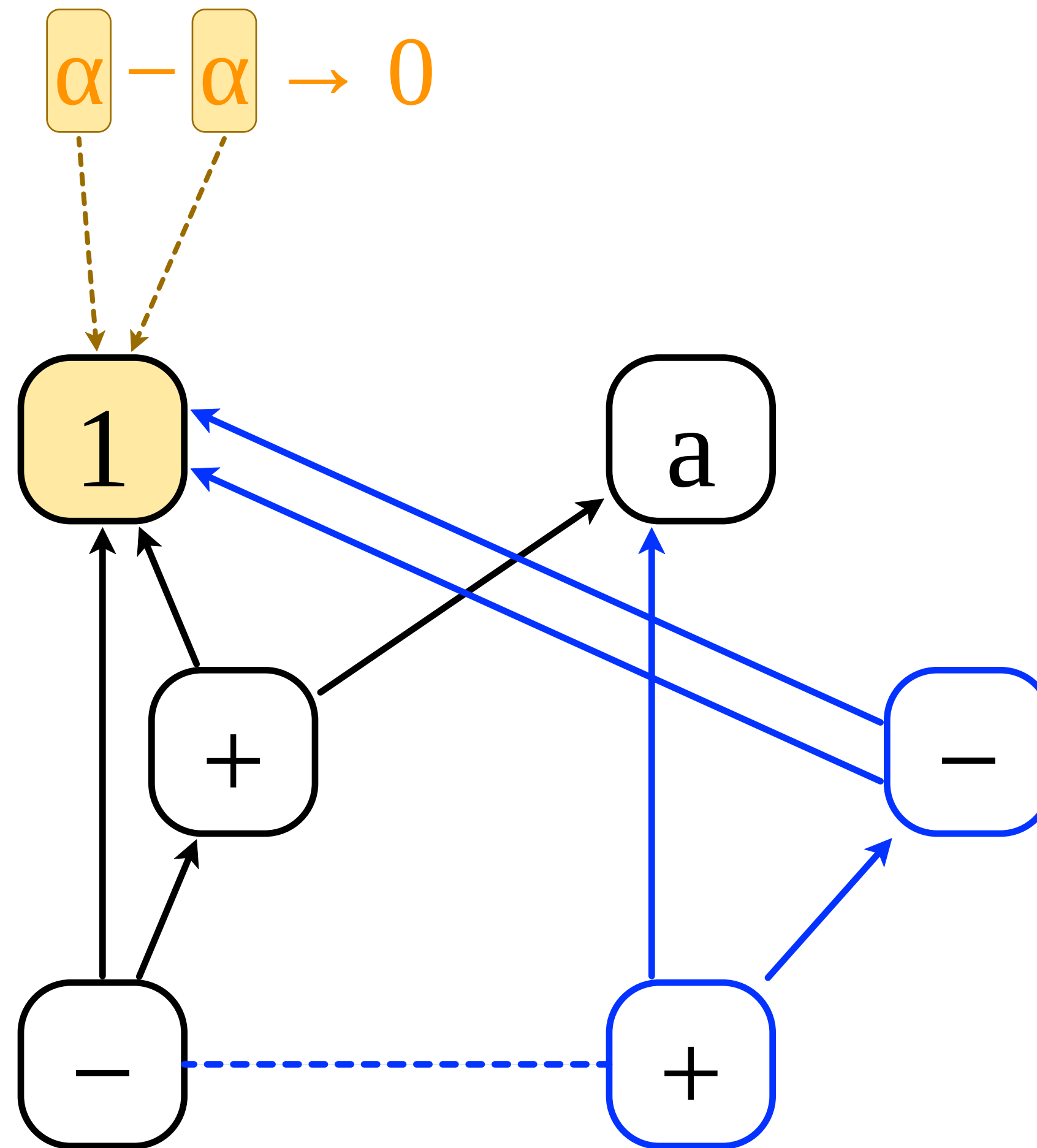


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

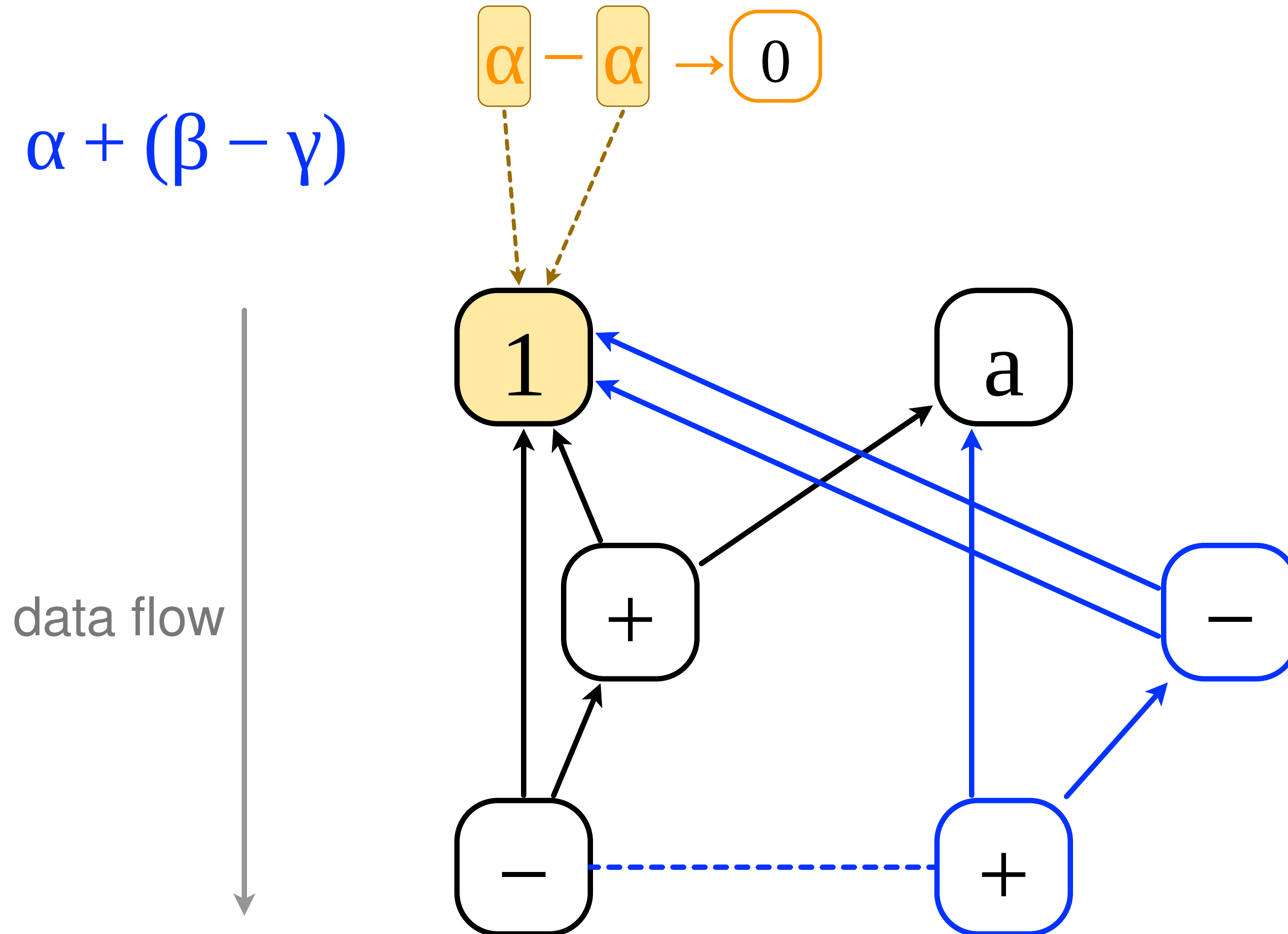
data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$



data flow

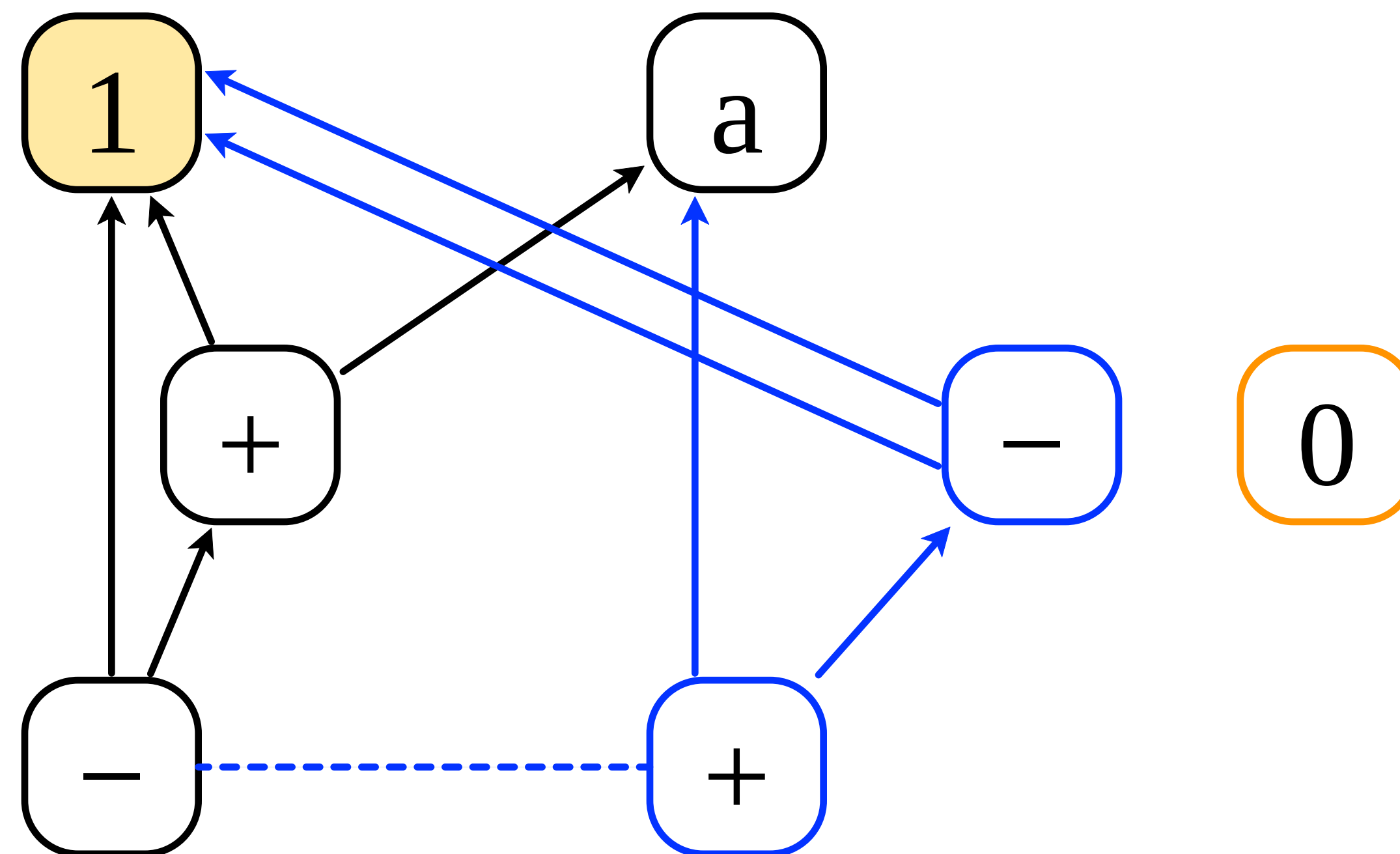
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow

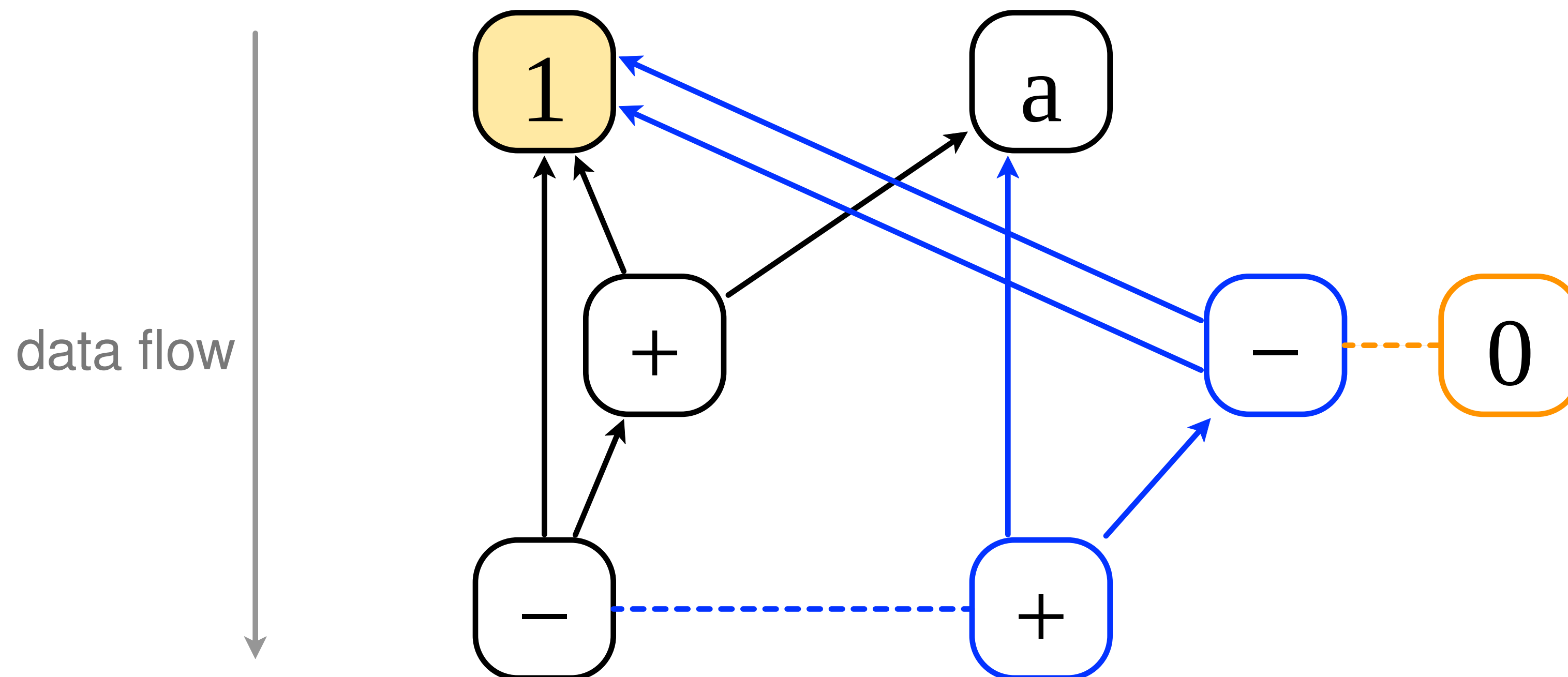


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

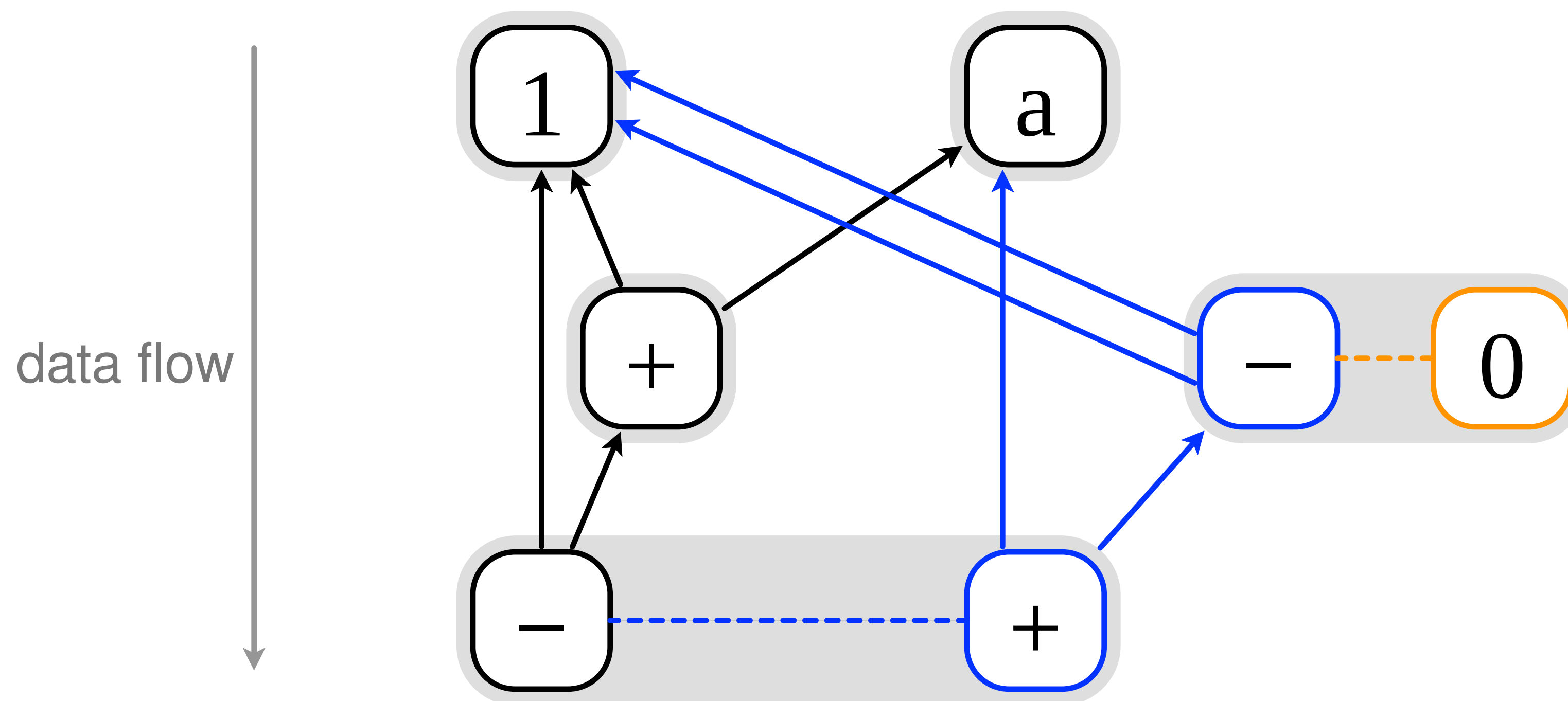


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

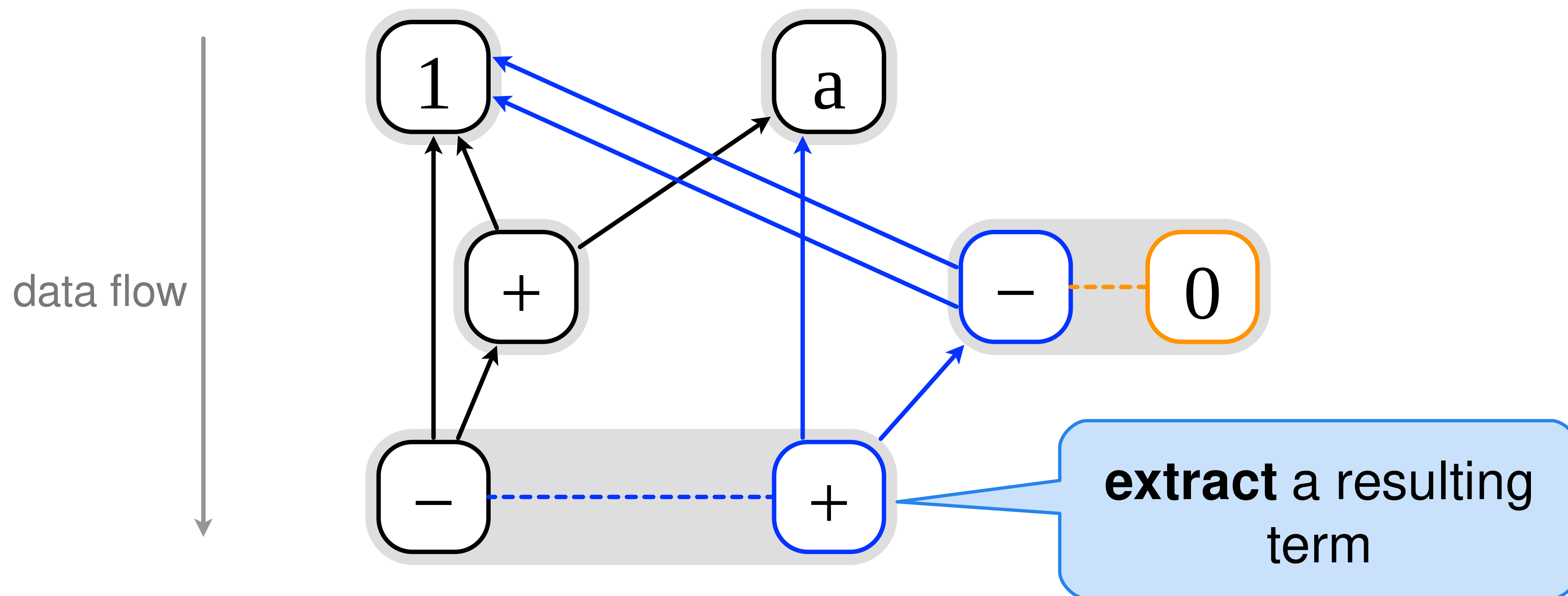


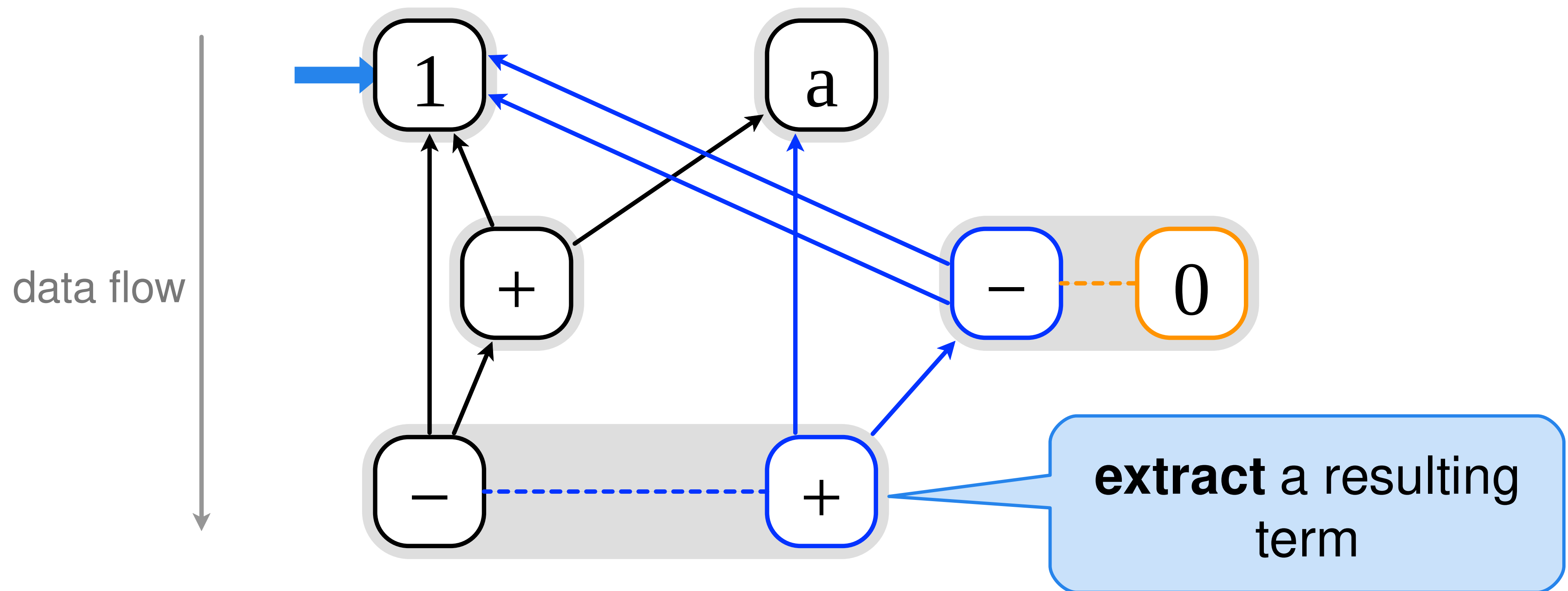
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$



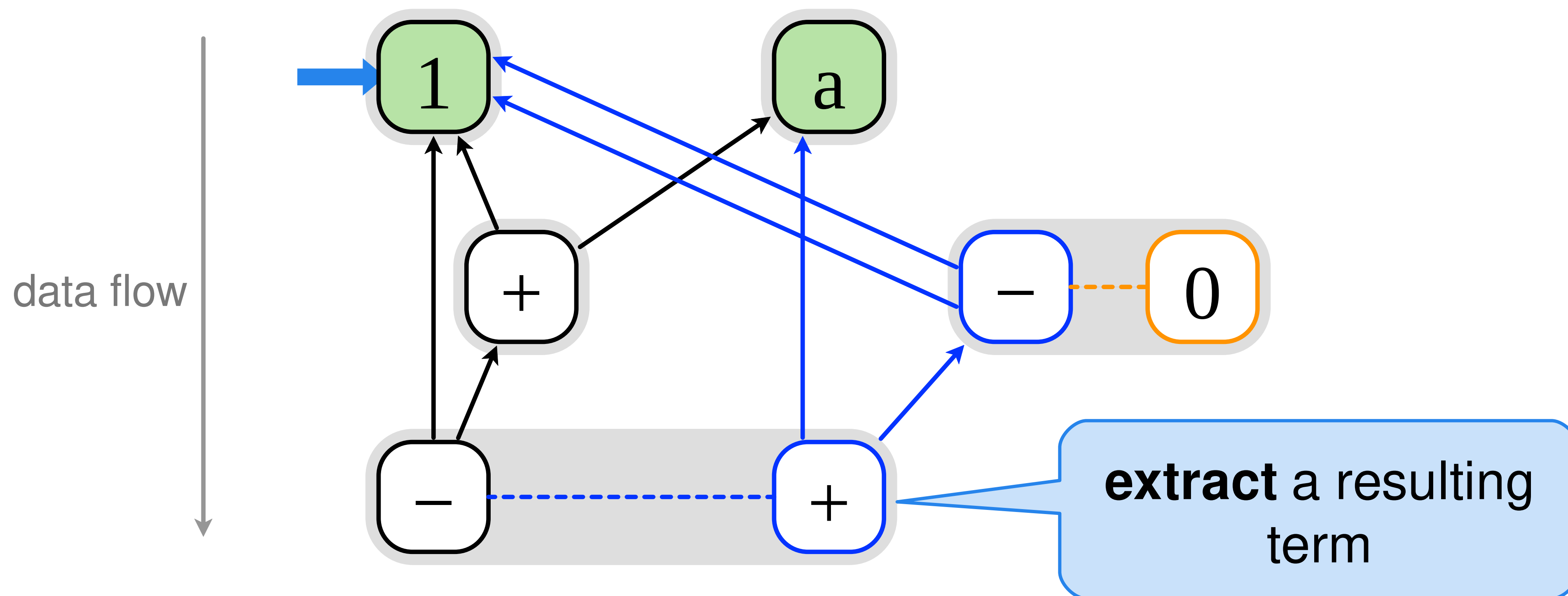


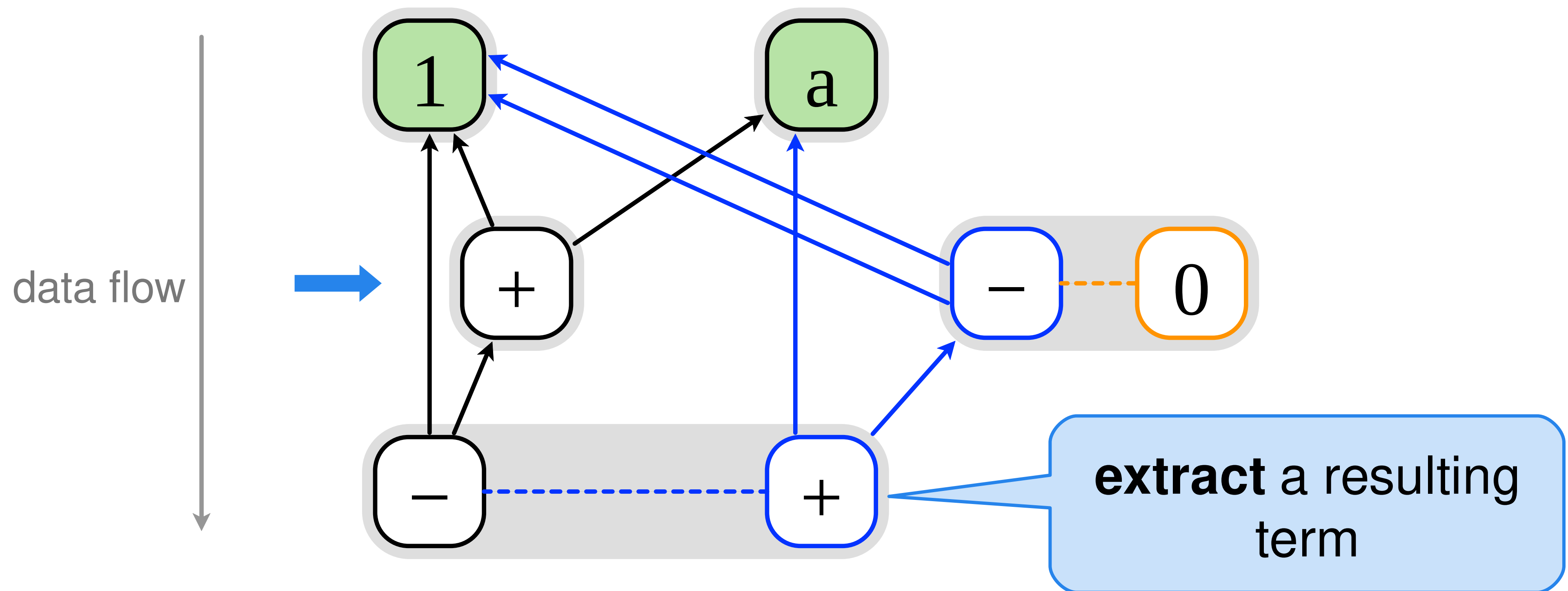
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$



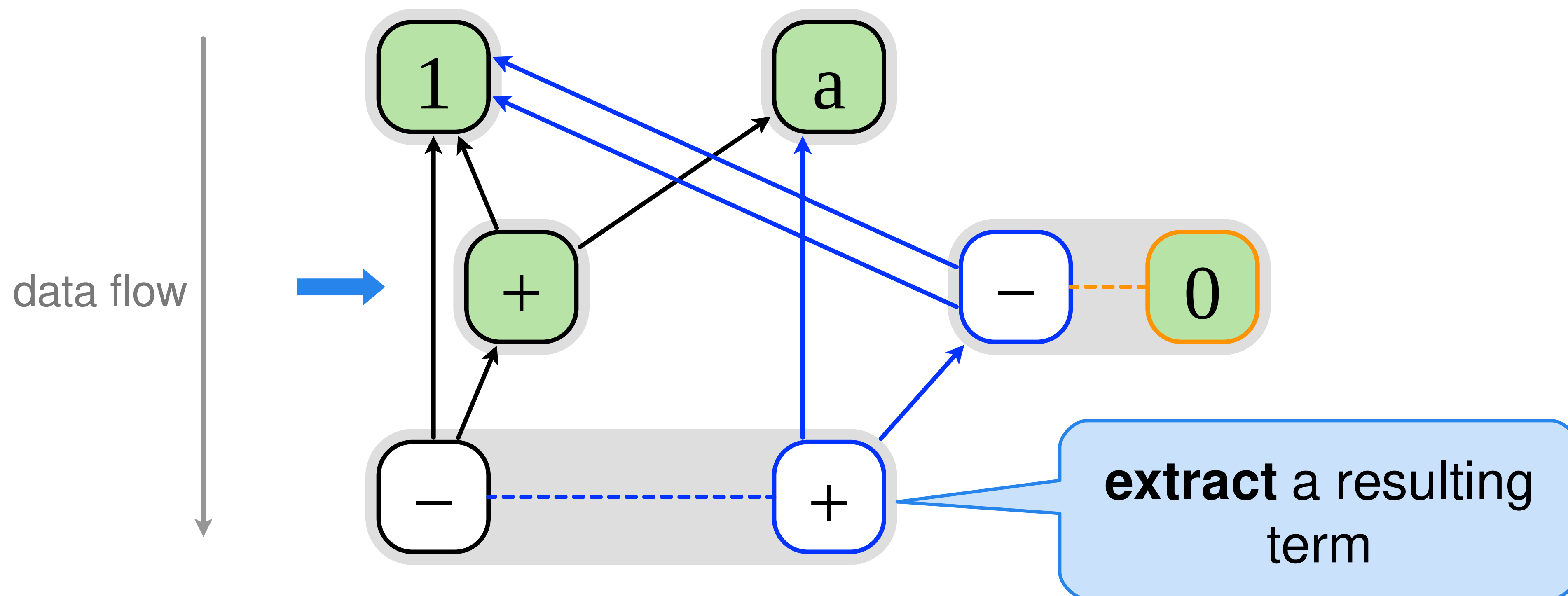


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

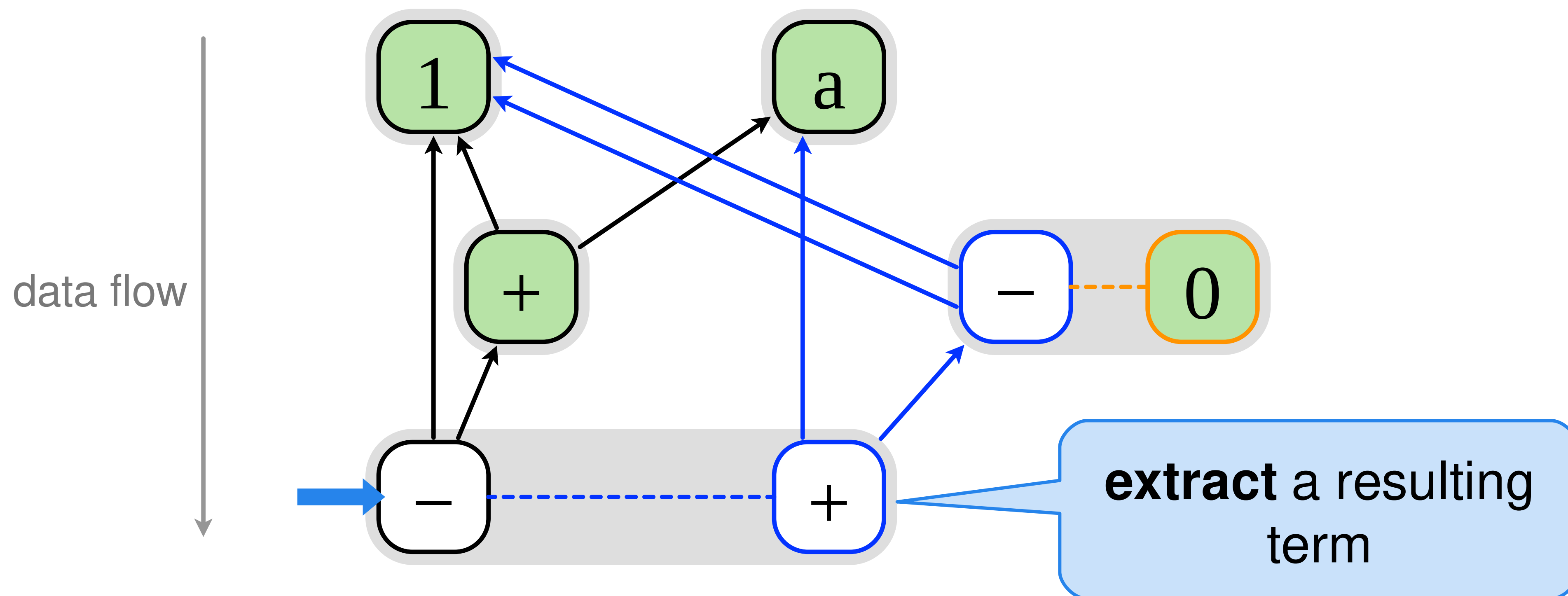


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

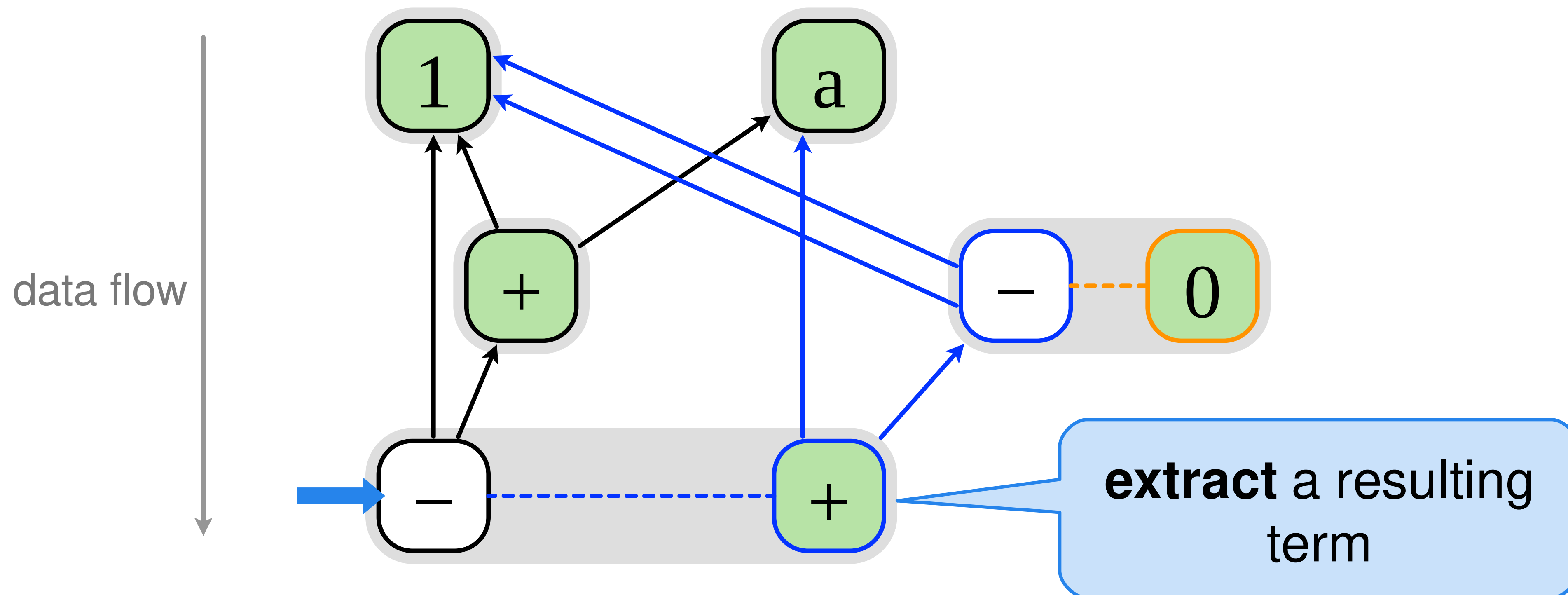


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

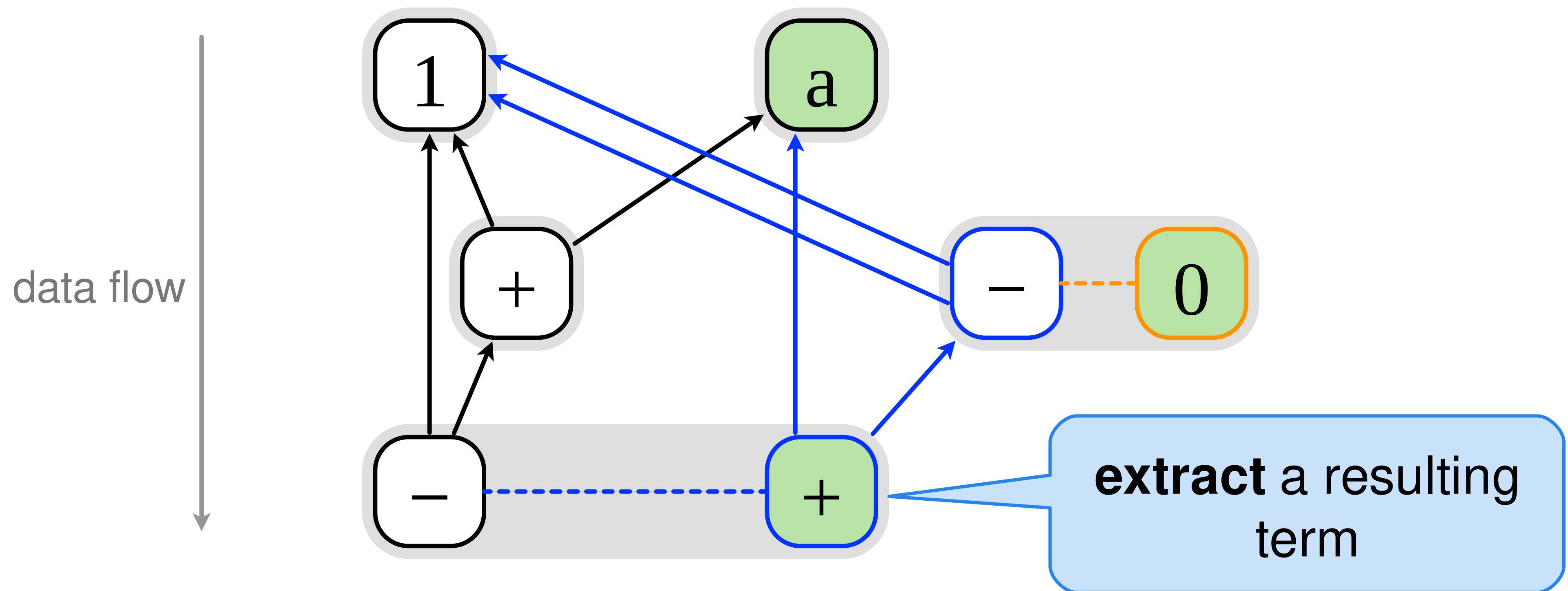


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

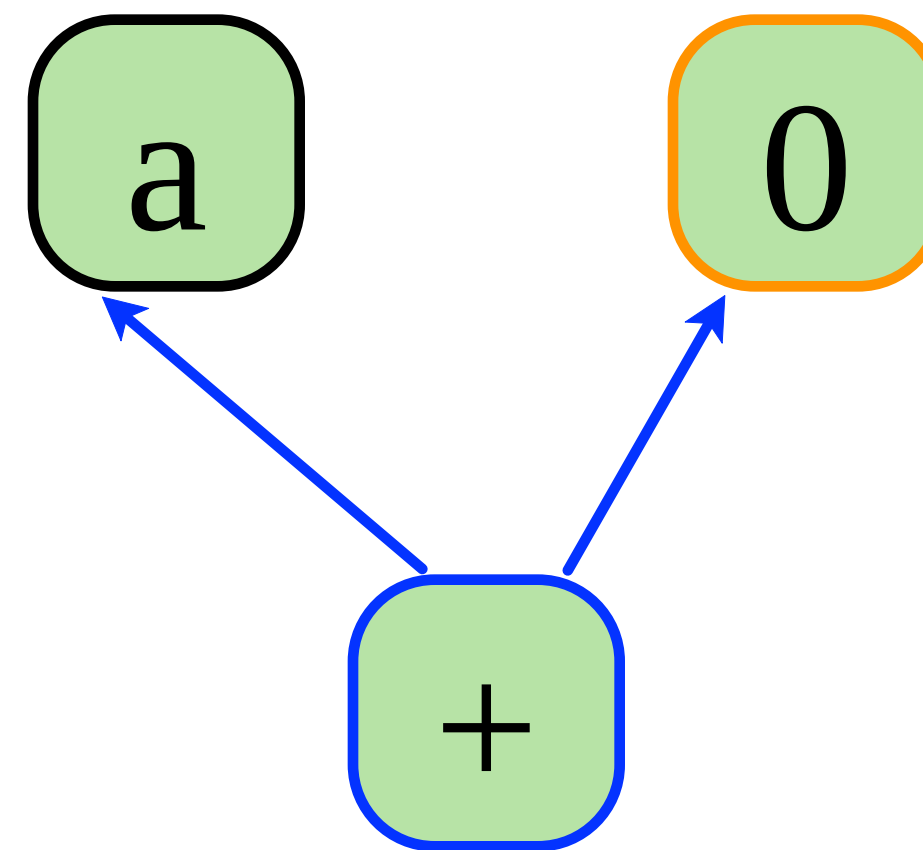


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

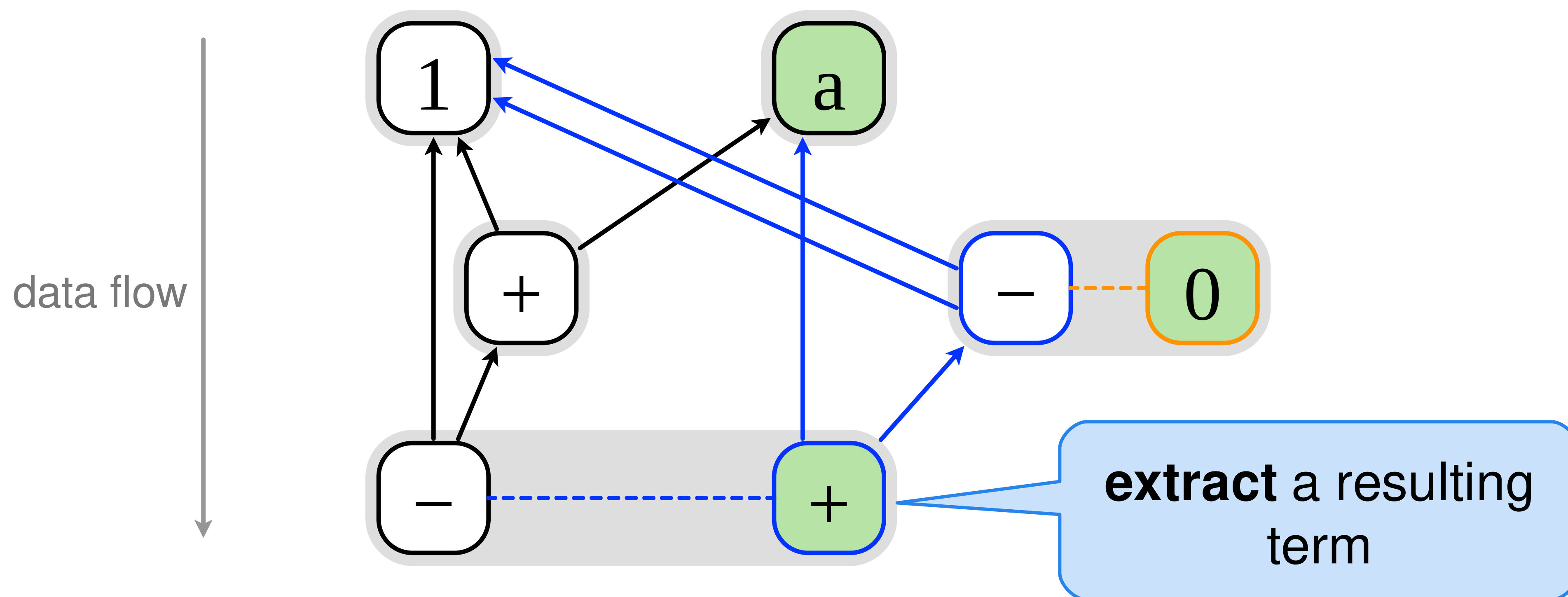


E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$



E-Graphs Explore a Big Space

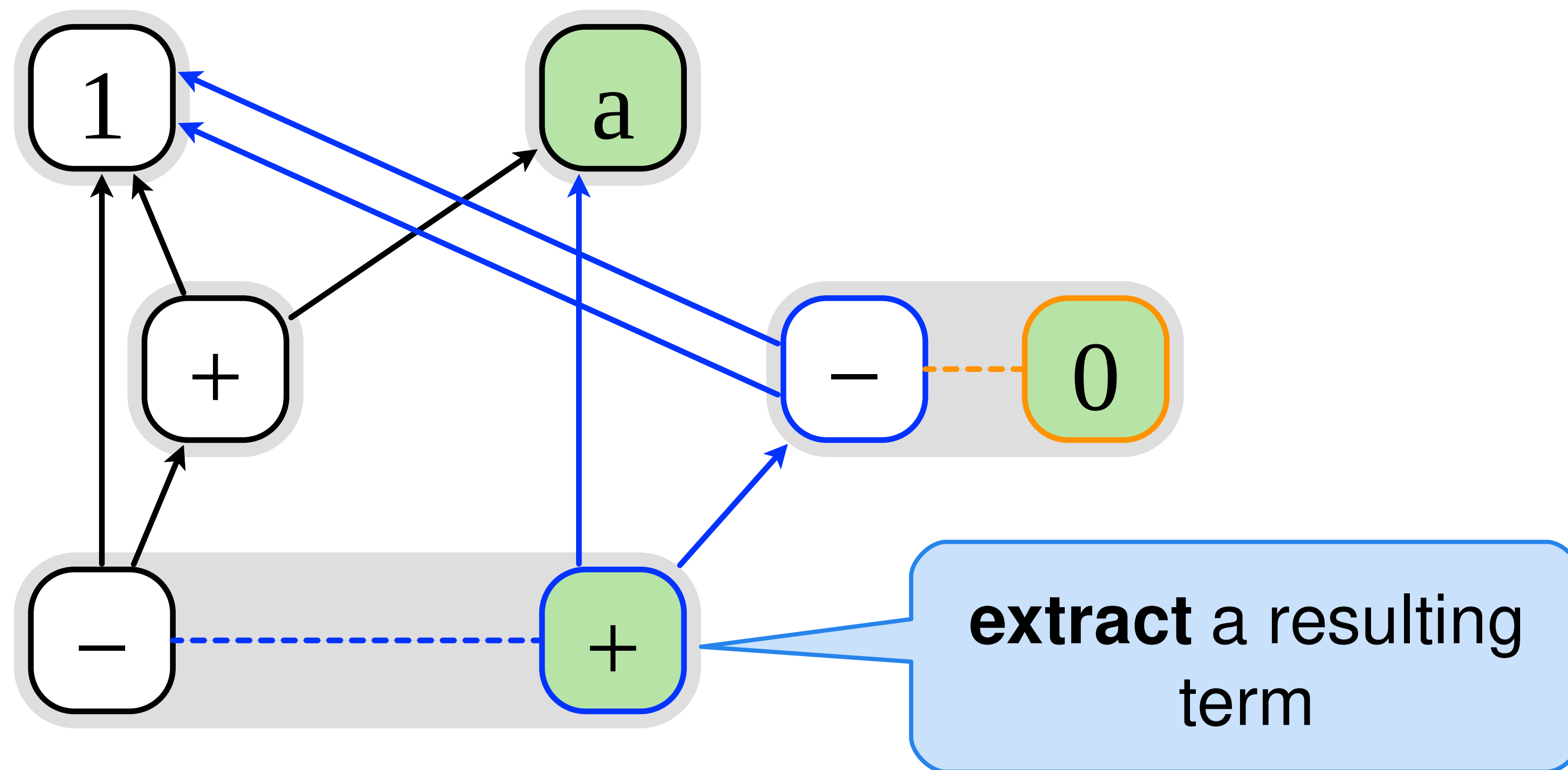
$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

data flow



E-Graphs Explore a Big Space

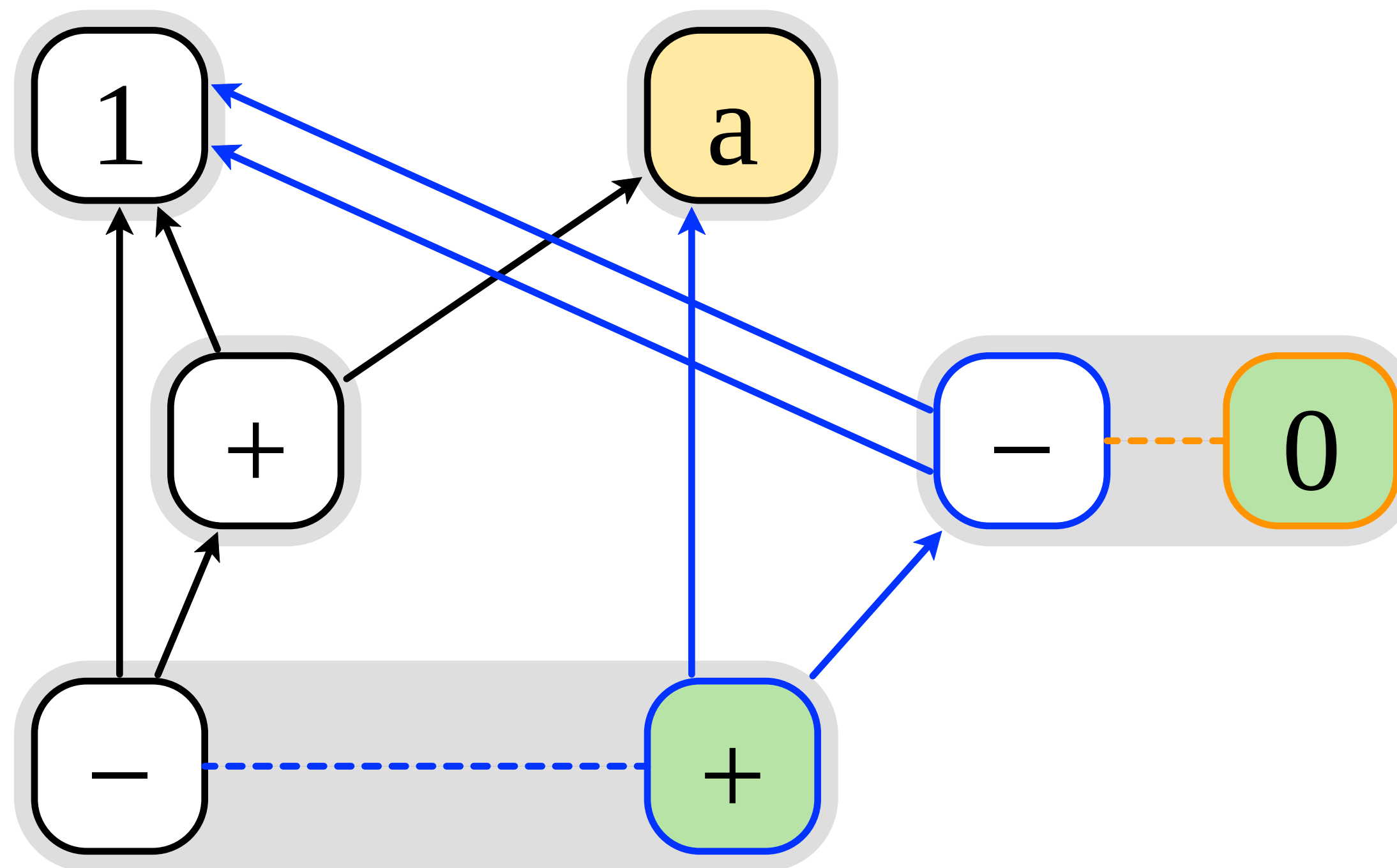
$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

data flow



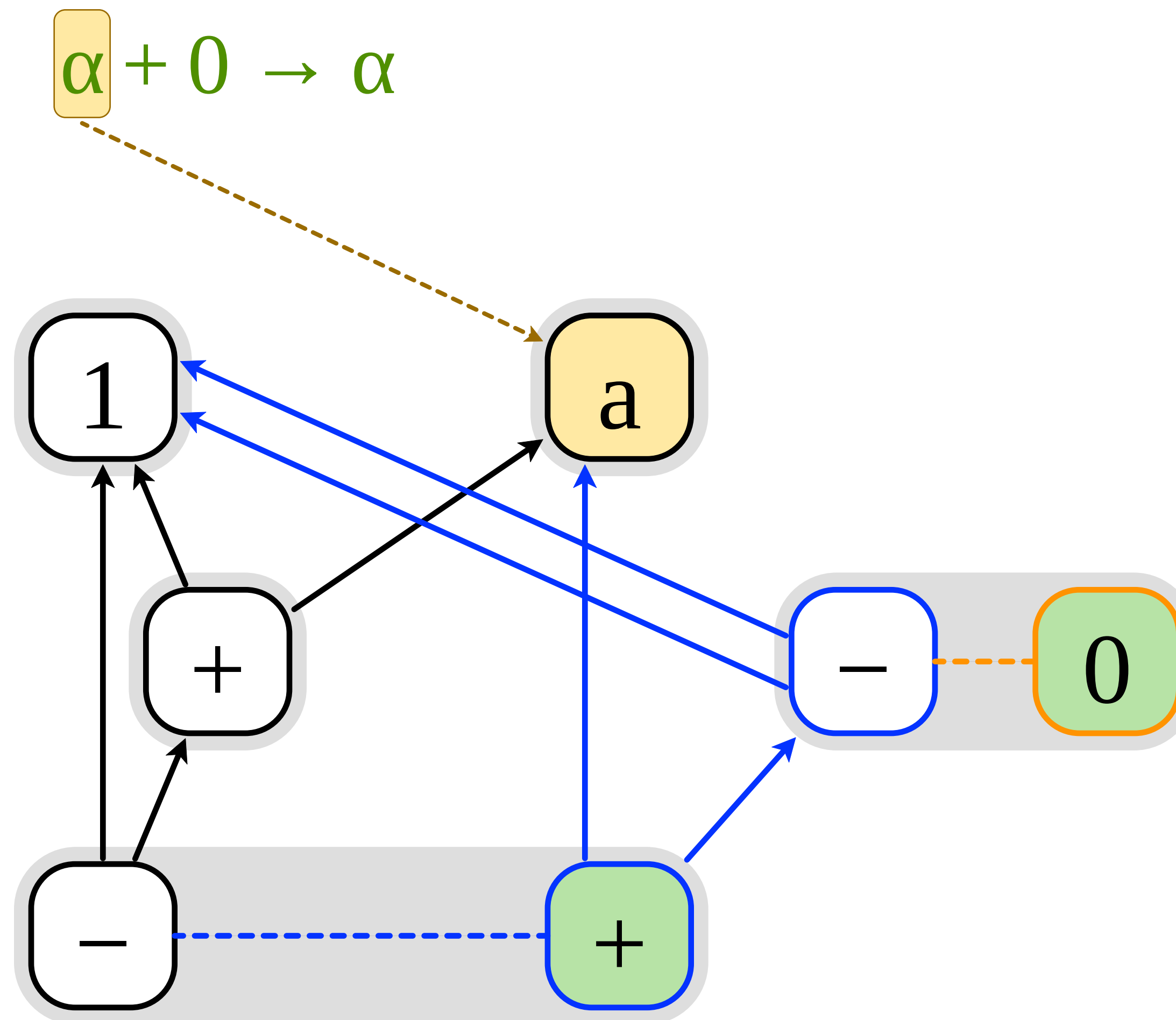
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



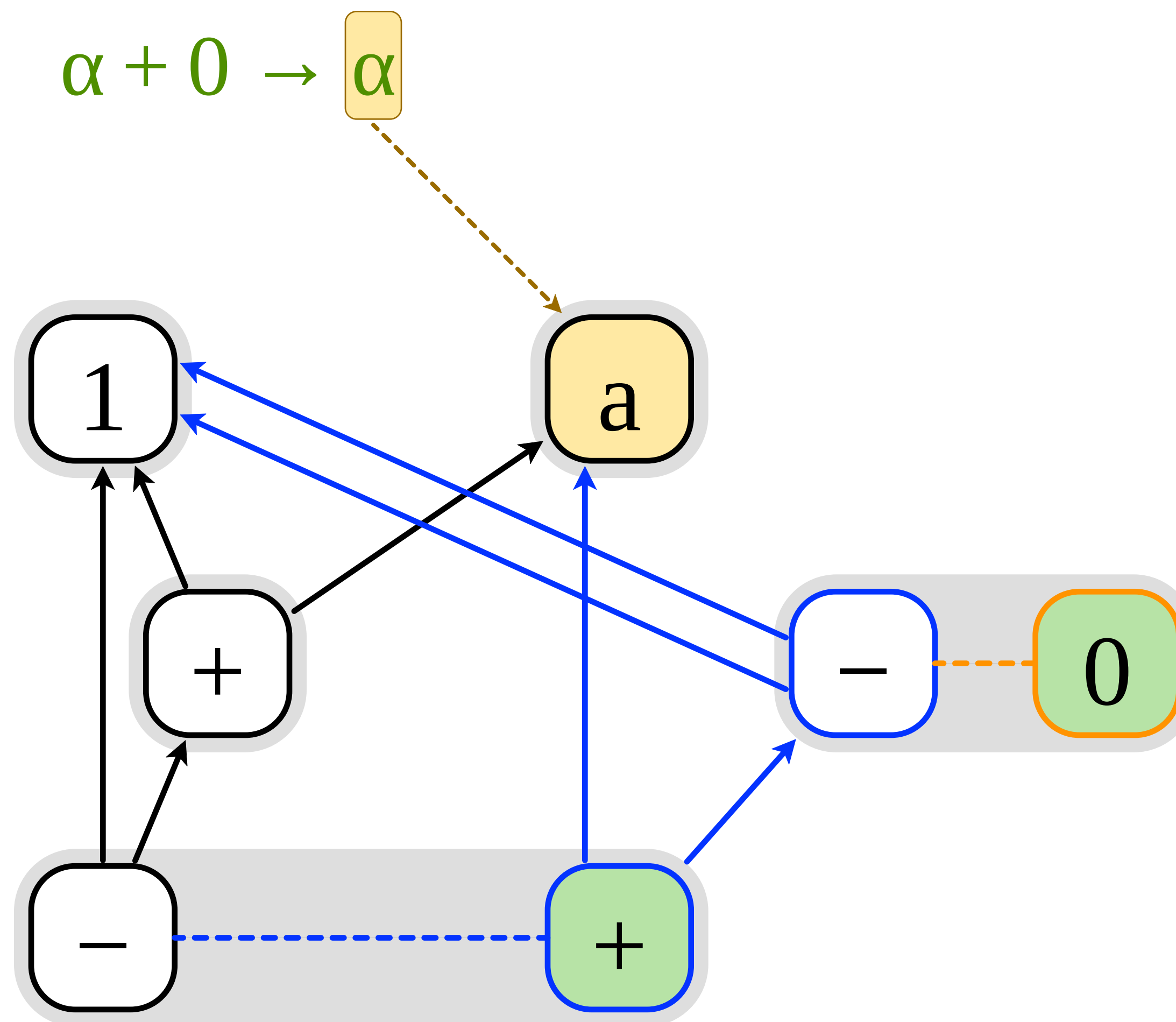
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

data flow



E-Graphs Explore a Big Space

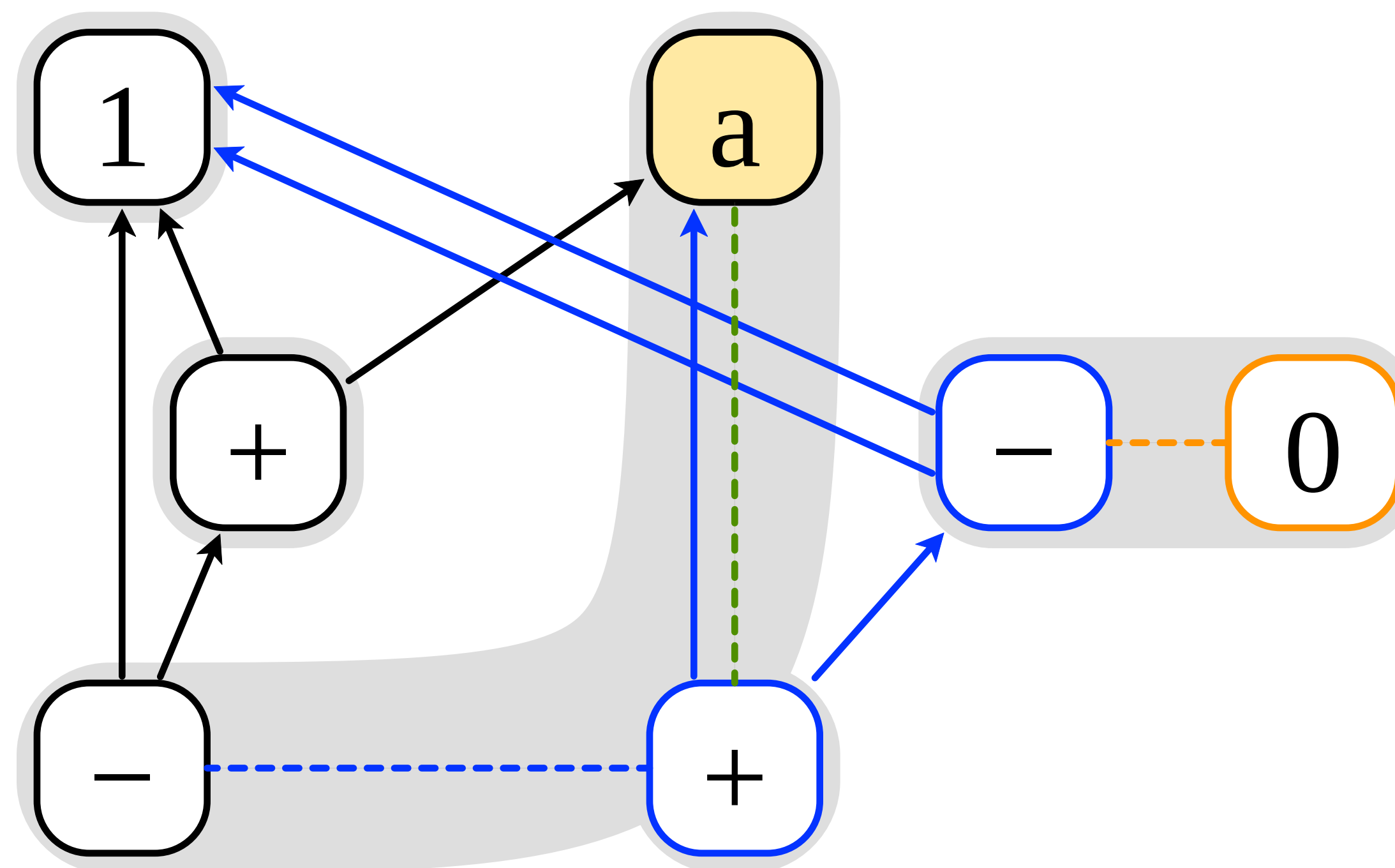
$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

data flow



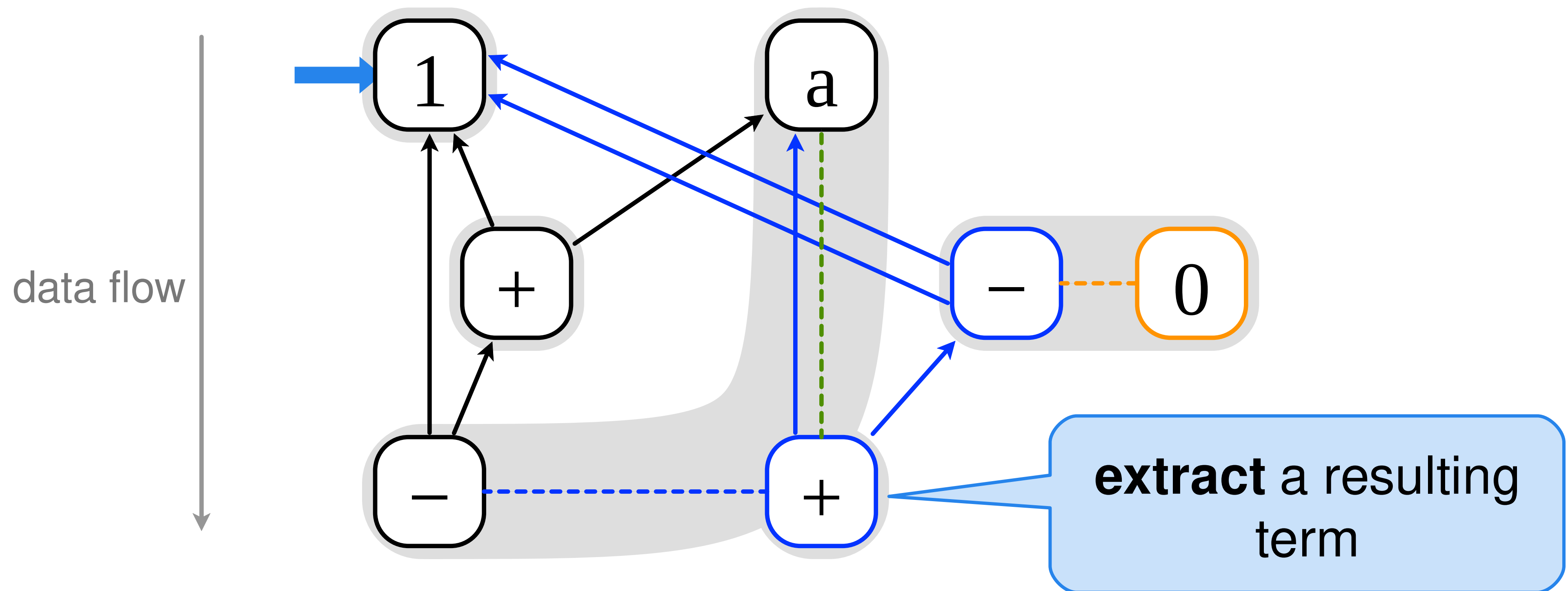
E-Graphs Explore a Big Space

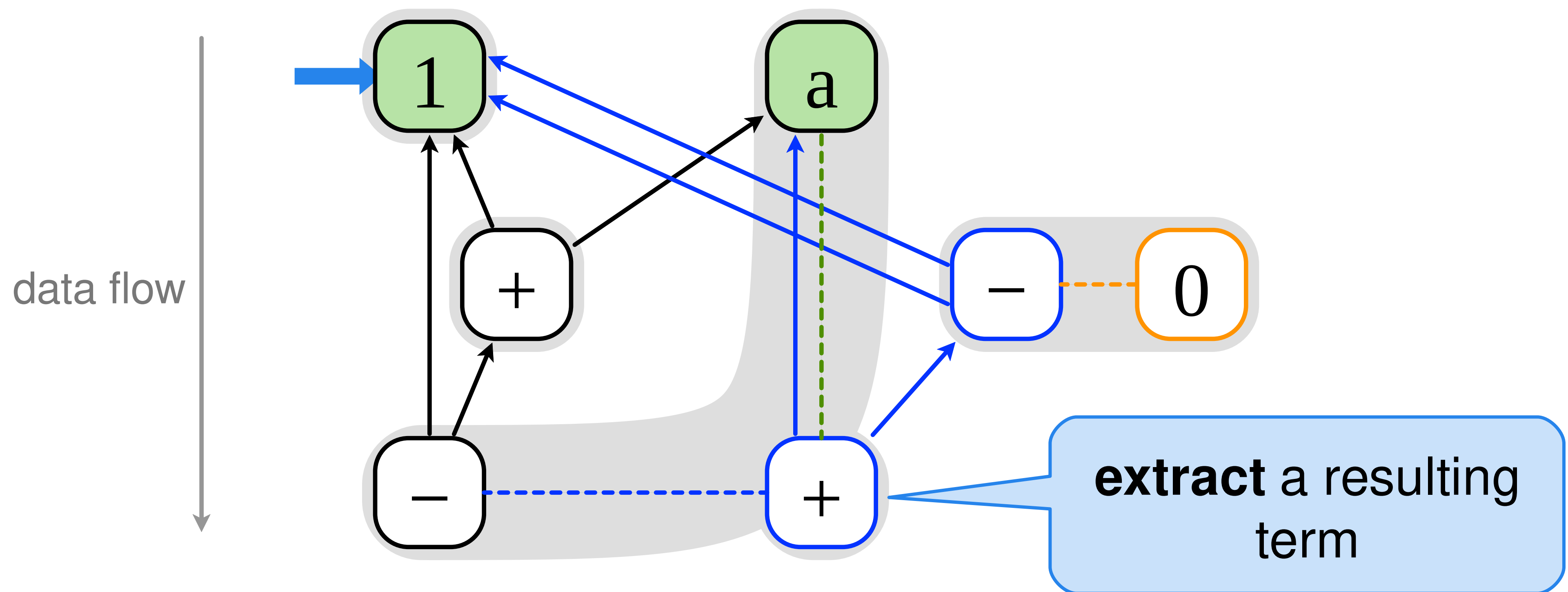
$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$





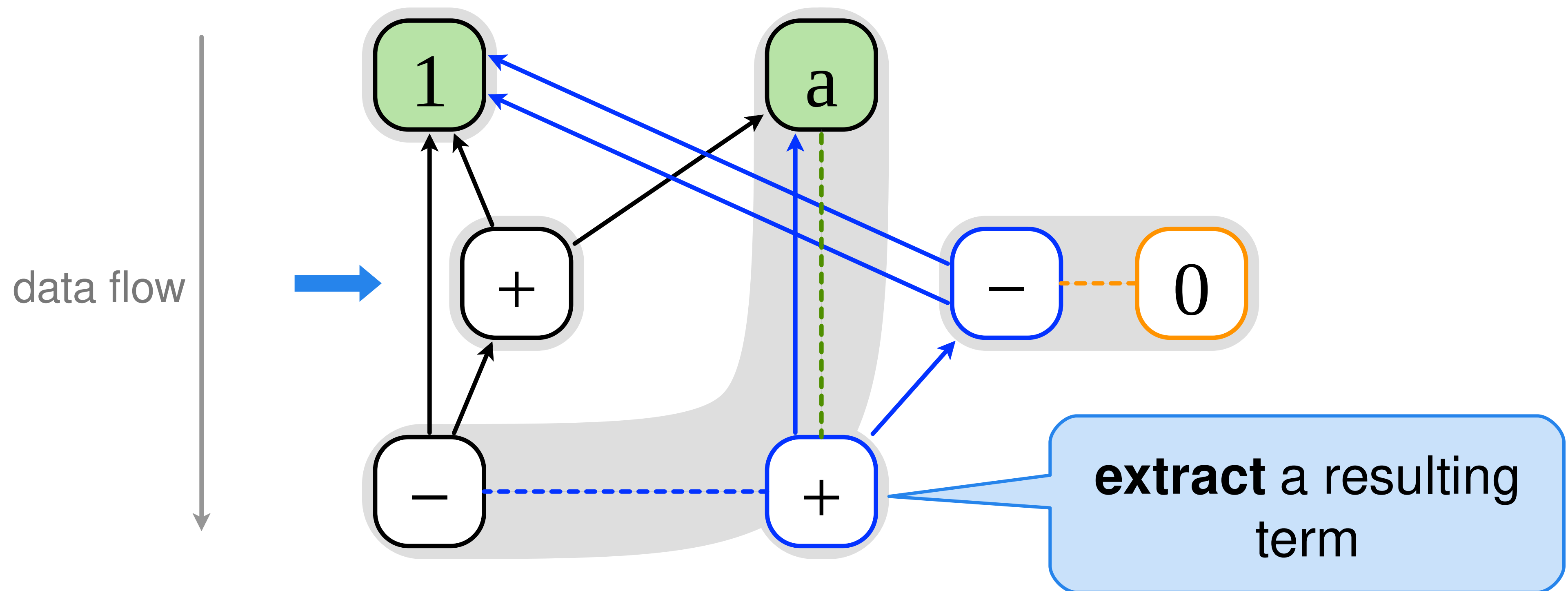
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



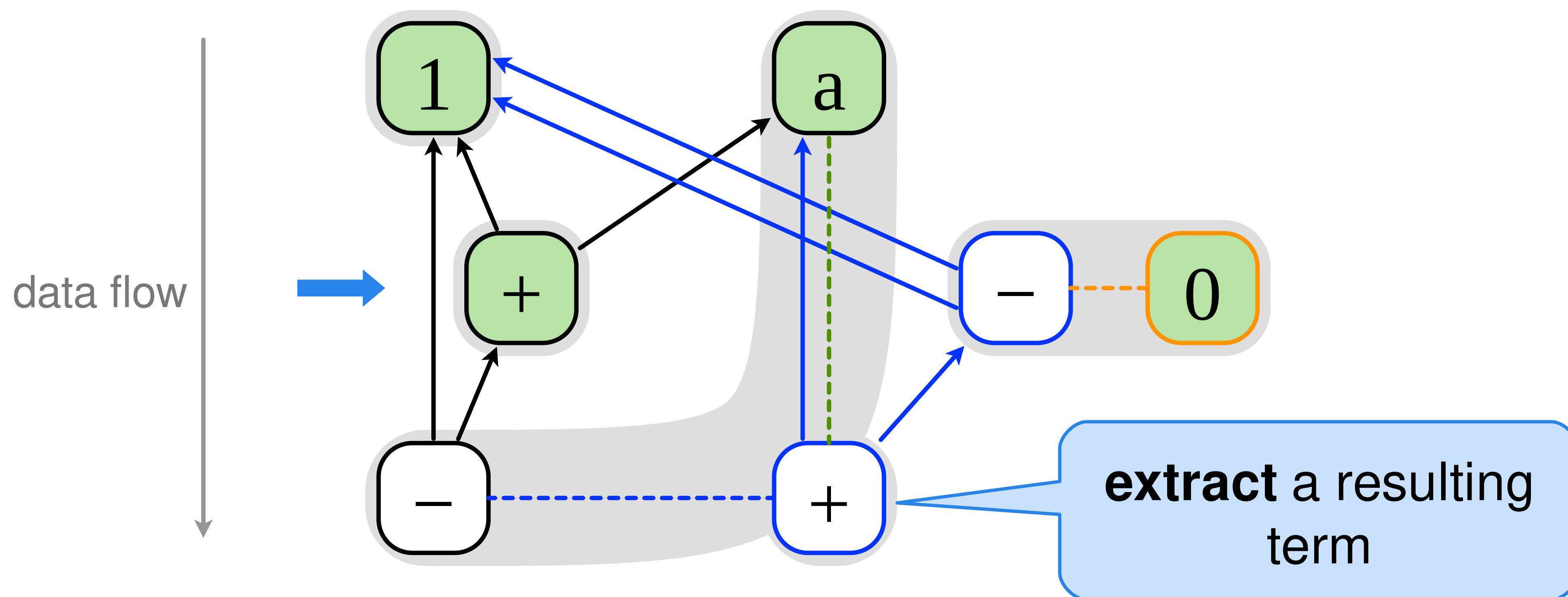
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



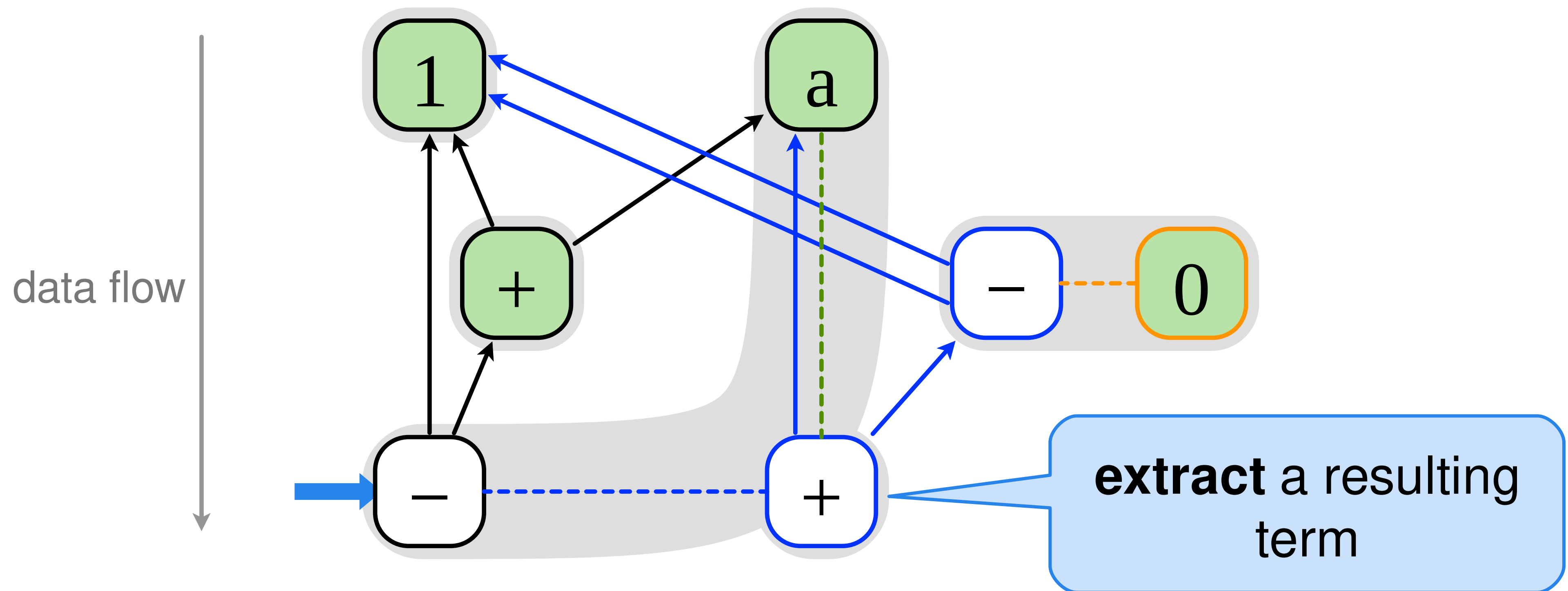
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



E-Graphs Explore a Big Space

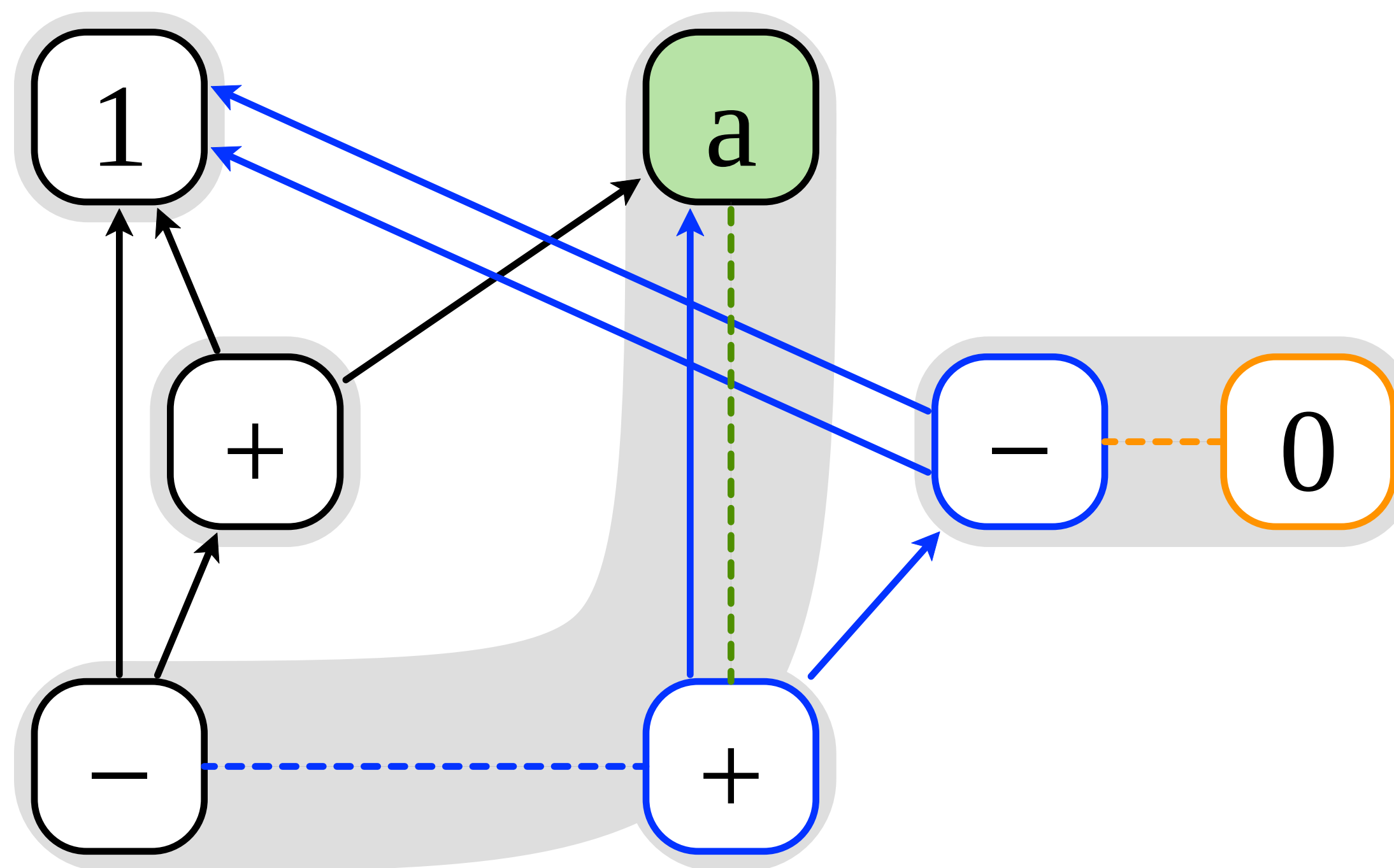
$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

data flow



1

2

3

4

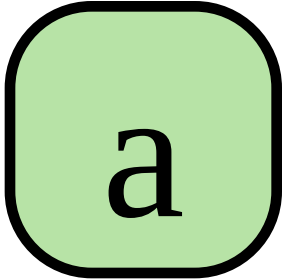
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

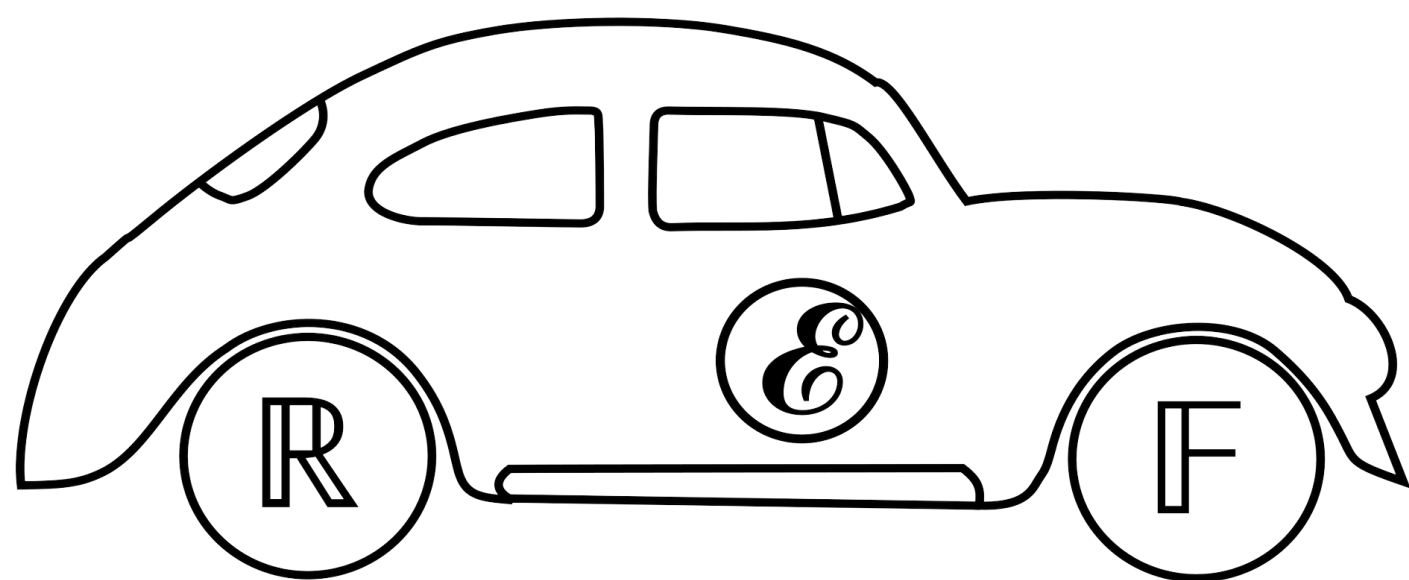
$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

a

E-Graphs are Super Useful!

Floating point

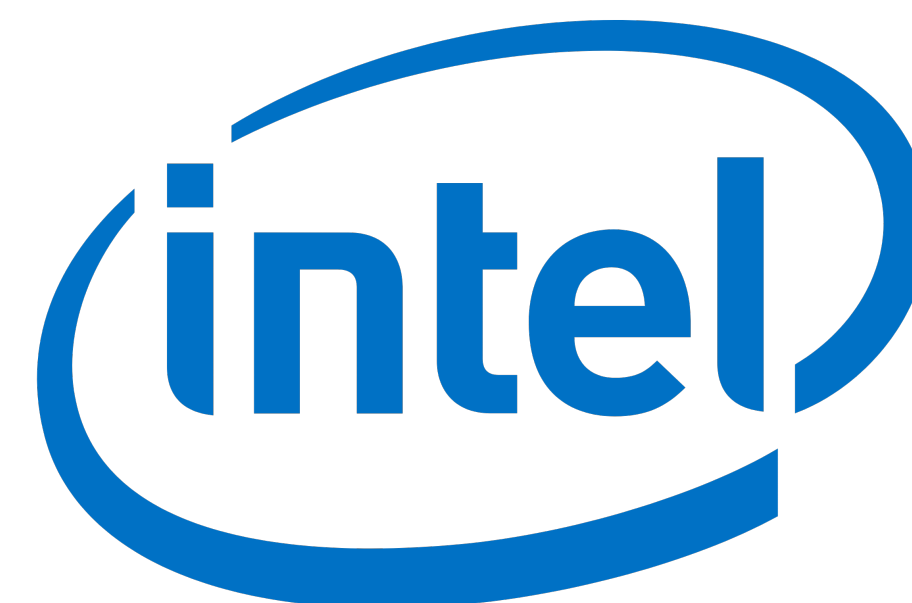


(Herbie)

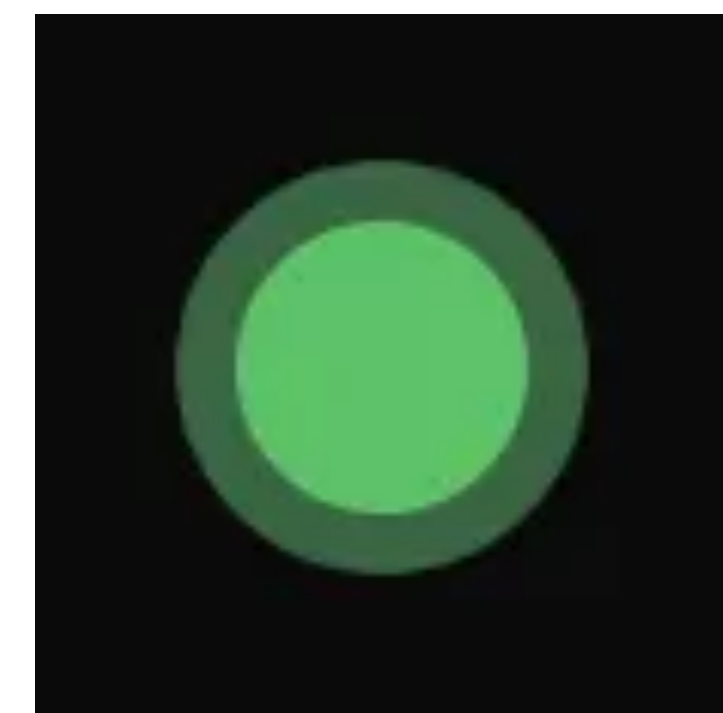
Python Simulations



RTL Optimization



ML Kernels



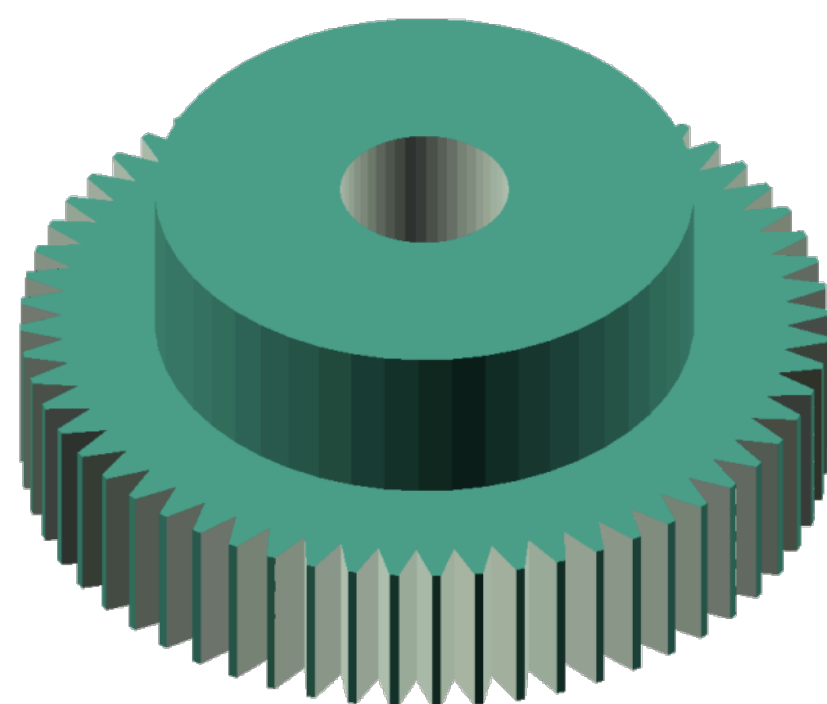
Luminal

SQL



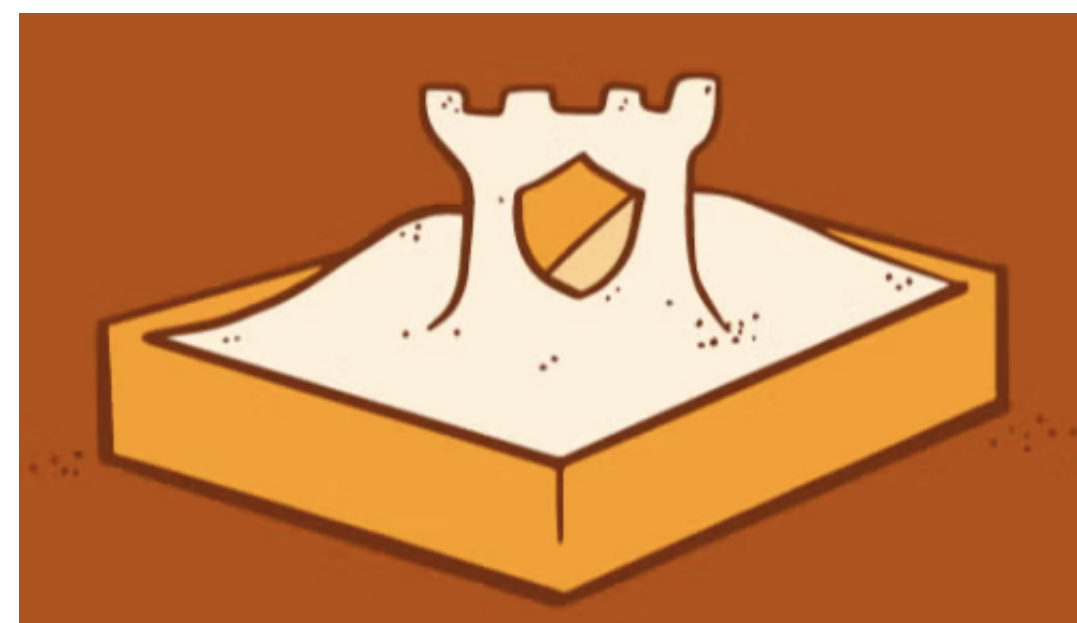
RisingWave

CAD



(Szalinski)

WASM



(Cranelift)

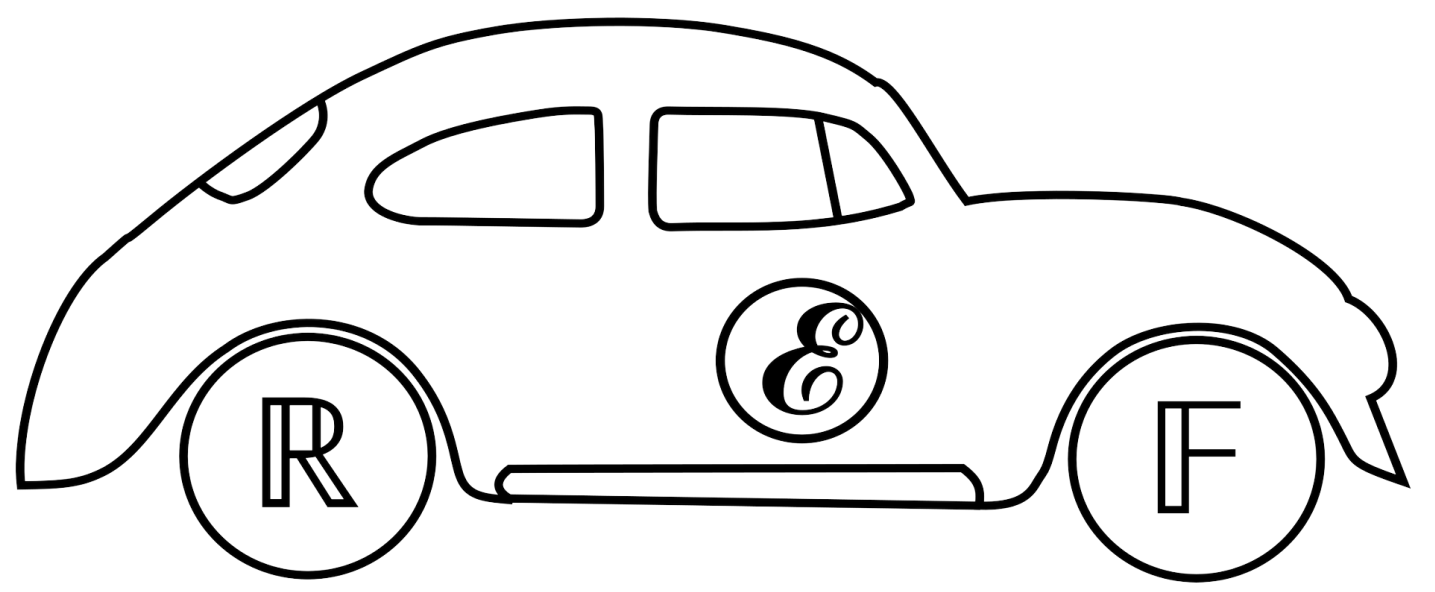
User DSL Optimizations



...and more!

E-Graphs are Super Useful!

Floating point

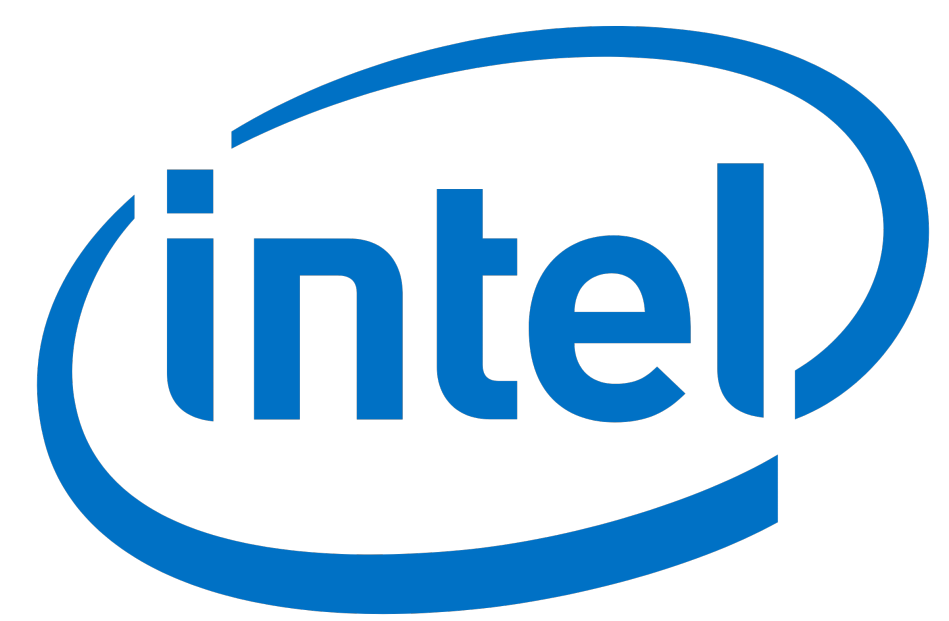


(Herbie)

Python Simulations



RTL Optimization



ML Kernels

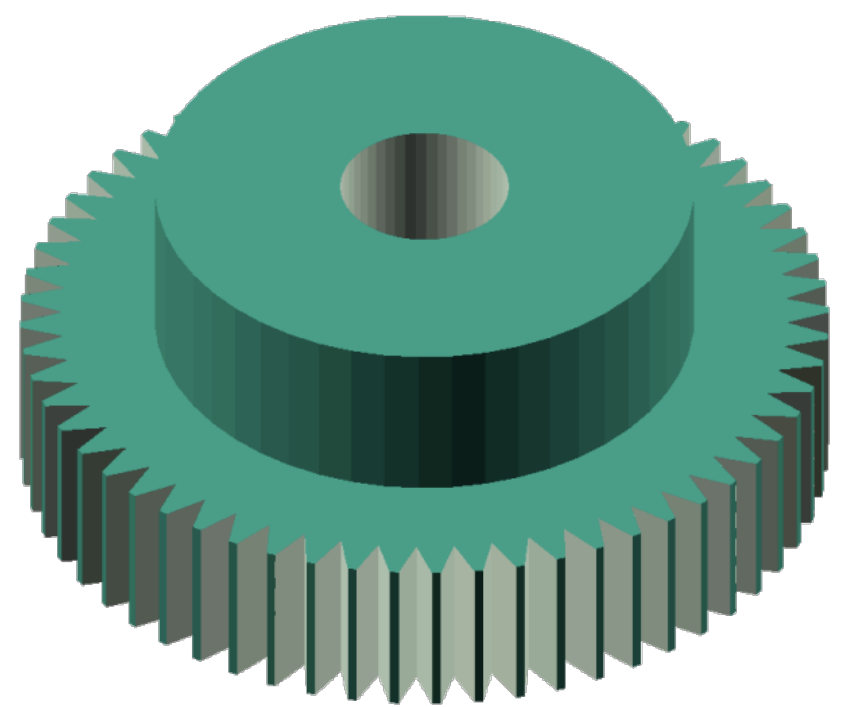


SQL



RisingWave

CAD

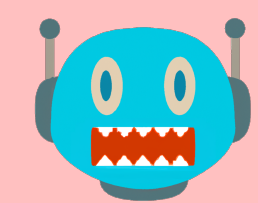


(Szalinski)

WASM



(Cranelift)



need side effects!

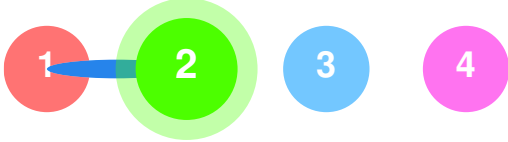
L Optimizations

Luminal

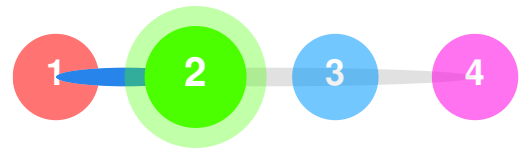


MLIR

...and more!



Example Effective E-Graph



Update

```
fun foo(a, x):  
  v1 = update(a, x)  
  v2 = update(a, x+1)  
  return v1
```

Update

```
fun foo(a, x):  
  v1 = update(a, x)  
  v2 = update(a, x+1)  
  return v1
```

Write **x** to address **a**,
return the previous value at **a**

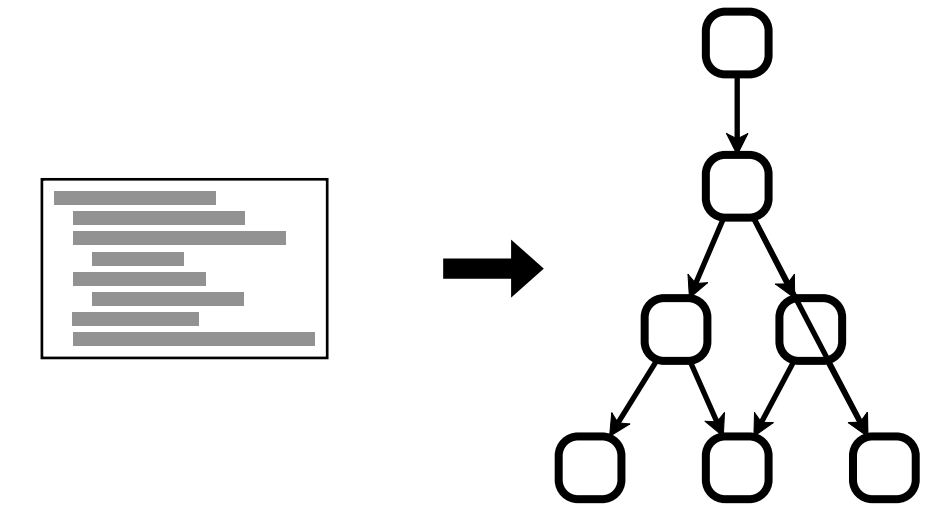
Update

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

ω_1 is the state of memory
after the update

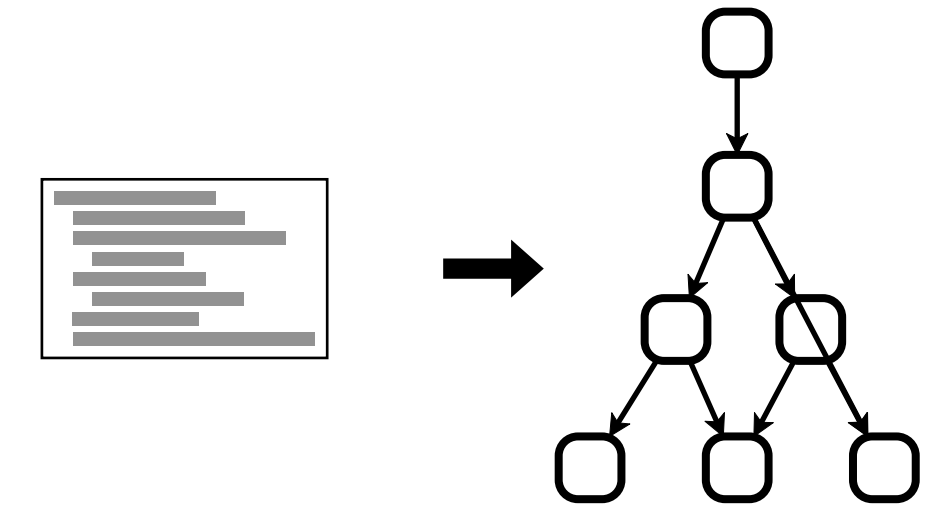
The Effectful E-Graph

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```



The Effectful E-Graph

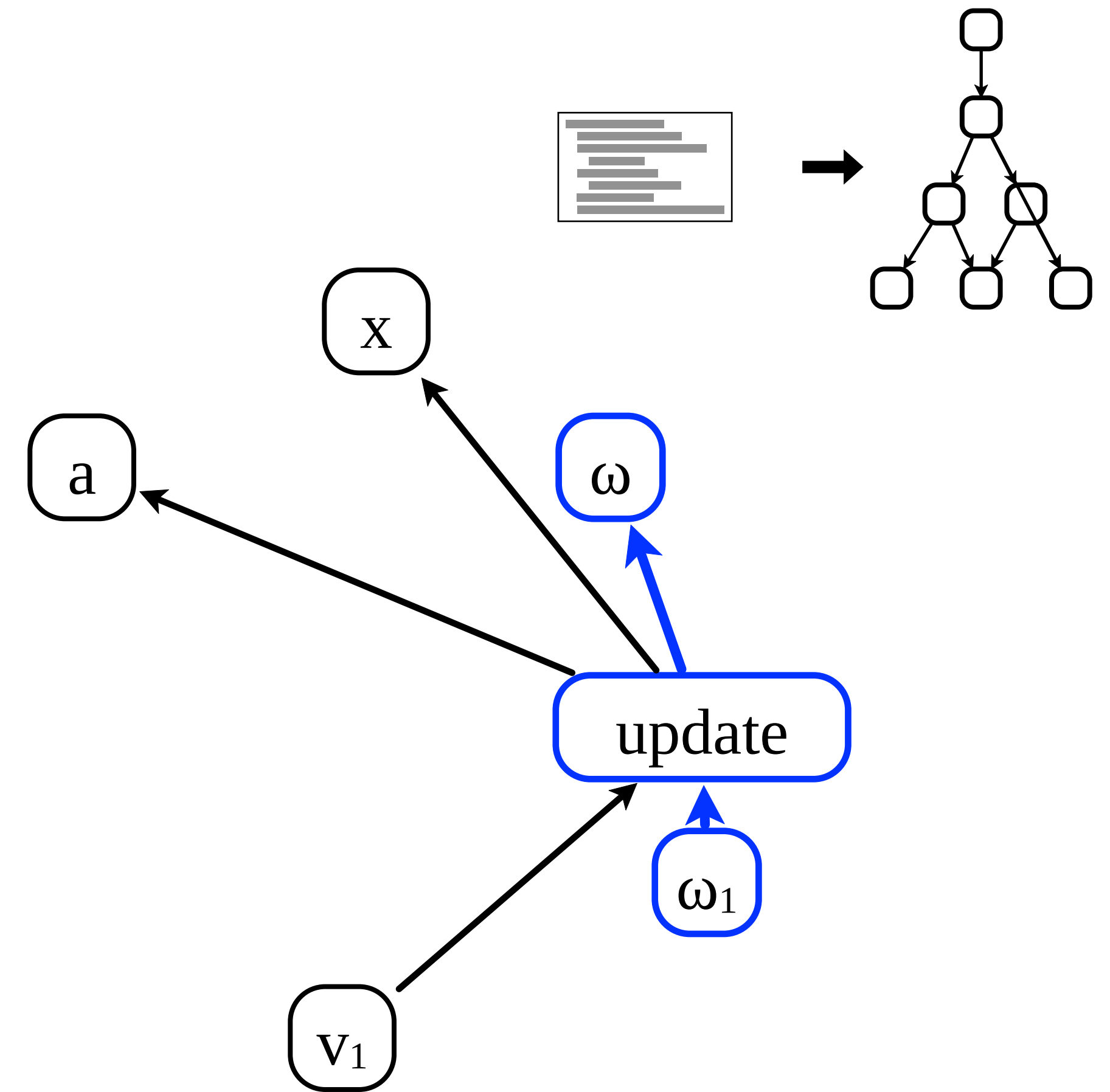
```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```



The Effective E-Graph

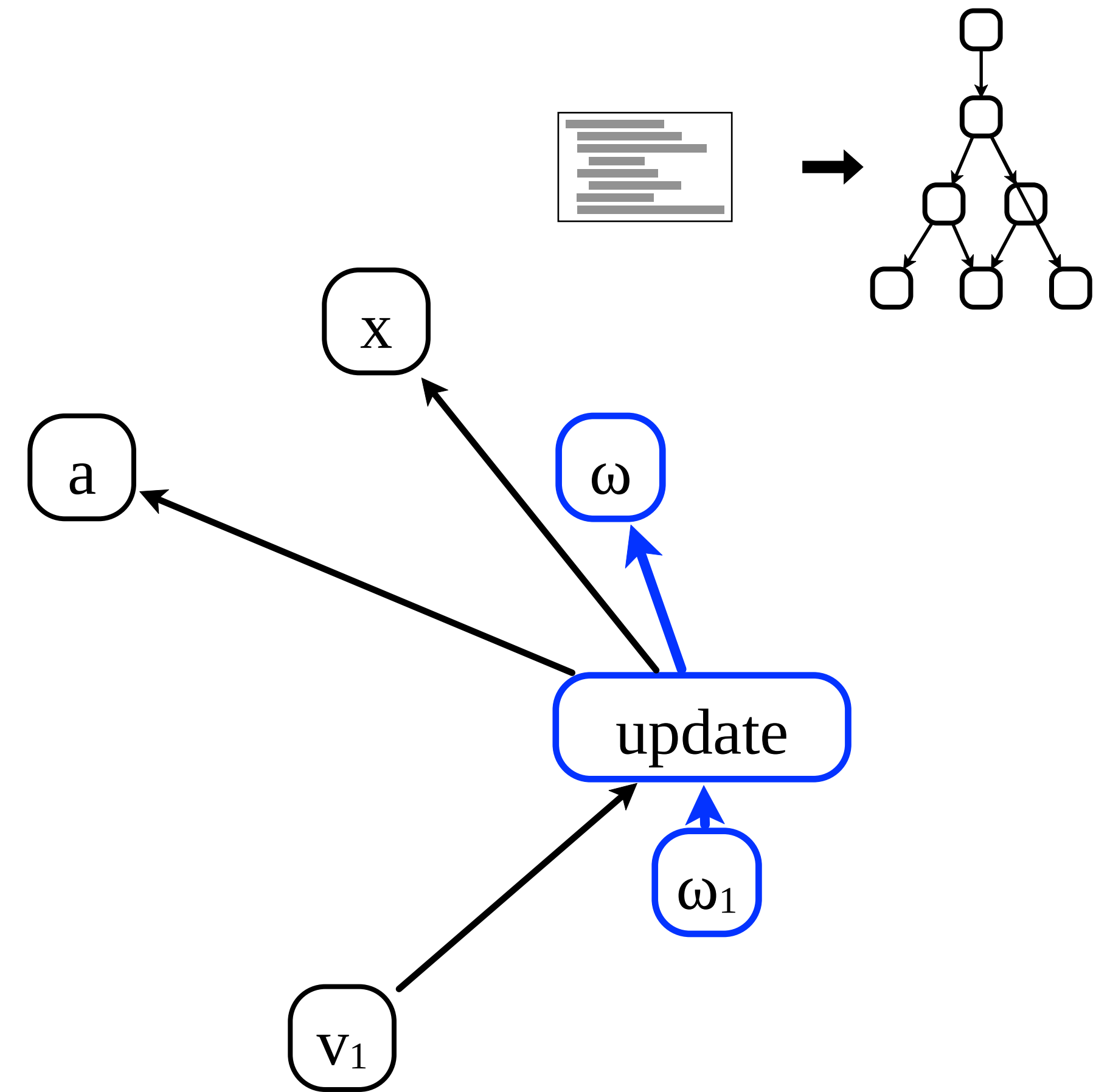
```

fun foo(a, x,  $\omega$ ):
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )
  return ( $v_1$ ,  $\omega_2$ )
  
```



The Effectful E-Graph

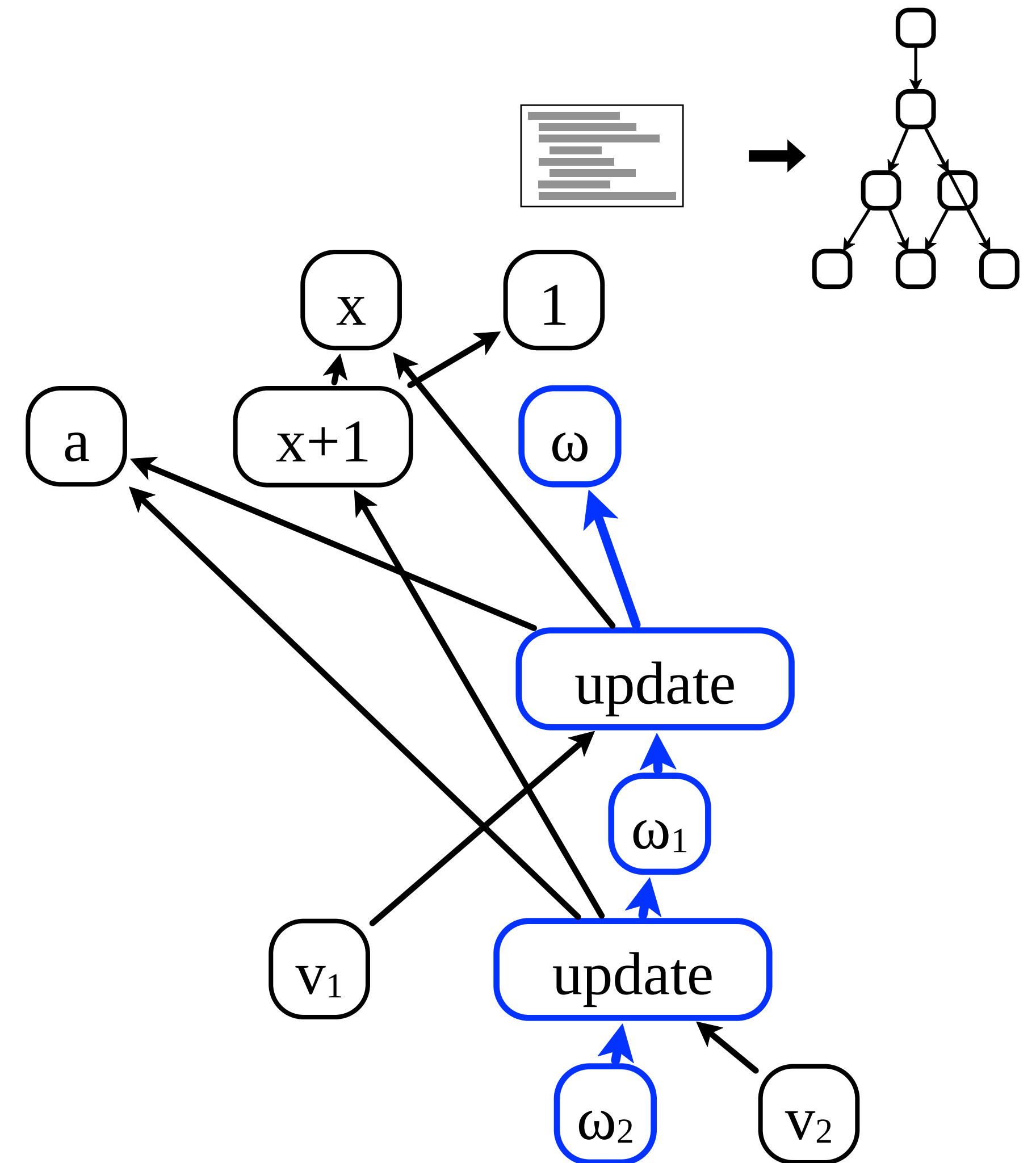
```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```



The Effectful E-Graph

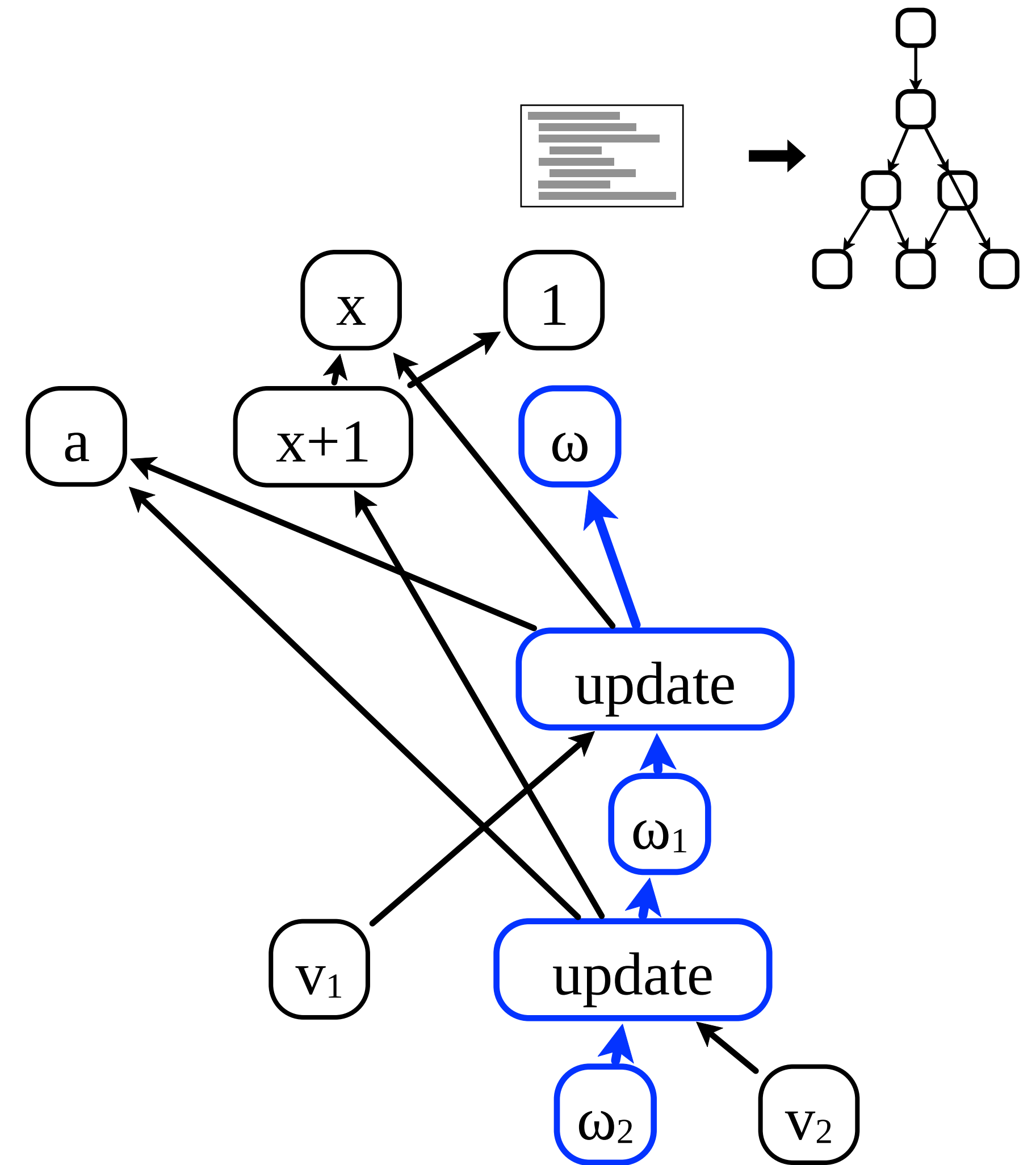
```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



The Effectful E-Graph

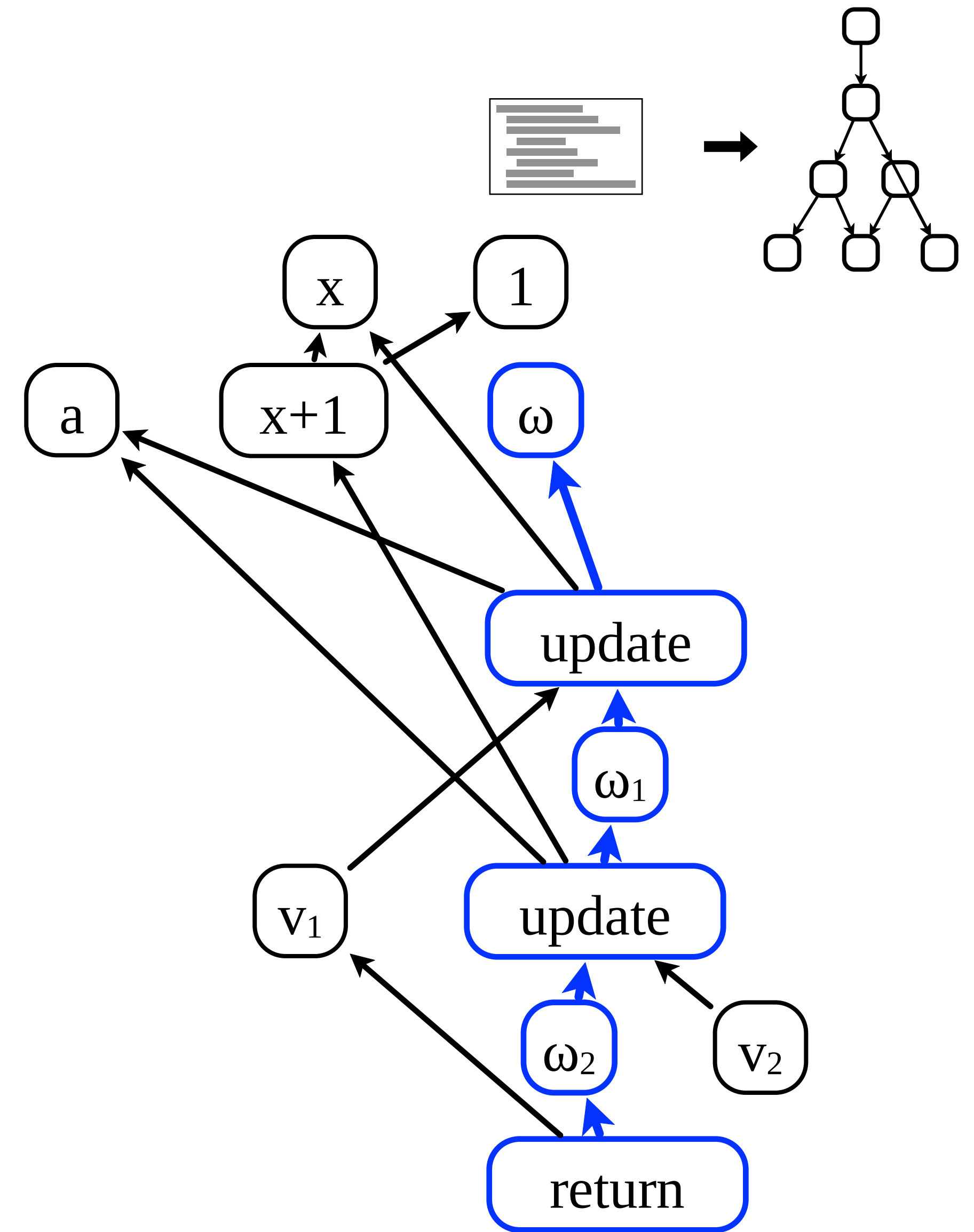
```
fun foo(a, x, ω):  
  (v1, ω1) = update(a, x, ω)  
  (v2, ω2) = update(a, x+1, ω1)  
  return (v1, ω2)
```



The Effectful E-Graph

```

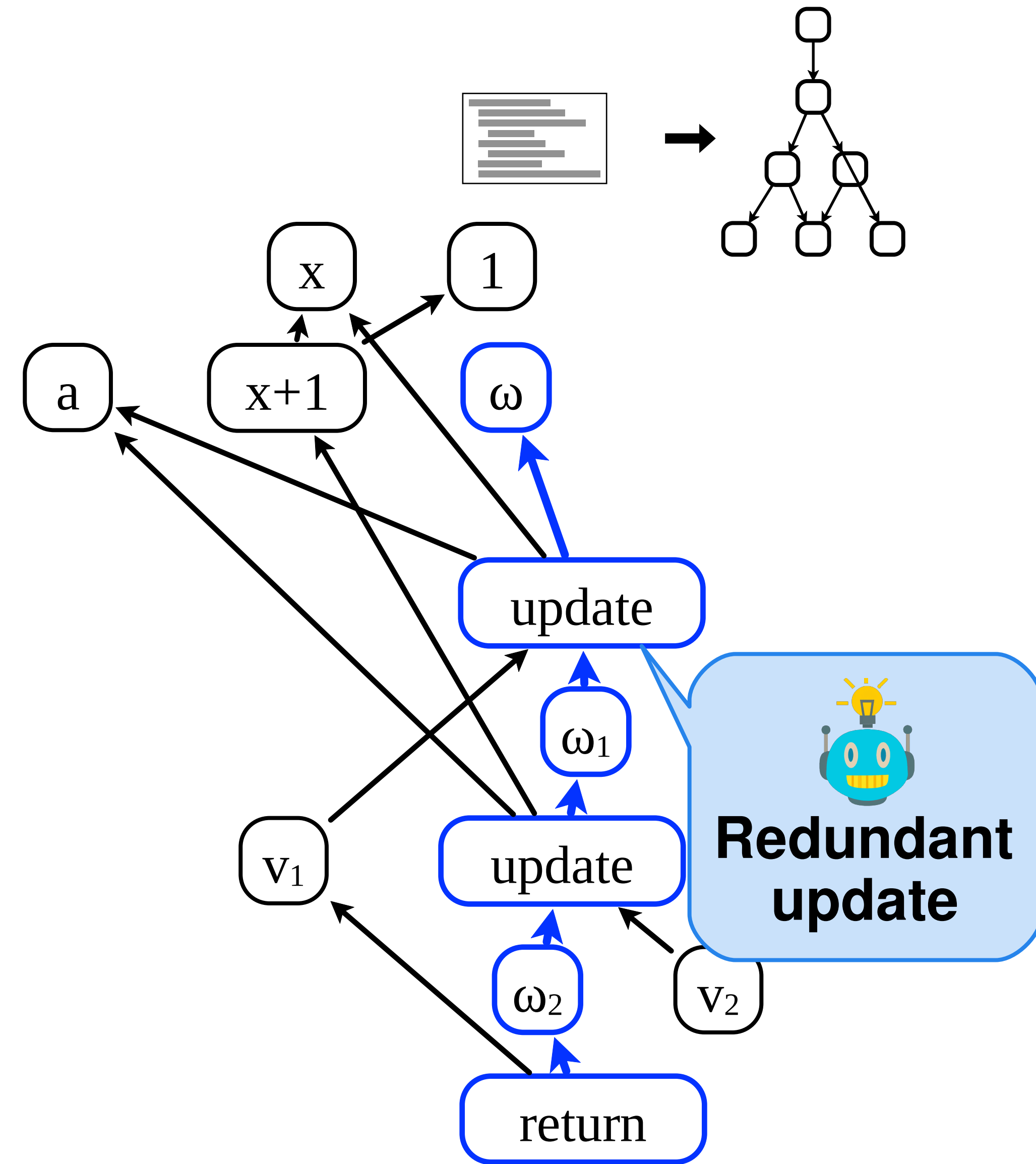
fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



The Effectful E-Graph

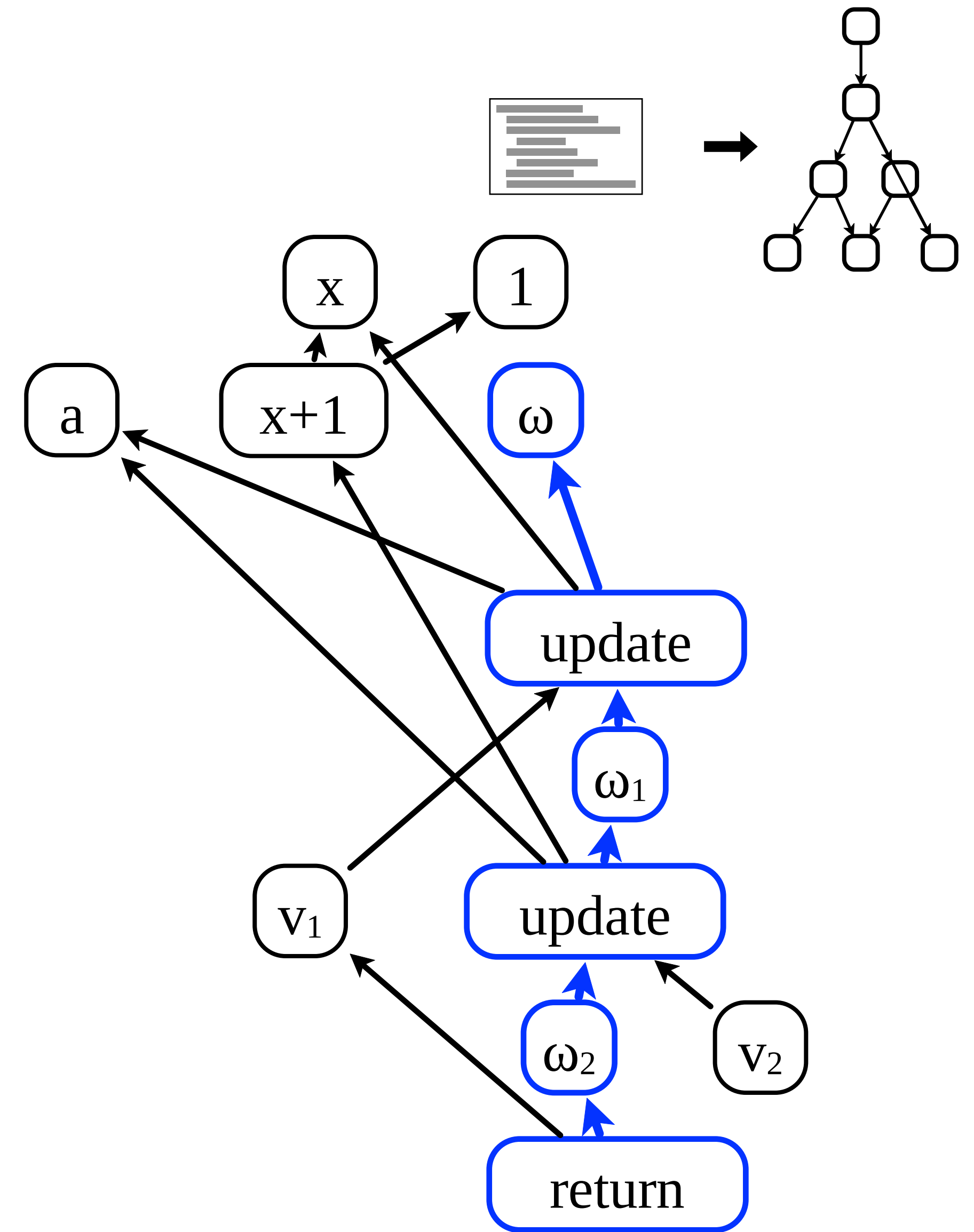
```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



Dead Store Elimination

```
fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
```



Dead Store Elimination

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

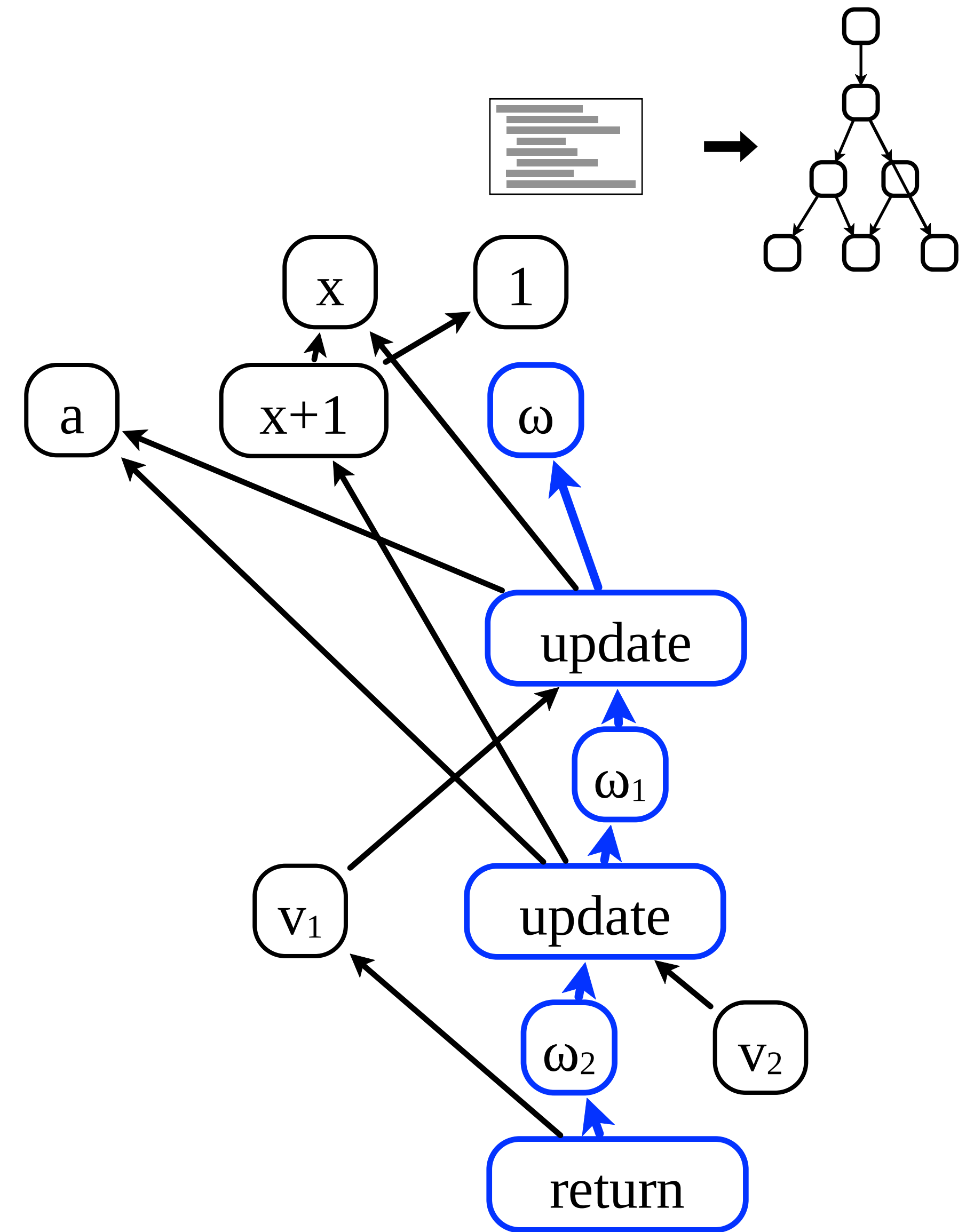
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Dead Store Elimination

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

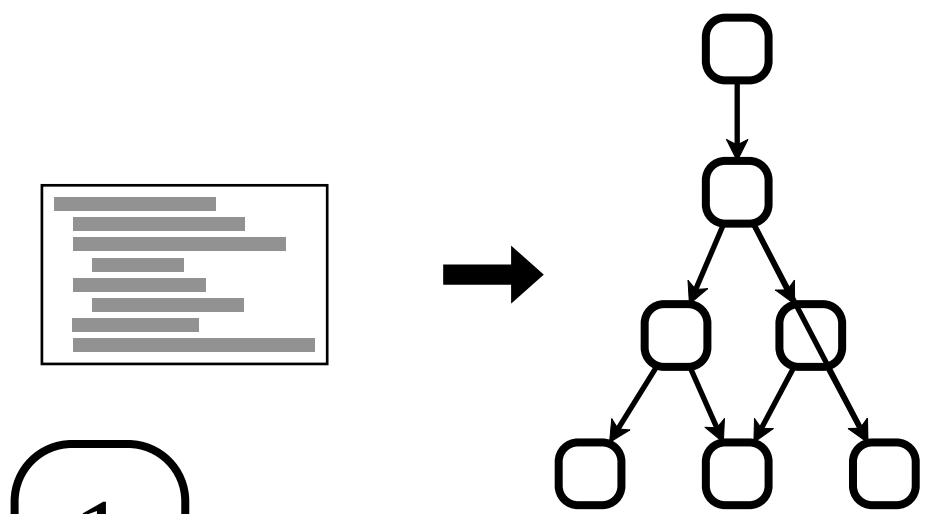
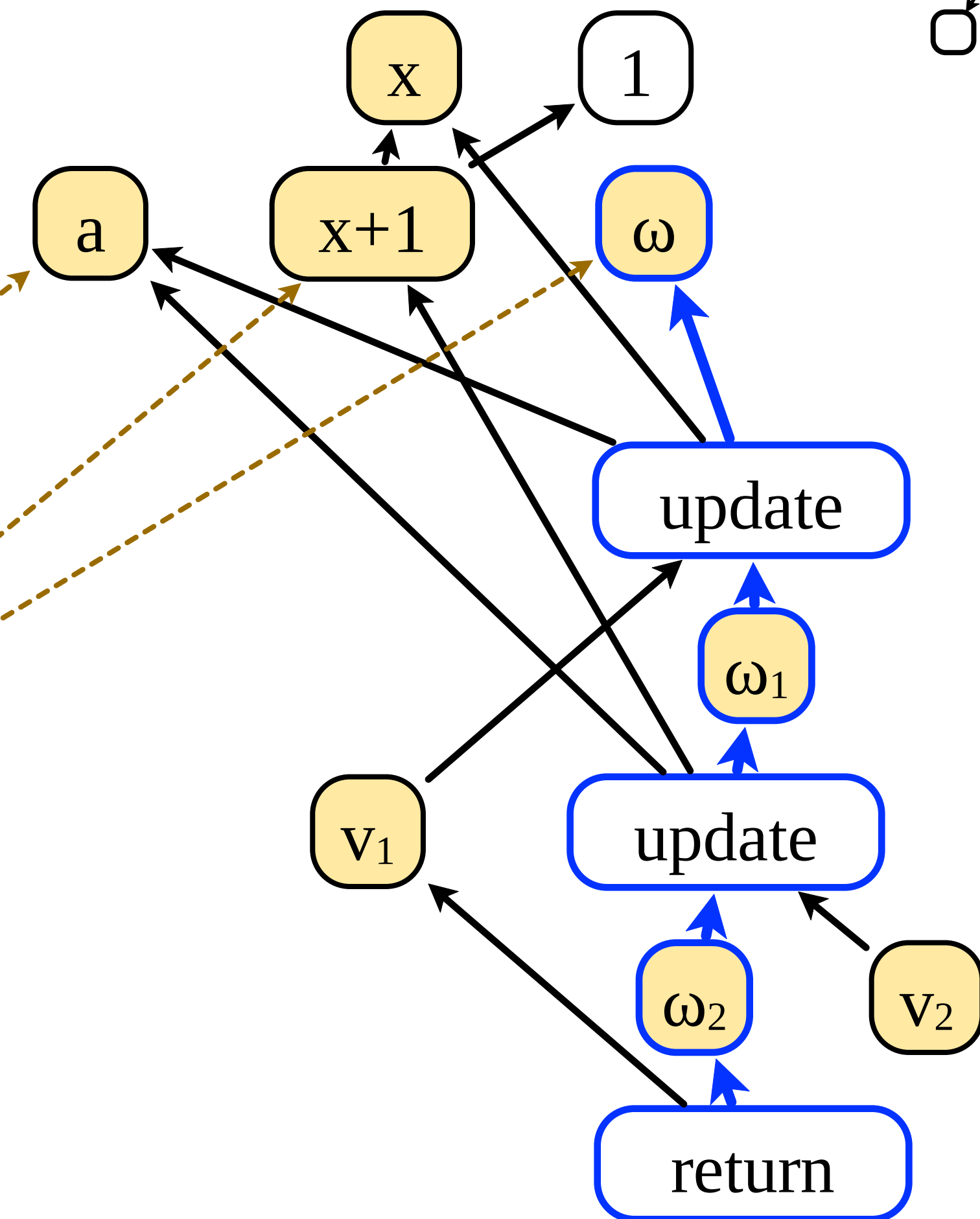
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Dead Store Elimination

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

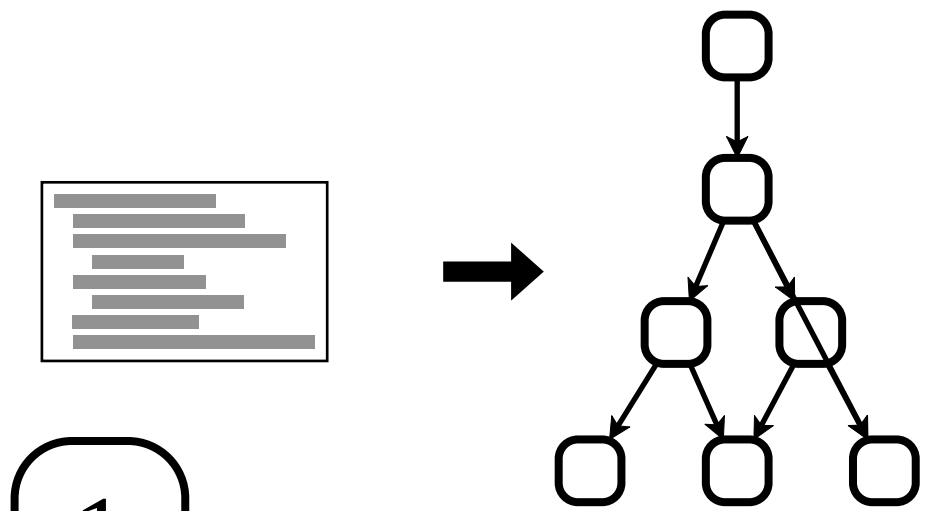
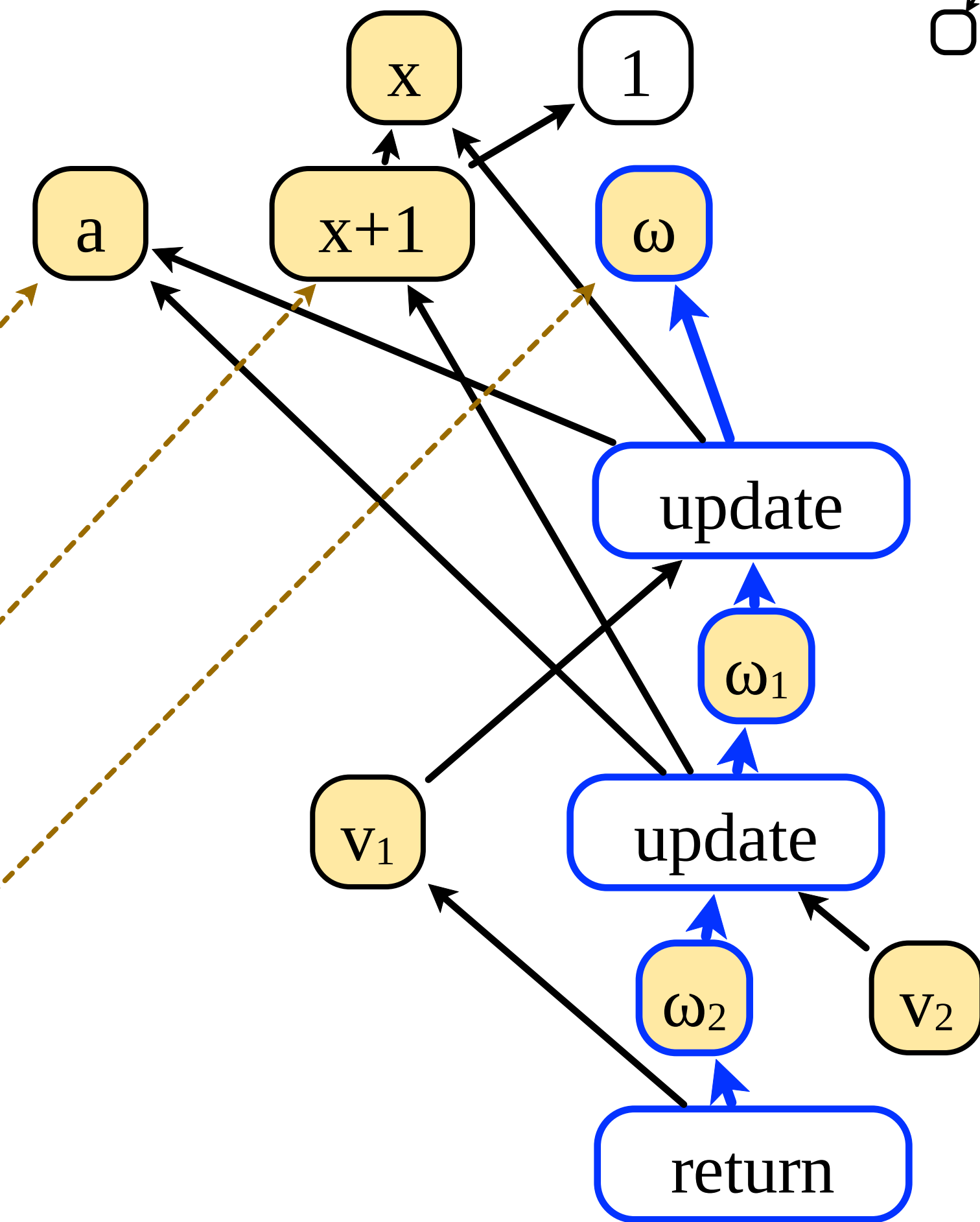
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
(v2, ω2) = update(a, x+1, ω1)
```

```
(v3, ω3) = update(a, x+1, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Dead Store Elimination

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

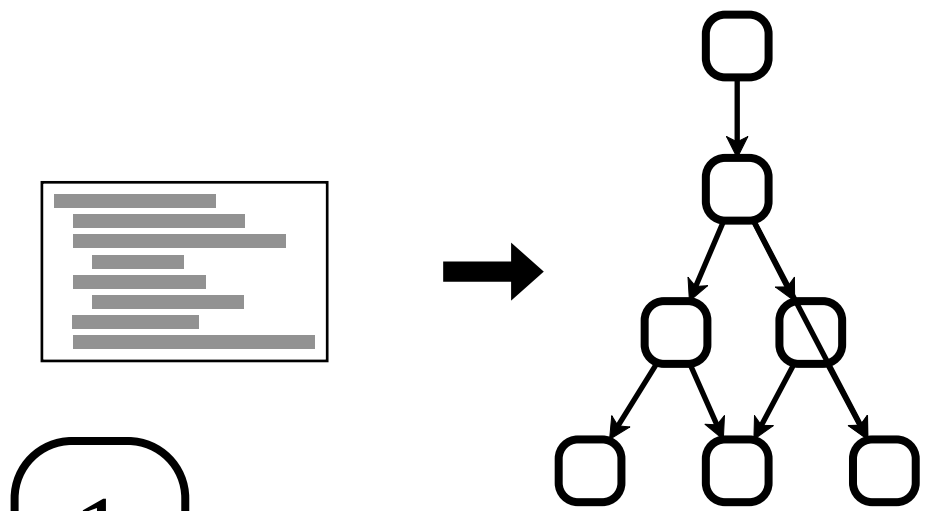
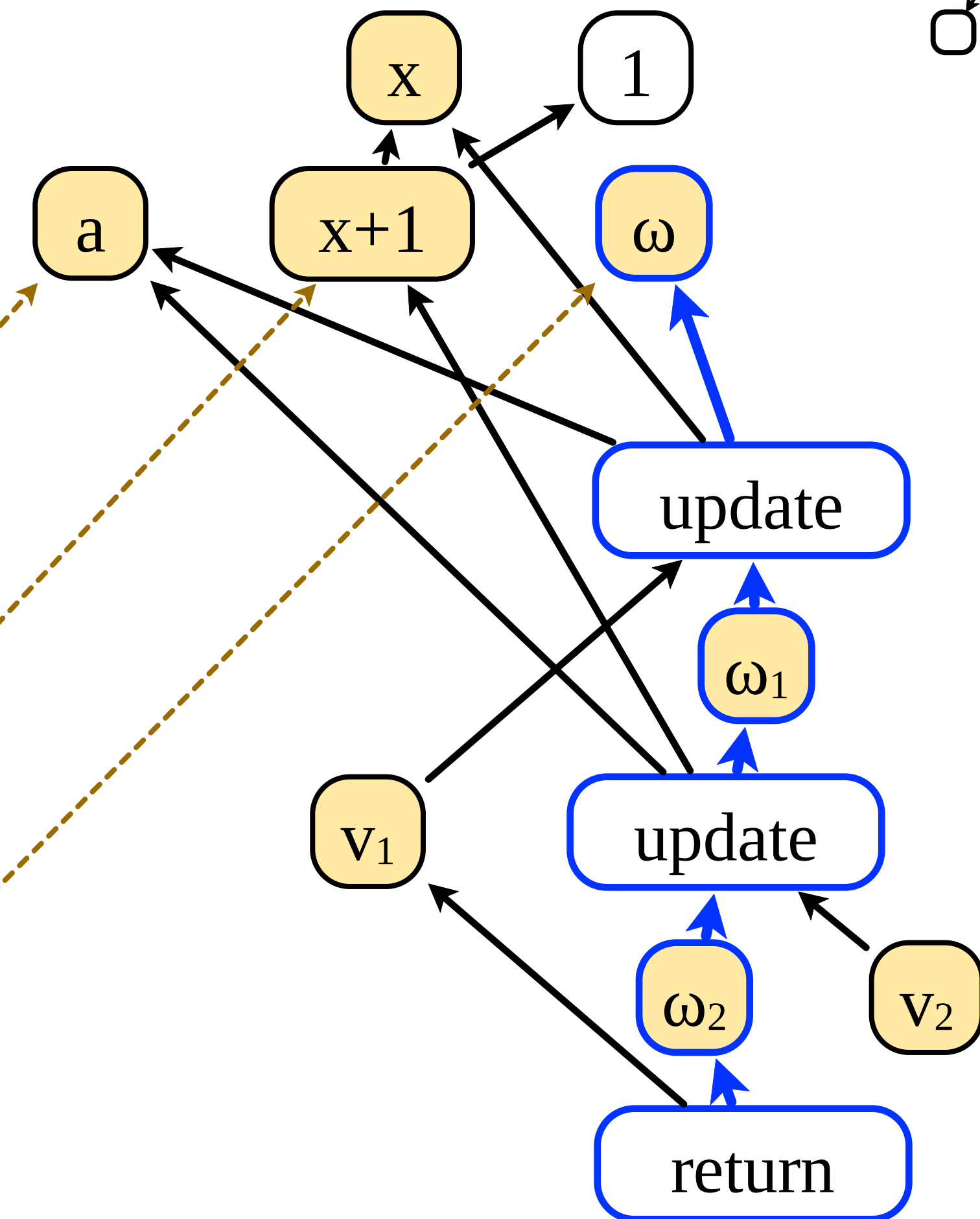
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
(v2, ω2) = update(a, x+1, ω1)
```

```
((v3, ω3)) = update(a, x+1, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Dead Store Elimination

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

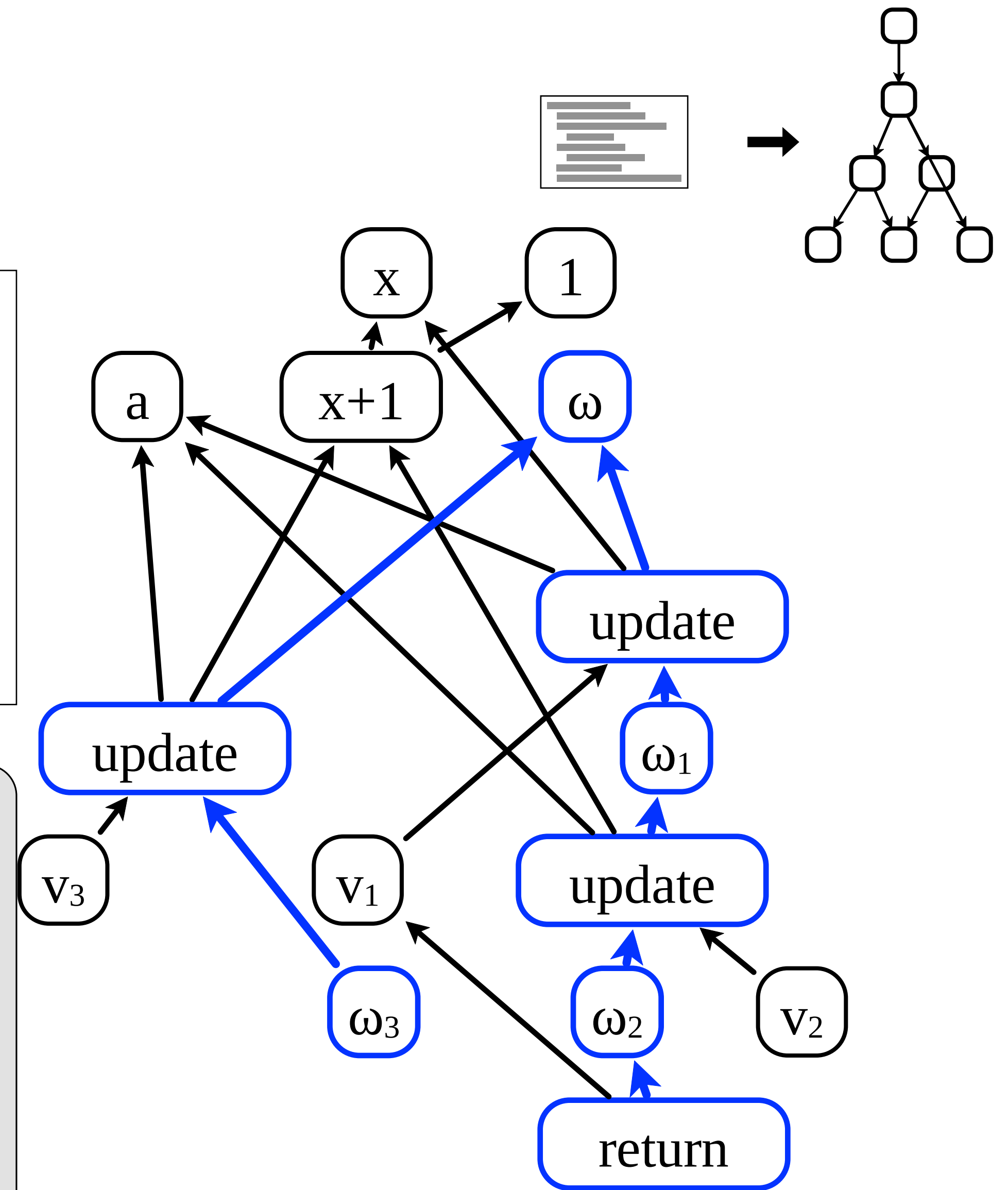
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Dead Store Elimination

```
fun foo(a, x, ω):
```

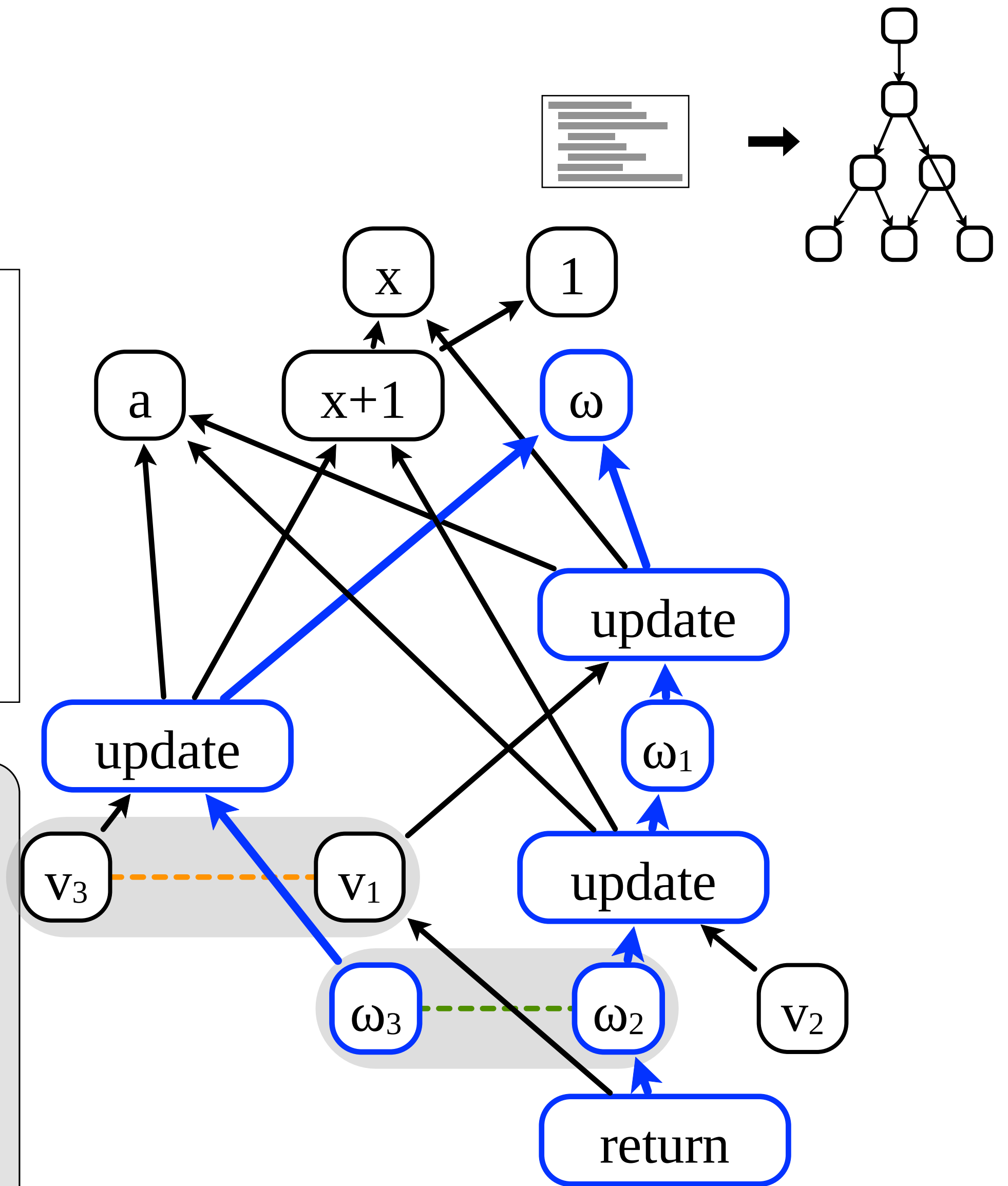
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

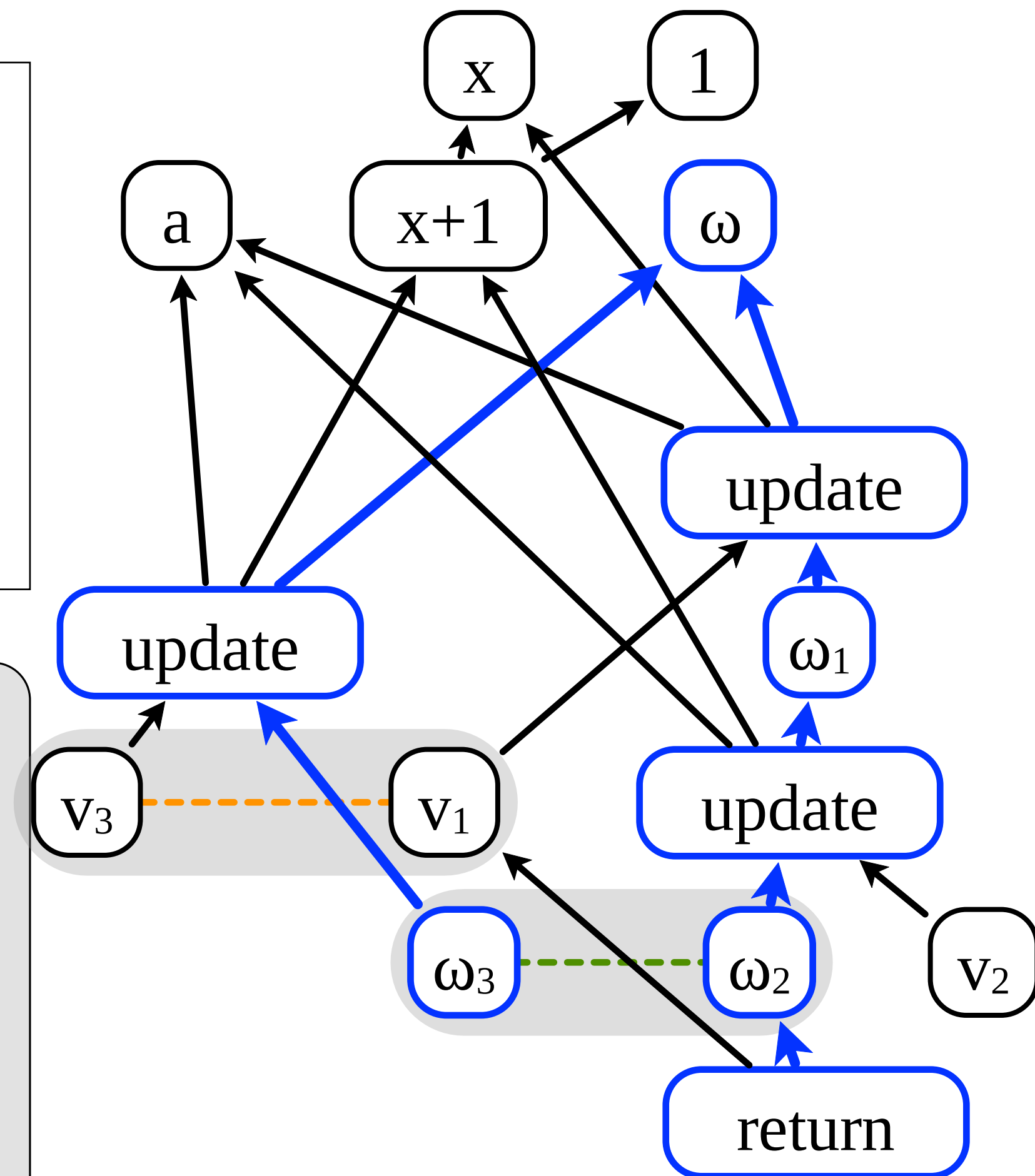
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

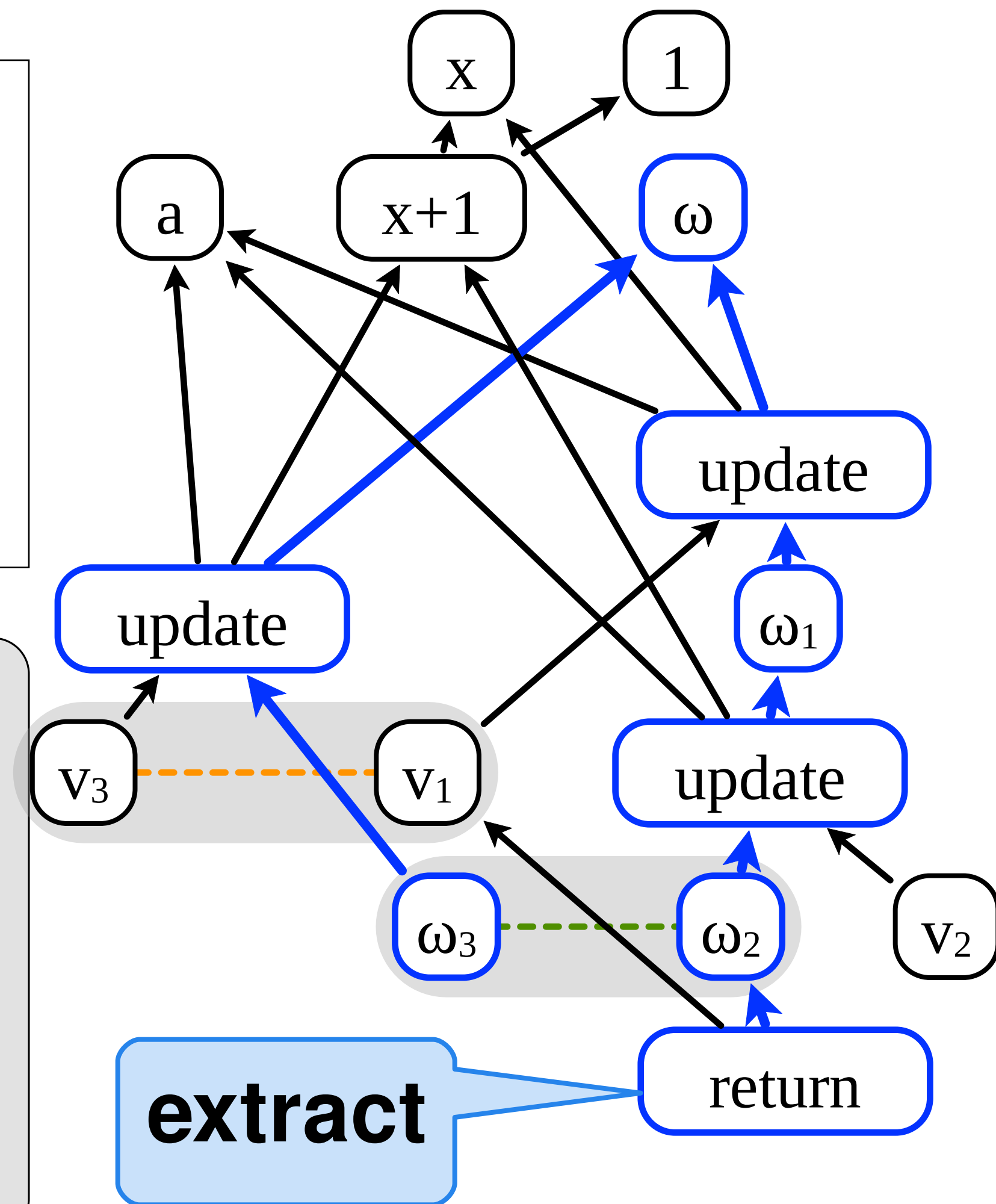
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

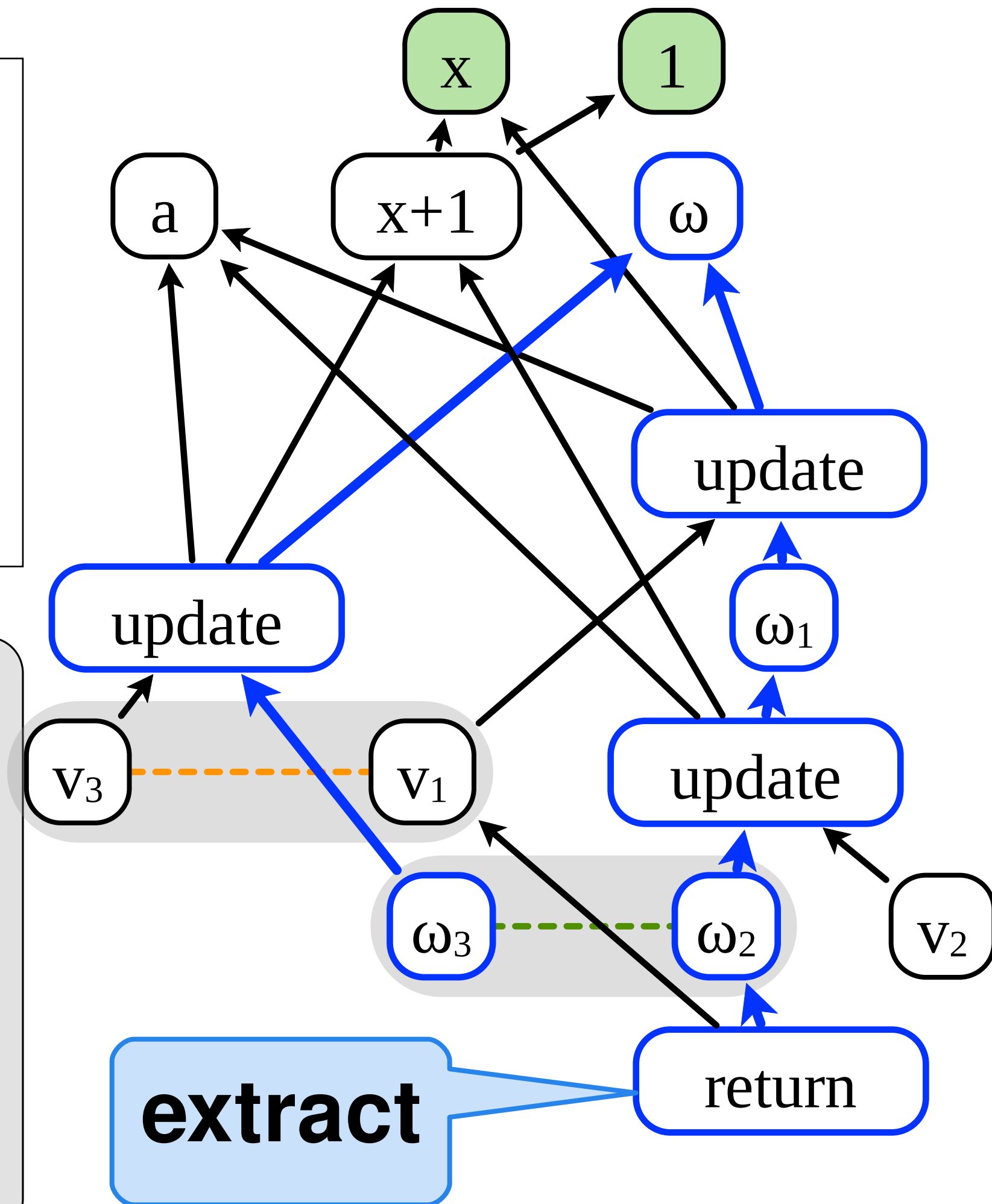
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

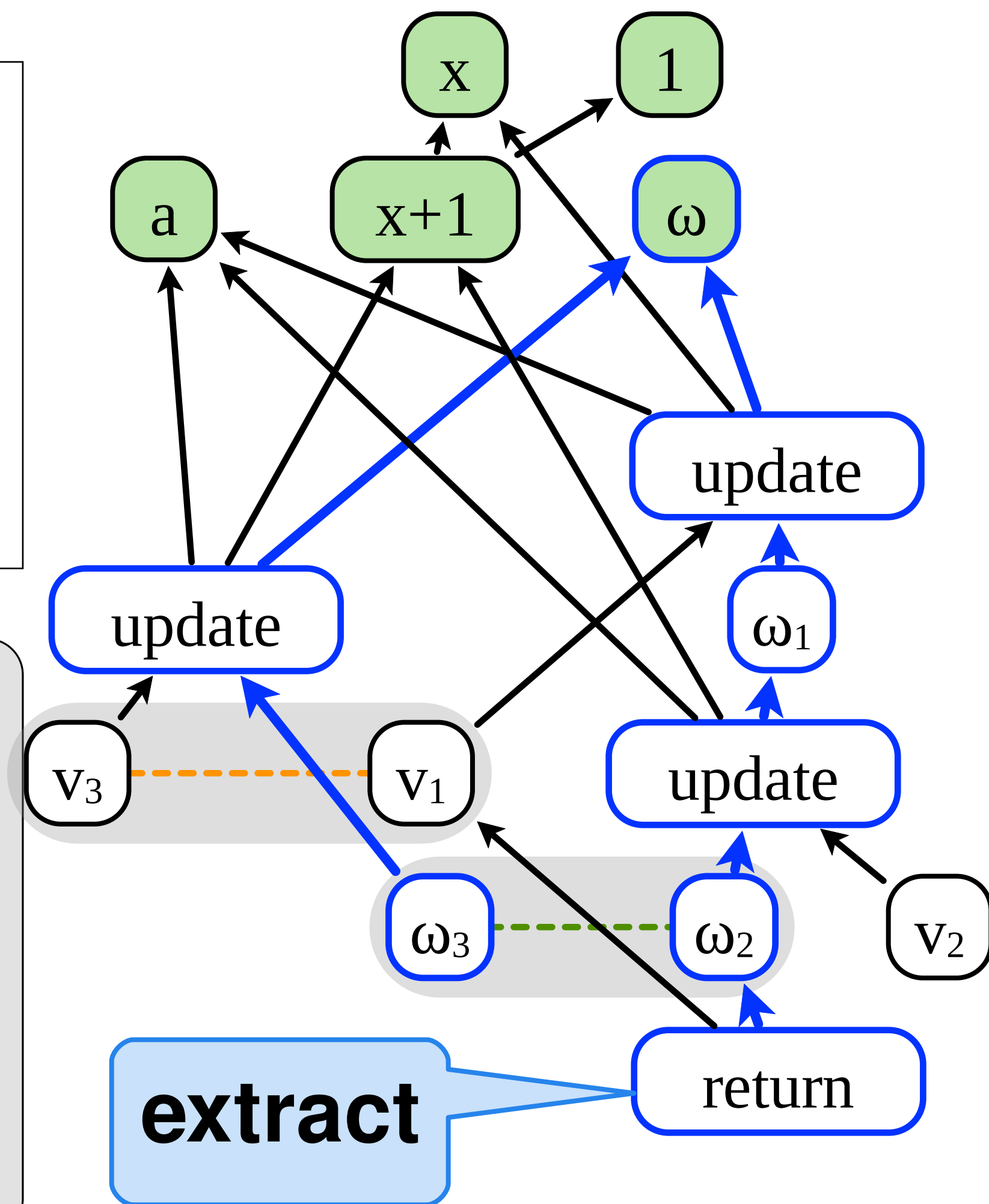
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

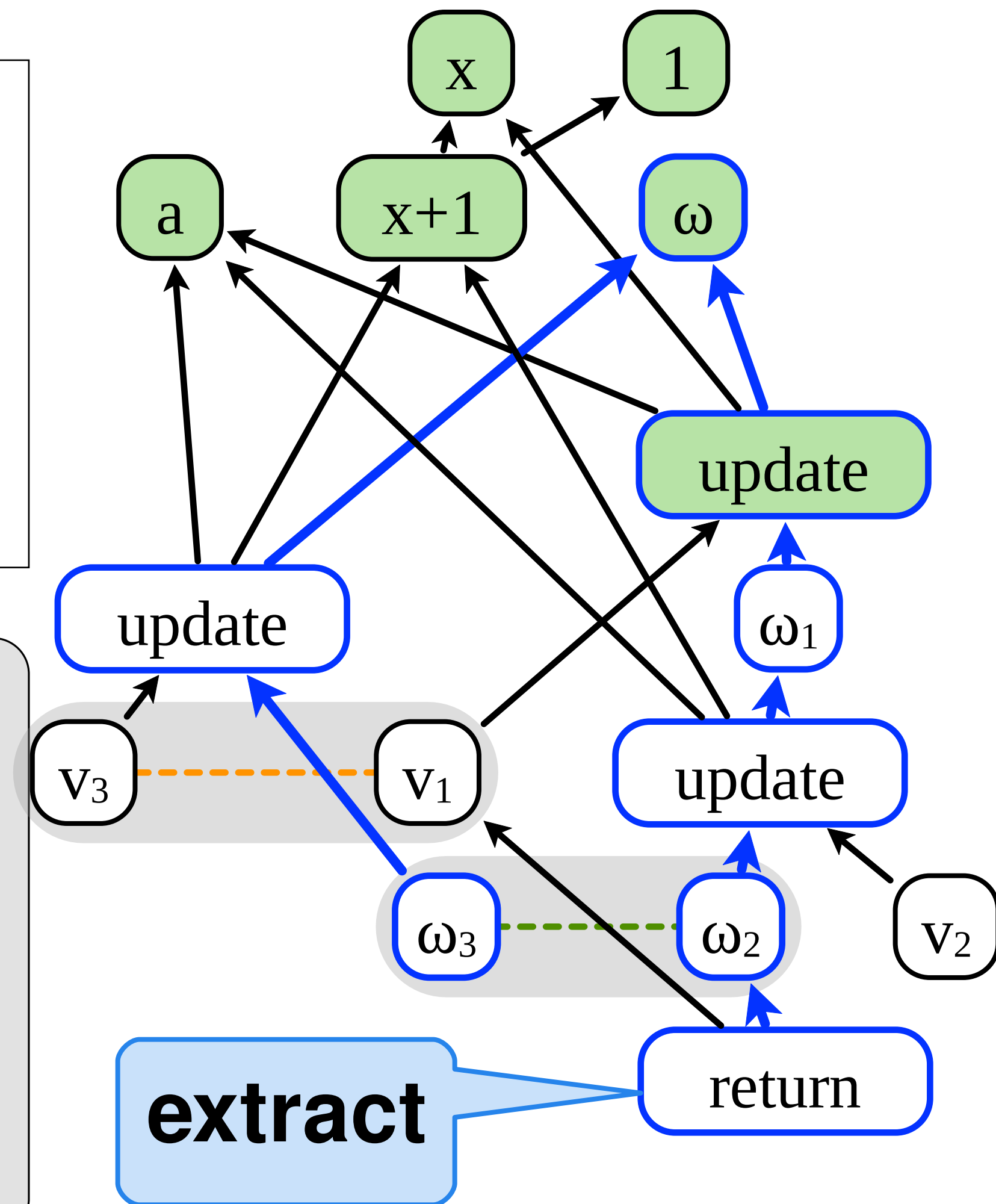
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

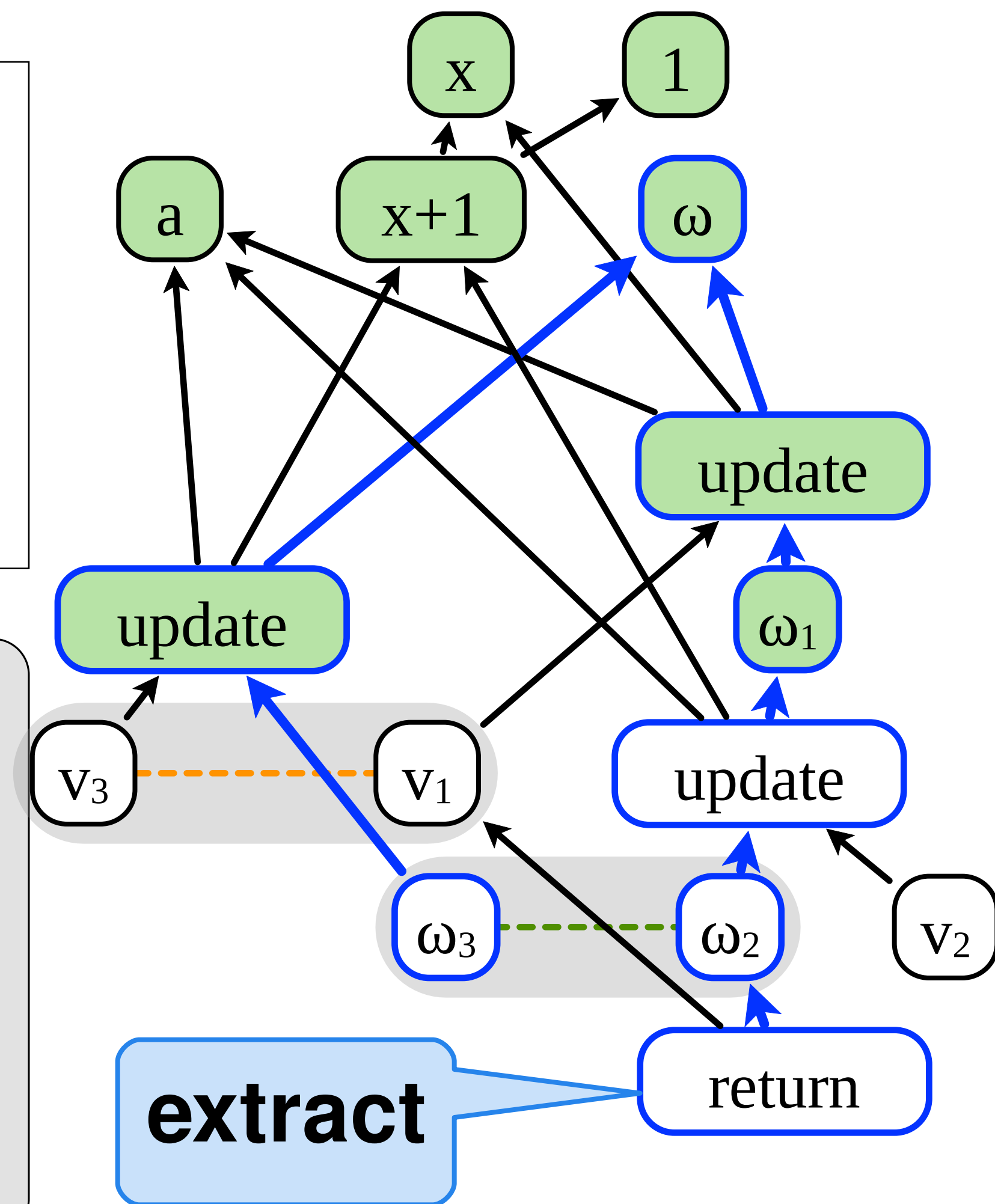
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

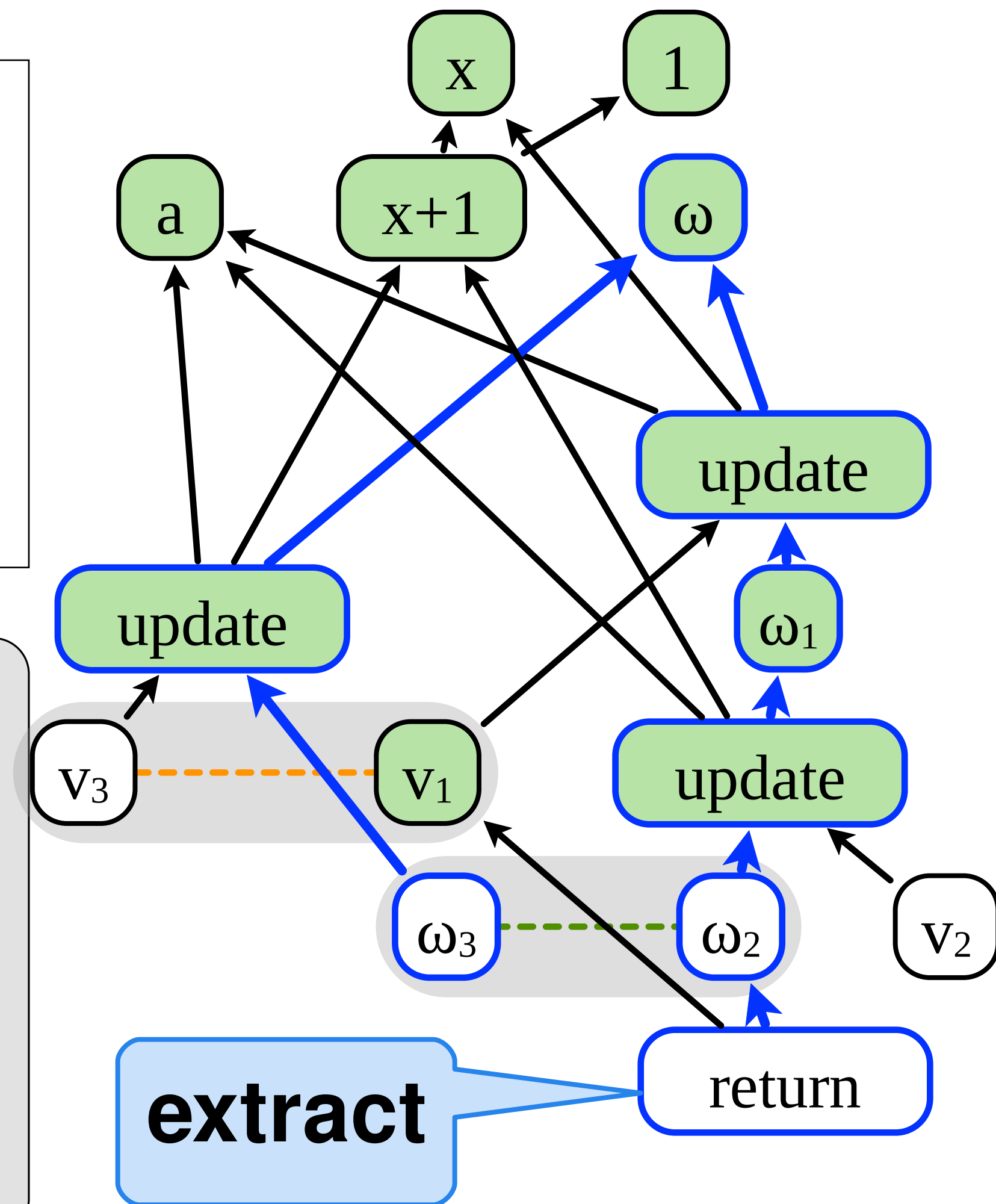
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

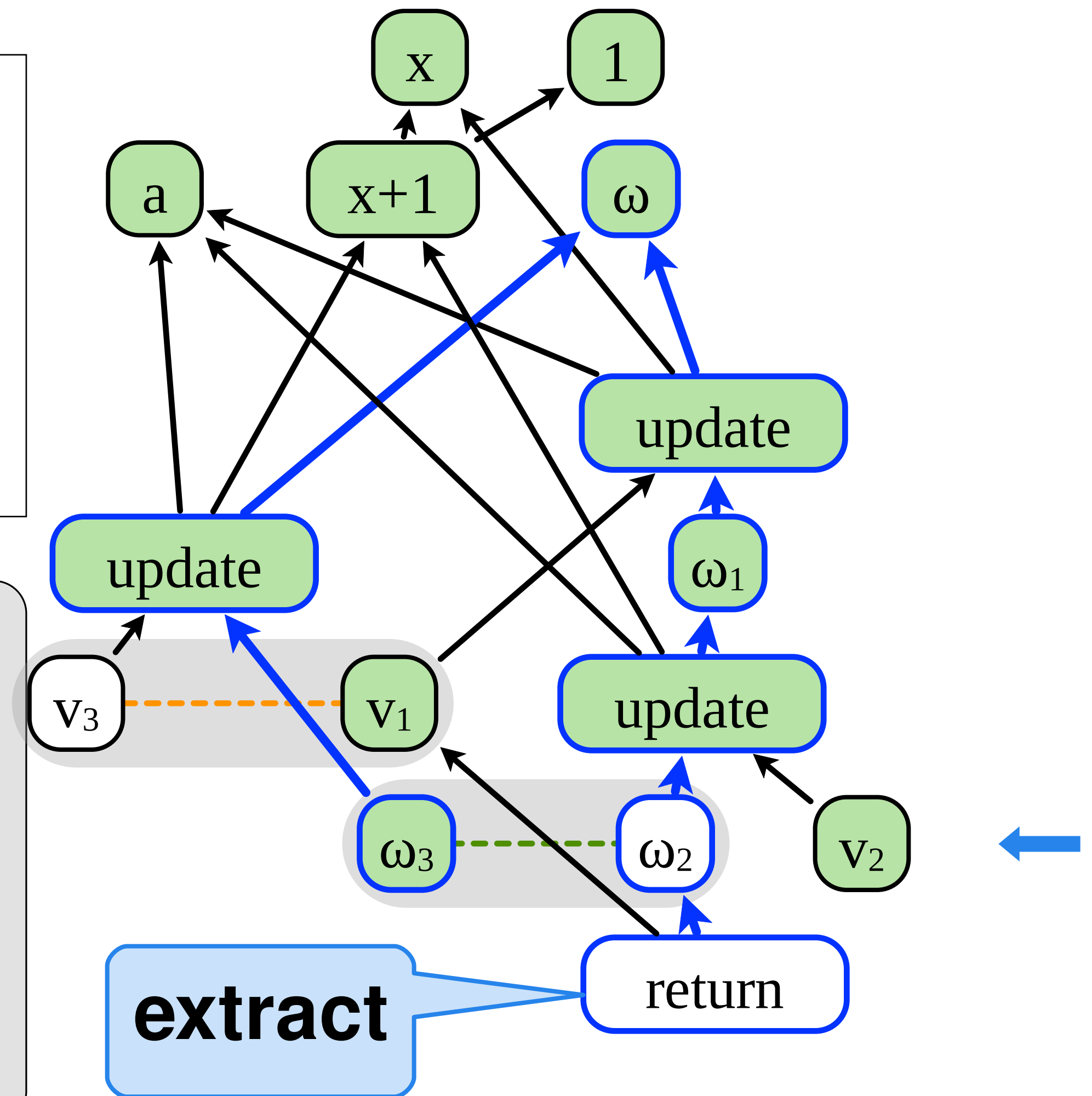
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

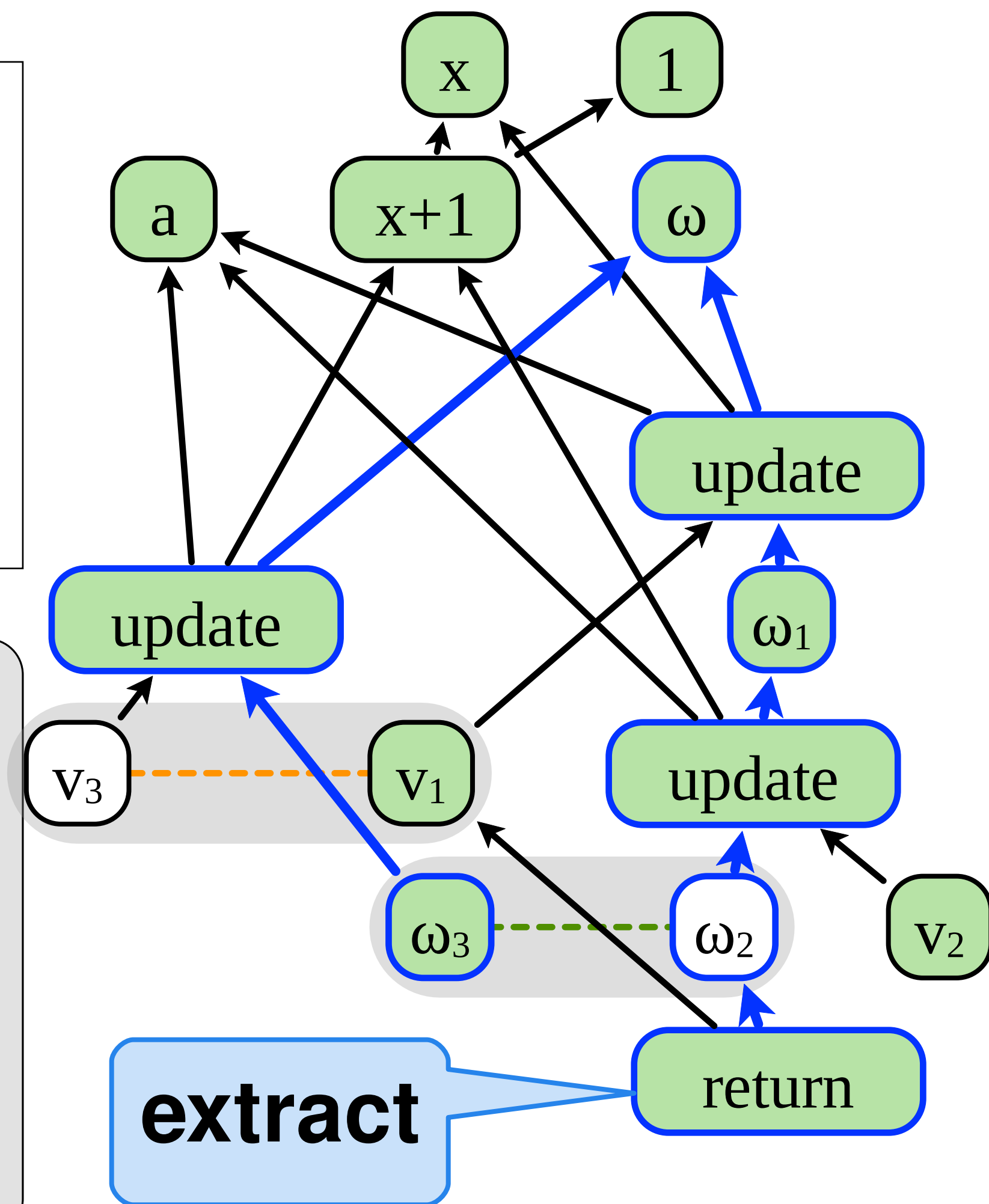
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

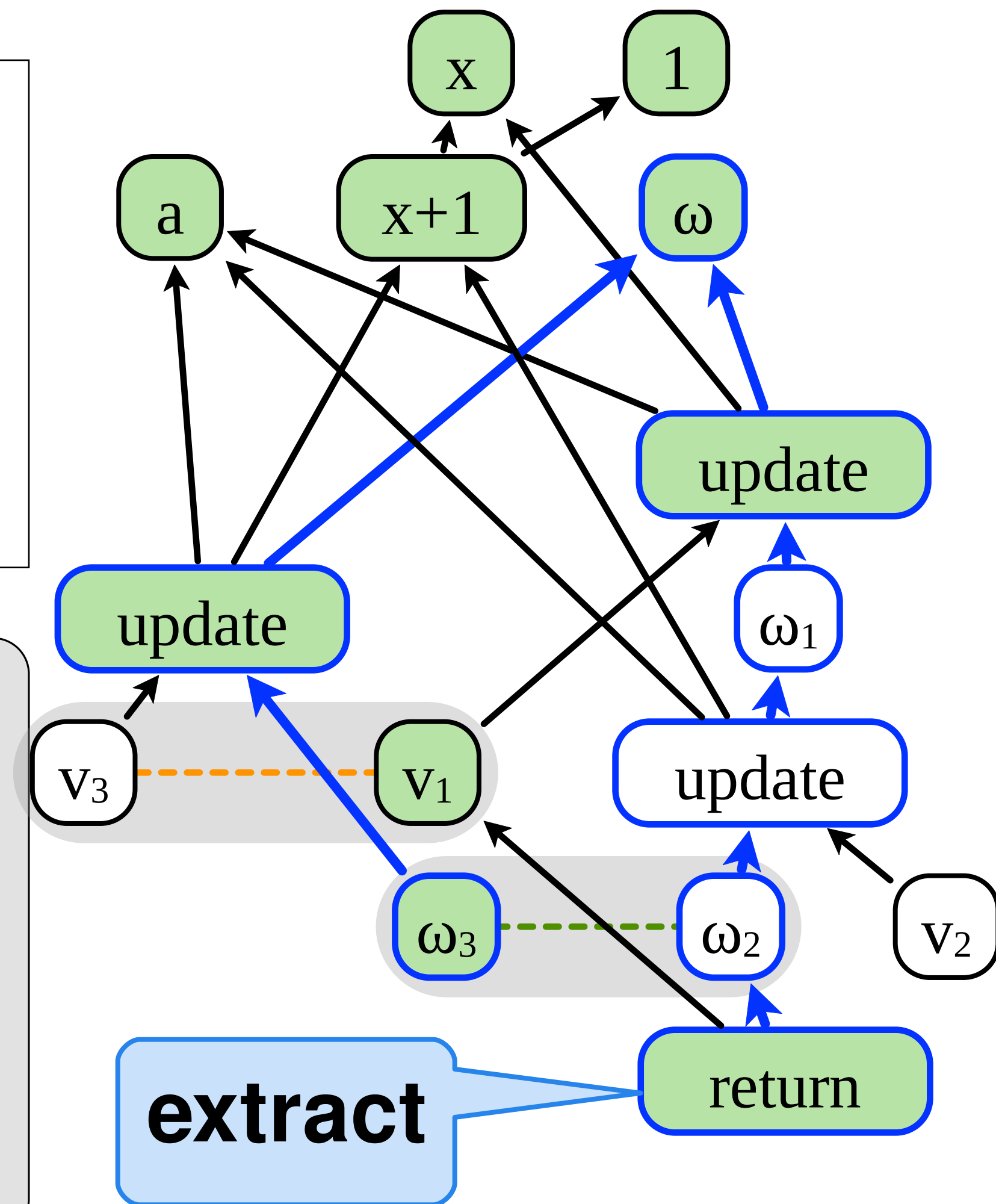
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

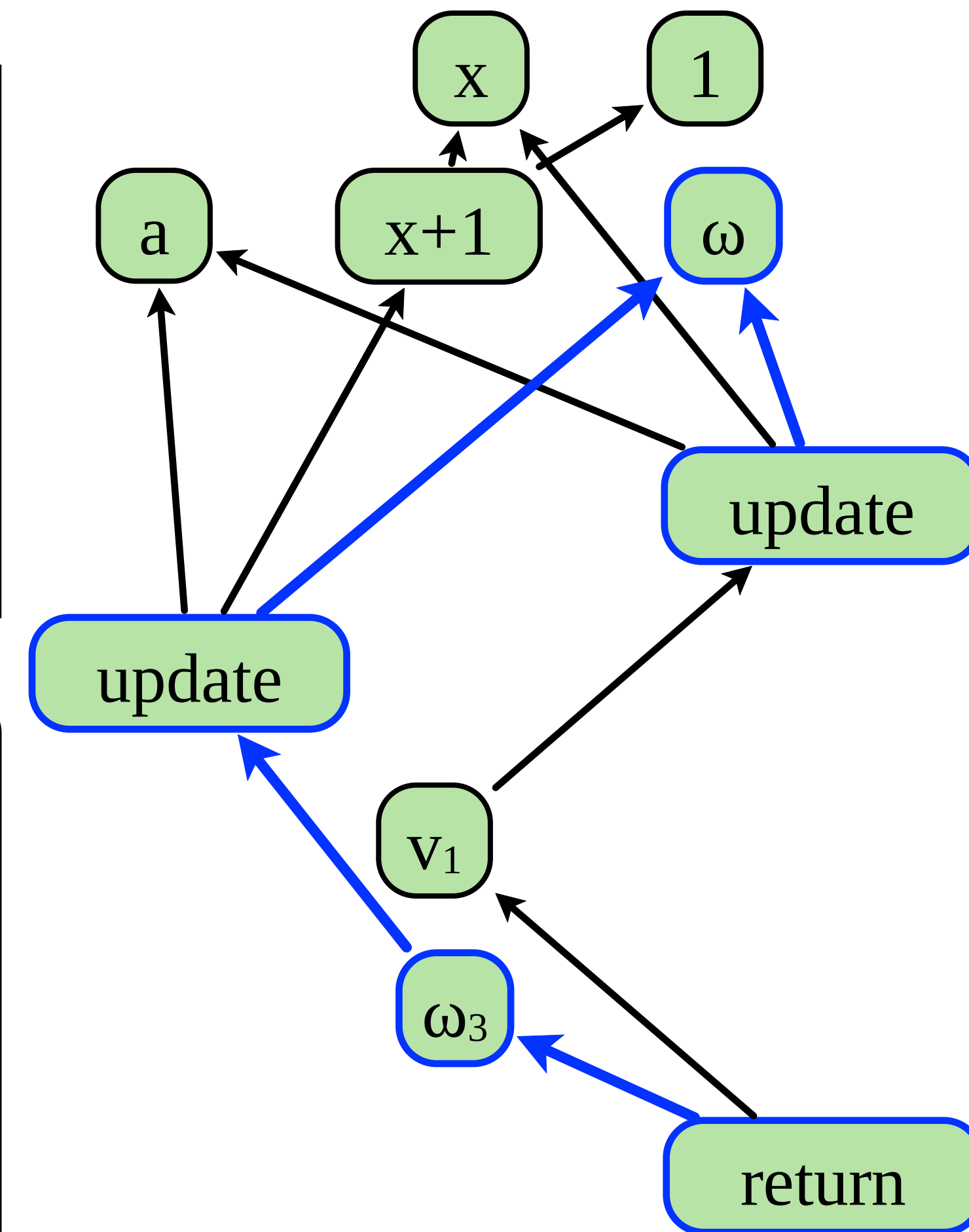
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

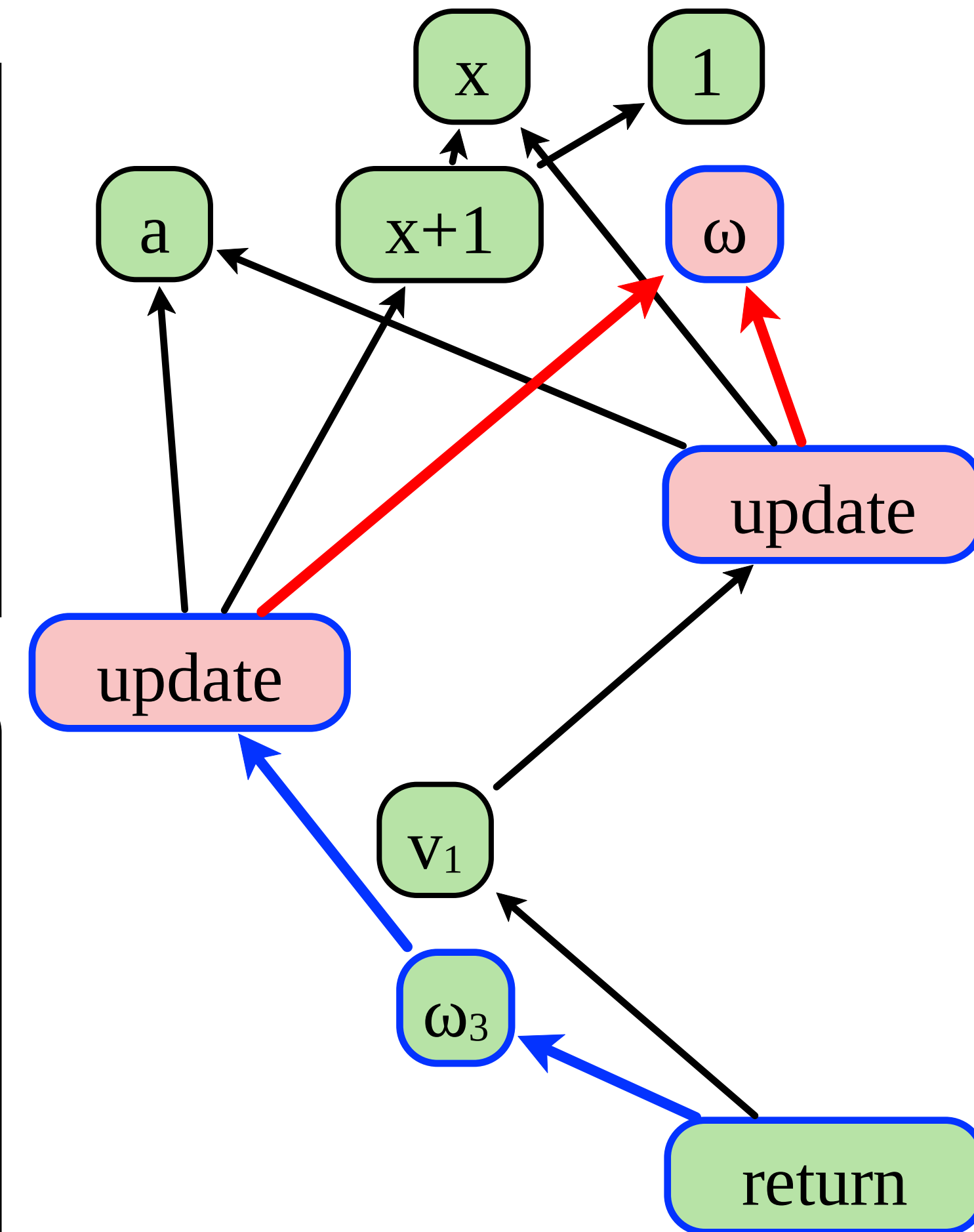
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

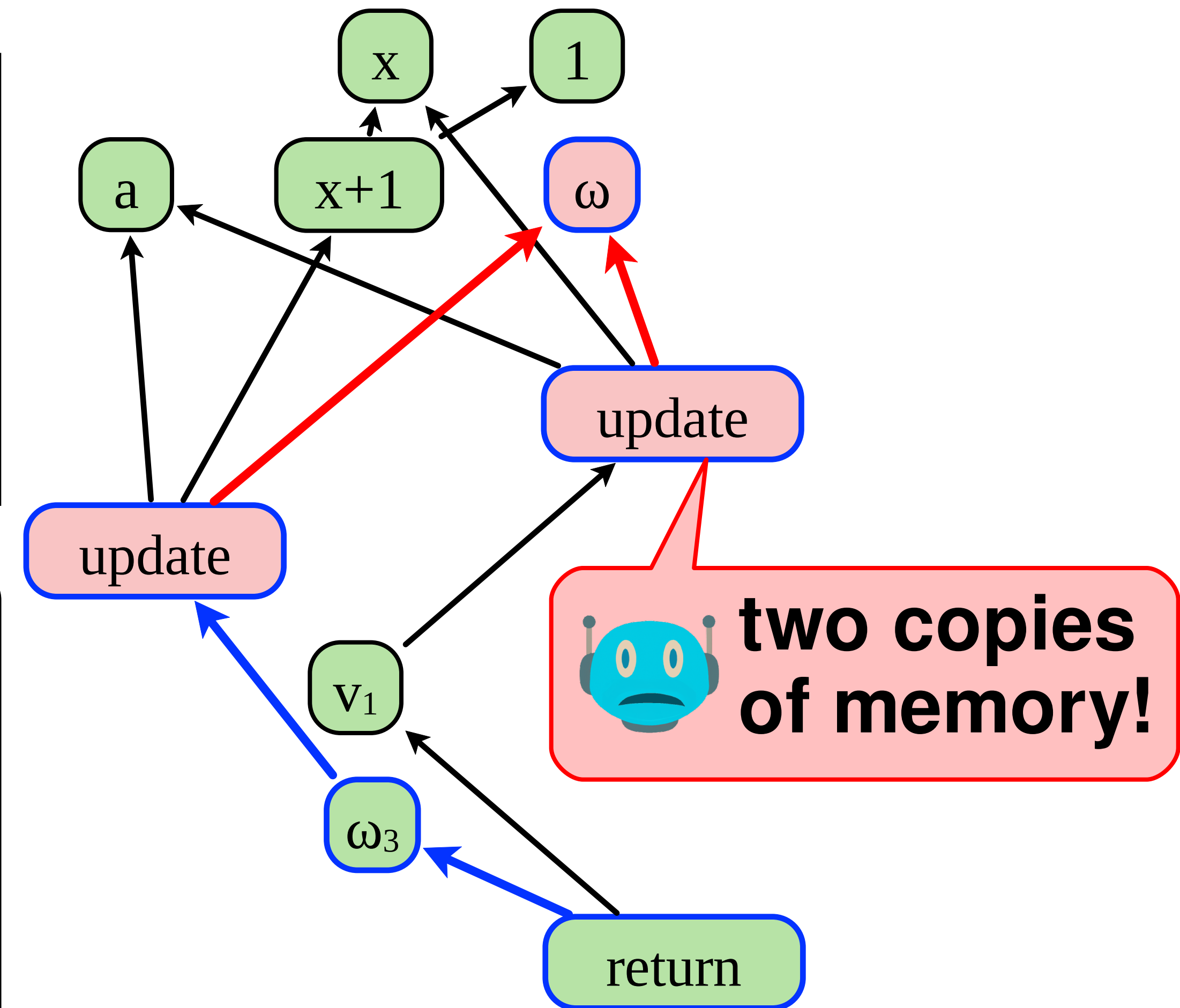
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
(v2, ω2) = update(a, x+1, ω1)
```

```
(v3, ω3) = update(a, x+1, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

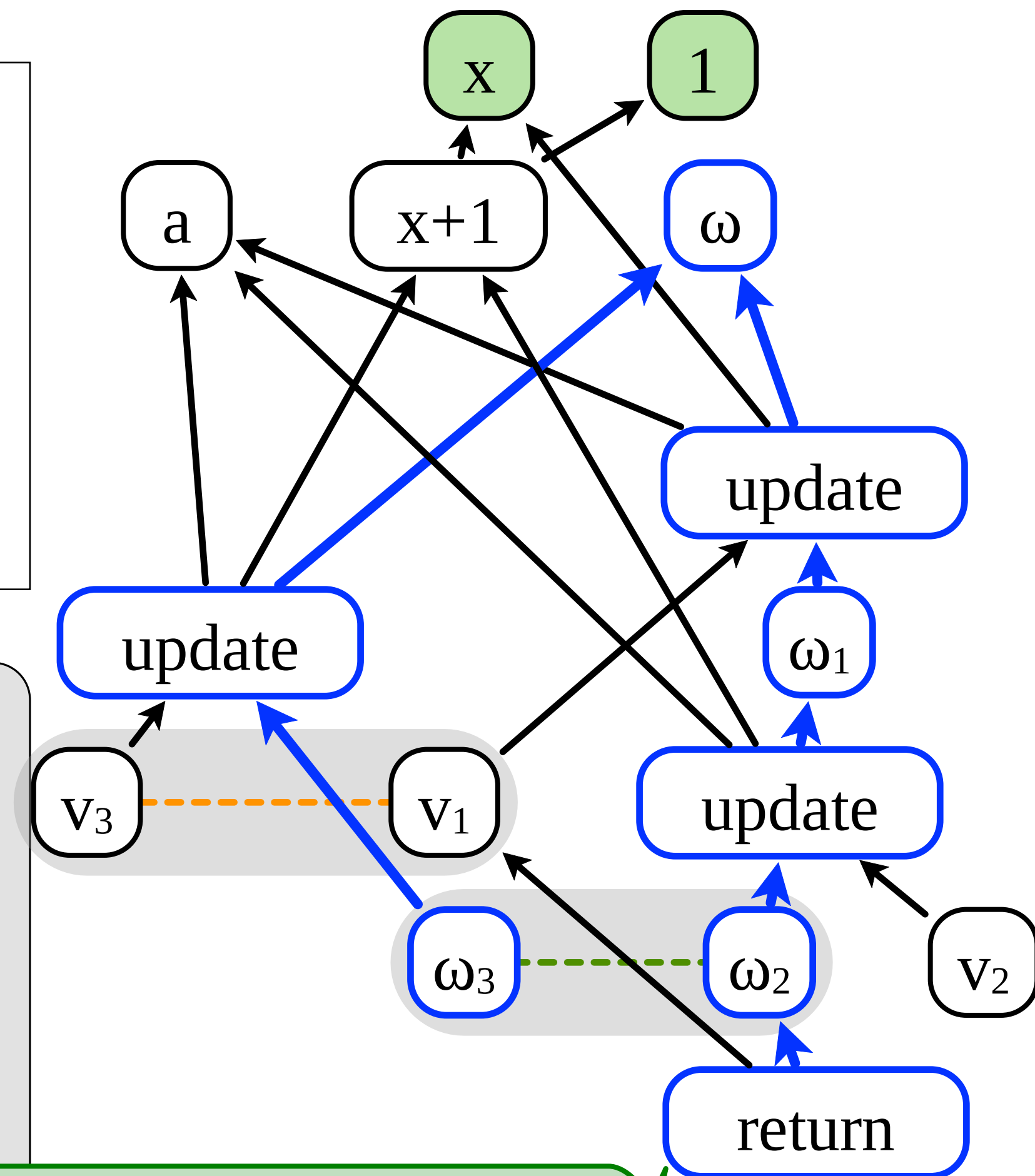
```
  (v2, ω2) = update(a, x+1, ω1)
```

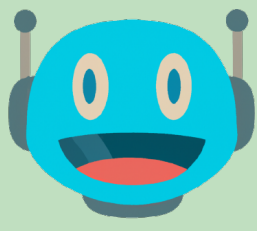
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

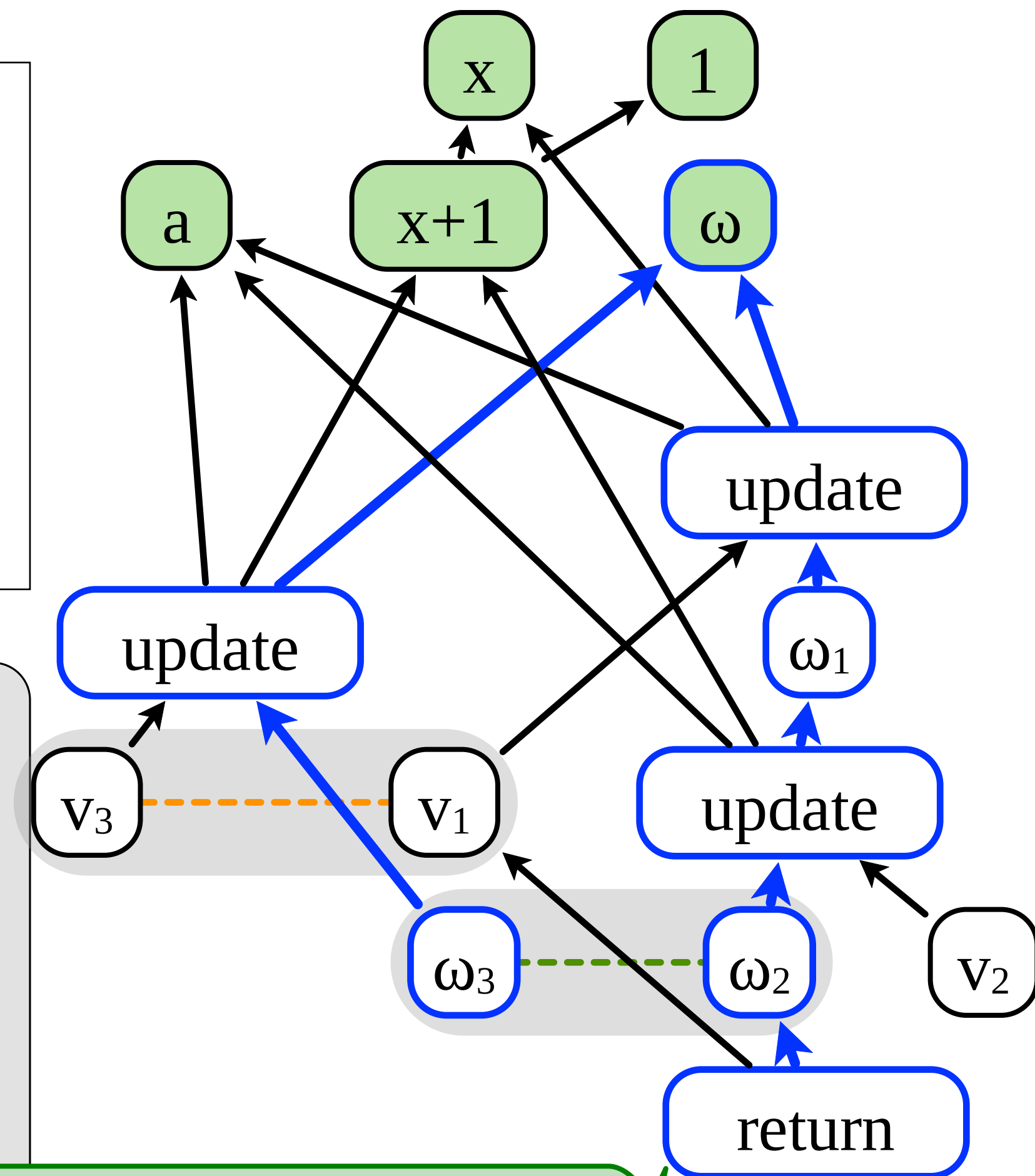
```
  (v2, ω2) = update(a, x+1, ω1)
```

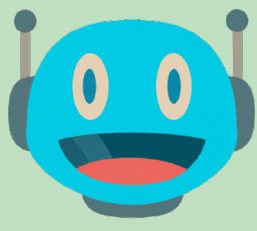
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

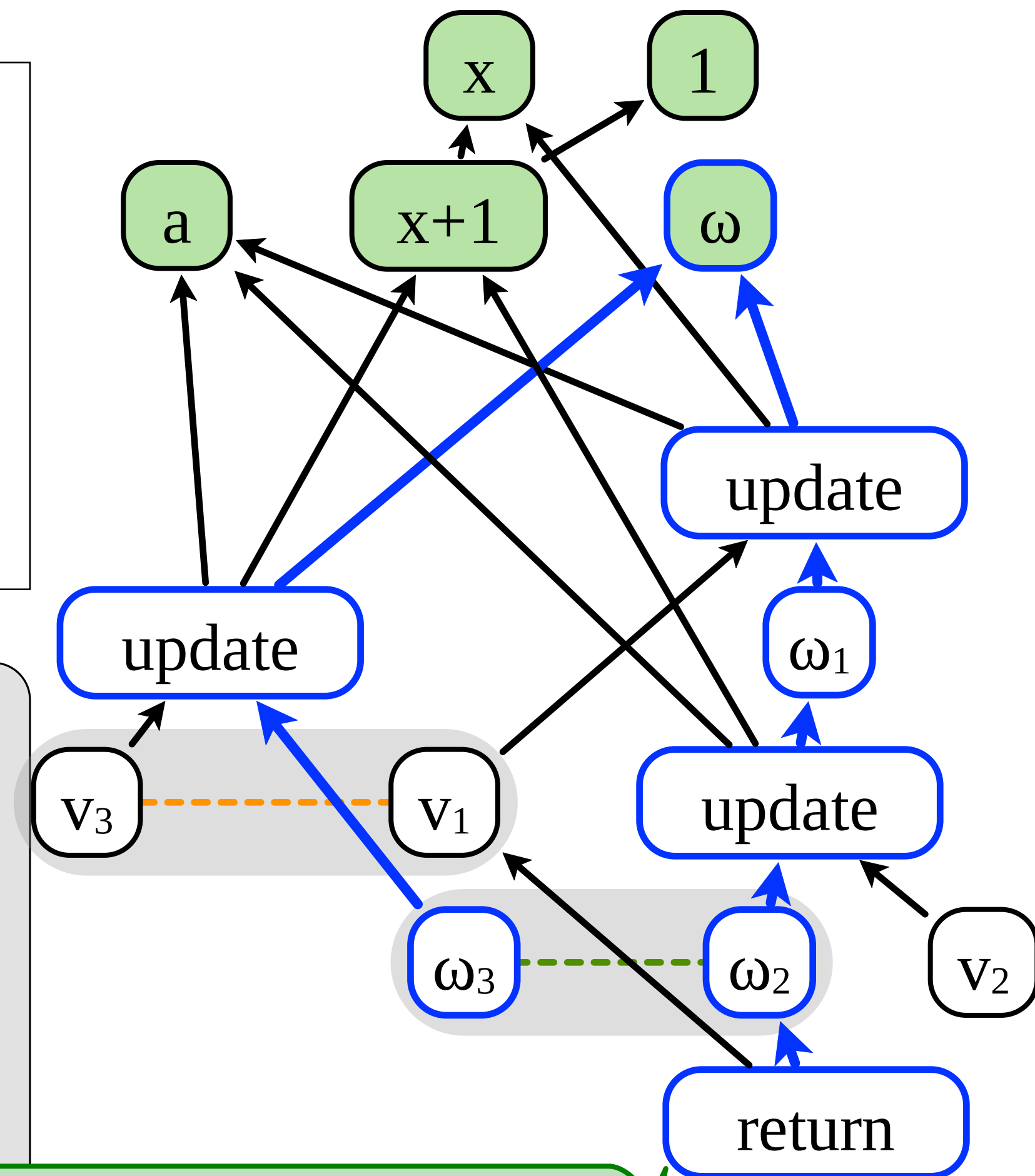
```
  (v2, ω2) = update(a, x+1, ω1)
```

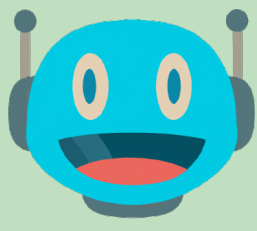
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

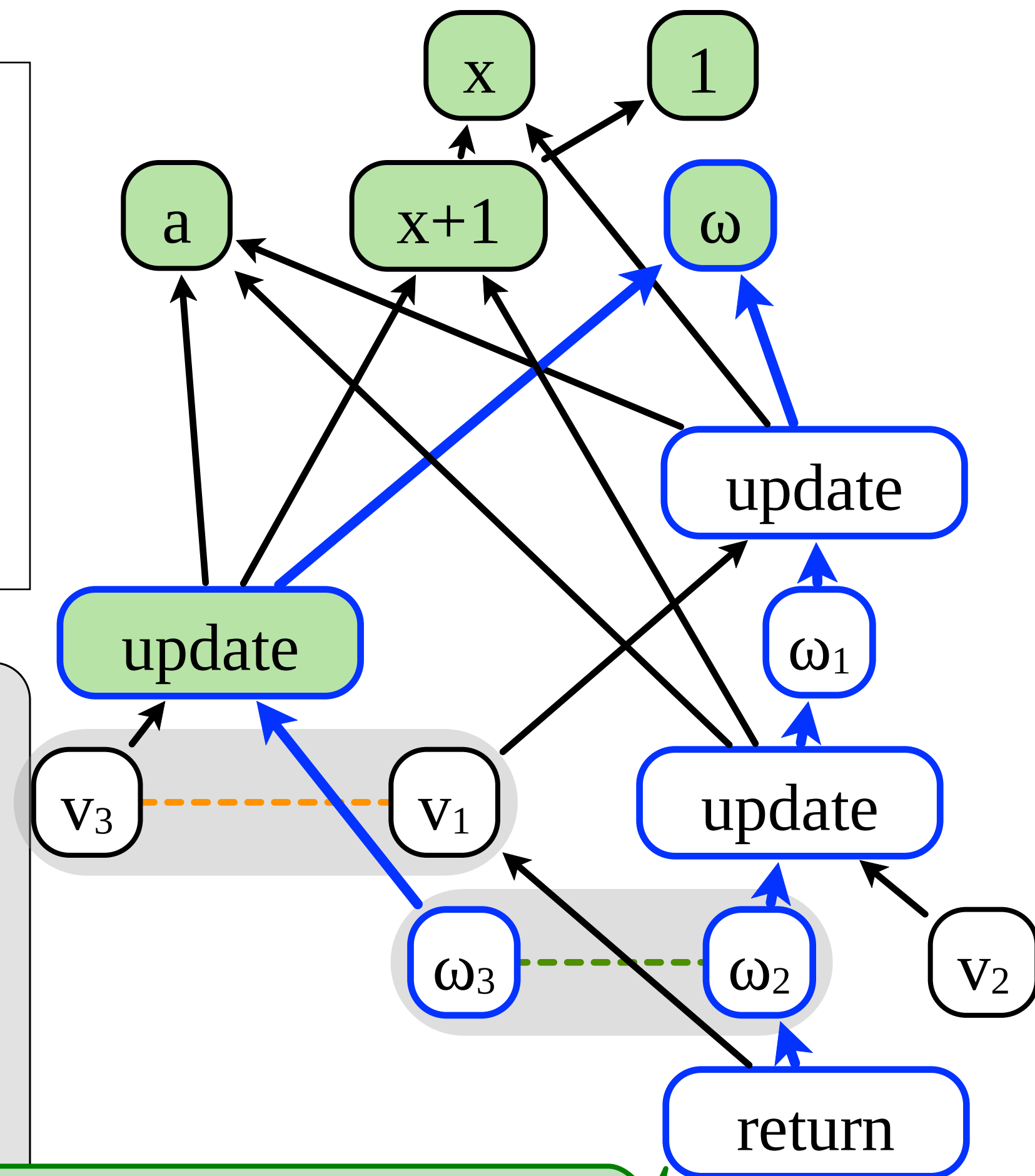
```
  (v2, ω2) = update(a, x+1, ω1)
```

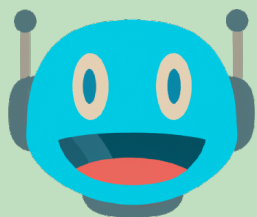
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

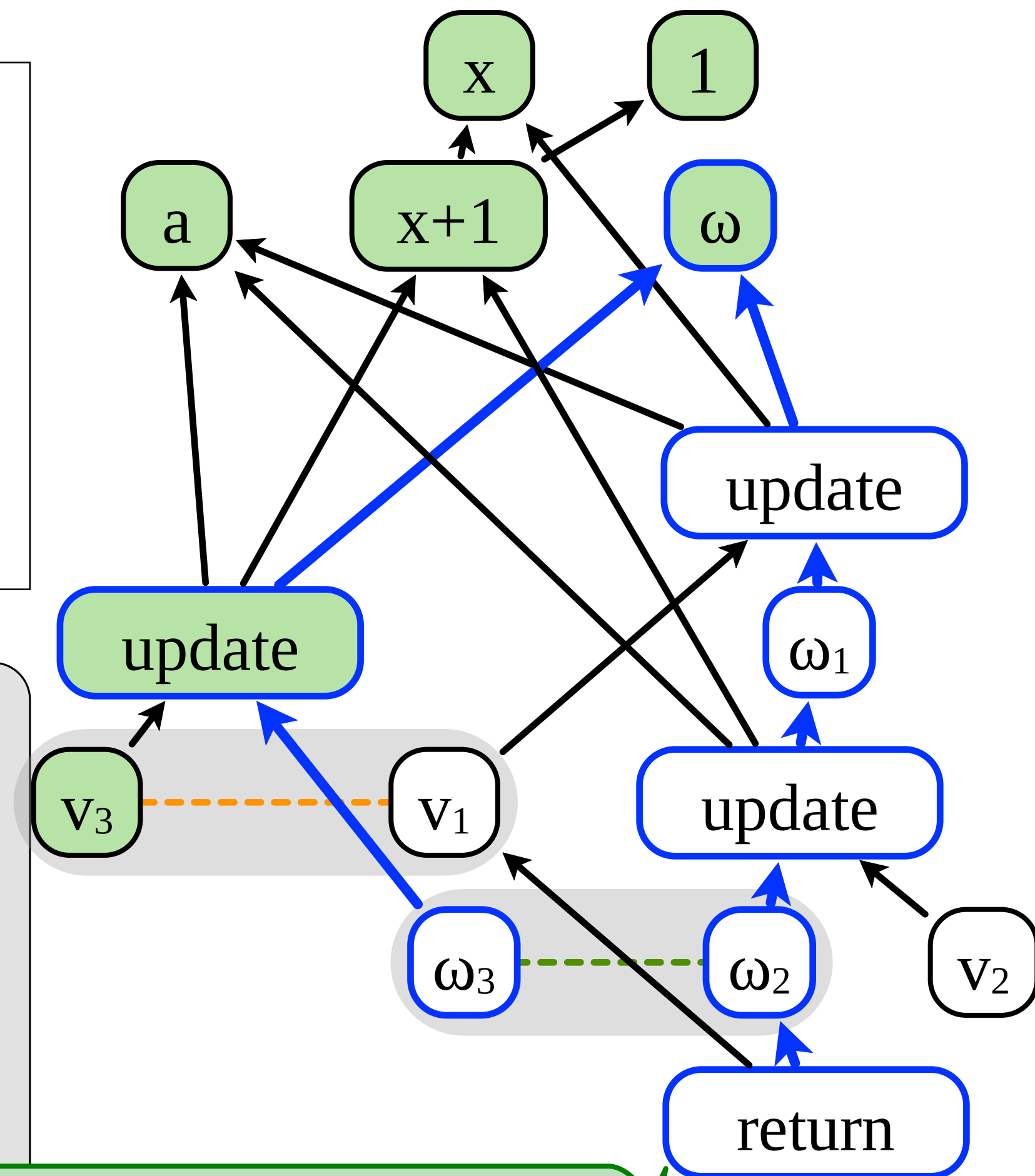
```
  (v2, ω2) = update(a, x+1, ω1)
```

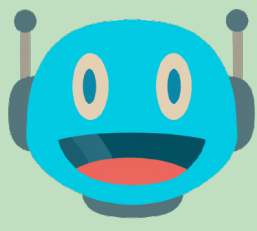
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

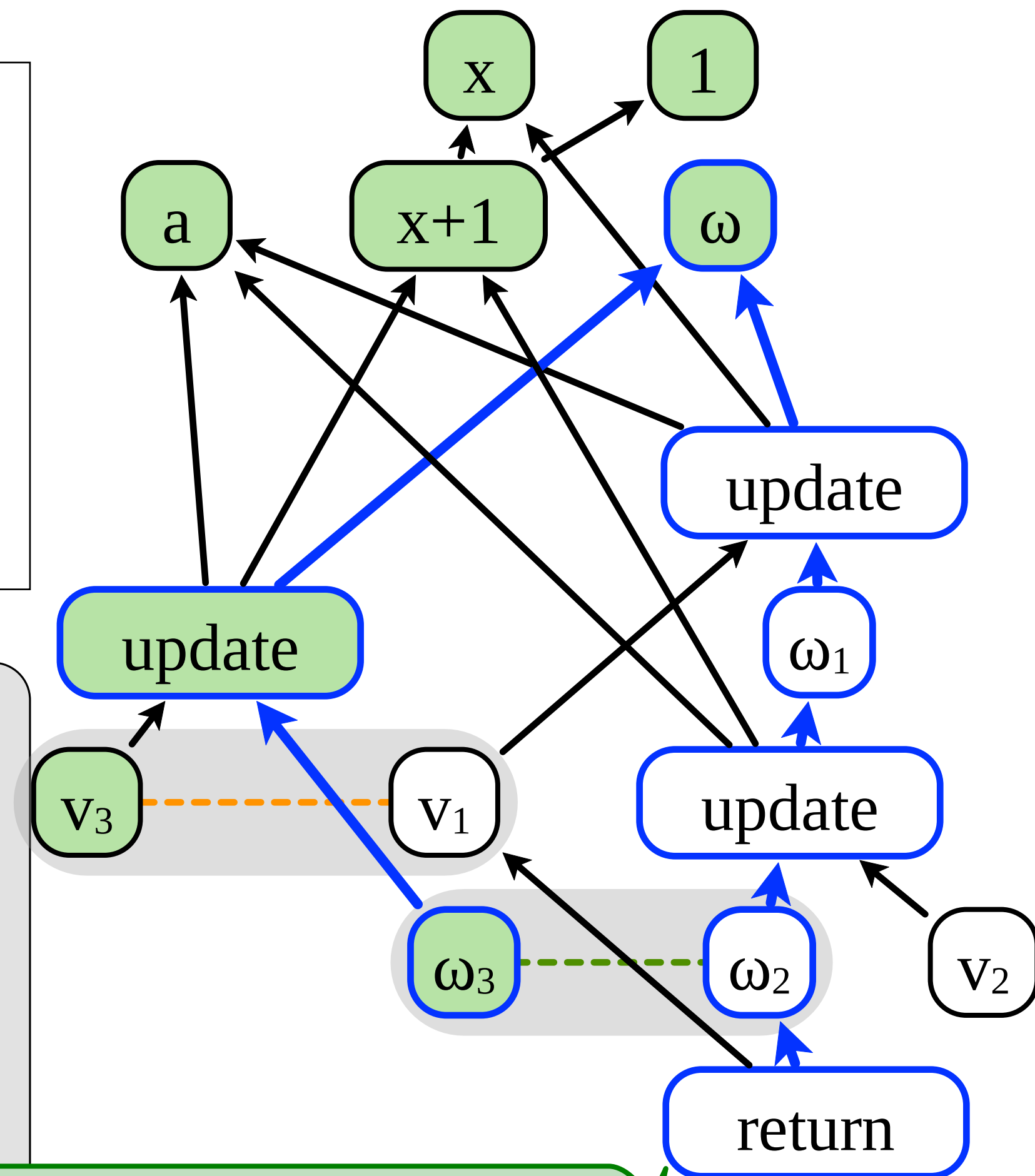
```
  return (v1, ω2)
```

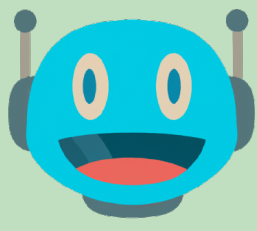
$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

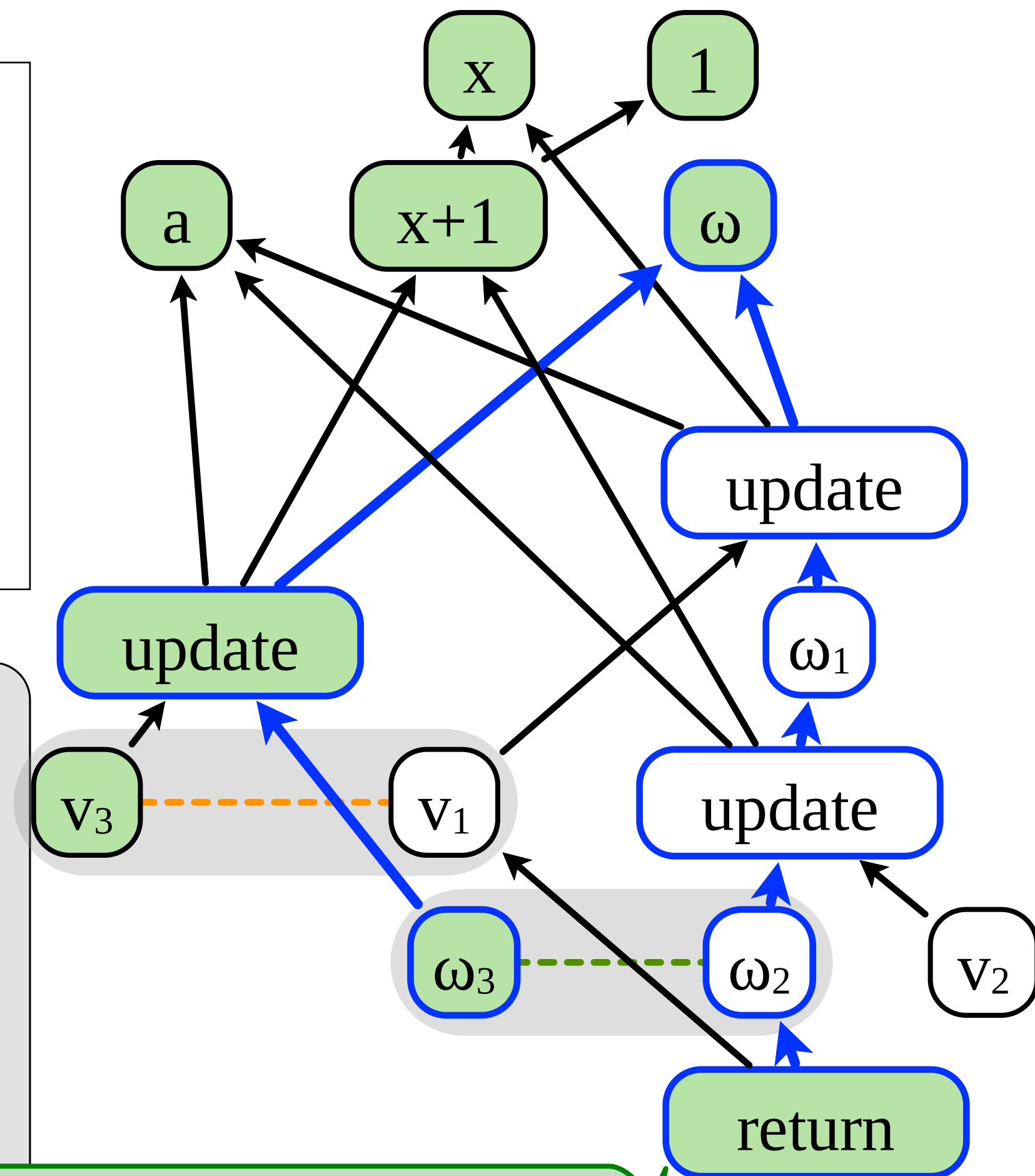
```
  (v2, ω2) = update(a, x+1, ω1)
```

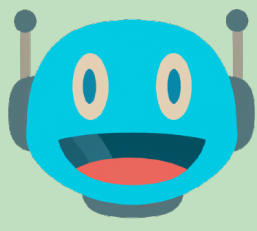
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

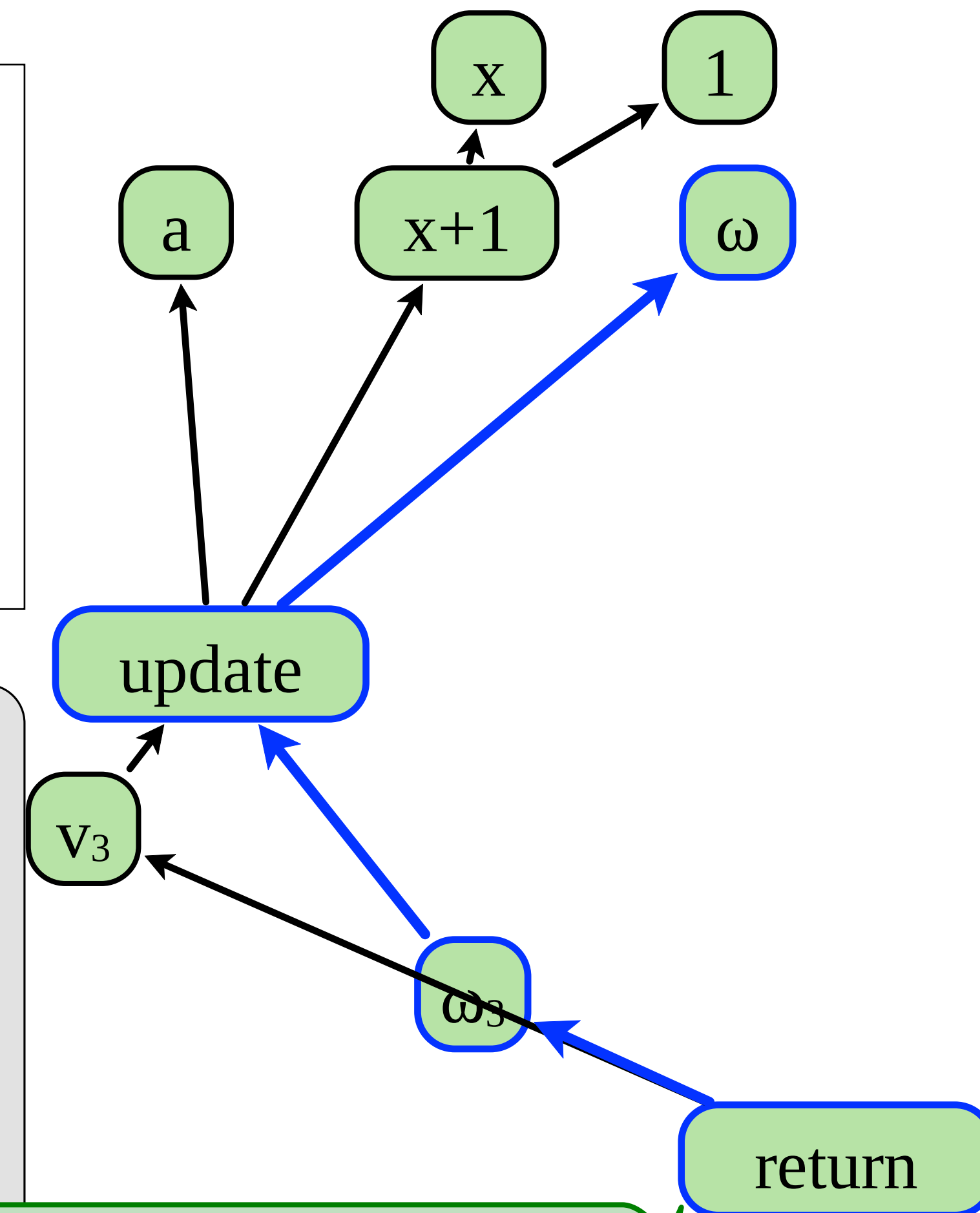
```
  return (v1, ω2)
```

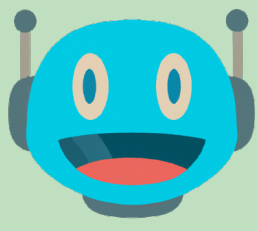
$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

Extraction Goes Wrong

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

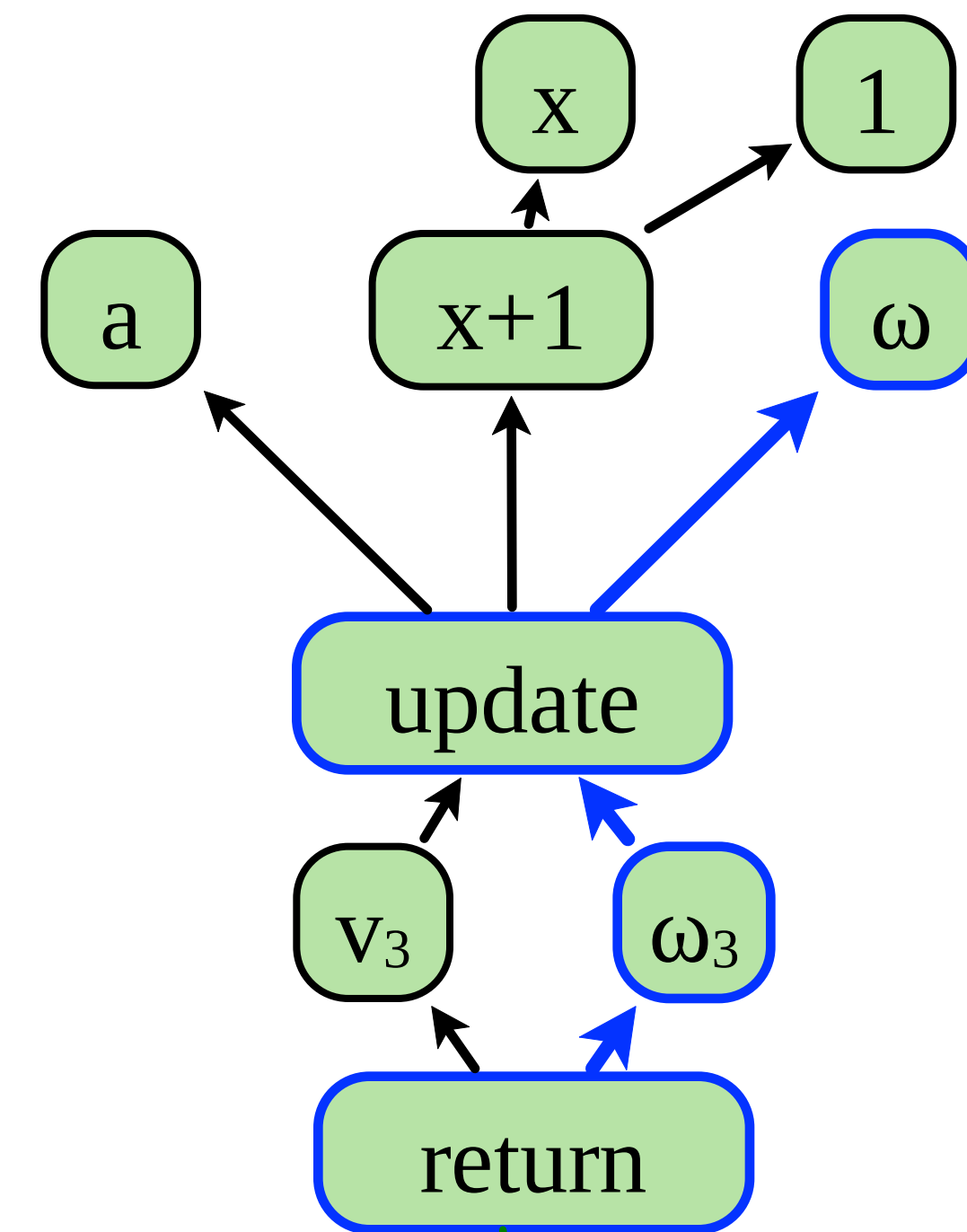
```
  return (v1, ω2)
```

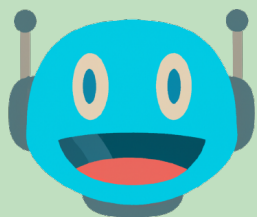
$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

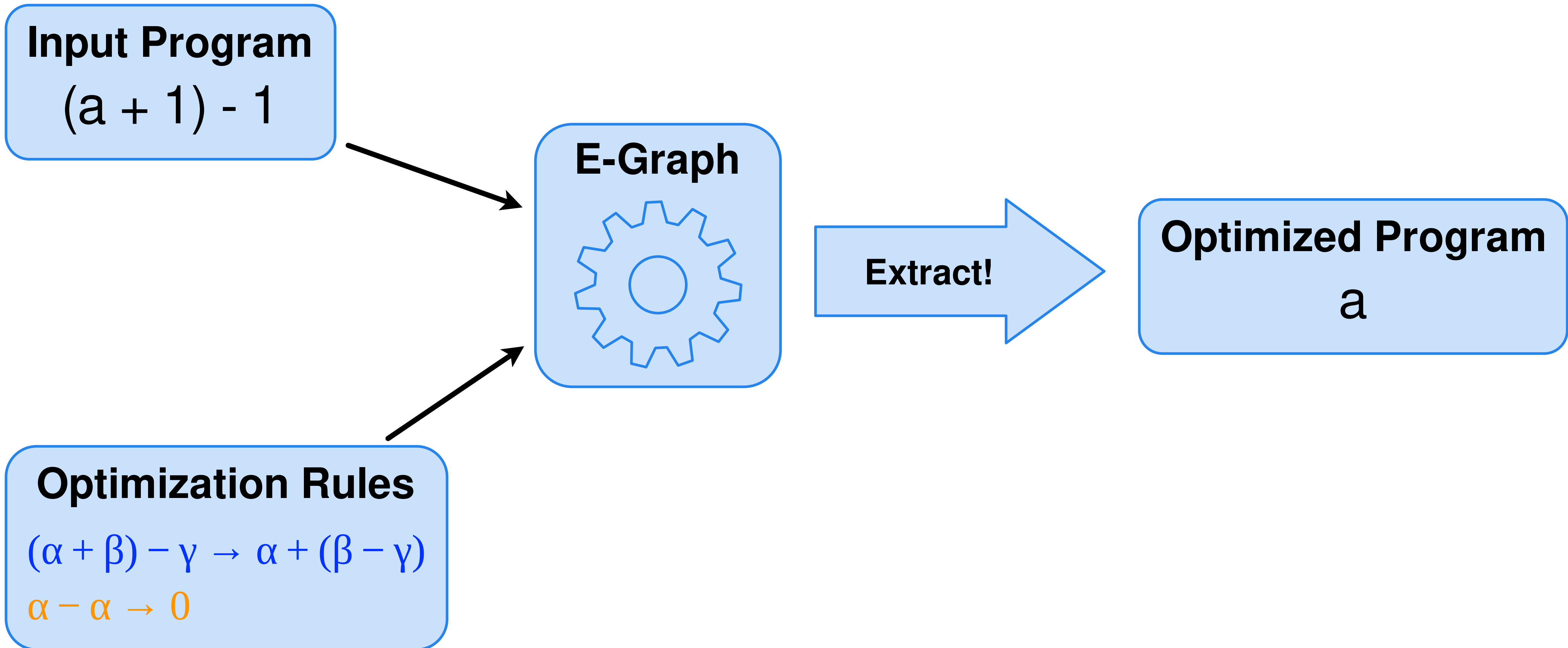
$$(v_3, \omega_3) = \text{update}(a, x+1, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

E-Graphs Explore a Big Space

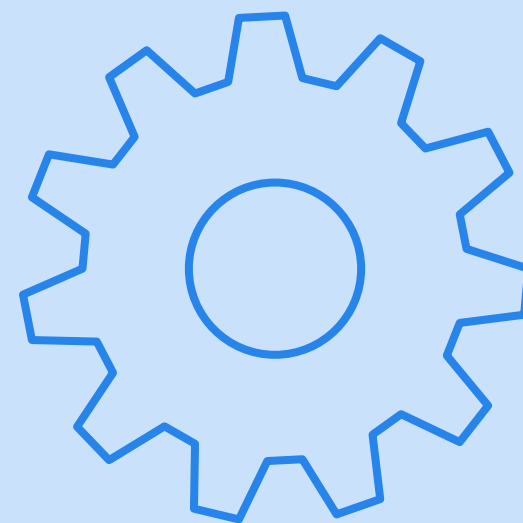


E-Graphs Explore a Big Space

Input Program

```
v1 = update(a, x)
v2 = update(a, x+1)
```

E-Graph



Extract!

Optimized Program

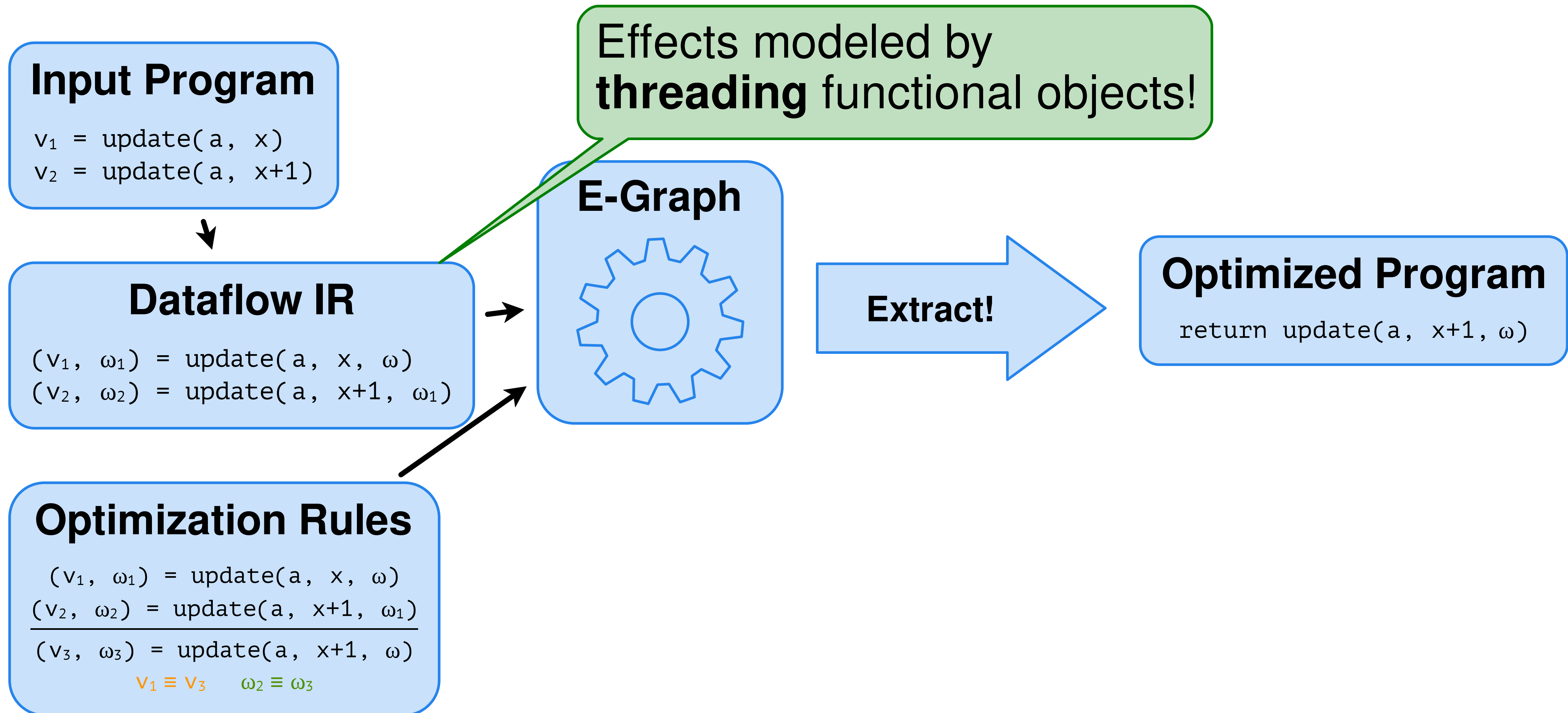
```
return update(a, x+1, ω)
```

Optimization Rules

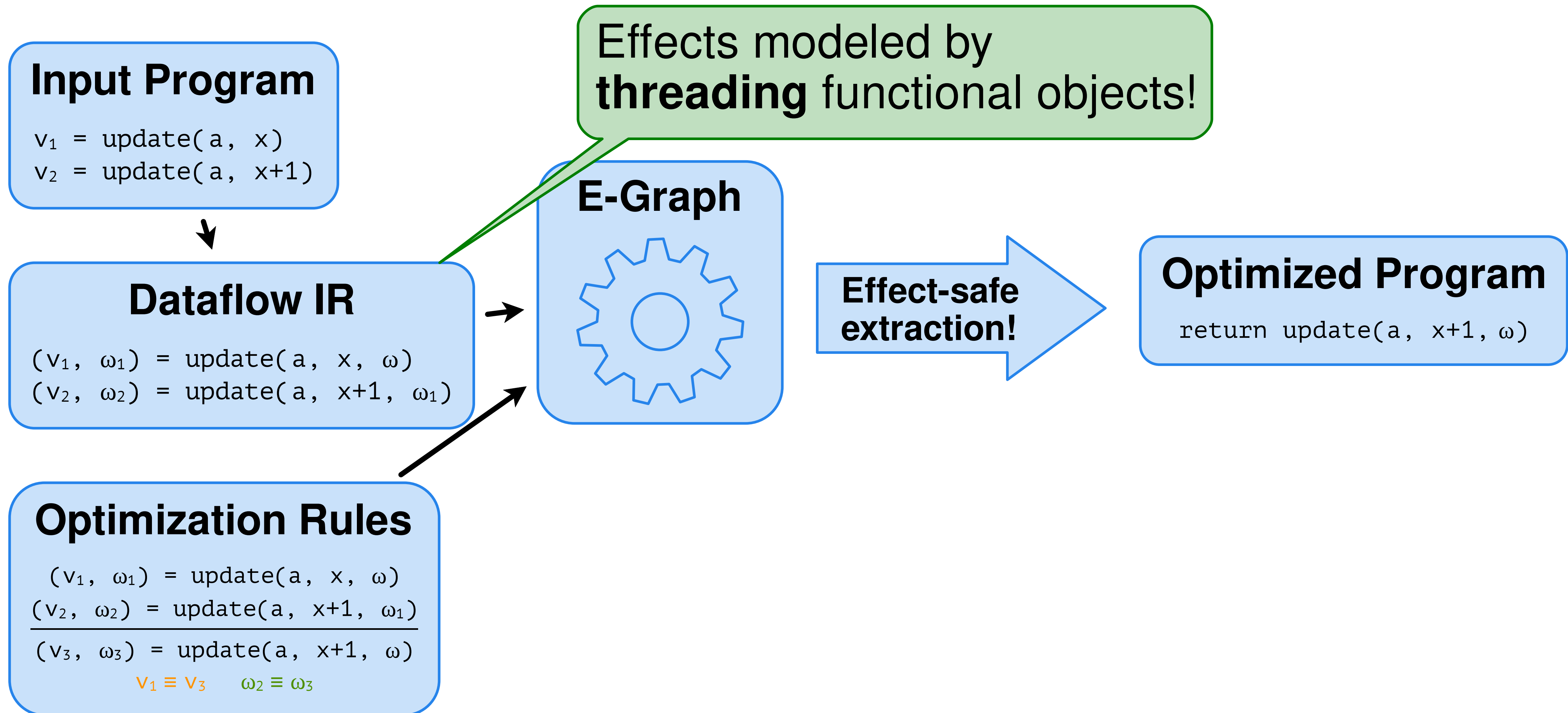
$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$

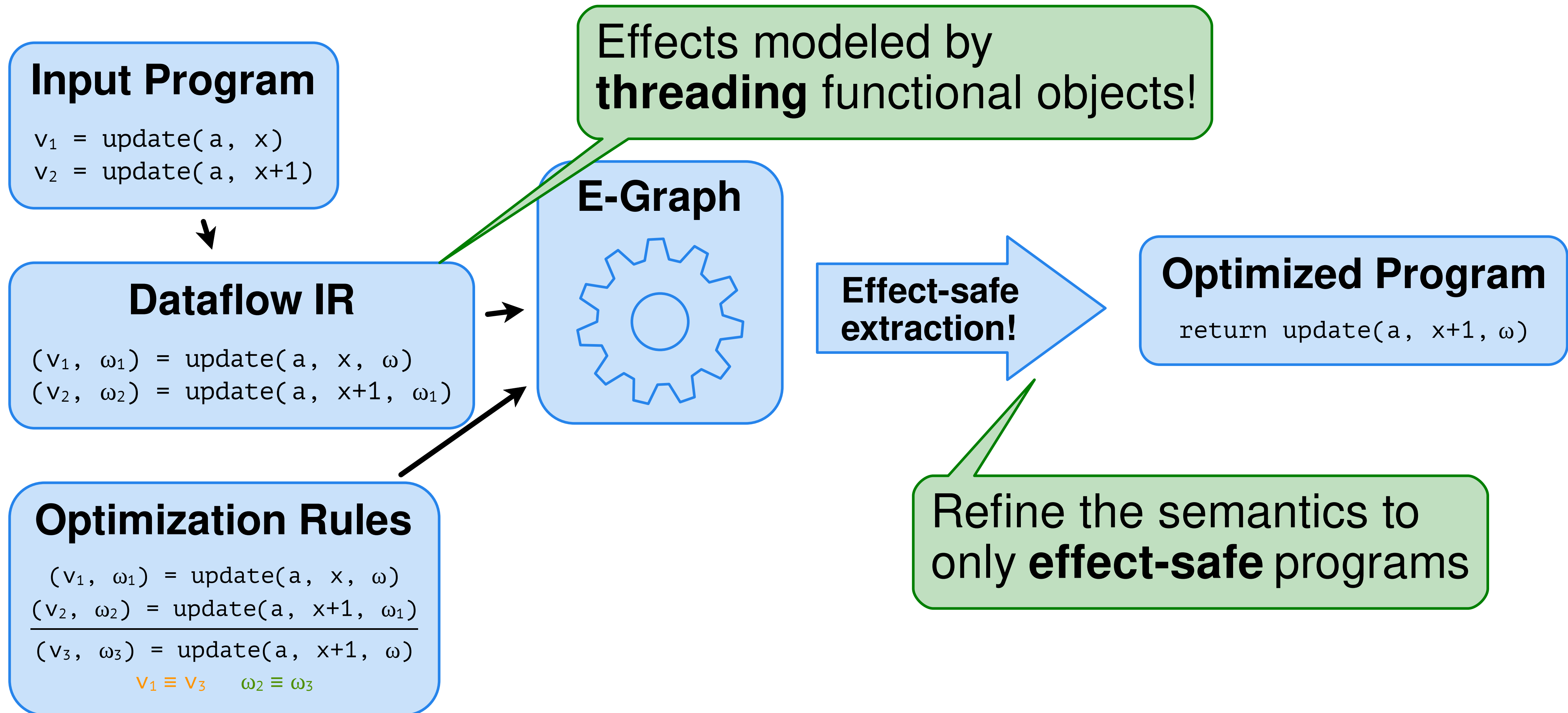
E-Graphs Explore a Big Space



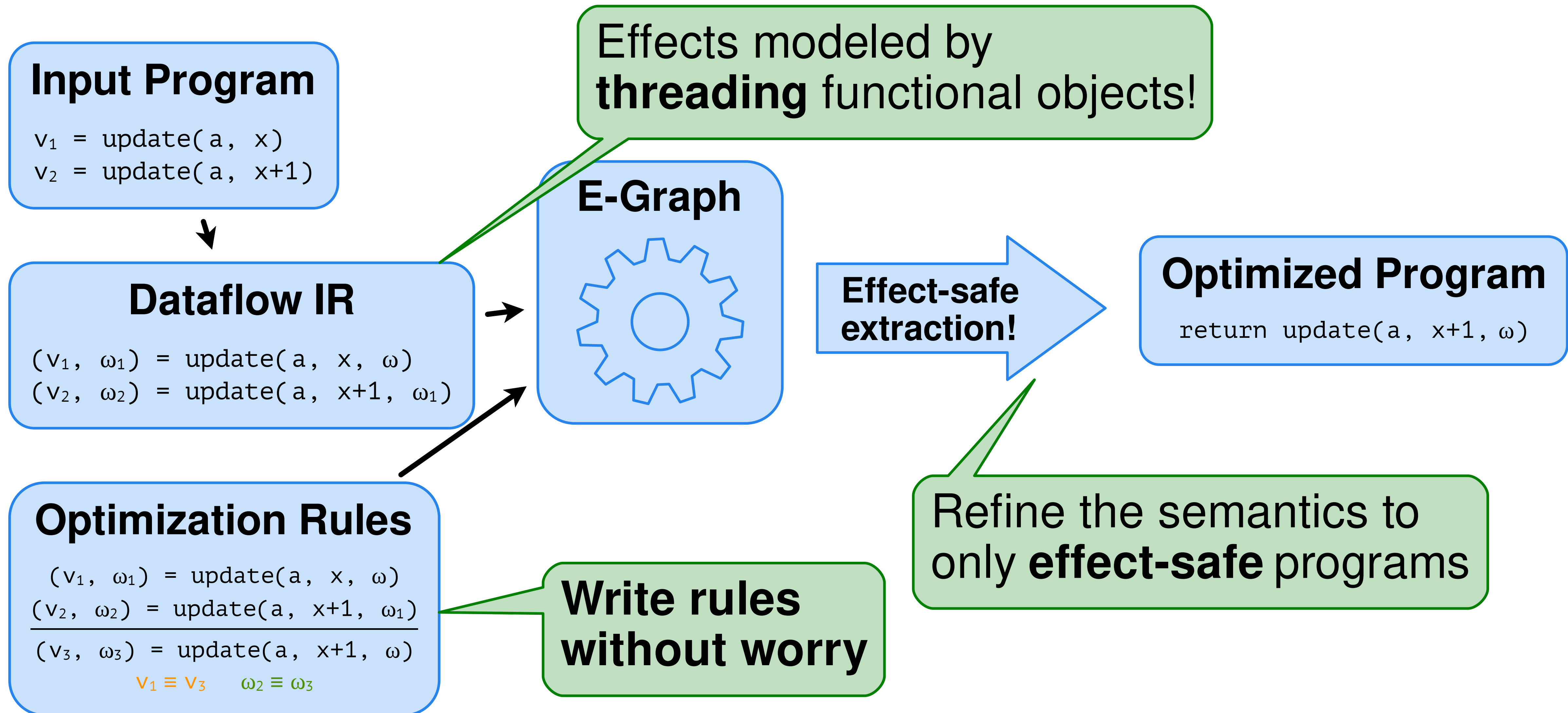
E-Graphs Explore a Big Space



E-Graphs Explore a Big Space



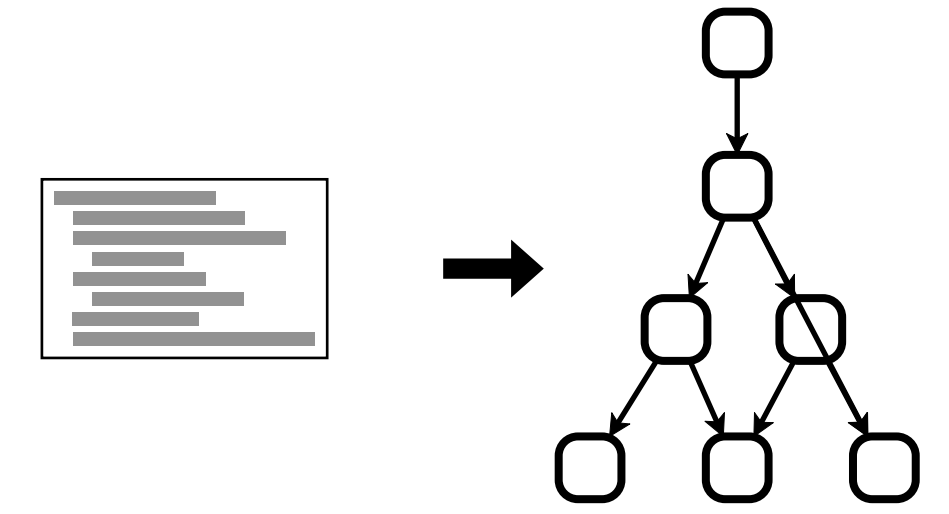
E-Graphs Explore a Big Space





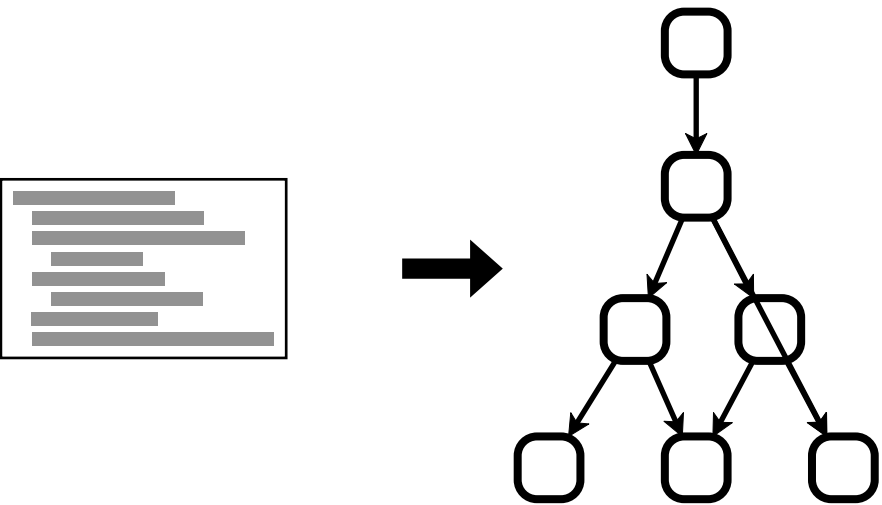
Effect-Safe Extraction

Effect-Safe Extraction is NP-Hard



$$\phi = \neg x_0 \wedge (\neg x_1 \vee x_0) \wedge (x_2 \vee x_1 \vee \neg x_2)$$

Effect-Safe Extraction is NP-Hard



$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

X_0

$\neg X_0$

X_1

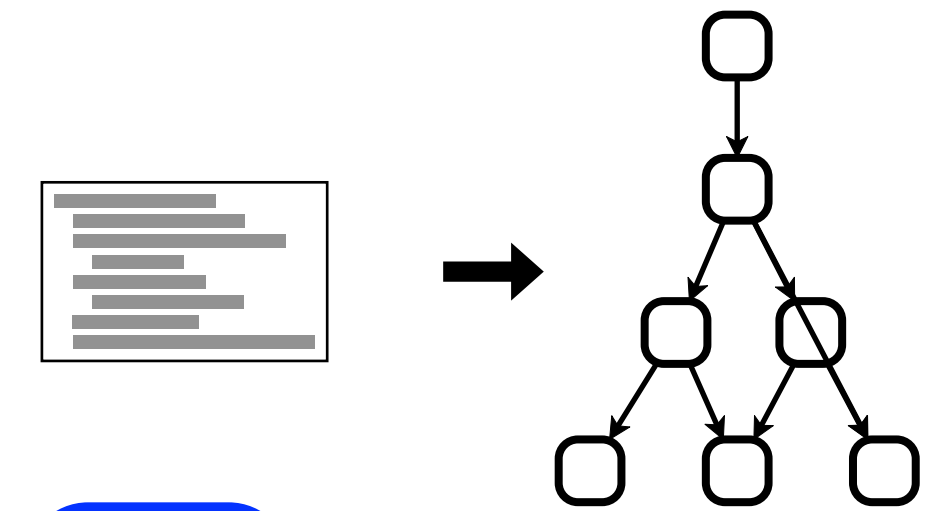
$\neg X_1$

X_2

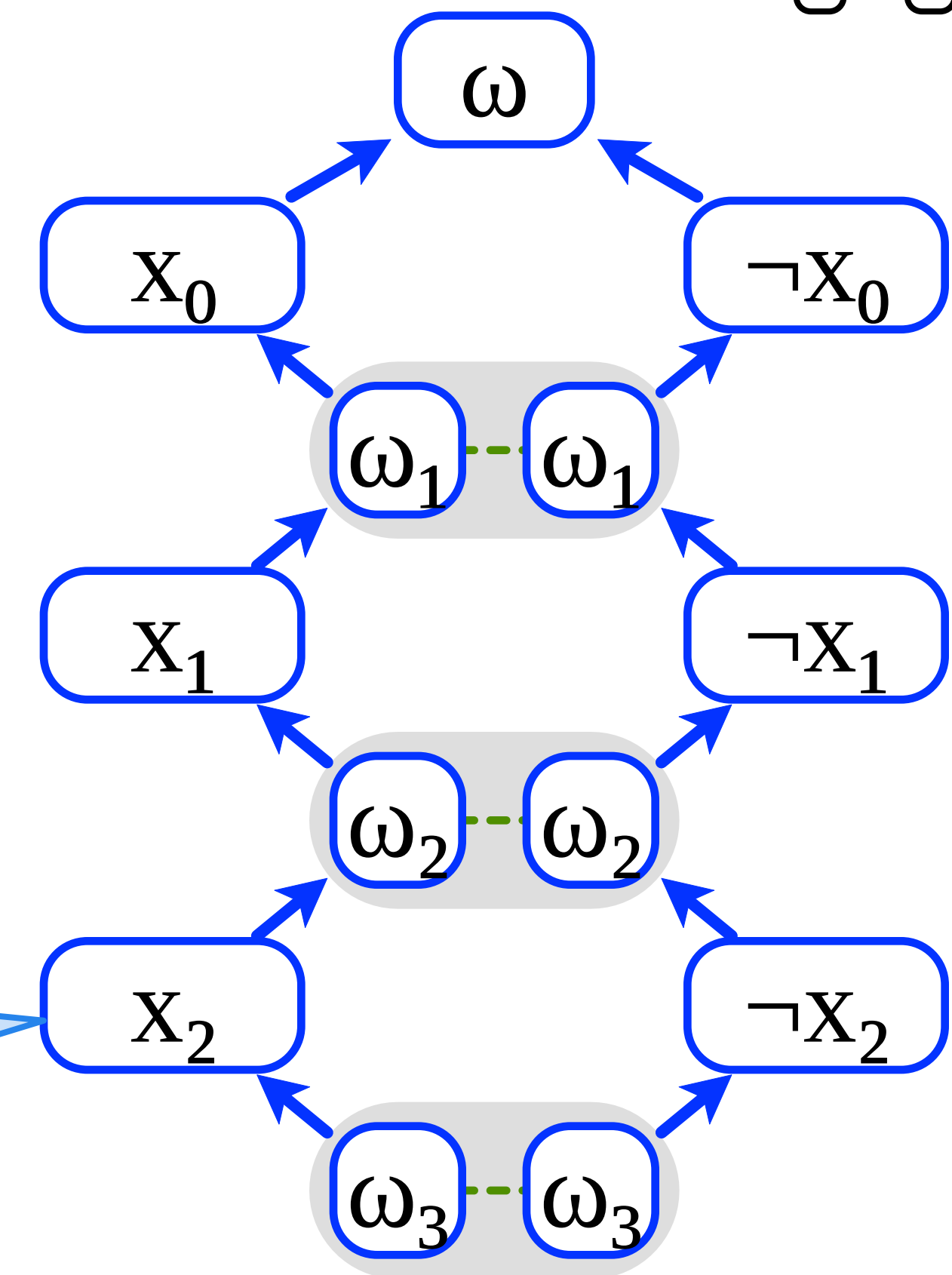
$\neg X_2$

Effect-Safe Extraction is NP-Hard

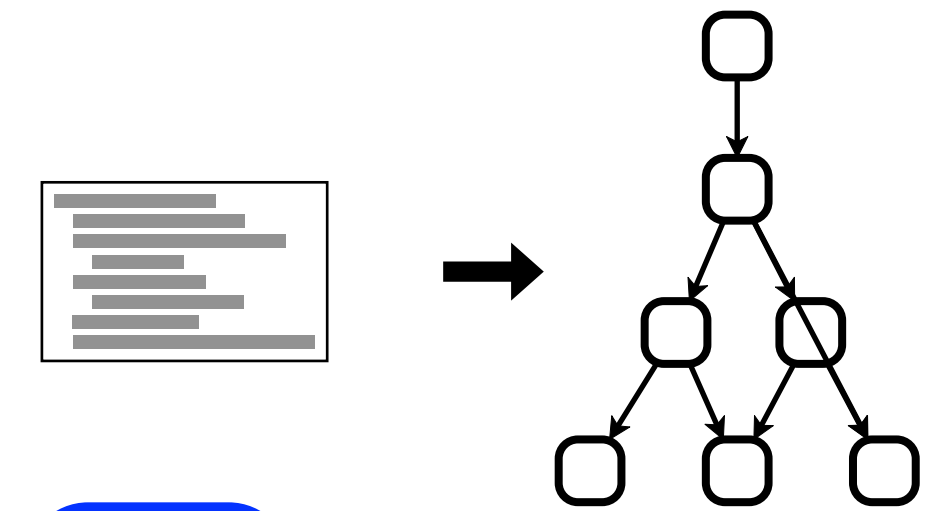
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



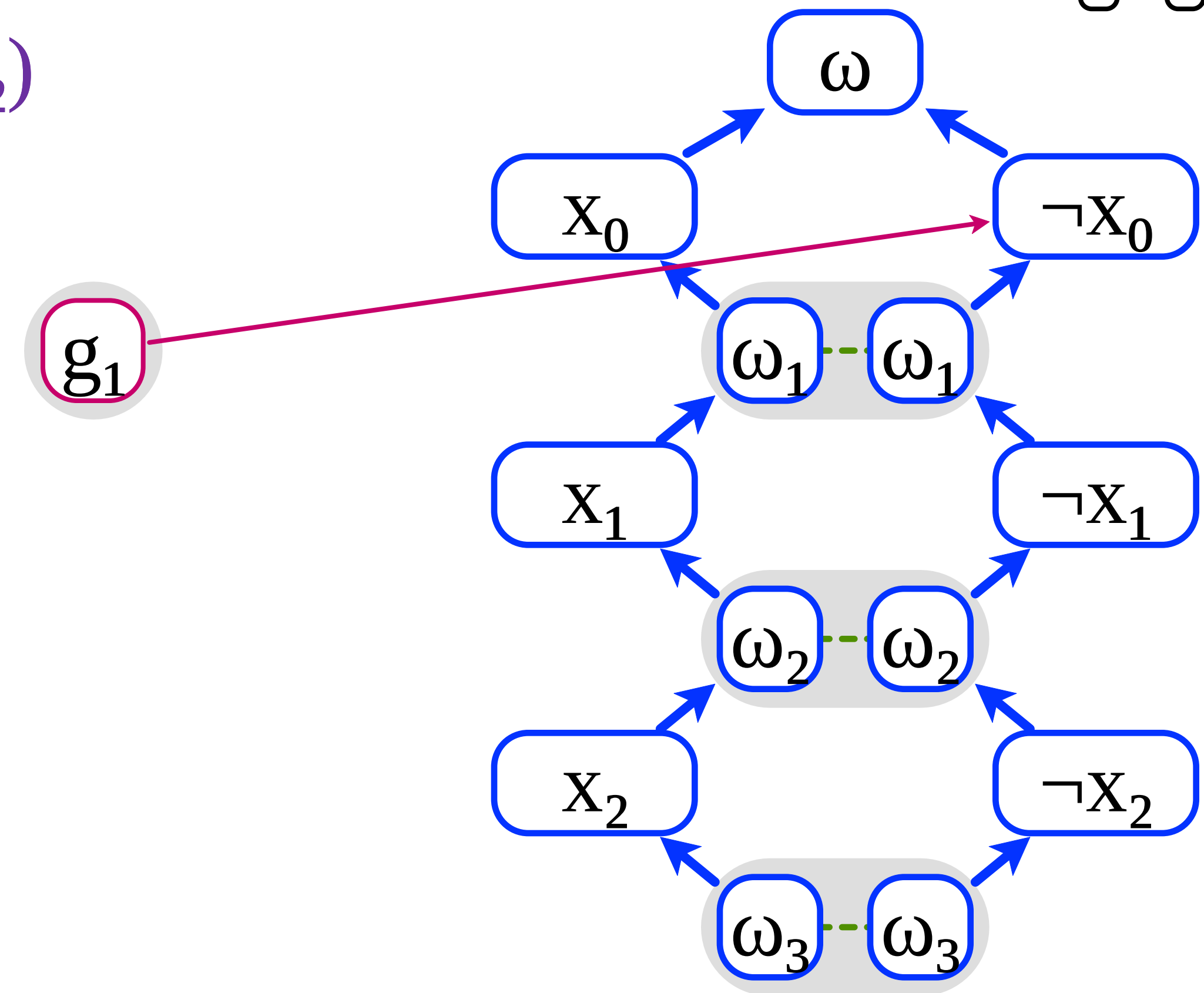
state walks encode assignments



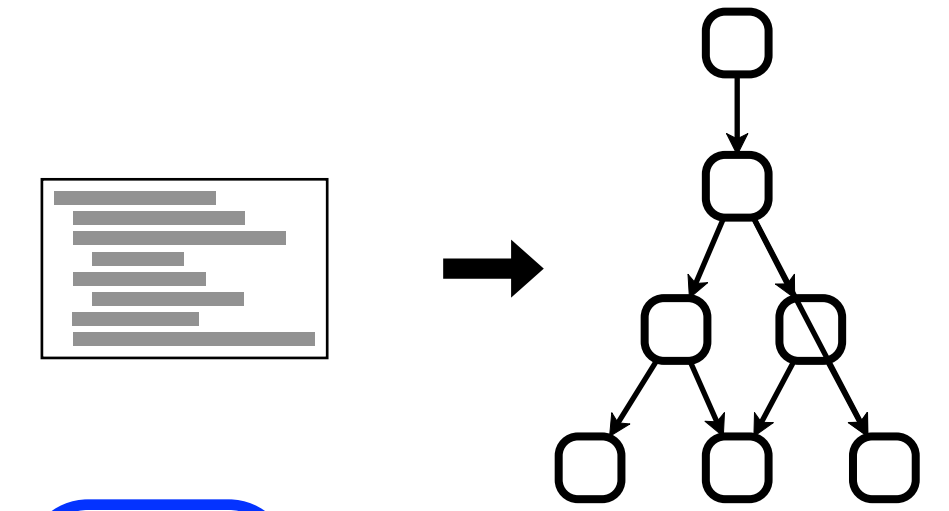
Effect-Safe Extraction is NP-Hard



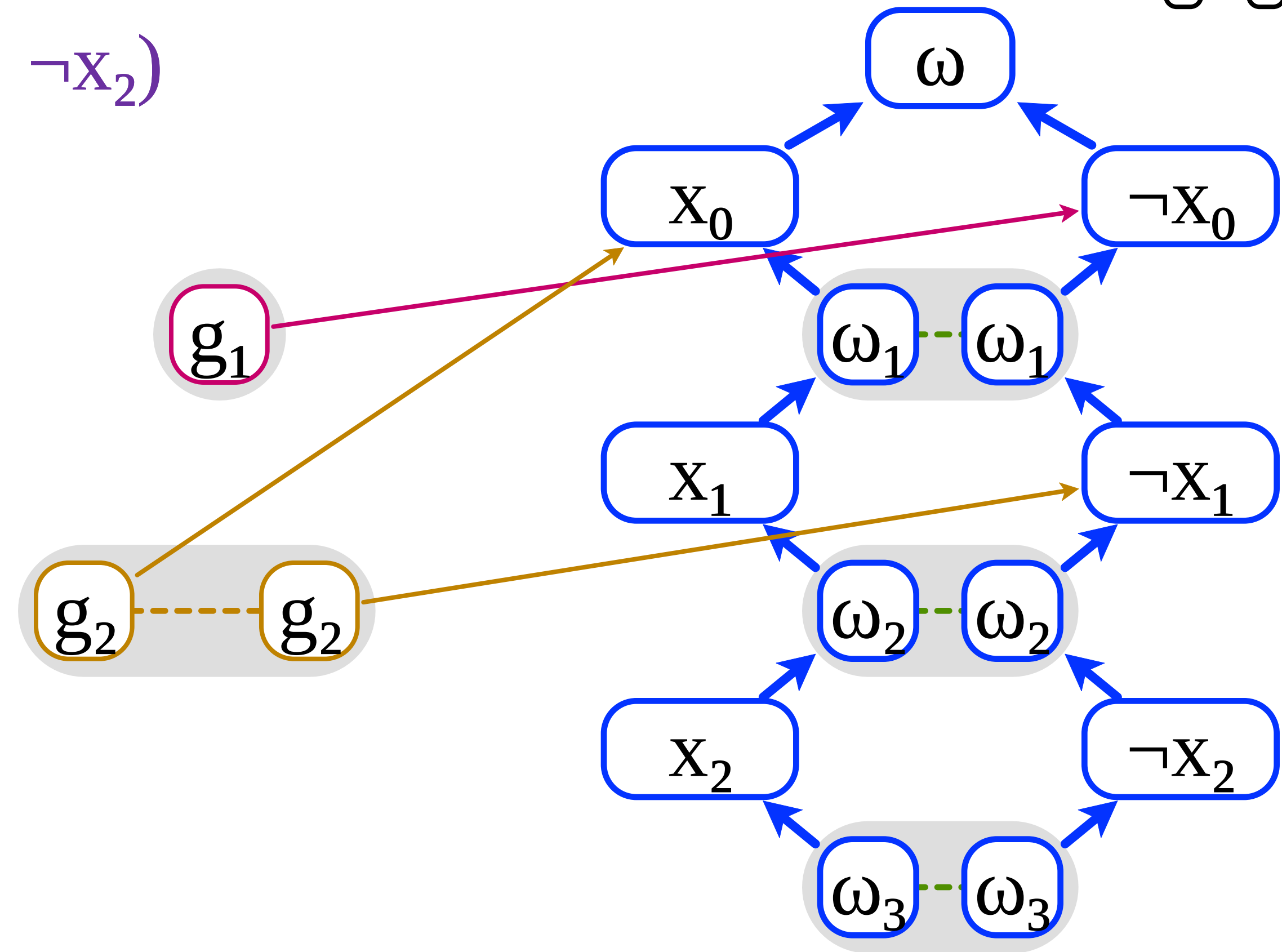
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



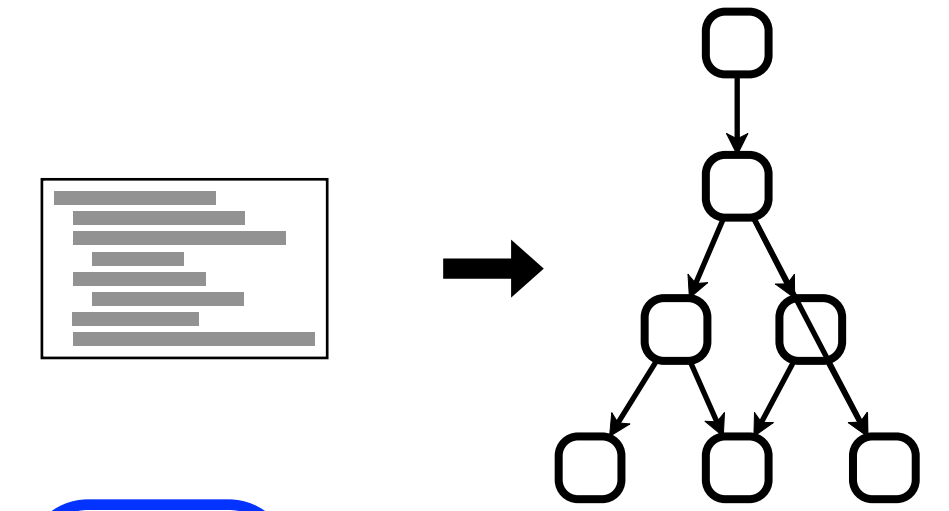
Effect-Safe Extraction is NP-Hard



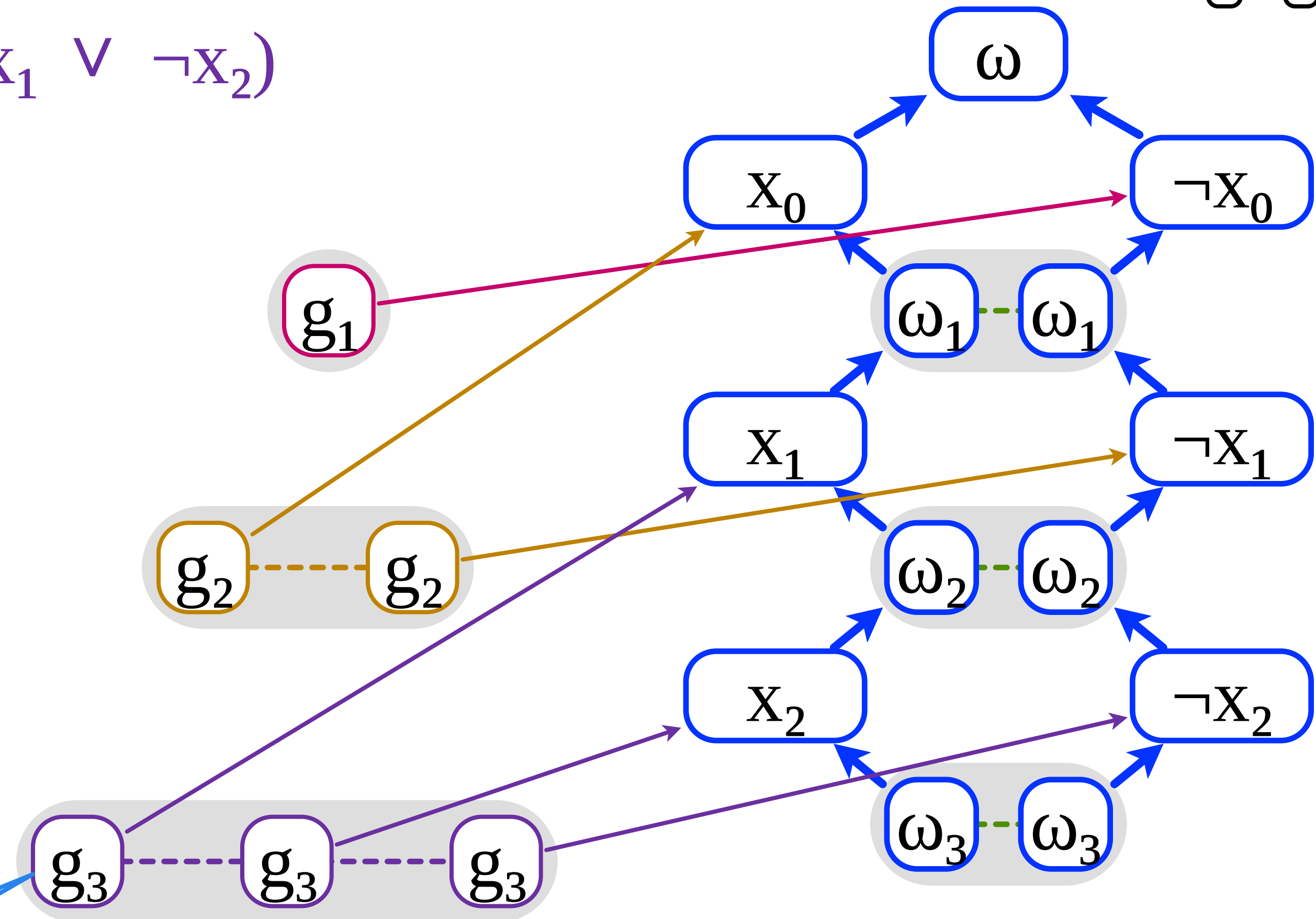
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



Effect-Safe Extraction is NP-Hard

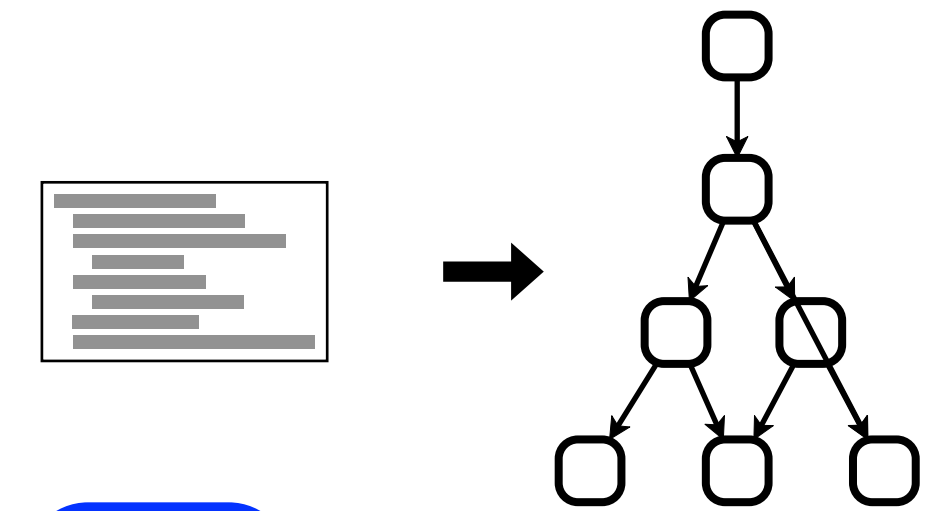


$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

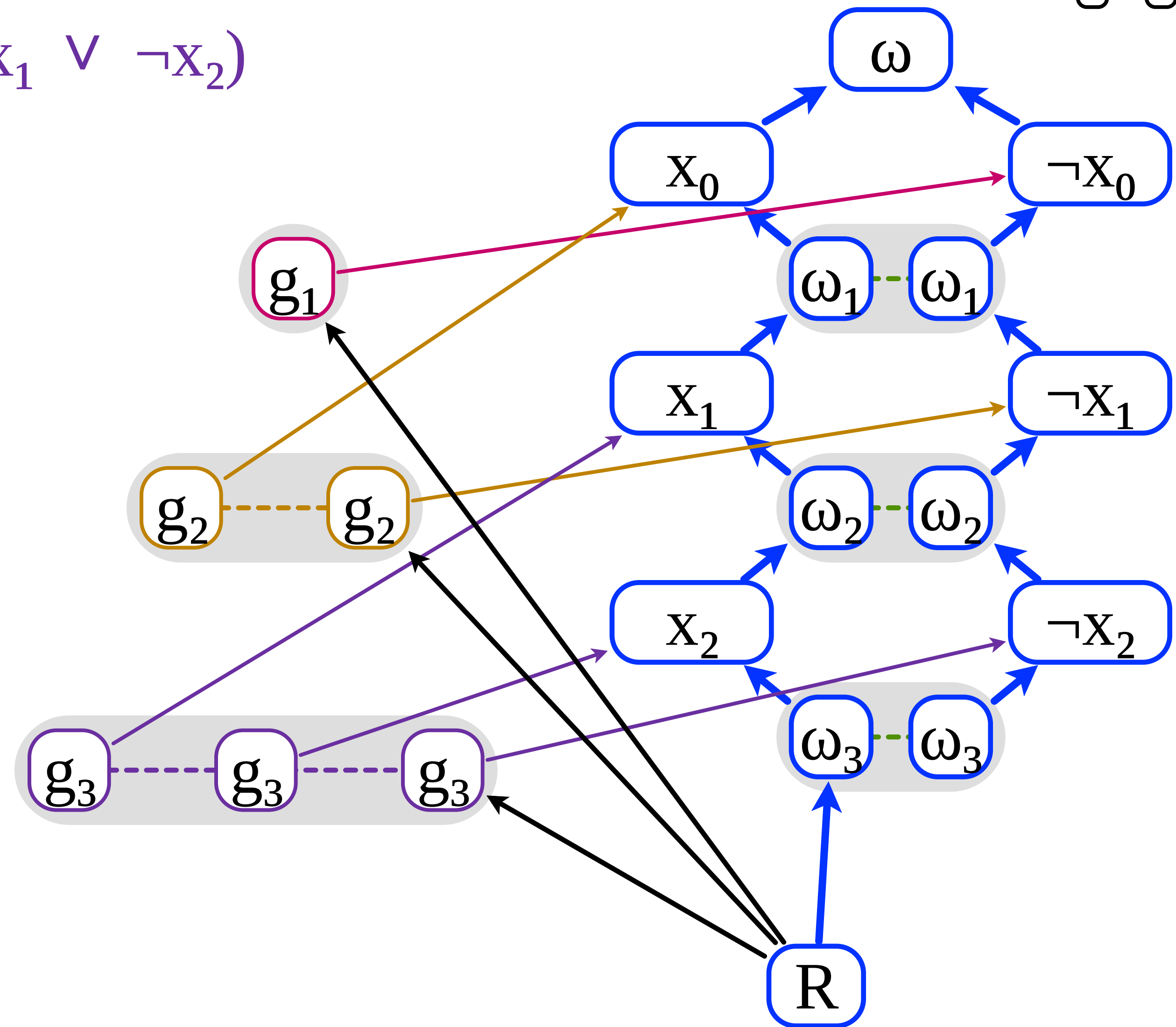


e-classes encode clauses

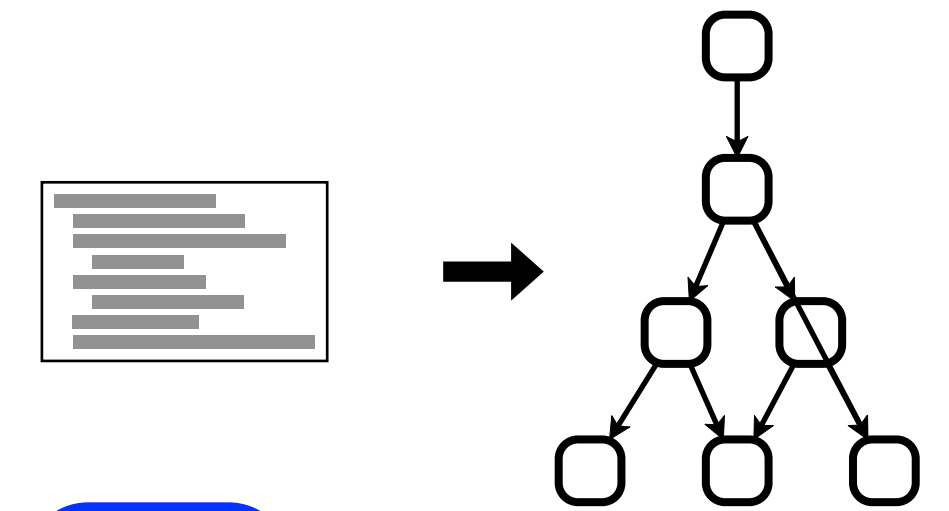
Effect-Safe Extraction is NP-Hard



$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



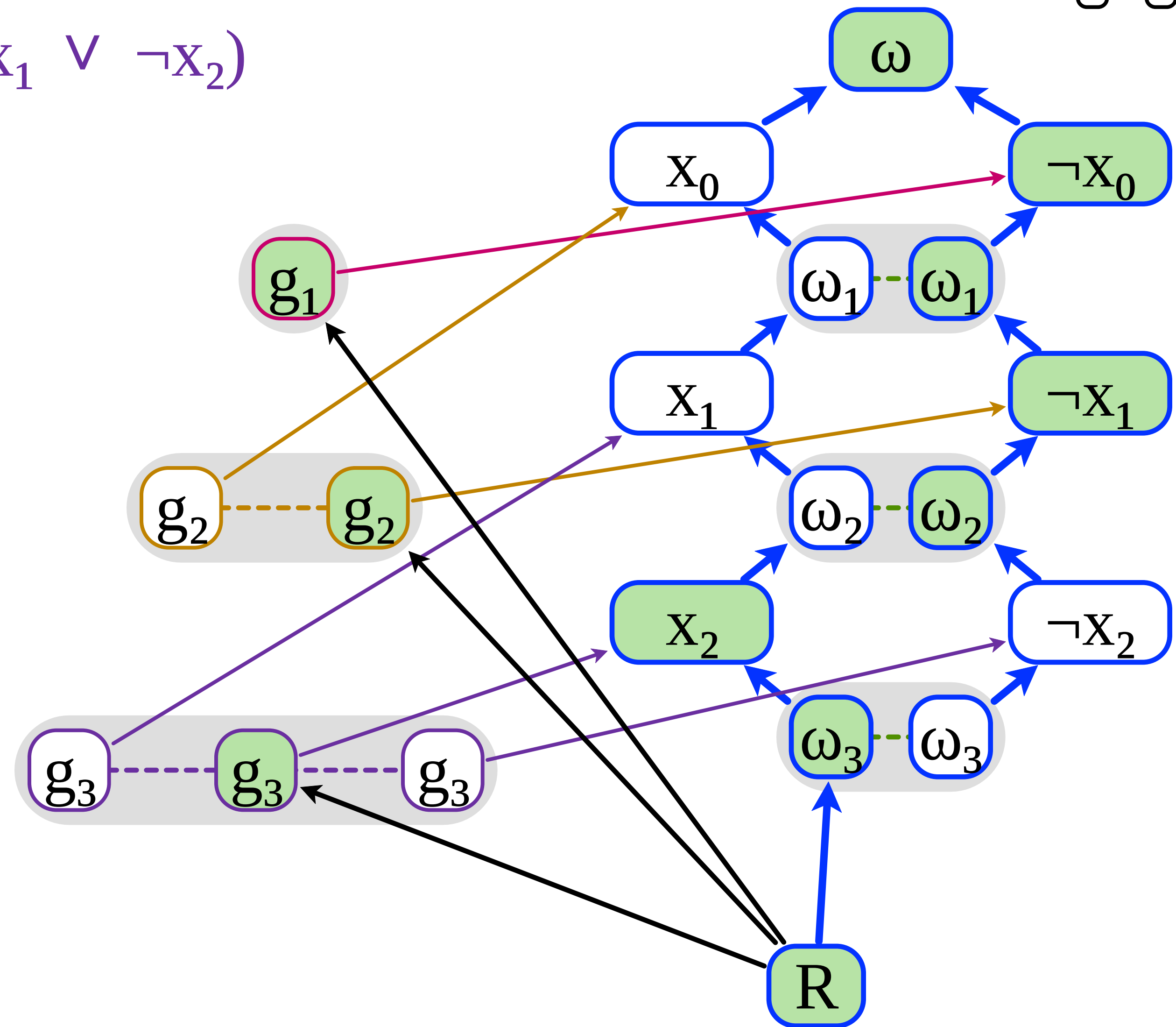
Effect-Safe Extraction is NP-Hard



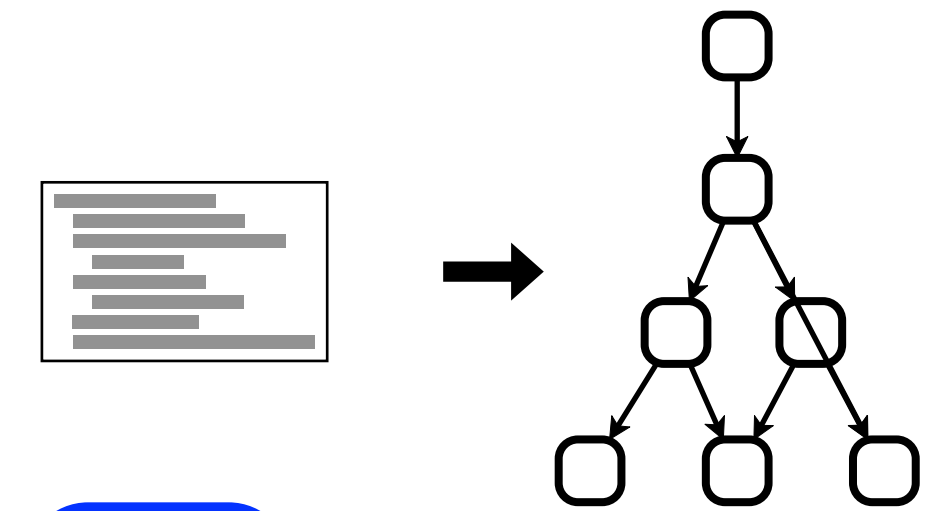
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

Satisfying assignment:

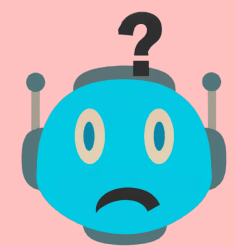
$$X_0 = \perp \quad X_1 = \perp \quad X_2 = \top$$



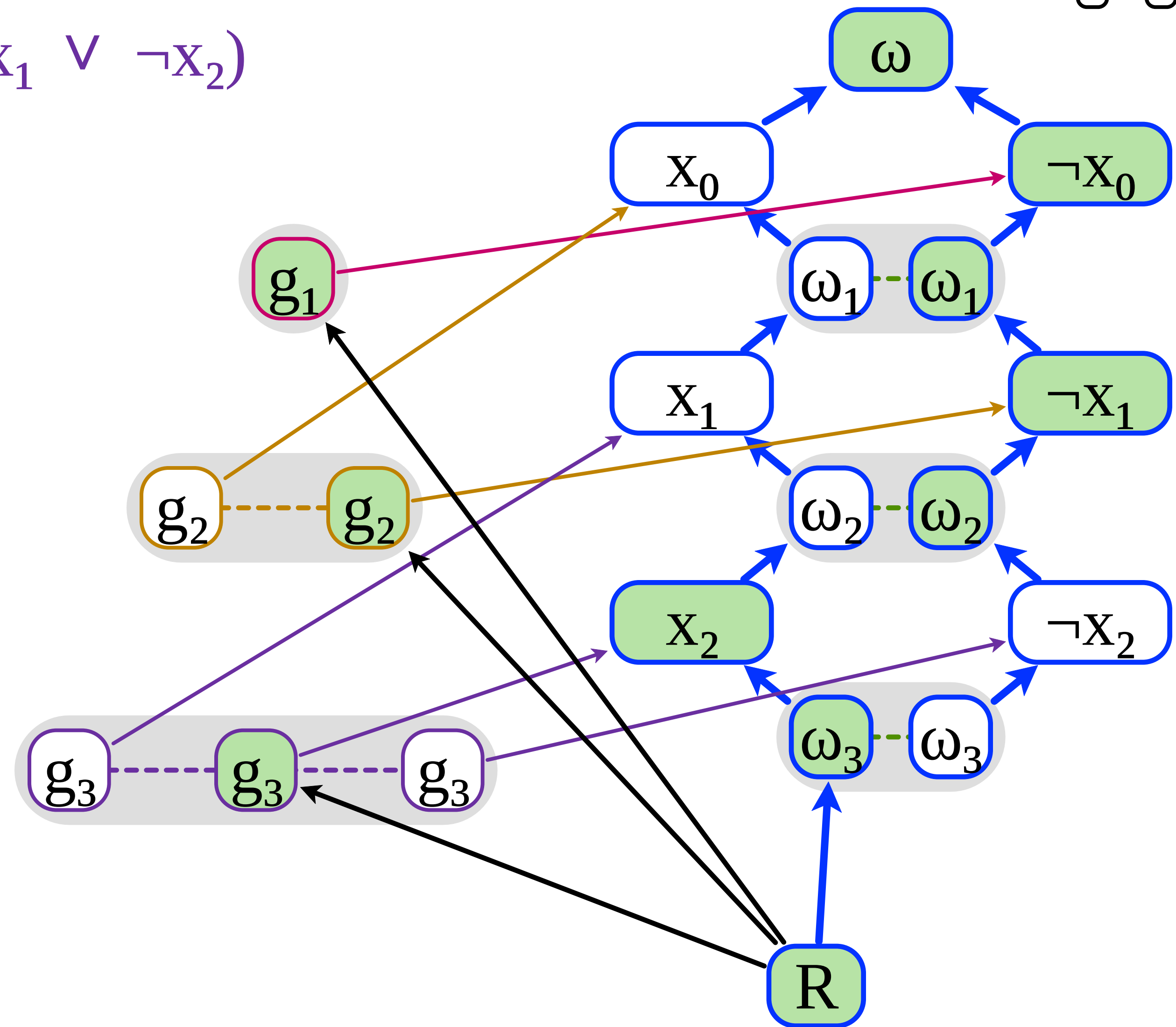
Effect-Safe Extraction is NP-Hard



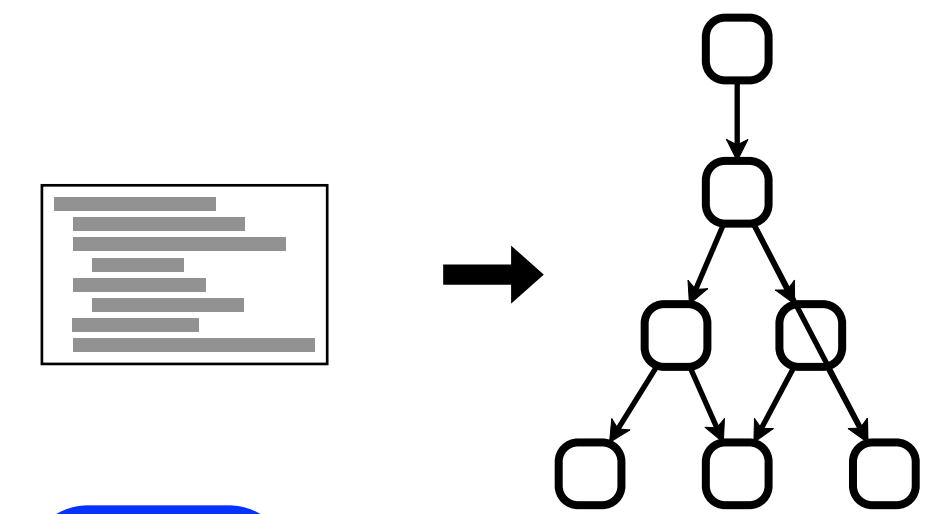
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



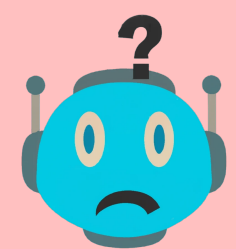
Problem is NP-hard!
Give up??



Effect-Safe Extraction is NP-Hard

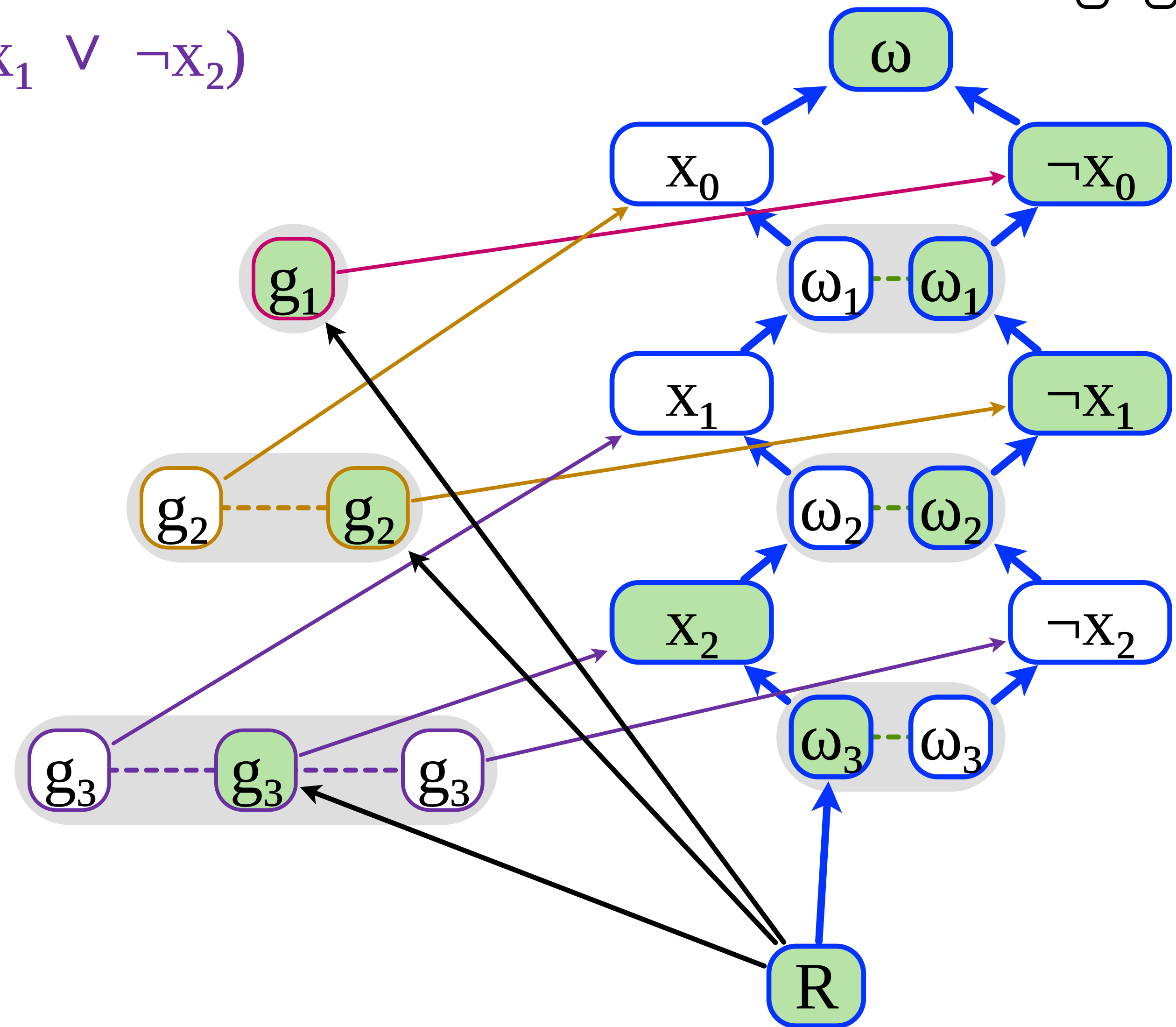


$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

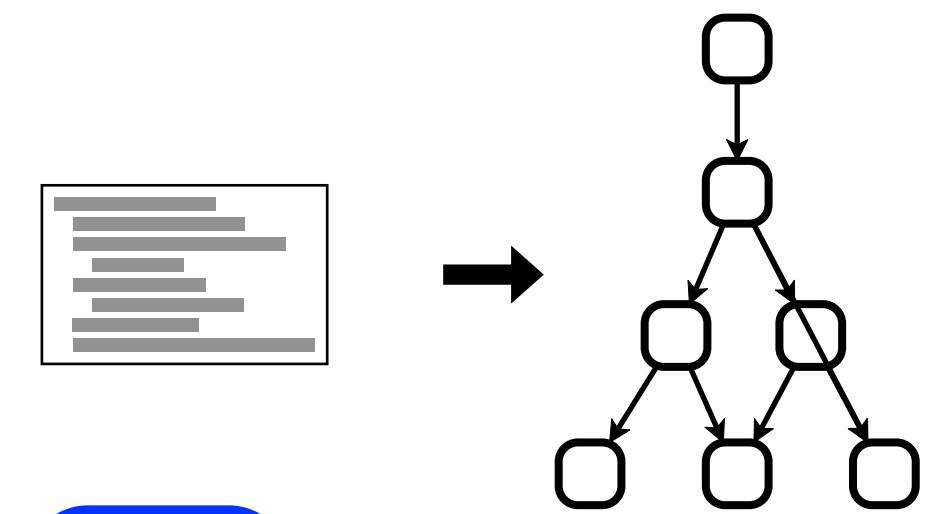


Problem is NP-hard!
Give up??

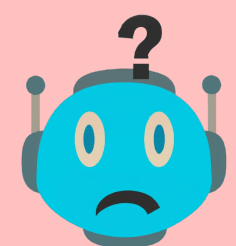
No! Real-world e-graphs are easier, measured with a *parameter*



Effect-Safe Extraction is NP-Hard



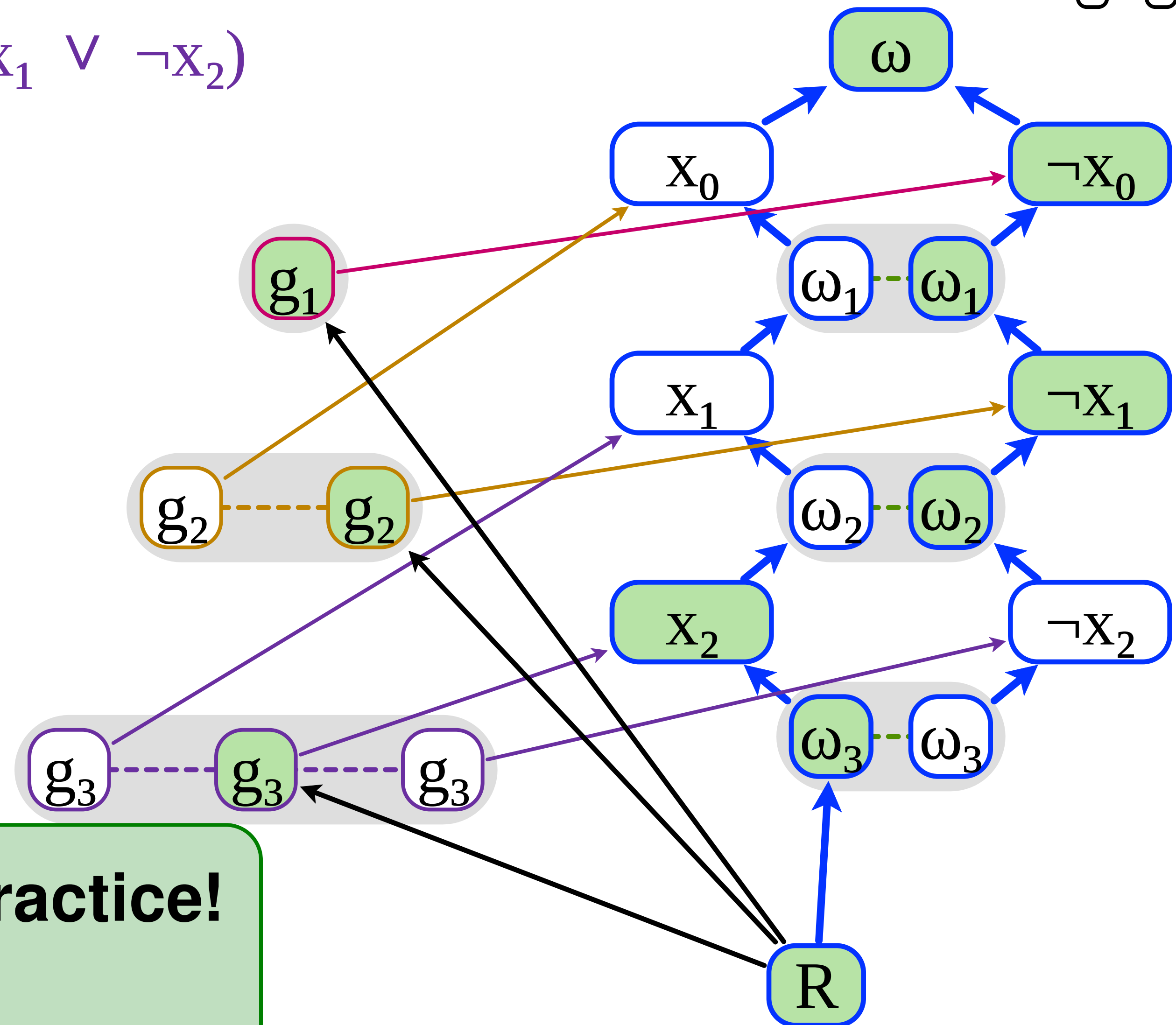
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



Problem is NP-hard!
Give up??

No! Real-world e-graphs are easier, measured with a *parameter*

***statewalk width* is low in practice!**
Complexity is $O(kn^2)$



The Statewalk DP Algorithm

```
fun foo(a, x, ω):
```

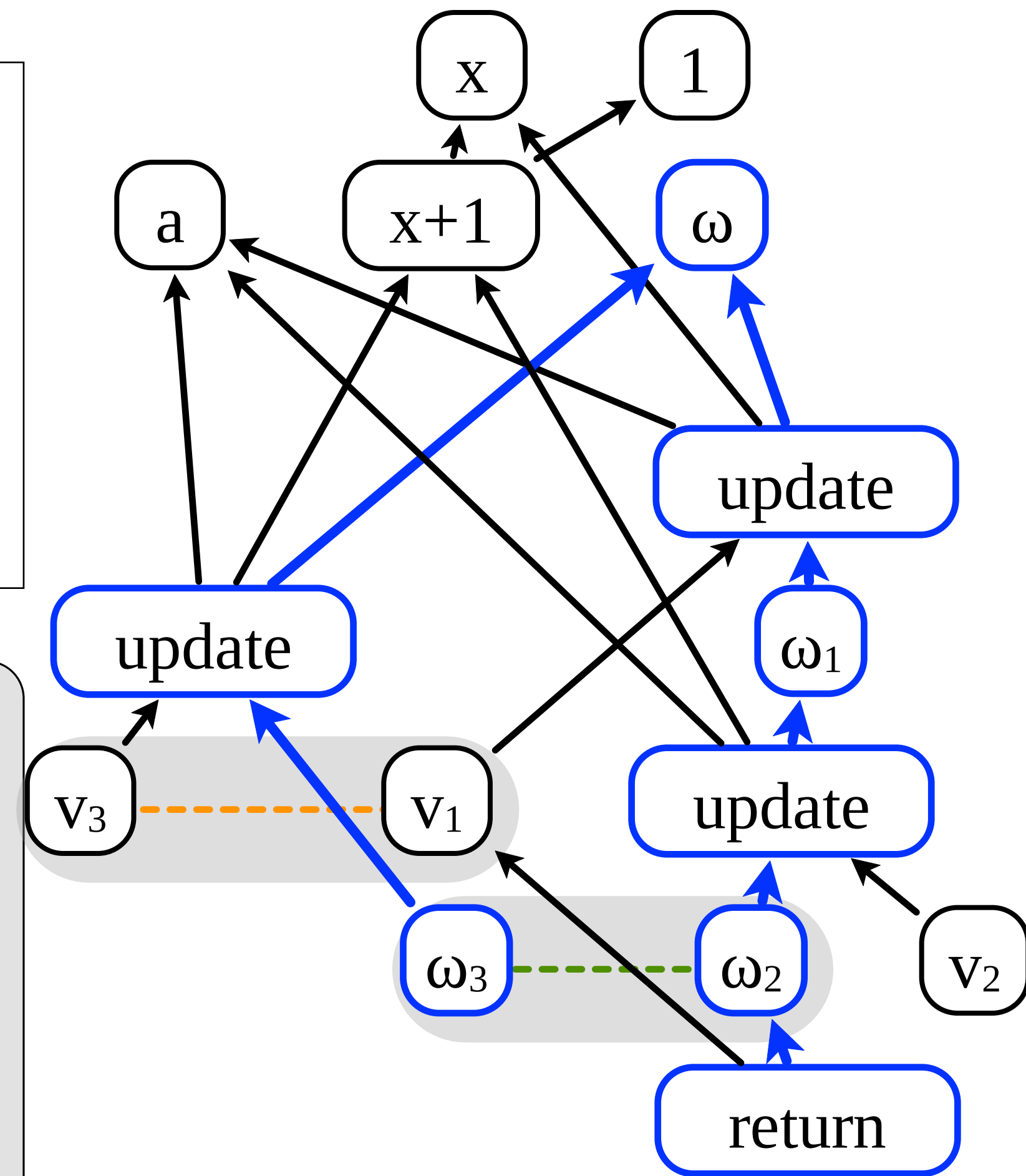
```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

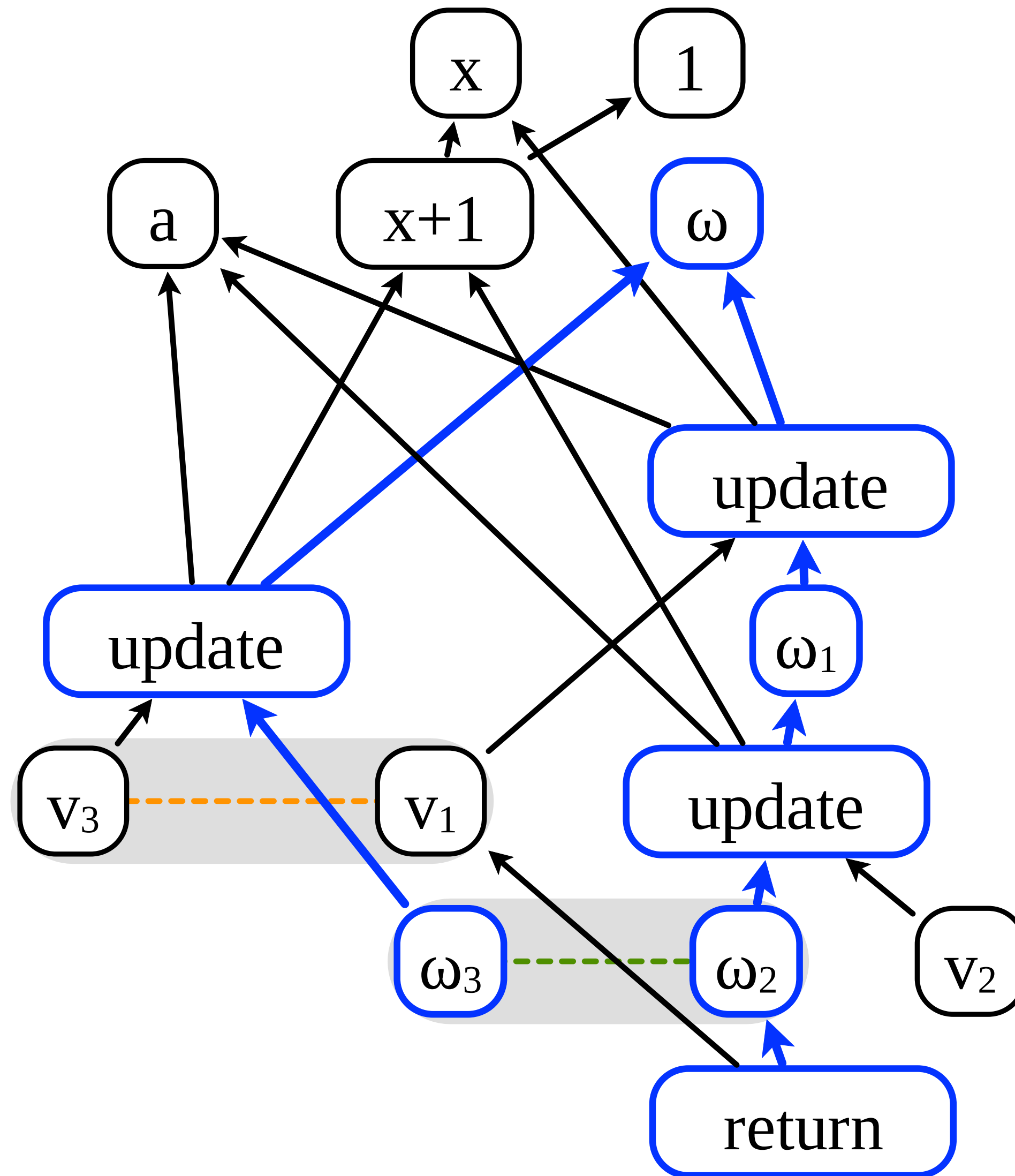
```
  return (v1, ω2)
```

$$\frac{\begin{array}{l} (v_1, \omega_1) = \text{update}(a, x, \omega) \\ (v_2, \omega_2) = \text{update}(a, x+1, \omega_1) \end{array}}{(v_3, \omega_3) = \text{update}(a, x+1, \omega)}$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$

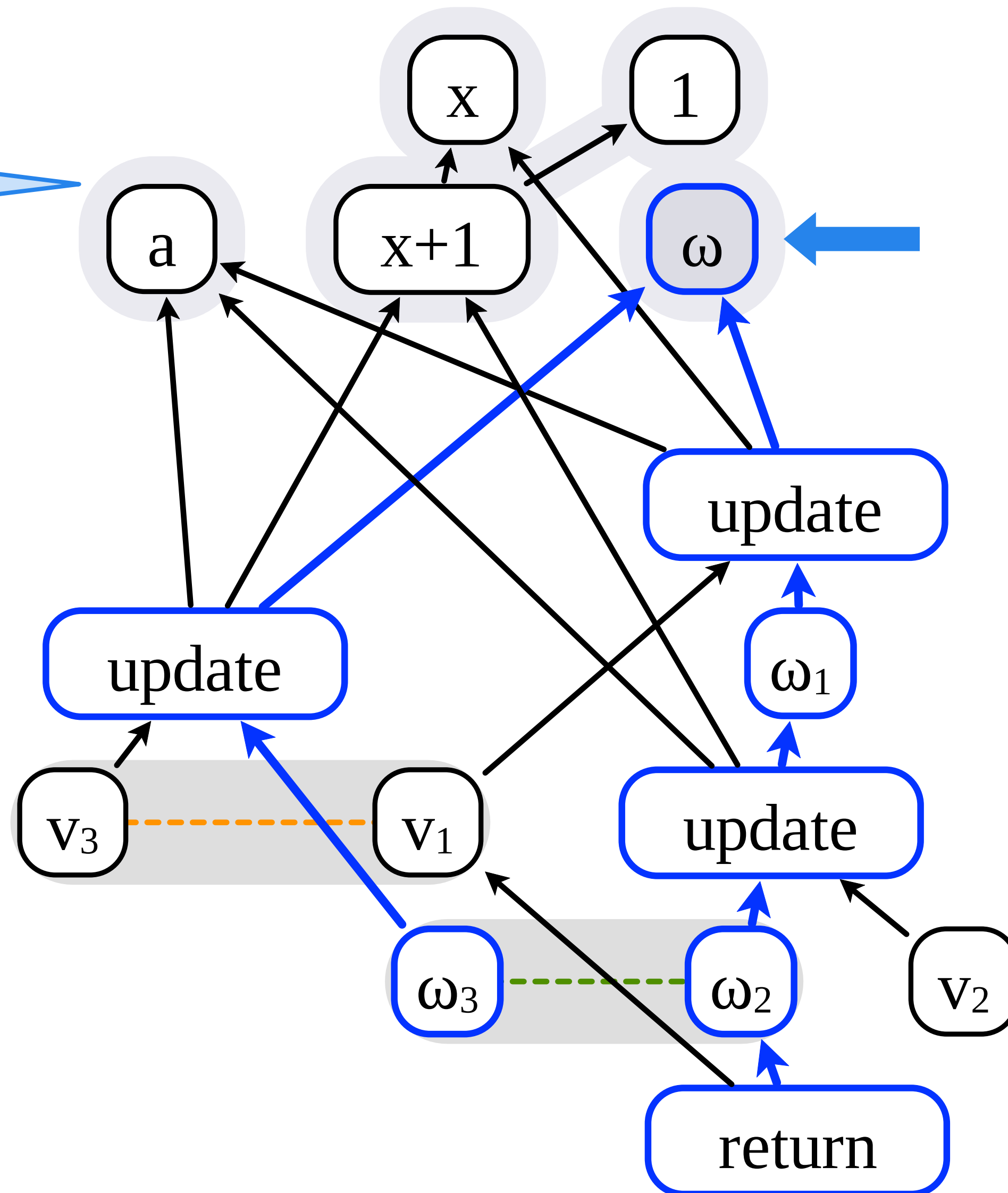


The Statewalk DP Algorithm

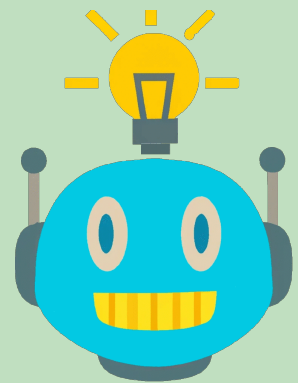


The Statewalk DP Algorithm

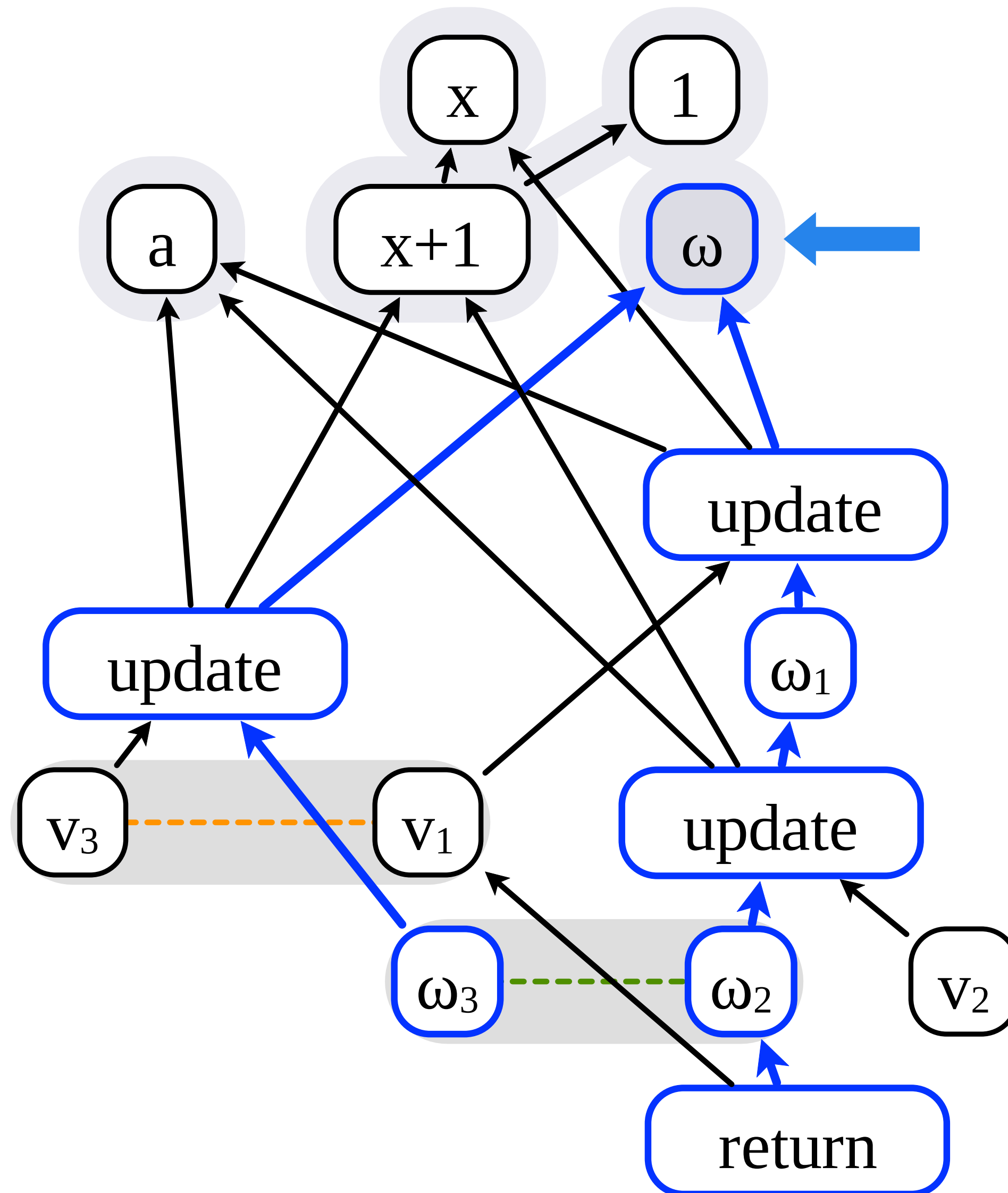
extractable set



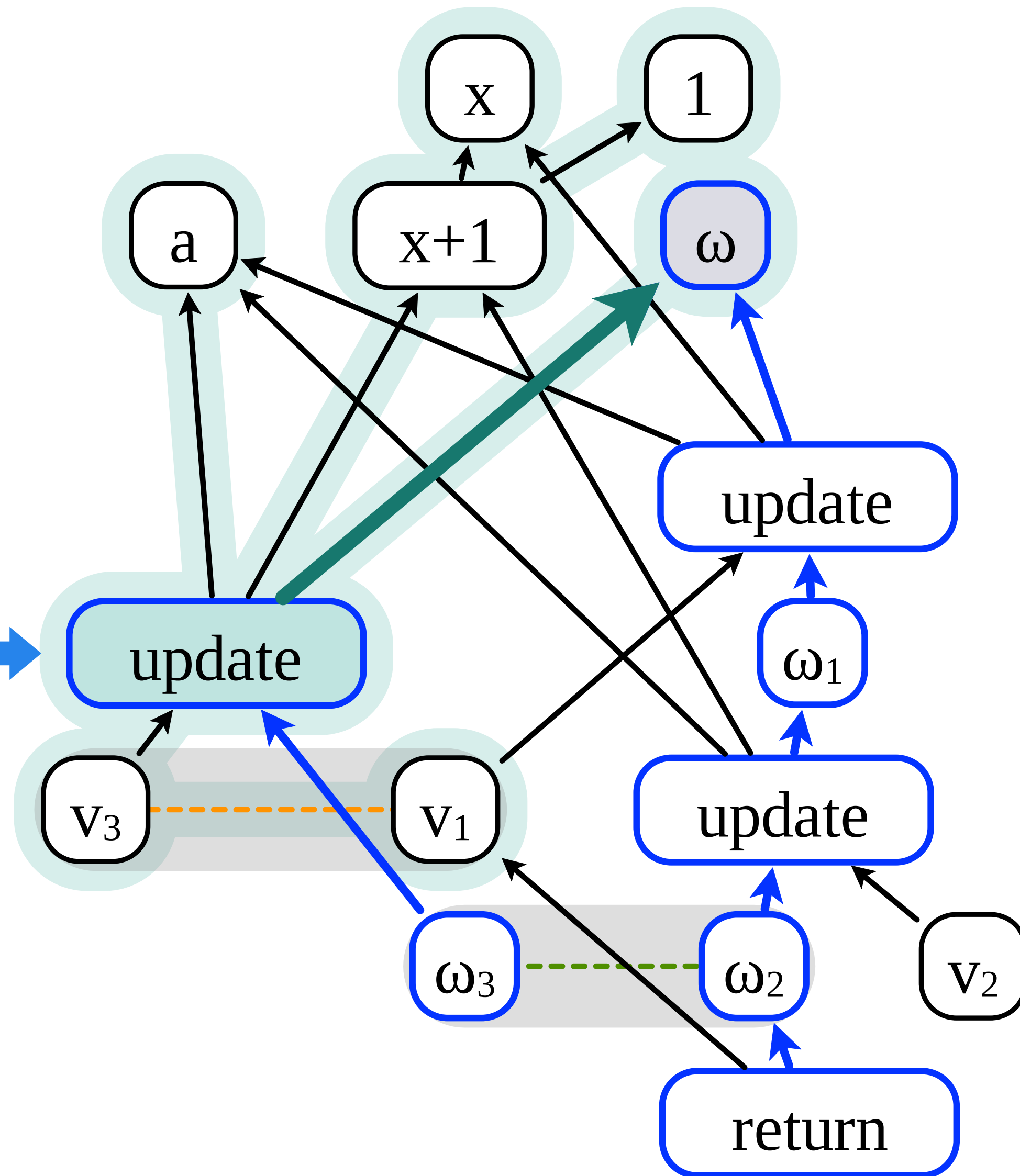
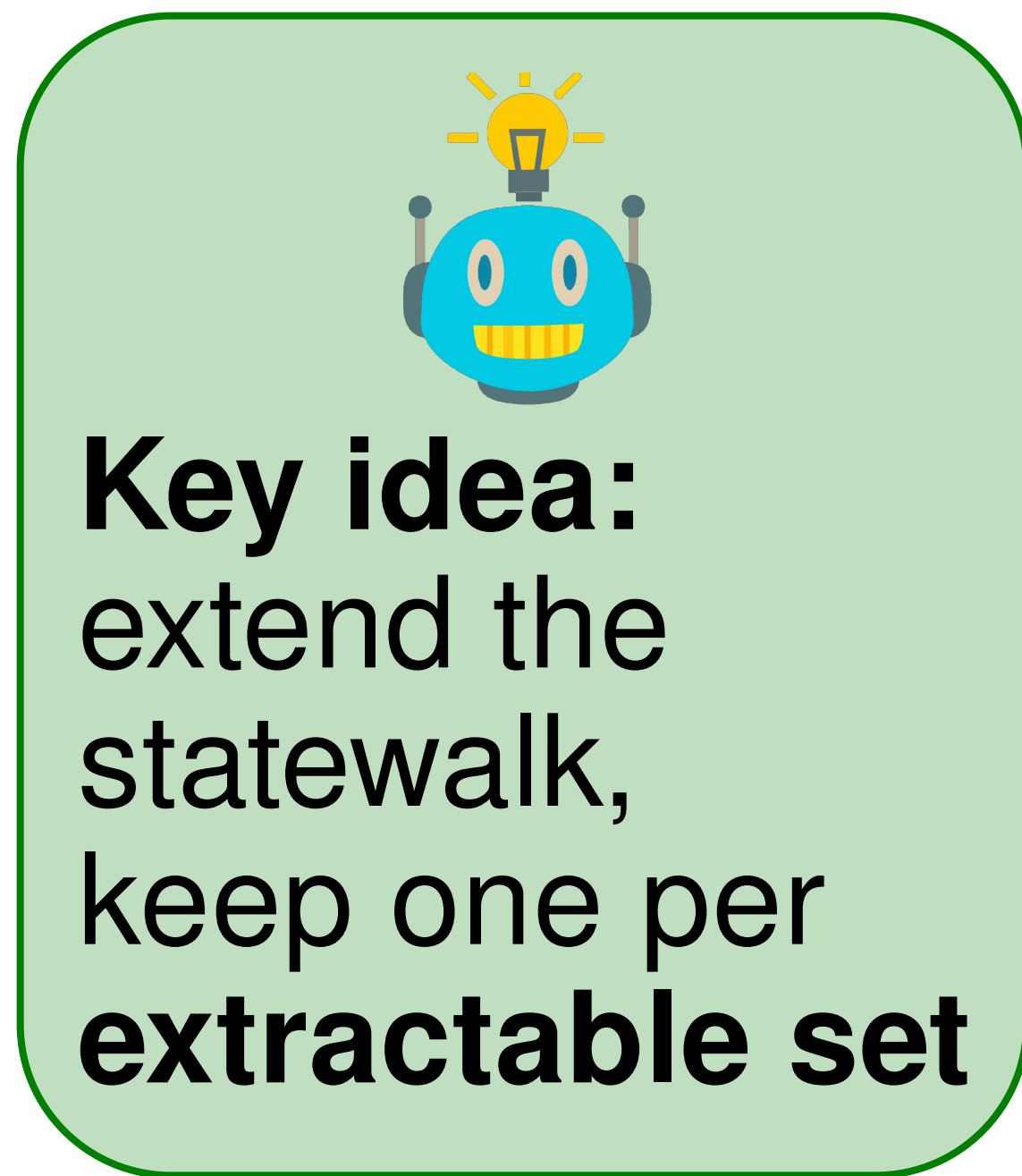
The Statewalk DP Algorithm



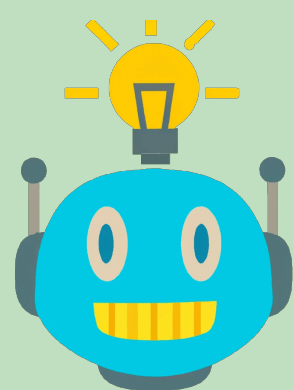
Key idea:
extend the
statewalk,
keep one per
extractable set



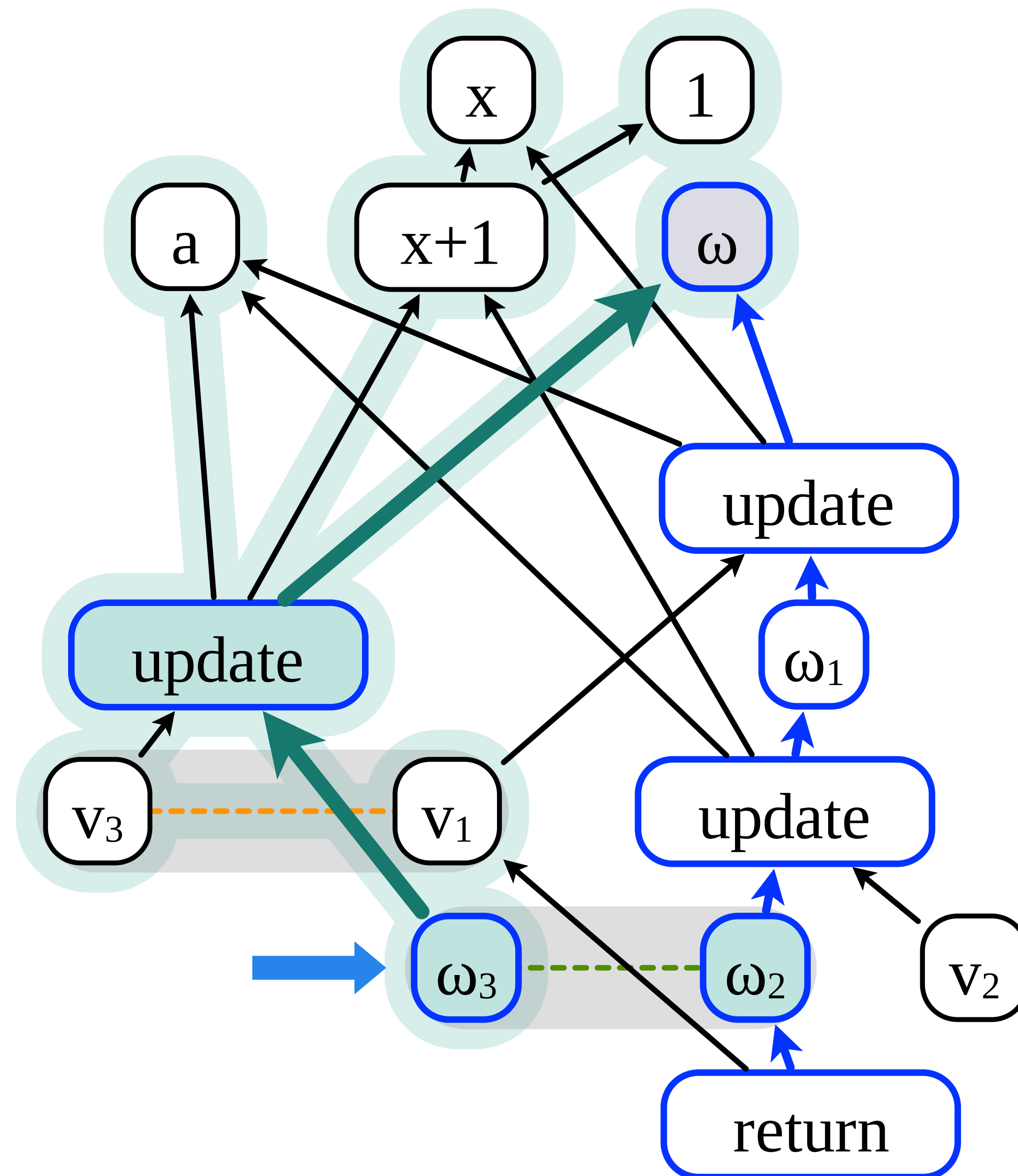
The Statewalk DP Algorithm



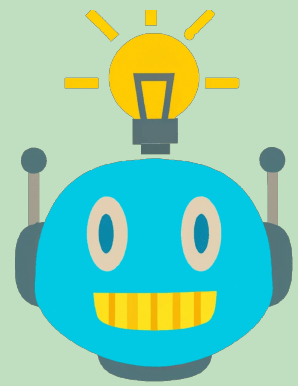
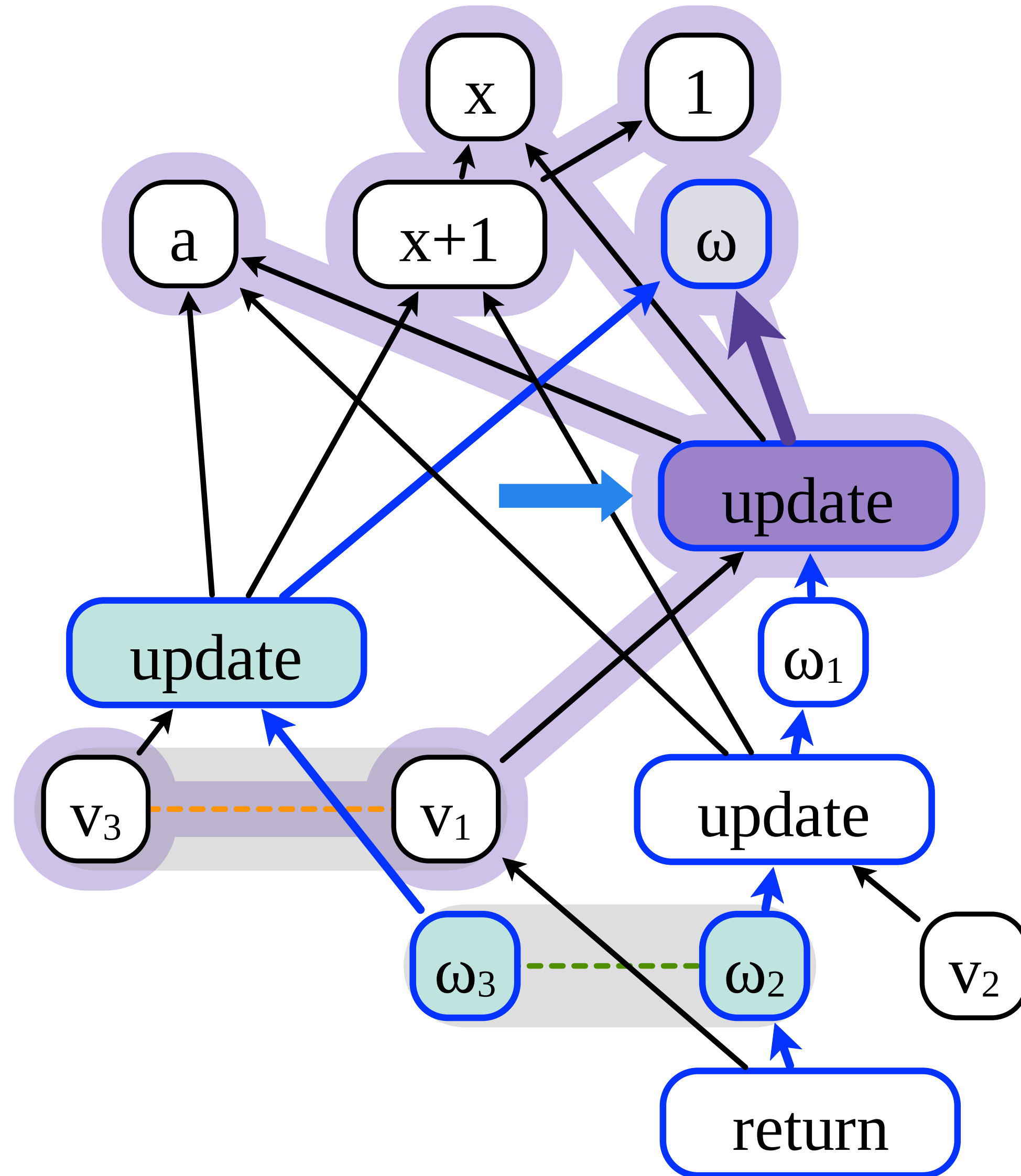
The Statewalk DP Algorithm



Key idea:
extend the
statewalk,
keep one per
extractable set

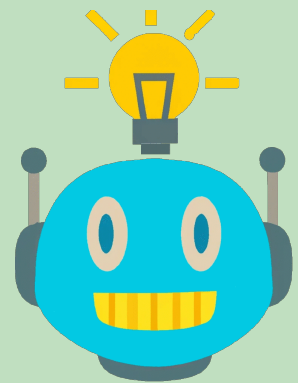


The Statewalk DP Algorithm

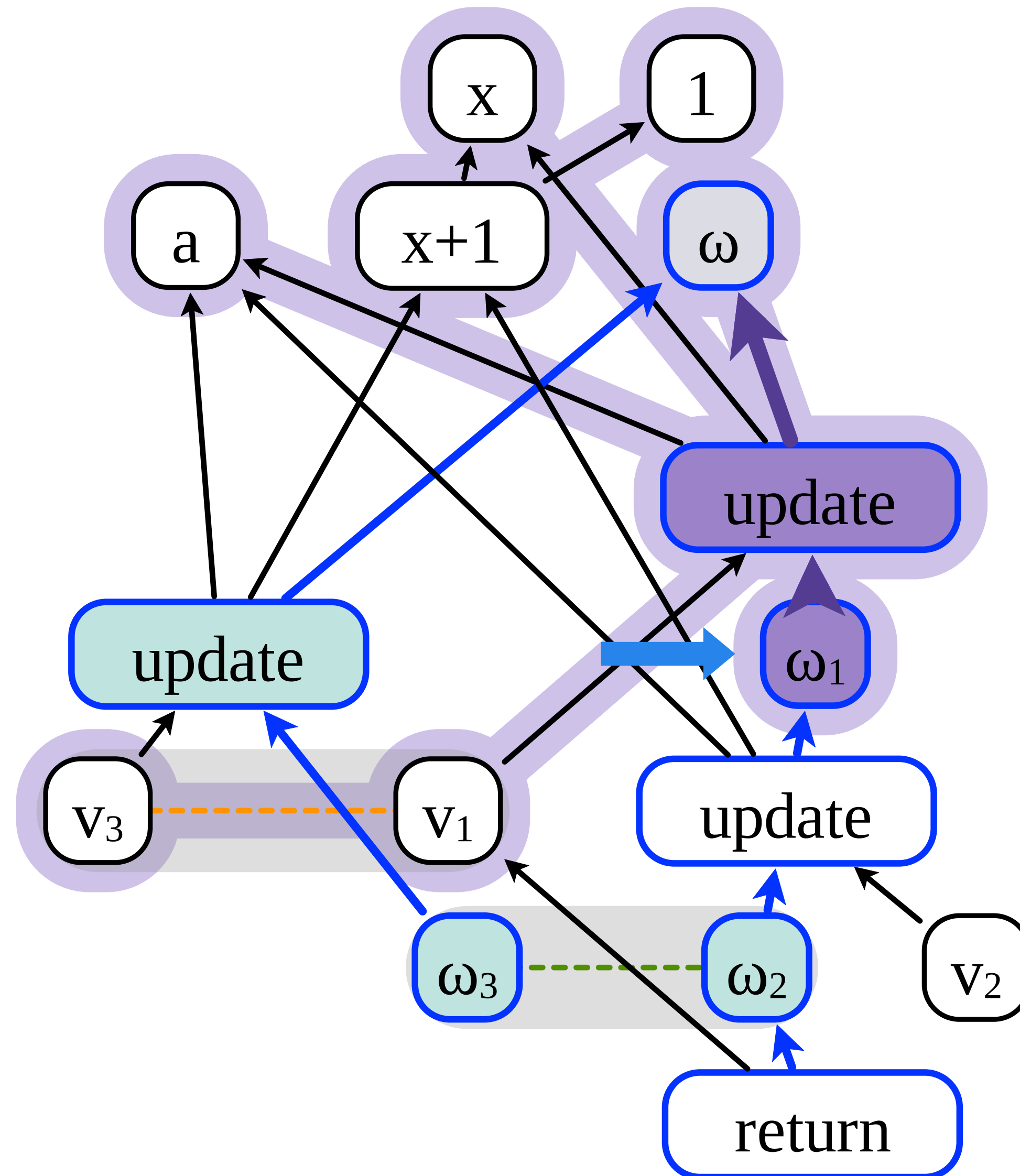


Key idea:
extend the
statewalk,
keep one per
extractable set

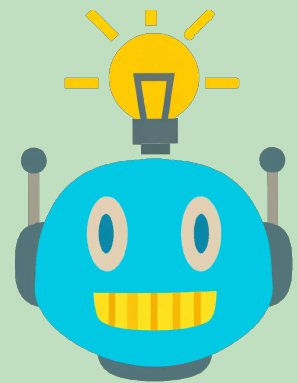
The Statewalk DP Algorithm



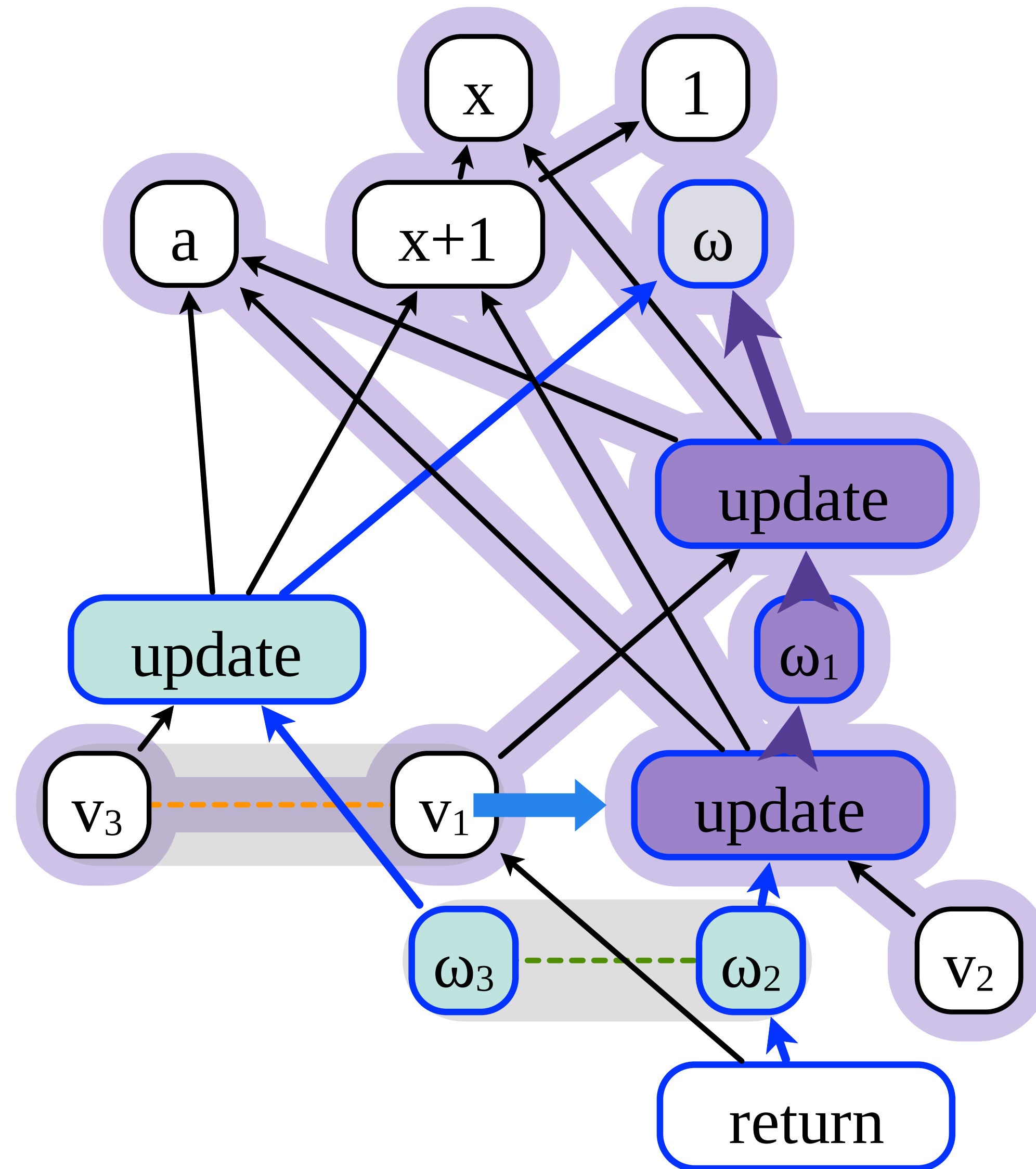
Key idea:
extend the
statewalk,
keep one per
extractable set



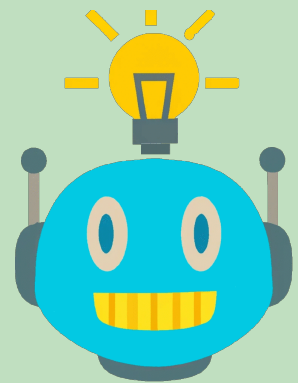
The Statewalk DP Algorithm



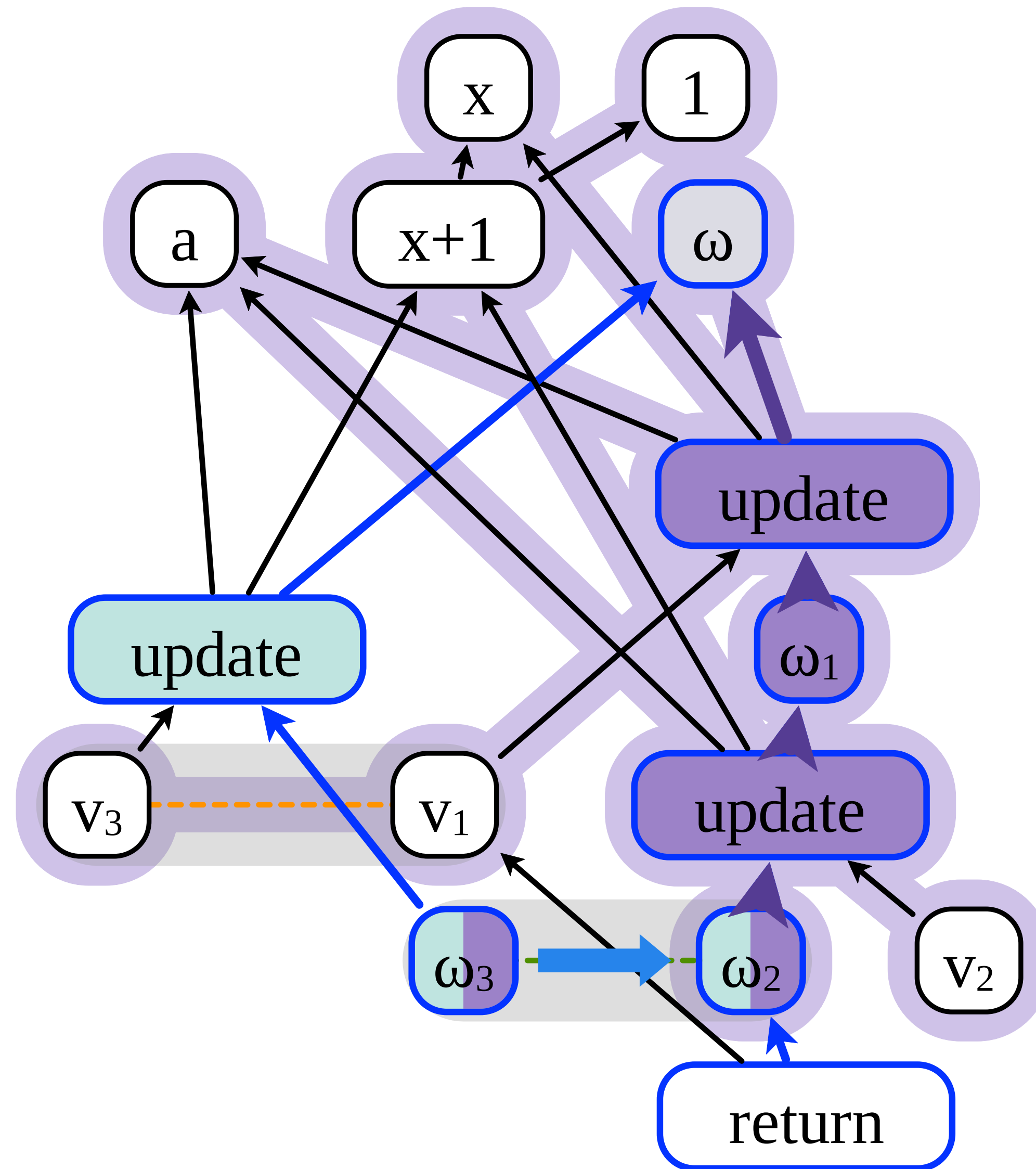
Key idea:
extend the
statewalk,
keep one per
extractable set



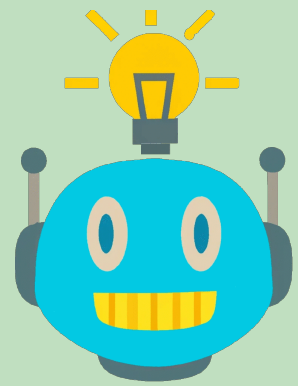
The Statewalk DP Algorithm



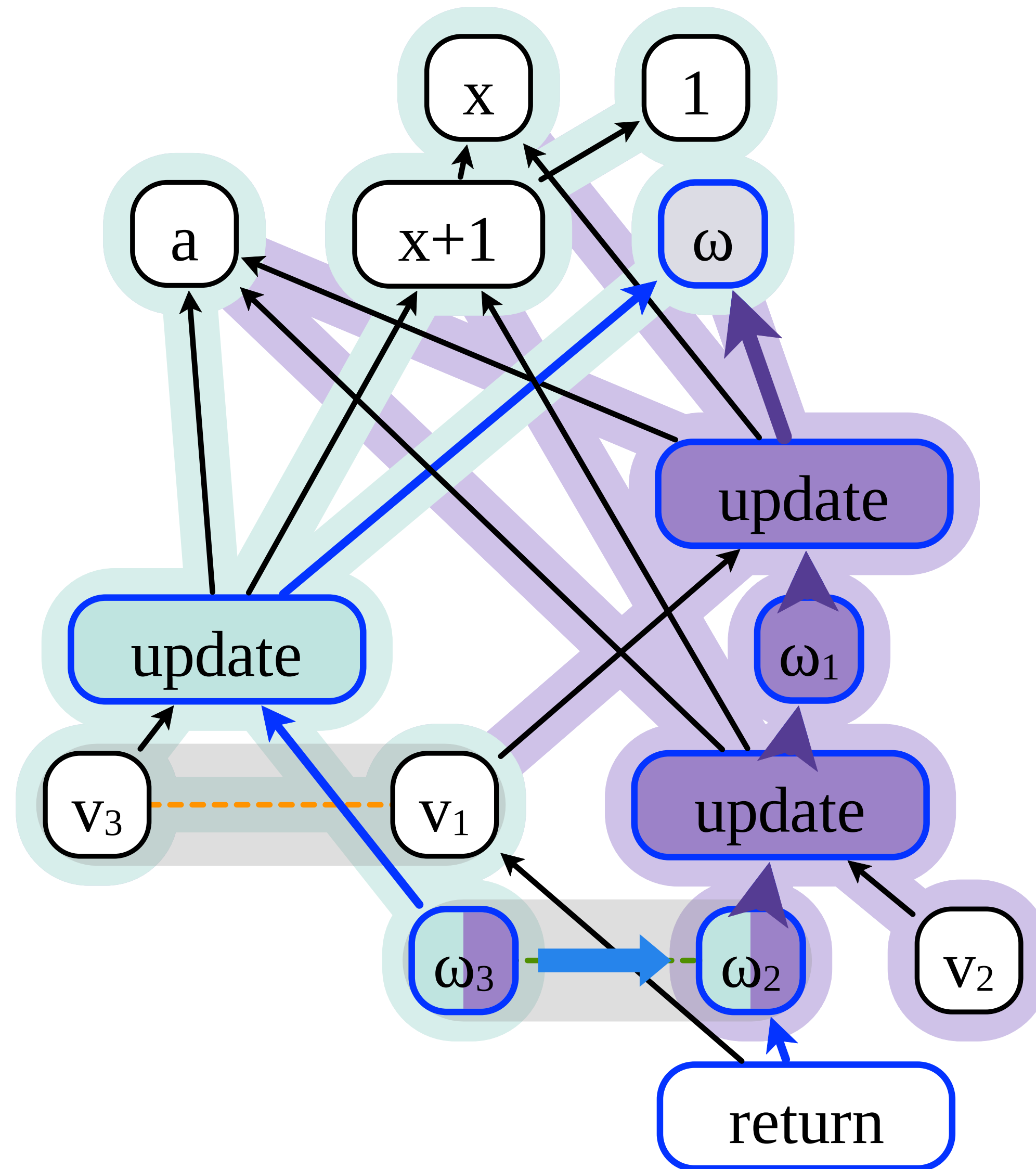
Key idea:
extend the
statewalk,
keep one per
extractable set



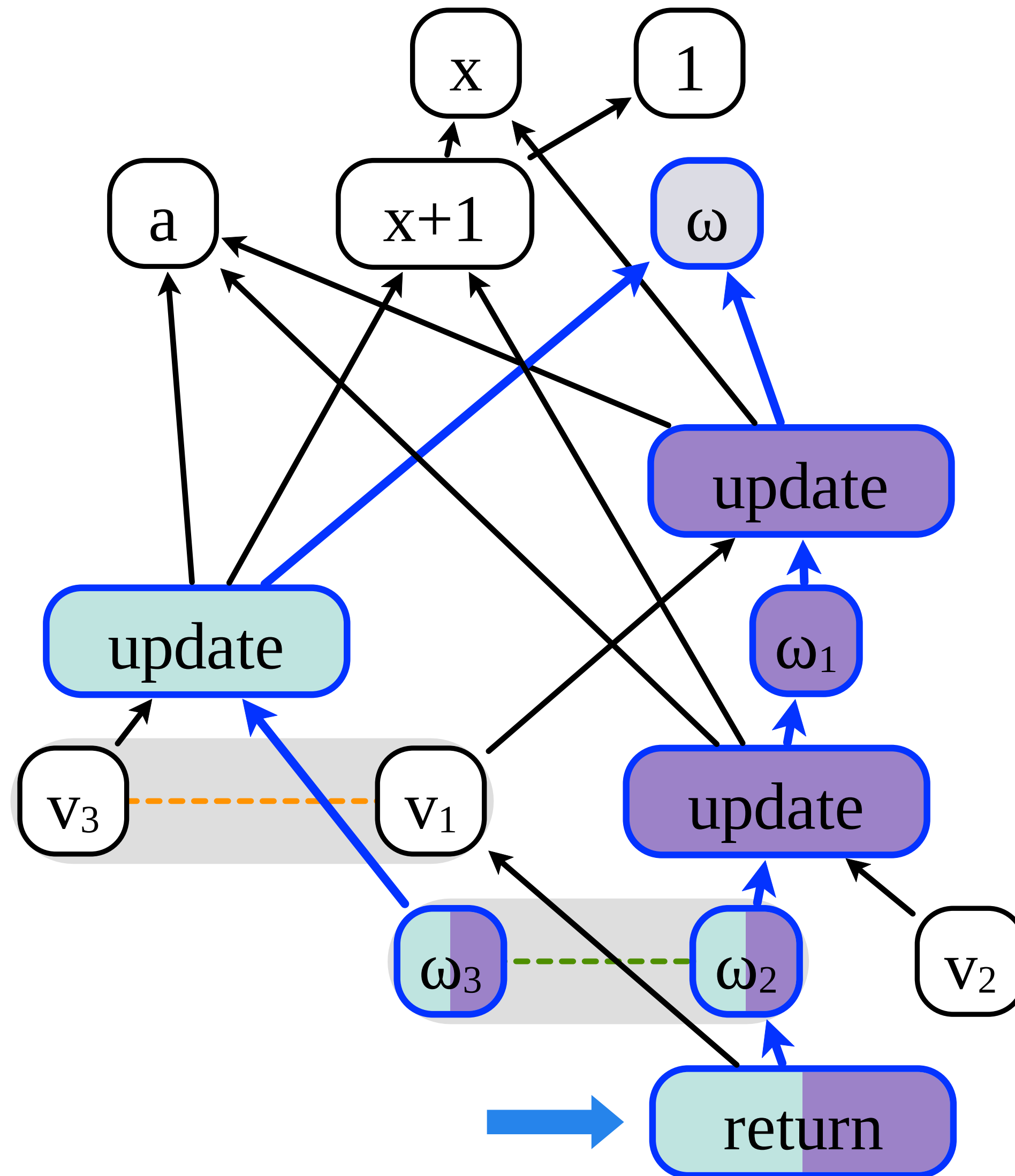
The Statewalk DP Algorithm



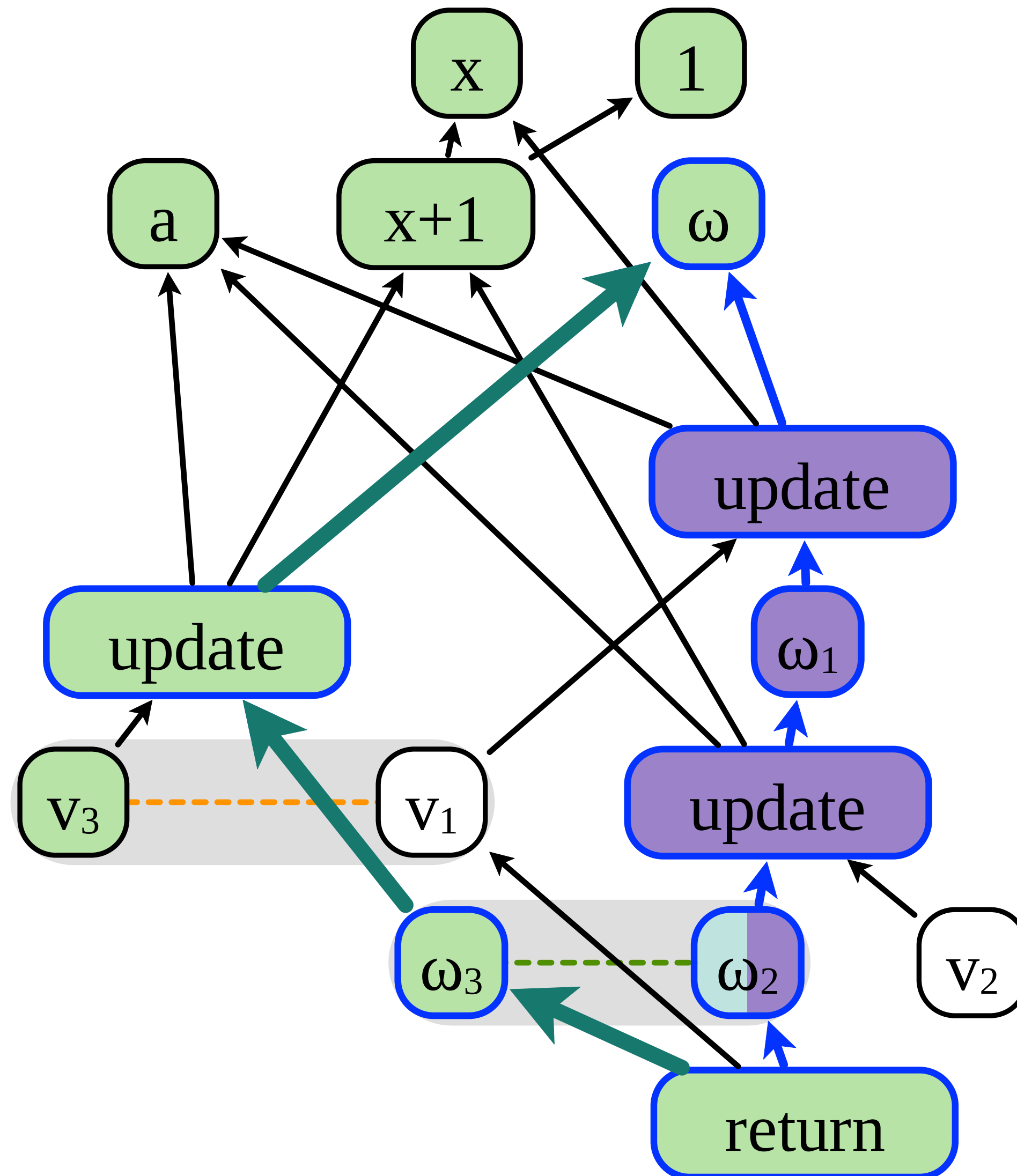
Key idea:
extend the
statewalk,
keep one per
extractable set



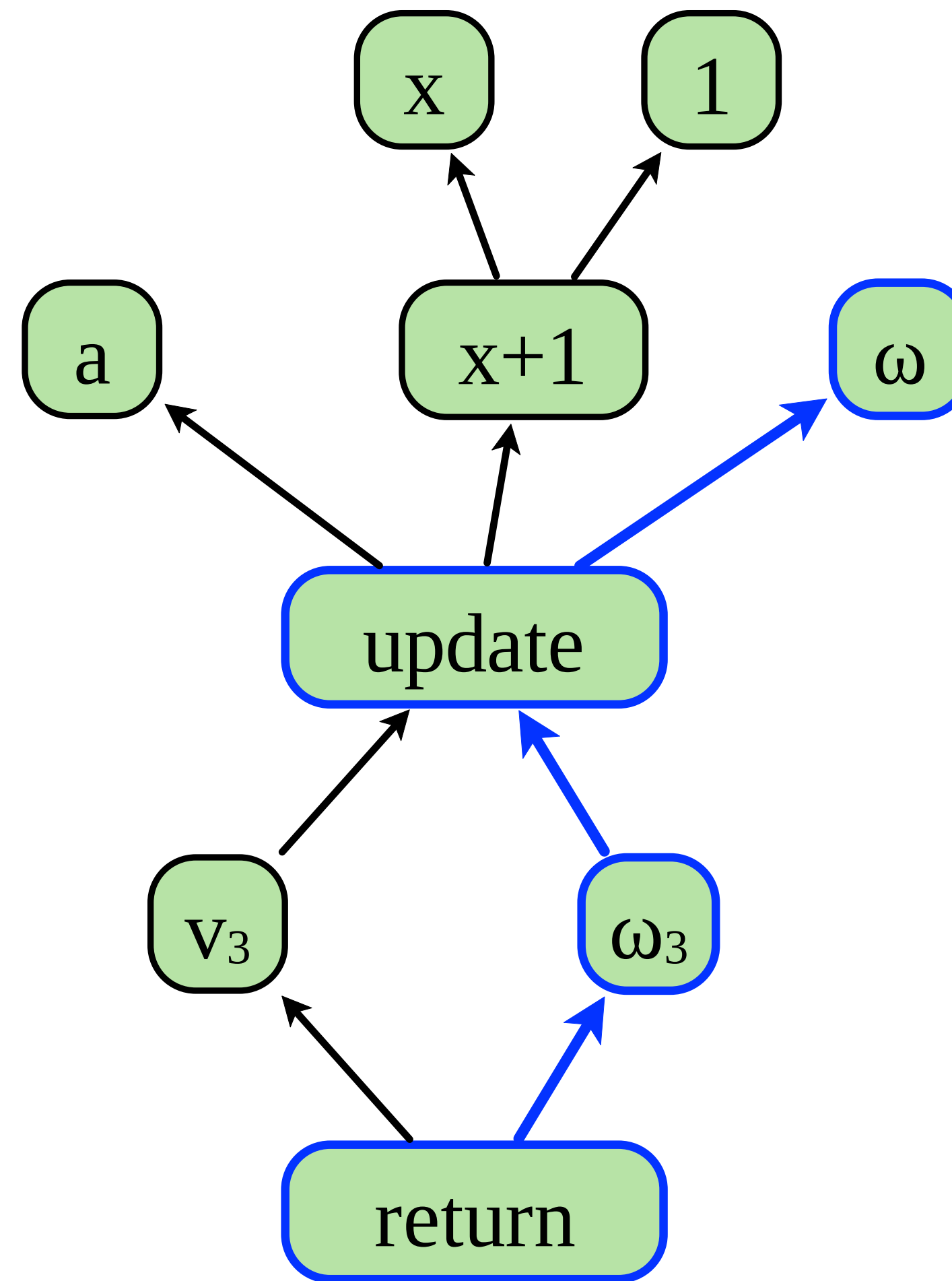
The Statewalk DP Algorithm

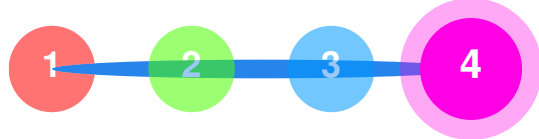


The Statewalk DP Algorithm



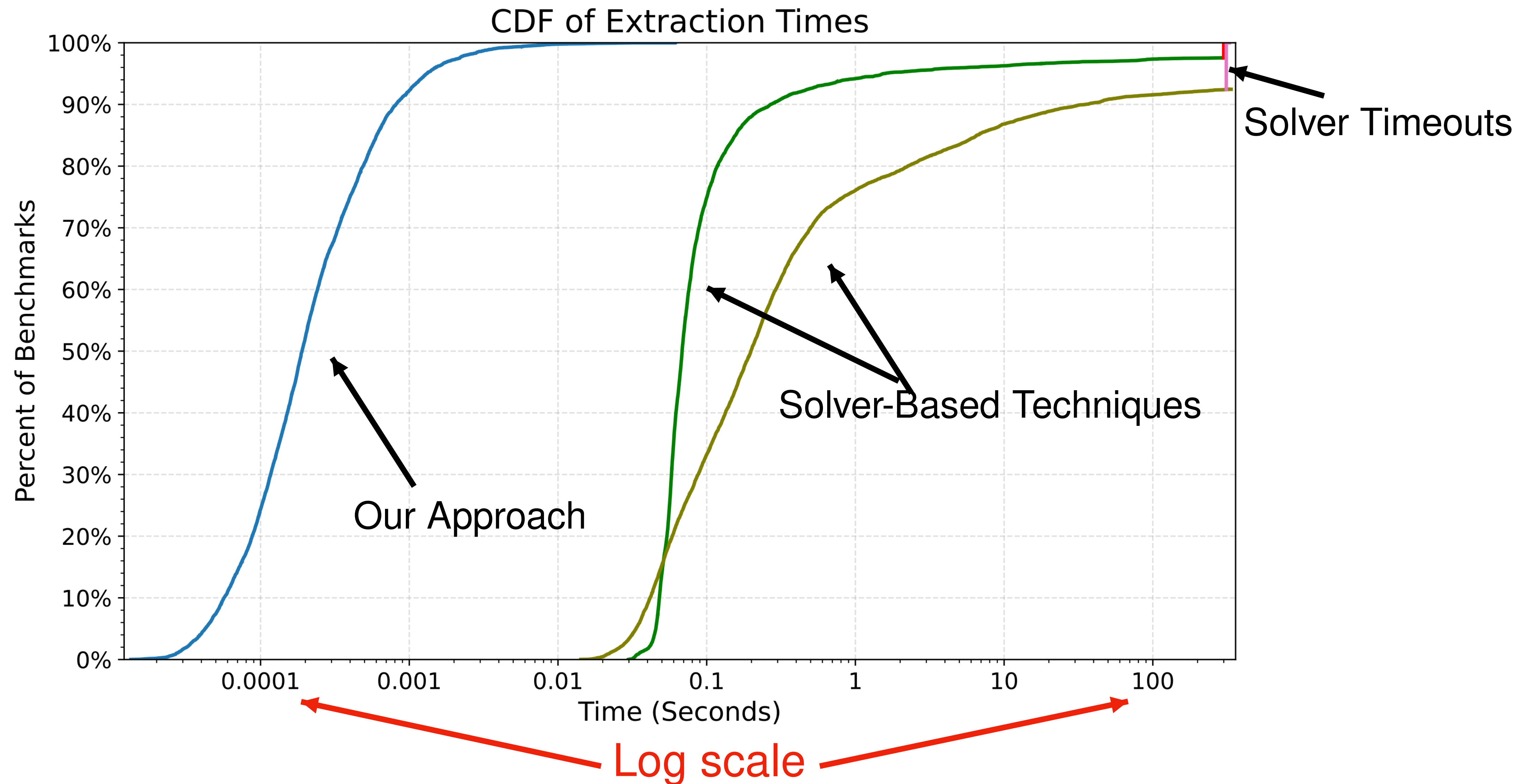
The Statewalk DP Algorithm

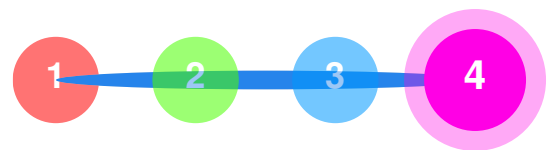




It Works!

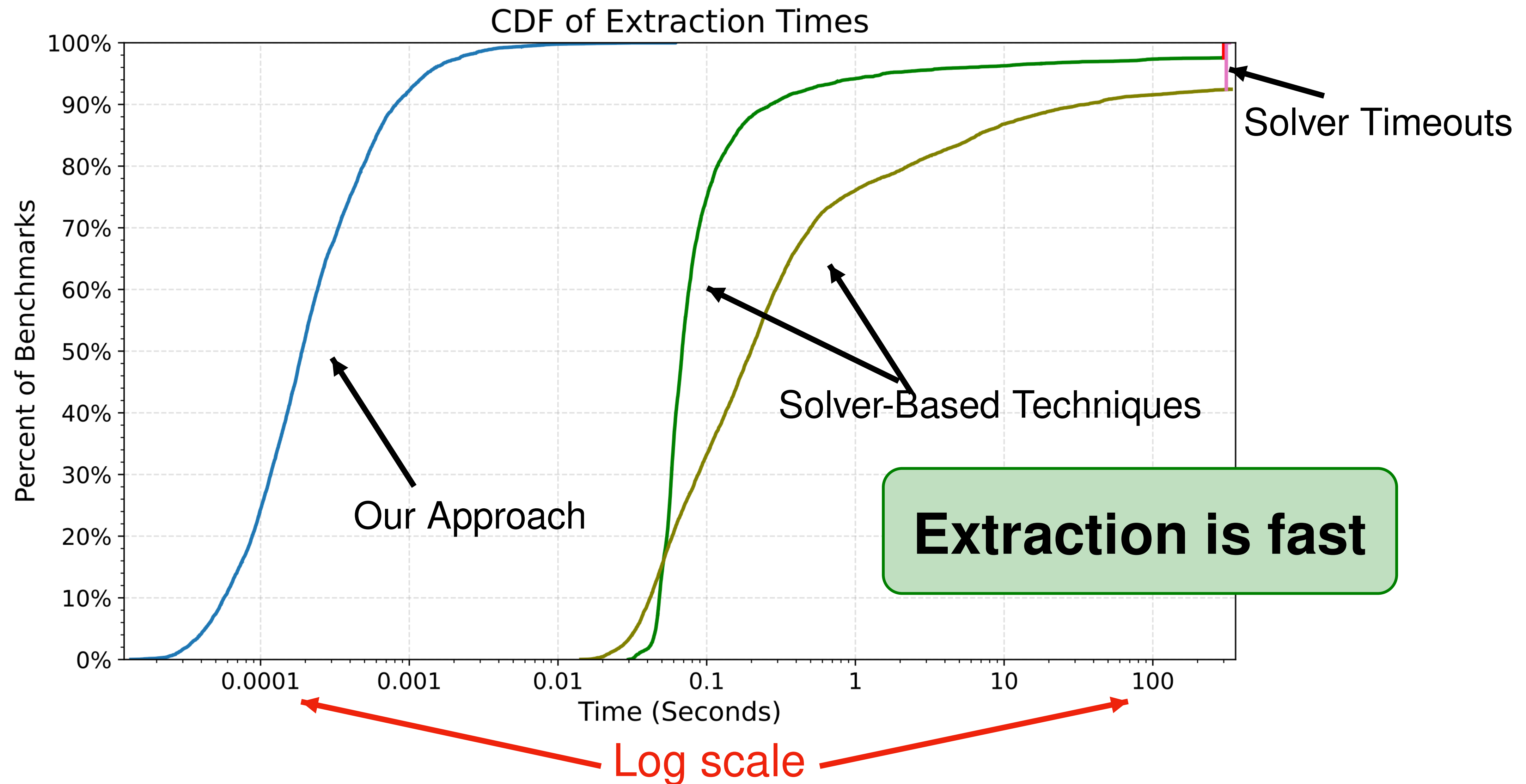
Fast Effect-safe Extraction in Practice



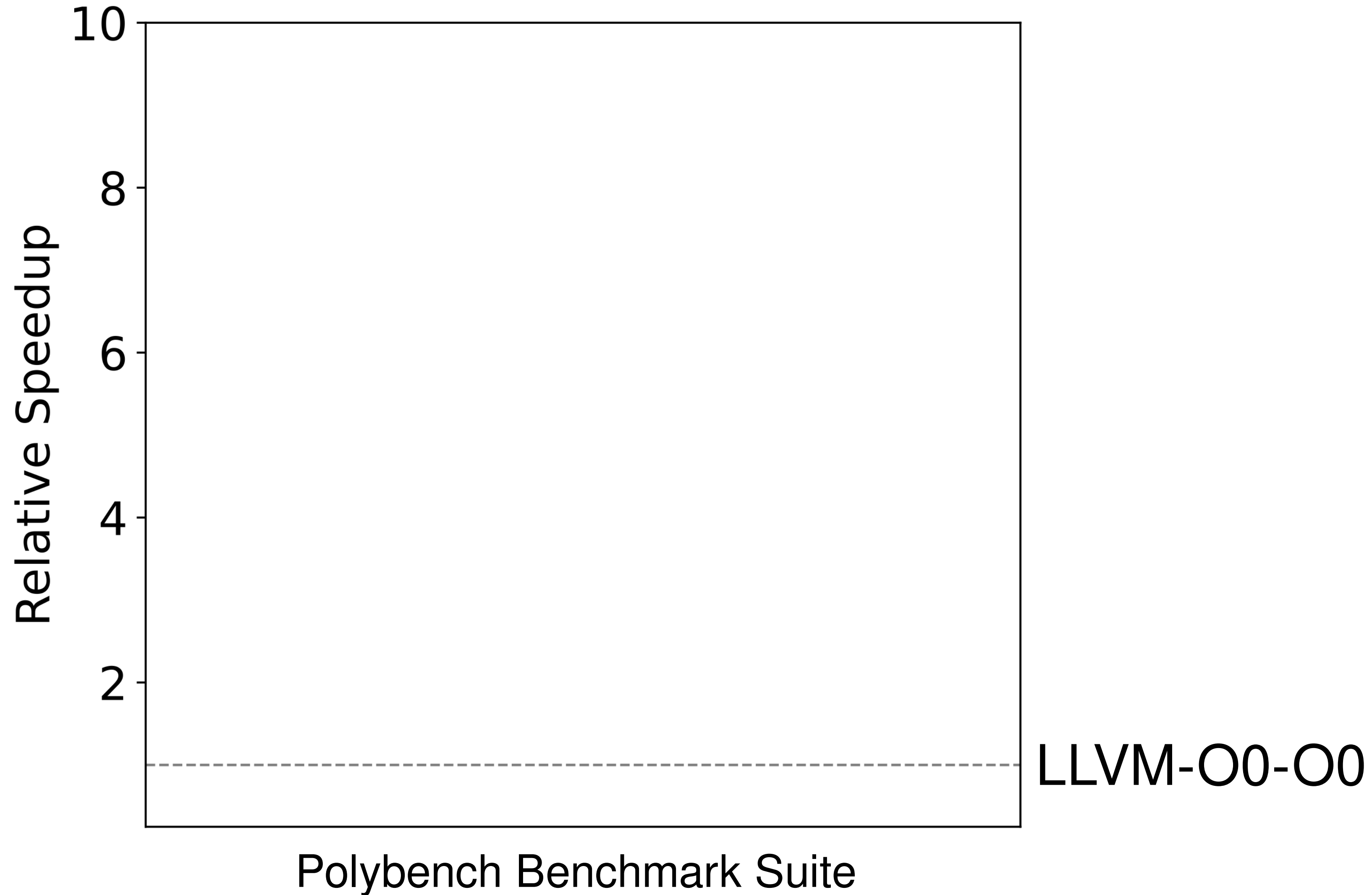


It Works!

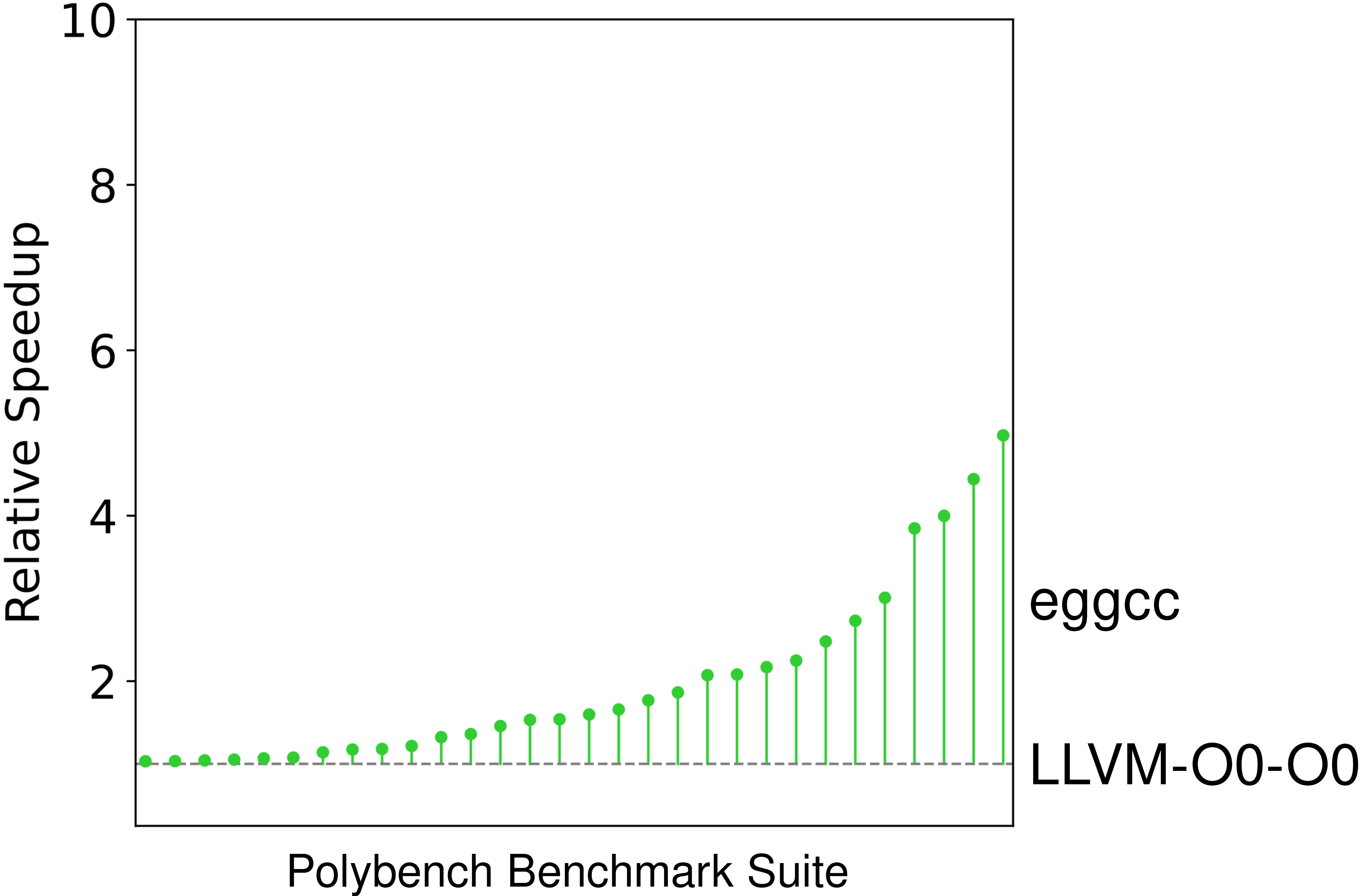
Fast Effect-safe Extraction in Practice



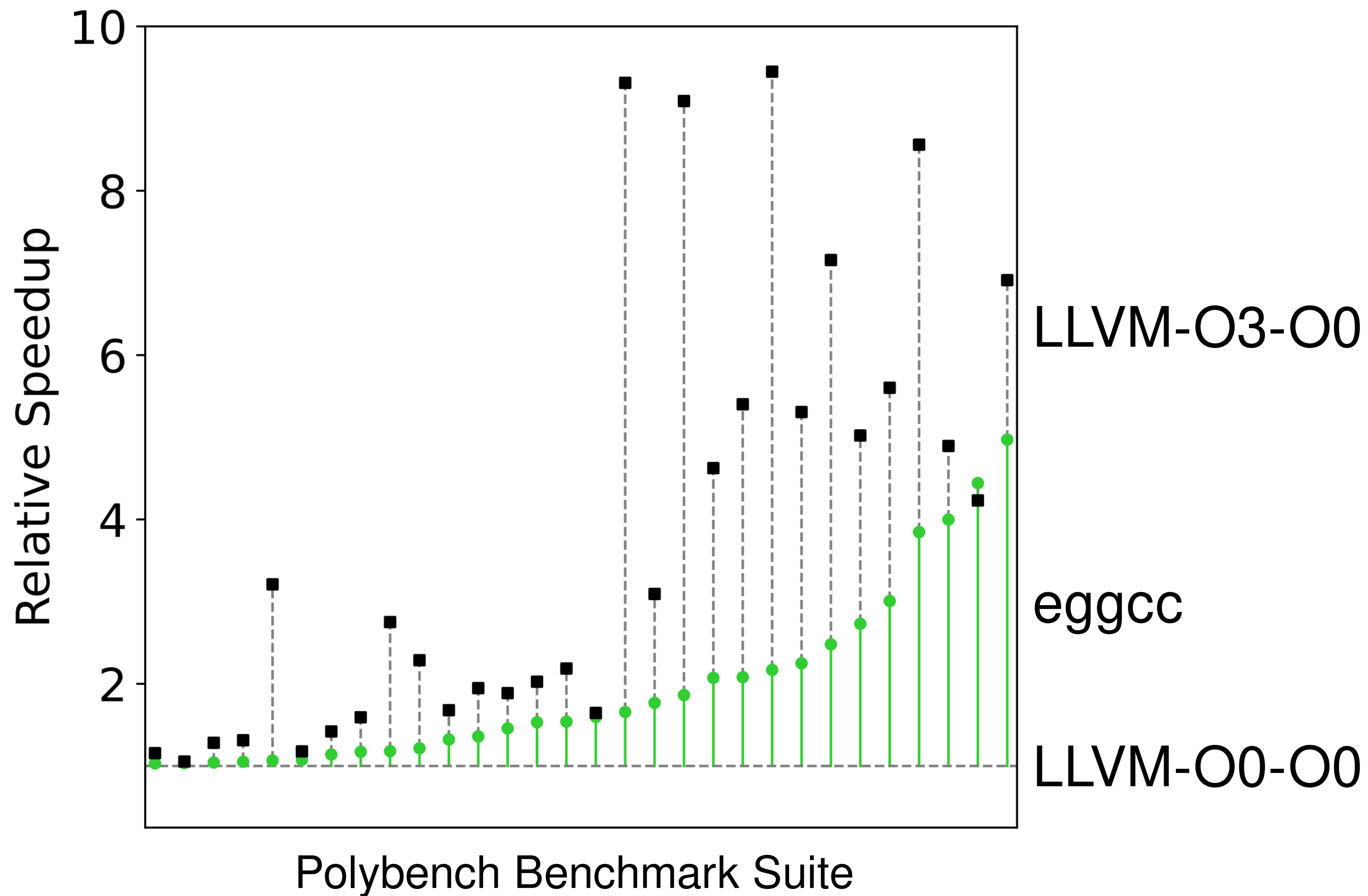
Fast Effect-safe Extraction in Practice



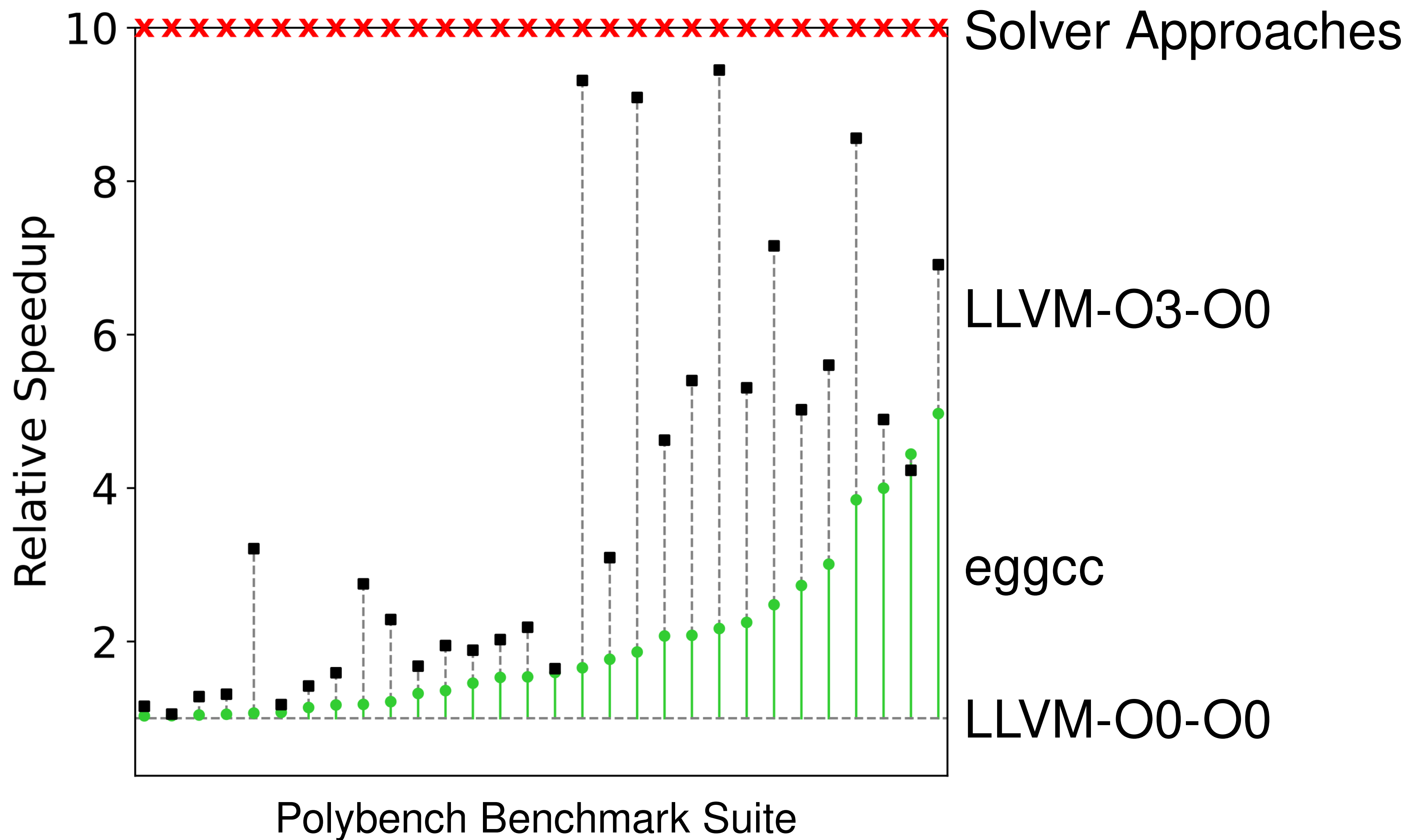
Fast Effect-safe Extraction in Practice



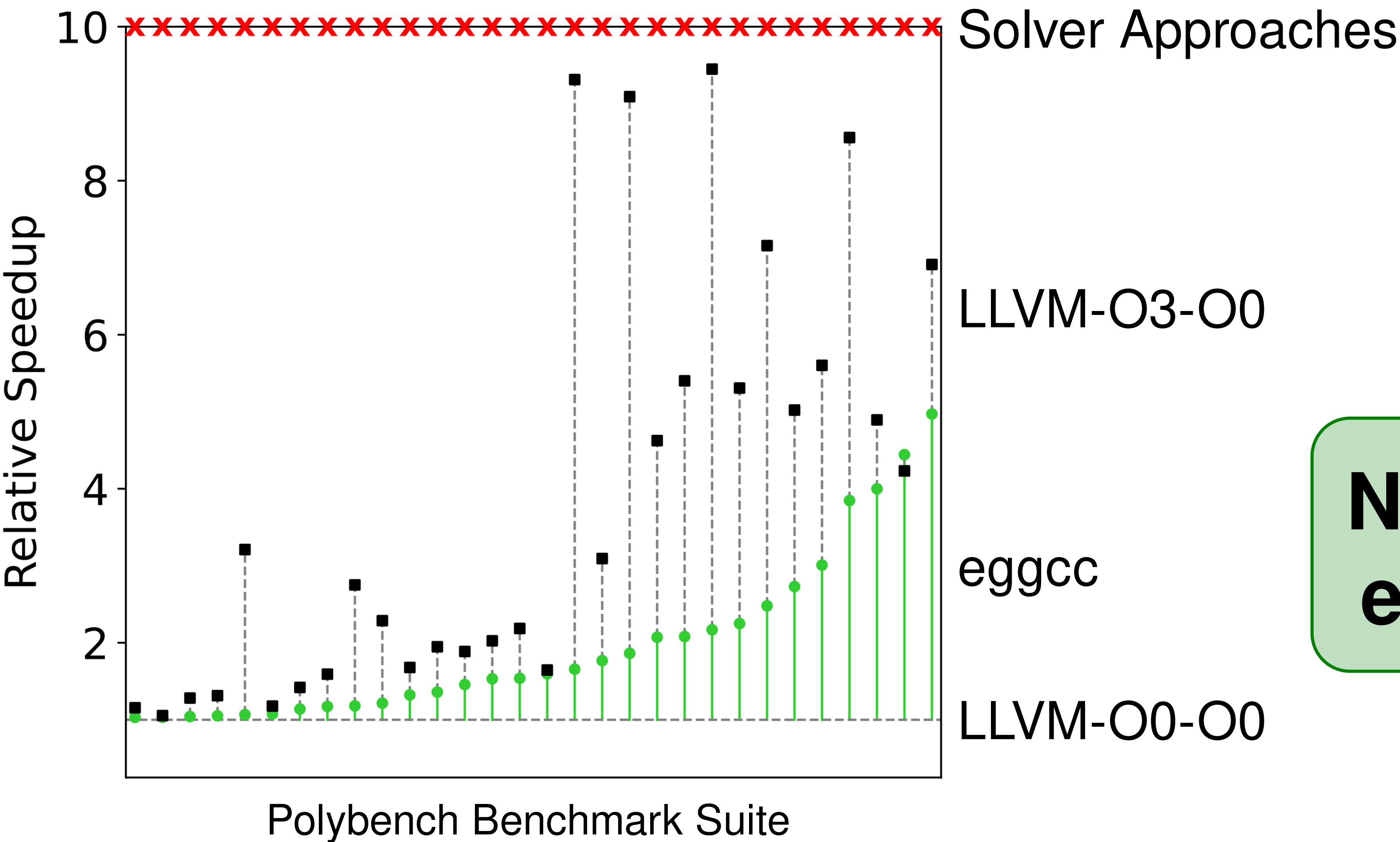
Fast Effect-safe Extraction in Practice



Fast Effect-safe Extraction in Practice



Fast Effect-safe Extraction in Practice

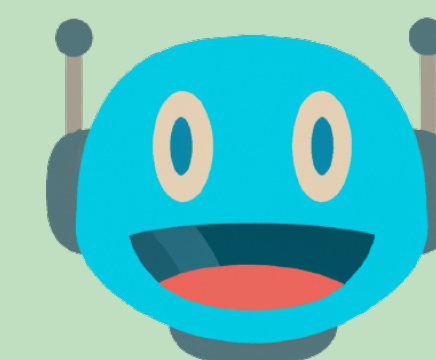


**Non-trivial
e-graphs!**

Efficient Extraction from Effectful E-Graphs



Oliver Flatt, Anjali Pal, Yihong Zhang, Ryan Tjoa, Kirsten Graham, Alex Fischman,
Chandrakana Nandi, Eli Rosenthal, Zachary Tatlock, Haobin Ni



Questions?

Efficient Extraction from Effectful E-Graphs



Oliver Flatt, Anjali Pal, Yihong Zhang, Ryan Tjoa, Kirsten Graham, Alex Fischman,
Chandrakana Nandi, Eli Rosenthal, Zachary Tatlock, Haobin Ni